Embodied AI

Comprehensive Lecture Notes

Vittorio Cava

A.Y. 2026/27

Contents

I Lecture 2 **2**

1 Representations for 3D *(slides slides 2–4)* **3**

2 Notation *(slides slide 5)* **4**

3 Manifolds *(slides slide 6)* **4**

4 Defining a Surface *(slides slide 7)* **5**

5 Meanwhile in 3D-land... *(slides slides 8–9)* **5**

6 Depth Maps *(slides slides 10–14)* **6**

6.1 Depth maps *(slides slides 10–12)* 6

6.2 Depth Maps and Normal Maps *(slides slide 13)* 6

6.3 Depth Maps — Representing the Visible *(slides slide 14)* 6

7 Point Clouds *(slides slides 15–21)* **7**

7.1 Point Clouds *(slides slides 15–18)* 7

7.2 Oriented Point Clouds *(slides slide 19)* 8

7.3 No explicit connectivity *(slides slides 20–21)* 8

8 Meshes *(slides slides 22–34)* **9**

8.1 Meshes: vertices and faces *(slides slides 22–25)* 9

8.2 Triangular Meshes *(slides slides 26–31)* 10

8.3 Triangular Meshes — Geometry vs Connectivity *(slides slides 32–34)* 12

9 Parametric Surfaces *(slides slides 35–41)* **14**

9.1 Parametric surfaces in 2D *(slides slide 35)* 14

9.2 Parametric surfaces *(slides slides 36–40)* 14

9.3 Parametric Surfaces — Pros and Cons *(slides slide 41)* 16

10 Surface Representations *(slides slide 42)* **16**

11 Volume Representations *(slides slides 43–46)* **17**
11.1 Voxelized 3D *(slides slides 44–46)* . . . 17

12 Implicit Surfaces *(slides slides 47–62)* **18**
12.1 Implicit surfaces *(slides slide 47)* . . . 18
12.2 Generate vs Test *(slides slide 48)* . . . 19
12.3 Implicit Surfaces: Examples *(slides slide 49)* . . . 19
12.4 When a Field Defines a Surface *(slides slide 50)* . . . 20
12.5 Normals for Free *(slides slide 51)* . . . 21
12.6 Boolean Operations *(slides slide 52)* . . . 21
12.7 Many Functions, One Surface *(slides slide 53)* . . . 22
12.8 Signed Distance Function *(slides slides 54–58)* . . . 22
12.9 Storing a Field on a Grid *(slides slide 59)* . . . 24
12.10 Storing a Field in a Network *(slides slide 60)* . . . 24
12.11 From Implicit Surface to Mesh *(slides slide 61)* . . . 26
12.12 Implicit Surfaces: Summary *(slides slide 62)* . . . 27

13 Boundary vs Volumetric Representations *(slides slide 63)* **27**

14 Density Fields *(slides slides 64–65)* **27**
14.1 Density fields *(slides slide 64)* . . . 27
14.2 Primitive-based Density Fields *(slides slide 65)* . . . 28

15 So? *(slides slide 66)* **28**

16 And many more... *(slides slides 67–71)* **29**
16.1 Octrees *(slides slide 68)* . . . 29
16.2 Layered Depth Images *(slides slide 69)* . . . 30
16.3 Multi-plane Images *(slides slide 70)* . . . 30
16.4 CAD Models and CSG Trees *(slides slide 71)* . . . 31

17 Summary *(slides slides 72–74)* **31**

II Lecture 3 **33**

18 From Image Formation to Measurement *(slides 1–3)* **34**

19 From Representations to Measurements *(slides 4–7)* **35**
19.1 Recovered and Ambiguous Quantities *(slides 5)* . . . 35
19.2 The Back-Projected Ray *(slides 6)* . . . 35
19.3 Scale-Depth Ambiguity *(slides 7)* . . . 36

20 Three Ways to Pay for Depth *(slides 8–12)* **38**
20.1 Two Measurement Principles *(slides 9)* . . . 38
20.2 Triangulation From Two Views *(slides 10)* . . . 38
20.3 The Three Currencies *(slides 11)* . . . 39
20.4 The Single-Image Claim *(slides 12)* . . . 39

21 Inside a LiDAR *(slides 13–17)* **40**
21.1 Pulsed Time of Flight *(slides 14)* . . . 40
21.2 Structured Sparsity *(slides 15)* . . . 40
21.3 Limitations of a LiDAR Point Cloud *(slides 16)* . . . 42
21.4 Two Error Laws *(slides 17)* . . . 43

22 From Point Cloud to Depth Image *(slides 18–19)* **43**
22.1 Projecting a Cloud Into the Image *(slides 19)* . . . 44

23 From Recovery to Prediction *(slides 20–22)* **45**
23.1 Monocular Cues in Human Vision *(slides 21)* . . . 46
23.2 Depth Prediction as a Prior *(slides 22)* . . . 46

24 Cues in Closed Form *(slides 23–26)* **47**
24.1 Vanishing Points *(slides 24)* . . . 47
24.2 Apparent Size *(slides 25)* . . . 48
24.3 Vertical Position *(slides 26)* . . . 49

25 Supervised Single-View Depth *(slides 27–35)* **49**
25.1 The Supervised Setup *(slides 28)* . . . 50
25.2 Three Datasets *(slides 29)* . . . 50
25.3 The Scale-Invariant Loss *(slides 30)* . . . 50
25.4 Output Spaces *(slides 31)* . . . 52
25.5 Affine Invariance in Disparity and in Depth *(slides 32)* . . . 52
25.6 MiDaS: Mixing the Datasets *(slides 33)* . . . 53
25.7 Depth Anything V2 *(slides 34)* . . . 54
25.8 Failure on Anamorphic Art *(slides 35)* . . . 55

26 Cues Used by the Network *(slides 36–39)* **56**
26.1 The Pasted-Car Experiment *(slides 37)* . . . 56
26.2 Pitch and Roll *(slides 38)* . . . 56
26.3 Dependence on the Training Camera *(slides 39)* . . . 58

Next: Self-Supervision and a Generative Prior *(slide 40)* **58**

Overview: Part I in One Slide *(slides 1–2)* **59**

27 Supervision without Ground Truth *(slides 3–6)* **59**

27.1 View Synthesis and the Backward Warp *(slides 4)* . . . 59
27.2 Image Reconstruction as Supervision *(slides 5)* . . . 60
27.3 Where the Loss Is Blind *(slides 6)* . . . 61

28 Diffusion, From Scratch *(slides 7–22)* **62**
28.1 Supervised Learning *(slides 8)* . . . 62
28.2 Generative Learning *(slides 9)* . . . 62
28.3 Many Outputs for One Input – Text to Image *(slides 10)* . . . 63
28.4 The Forward and Reverse Process *(slides 11)* . . . 63
28.5 Training by Noise Prediction *(slides 12)* . . . 64
28.6 The Spiral Toy Model *(slides 13)* . . . 65
28.7 Training the Toy Model *(slides 14)* . . . 65
28.8 The Ideal Denoiser *(slides 15)* . . . 66
28.9 The Smoothed Distance Function *(slides 16)* . . . 67
28.10 Sampling From a Diffusion Model – Why iterative? *(slides 17)* . . . 68
28.11 The DDIM Sampler *(slides 18)* . . . 68
28.12 Probabilistic Sampling *(slides 19)* . . . 69
28.13 Three Uses of a Prior *(slides 20)* . . . 69
28.14 The Autoencoder and Latent Diffusion *(slides 21)* . . . 69
28.15 Conditioning and Guidance *(slides 22)* . . . 70

29 Marigold *(slides 23–31)* **70**
29.1 The Marigold Method *(slides 24)* . . . 70
29.2 From the Spiral to Marigold *(slides 25)* . . . 71
29.3 The Source of Affine Invariance *(slides 26)* . . . 71
29.4 Conditioning on the Image *(slides 27)* . . . 72
29.5 Training Marigold *(slides 28)* . . . 73
29.6 Inference *(slides 29)* . . . 74
29.7 Ensembling Over Samples *(slides 30)* . . . 74
29.8 Limitations of Marigold *(slides 31)* . . . 74

30 Back to 3D *(slides 32–34)* **75**
30.1 Effect of Scale and Shift Errors *(slides 33)* . . . 75
30.2 Sources of Scale and Shift *(slides 34)* . . . 75

31 Marigold-DC *(slides 35–38)* **76**
31.1 The Depth Completion Task *(slides 36)* . . . 76
31.2 Steering the Loop *(slides 37)* . . . 77
31.3 Nothing Retrained *(slides 38)* . . . 78

Summary *(slides 39–40)* **80**

A Supplementary derivations (not on the slides) **82**

III Lecture 4 **83**

Agenda and Learning Objectives (slides 1–2) **84**

B Two-View Geometry *(slides 3–13)* **84**
B.1 Depth Cues and Binocular Stereopsis *(slides 4)* . . . 84
B.2 Two-View Stereo Matching *(slides 5)* . . . 84
B.3 The Classical Stereo Pipeline *(slides 6)* . . . 85
B.4 Epipolar Geometry *(slides 7)* . . . 85
B.5 Image Rectification *(slides 8)* . . . 86
B.6 Rectified Epipolar Constraint *(slides 9)* . . . 87
B.7 Calculating the Rectifying Rotation Matrix *(slides 10)* . . . 87
B.8 Rectified Pair and Disparity *(slides 11)* . . . 88
B.9 Disparity to Depth *(slides 12)* . . . 88
B.10 Worked Example: Baseline and Depth Error *(slides 13)* . . . 89

C Block Matching *(slides 14–21)* **90**
C.1 Correspondence Ambiguity *(slides 15)* . . . 90
C.2 Similarity Metrics *(slides 16)* . . . 90
C.3 Block Matching *(slides 17)* . . . 91
C.4 Block Matching: Half Occlusions *(slides 18)* . . . 92
C.5 Block Matching: Assumption Violations *(slides 19)* . . . 93
C.6 Effect of Window Size *(slides 20)* . . . 94
C.7 Left-Right Consistency Test *(slides 21)* . . . 94

D Siamese Networks *(slides 22–28)* **95**
D.1 Siamese Networks for Stereo Matching *(slides 23)* . . . 95
D.2 Siamese Network Architecture *(slides 24)* . . . 96
D.3 Training Triplets *(slides 25)* . . . 97
D.4 Hinge Loss *(slides 26)* . . . 97
D.5 Comparisons *(slides 27)* . . . 98
D.6 Similarity Constraint: Failure Cases *(slides 28)* . . . 99

E End-to-End Learning *(slides 29–31)* **99**
E.1 DispNet and Synthetic Data *(slides 30)* . . . 99
E.2 GC-Net *(slides 31)* . . . 100

F Stereo Without Calibration *(slides 32–45)* **101**
F.1 The Cost of Calibration *(slides 33)* . . . 101
F.2 Limits of Rectification and Local Matching *(slides 34)* . . . 102

F.3 Pointmaps *(slides 35)* . . . 102
F.4 The DUSt3R Architecture *(slides 36)* . . . 103
F.5 MASt3R Network Output *(slides 37)* . . . 104
F.6 Training Objective and Metric Scale *(slides 38)* . . . 104
F.7 The Matching Head *(slides 39)* . . . 105
F.8 InfoNCE and the Hinge Loss *(slides 40)* . . . 105
F.9 Effect of 3D Grounding *(slides 41)* . . . 106
F.10 Reciprocal Matching *(slides 42)* . . . 107
F.11 MASt3R Training Details *(slides 43)* . . . 107
F.12 Results *(slides 44)* . . . 108
F.13 The Pipeline Revisited *(slides 45)* . . . 108

Summary **109**
Comparison of the methods covered . . . 109
Key take-aways . . . 109

IV Lecture 5 **111**

Agenda and Learning Objectives *(slides 1–2)* **112**

G Camera Calibration *(slides 3–7)* **112**
G.1 What calibration is and how it is done *(slides 4–7)* . . . 112

H Feature Detection and Description *(slides 8–20)* **114**
H.1 Point Features *(slides 9–11)* . . . 114
H.2 Scale Invariant Feature Transform (SIFT) *(slides 12–15)* . . . 115
H.3 Matching features *(slides 14–15)* . . . 116
H.4 SuperPoint: Learned Interest Points and Descriptors *(slides 16–20)* . . . 117

I Two-Frame Structure-from-Motion *(slides 21–35)* **120**
I.1 Epipolar Geometry *(slides 22–23)* . . . 120
I.2 Structure of the Essential Matrix *(slides 24)* . . . 121
I.3 Estimating the Essential Matrix: the 8-point algorithm *(slides 25)* . . . 122
I.4 Decomposing the Essential Matrix *(slides 26)* . . . 123
I.5 Geometry of the Four Solutions *(slides 27)* . . . 123
I.6 Disambiguation by Cheirality *(slides 28)* . . . 124
I.7 Estimating the Epipolar Geometry with Unknown Intrinsics *(slides 29–30)* . . . 125
I.8 Triangulation *(slides 31–35)* . . . 125

J Bundle Adjustment *(slides 36–54)* **128**
J.1 Bundle Adjustment *(slides 37–39)* . . . 128
J.2 Challenges of Bundle Adjustment *(slides 40)* . . . 128

J.3 Bundle Adjustment as Nonlinear Least Squares *(slides 41)* . . . 129
J.4 Updates in the Tangent Space *(slides 42)* . . . 129
J.5 The Gauss–Newton Algorithm *(slides 43)* . . . 130
J.6 The Schur Complement *(slides 44)* . . . 131
J.7 Incremental Structure-from-Motion *(slides 45–51)* . . . 132
J.8 Building Rome in a Day, COLMAP SfM and COLMAP MVS *(slides 52–54)* . . . 134

K Feed-Forward Structure-from-Motion *(slides 55–68)* 134
K.1 Limits of Incremental Structure-from-Motion *(slides 56)* . . . 134
K.2 From Learned Components to Learned Reconstruction *(slides 57)* . . . 135
K.3 Visual Geometry Grounded Transformer (VGGT) *(slides 58)* . . . 136
K.4 Problem Definition *(slides 59)* . . . 136
K.5 Architecture: Alternating Attention *(slides 60)* . . . 136
K.6 Prediction Heads and Tracking *(slides 61)* . . . 137
K.7 Training Objective and Normalization *(slides 62)* . . . 137
K.8 Results: Camera Pose Estimation *(slides 63)* . . . 138
K.9 Results: Dense Geometry *(slides 64)* . . . 139
K.10 Results: Matching and Tracking *(slides 65)* . . . 139
K.11 Bundle Adjustment from VGGT Predictions *(slides 66)* . . . 140
K.12 Limitations *(slides 67)* . . . 140
K.13 Comparison of Incremental and Feed-Forward SfM *(slides 68)* . . . 141

Summary *(slide 69)* 142

V Lecture 6 144

L The SLAM Problem *(slides 3–6)* 144
L.1 The SLAM Problem 3 . . . 144
L.2 Cameras for SLAM 4 . . . 145
L.3 SLAM and Structure-from-Motion 5 . . . 145
L.4 Running Example: VBR Campus 6 . . . 146

M Filtering: From the Bayes Filter to EKF-SLAM *(slides 7–13)* 146
M.1 SLAM as a Graphical Model 8 . . . 146
M.2 The Bayes Filter 9 . . . 147
M.3 Kalman Filter and Extended Kalman Filter 10 . . . 147
M.4 The EKF-SLAM State 11 . . . 148
M.5 Example: MonoSLAM 12 . . . 149
M.6 Limits of Filtering 13 . . . 149

N Graph-Based SLAM: Factor Graphs and Least Squares *(slides 14–22)* 150
N.1 Graph-Based SLAM 15 . . . 150

N.2 Factor Graphs 16 . . . 150
N.3 Visual SLAM as a Factor Graph 17 . . . 151
N.4 The Pose Graph 18 . . . 151
N.5 From Factors to Least Squares 19 . . . 152
N.6 Error and Information Matrix of an Edge 20 . . . 152
N.7 Gauss-Newton on the Pose Graph 21 . . . 152
N.8 Sparsity of the Normal Equations 22 . . . 153

O The Visual SLAM Pipeline: ORB-SLAM *(slides 23–32)* **154**
O.1 Pipeline Overview 24 . . . 154
O.2 ORB Features and Matching 25 . . . 154
O.3 Map Initialization 26 . . . 155
O.4 Tracking 27 . . . 155
O.5 Keyframes 28 . . . 156
O.6 Local Mapping 29 . . . 156
O.7 Loop Detection with Bags of Binary Words 30 . . . 157
O.8 Loop Correction and Scale Drift 31 . . . 158
O.9 Relocalization 32 . . . 158

P Research Directions *(slides 33–40)* **159**
P.1 Continuous Time: Rolling-Shutter Cameras 34 . . . 159
P.2 Continuous Time: Cumulative B-Splines on 35 . . . 159
P.3 Active SLAM 36 . . . 160
P.4 Active SLAM with an Active Stereo Head 37 . . . 160
P.5 Active Localization: Learning Where to Look 38 . . . 161
P.6 Active View Selection with Radiance Fields 39 . . . 162
P.7 Outlook: Learned Front Ends 40 . . . 162

Q Summary 41 **163**

Part I

Lecture 2

Abstract

These notes follow the slide deck of Lecture 02 (75 slides) in the order in which the slides present the material. Every section header carries the range of slides it covers. Formal statements are typeset in boxes (**Definition**, **Proposition**, **Algorithm**, ...), and every result or non-obvious idea is followed by a blue *Intuition* box. All figures are original TikZ/pgfplots redrawings, so the document is self-contained. The last slide of the file (slide 75, a backup slide placed after the acknowledgments) is collected in Appendix **??**.

1 Representations for 3D *(slides slides 2–4)*

The deck opens with the same bunny (and a knot) shown as a triangle mesh, a point cloud, a smooth surface and a voxel grid: the *same shape*, four very different data structures.

Life is nice in 2D-land… *(slides slide 3)*

In 2D, every camera (an old box camera, a compact camera, a phone) produces the same object: a regular grid of pixels. That single representation is shared across *sensors*, *tools* (Photoshop) and *learning frameworks* (PyTorch, TensorFlow). Nobody asks "which image representation is best".

Meanwhile in 3D land… *(slides slide 4)*

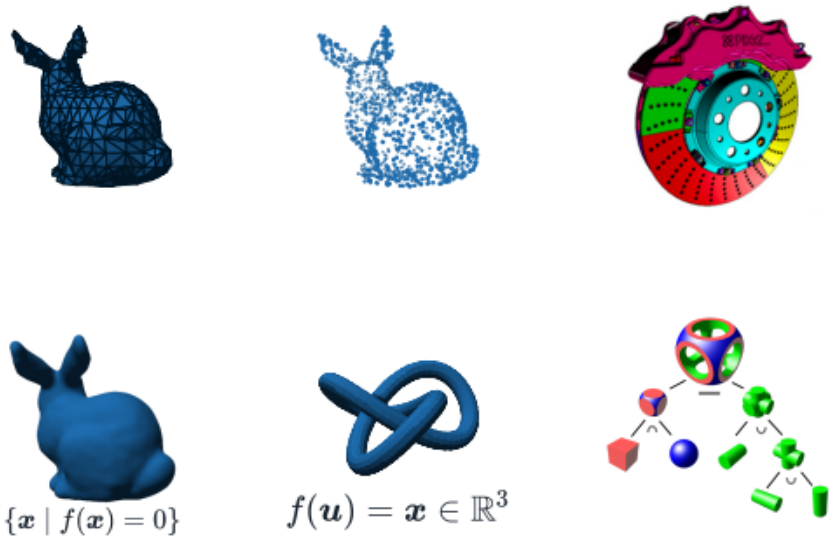


Figure 1: The same shape represented in four different ways

In 3D there is no such consensus. The slide lists: triangle meshes, point clouds, CAD models, surfaces given as a zero level set $\{\boldsymbol{x} \mid f(\boldsymbol{x}) = 0\}$, surfaces given by a parametrization $f(\boldsymbol{u}) = \boldsymbol{x} \in \mathbb{R}^3$, CSG trees, "and many more".

Intuition

The 2D grid is universal because image sensors *measure* on a grid and every algorithm can assume that grid. In 3D, different tasks pull in different directions: a LiDAR returns unordered points, a designer edits smooth curves, a physics engine needs inside/outside tests, a neural network prefers fixed-size arrays. Each 3D representation is a different compromise. The rest of the lecture is a tour of these compromises.

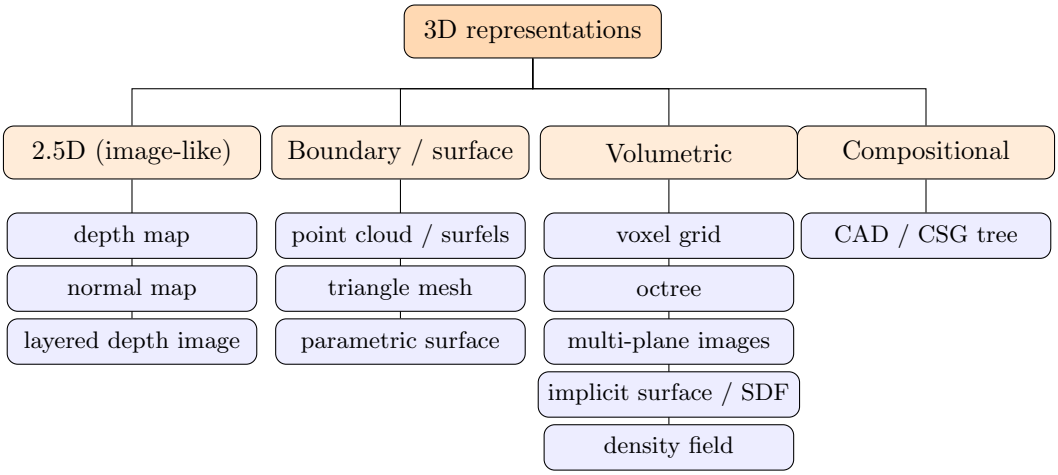


Figure 2: A map of the representations met in the lecture (our grouping, following slides 43 and 63: points, meshes and parametric surfaces are *boundary* representations; voxels and implicit fields are *volumetric*).

2 Notation *(slides slide 5)*

Definition 2.1 (Notation)**.** A point in 3D space is written in inhomogeneous coordinates as

$$\boldsymbol{x} = \begin{pmatrix} x \\ y \\ z \end{pmatrix} \in \mathbb{R}^3.$$

A point on the image plane carries the subscript s: $\boldsymbol{x}_s \in \mathbb{R}^2$. Discrete maps are *indexed* rather than evaluated, and brackets denote indexing:

$$D[\boldsymbol{x}_s] \in \mathbb{R}^+ \quad \text{(depth)}, \qquad N[\boldsymbol{x}_s] \in \mathbb{S}^2 \quad \text{(unit normal)}, \qquad V[i,j,k] \in [0,1] \quad \text{(occupancy)}.$$

Remark 2.2. The subscript denotes the *coordinate frame*, not the dimension of the space. Both $\boldsymbol{x}$ and $\boldsymbol{x}_s$ denote points.

Intuition

Parentheses mean "compute" ($f(\boldsymbol{x})$ runs a formula or a network); brackets mean "look up" ($D[\boldsymbol{x}_s]$ reads a pixel). The letter s is a hint that $\boldsymbol{x}_s$ lives on the *s*creen; it is a name tag, not an exponent.

3 Manifolds *(slides slide 6)*

Definition 3.1 (Manifold)**.** An n-dimensional *manifold* is a topological space that locally resembles Euclidean space: every point has a neighborhood homeomorphic to an open subset of $\mathbb{R}^n$; n is its dimension.

Lines and circles are 1-manifolds. Two-dimensional manifolds are called *surfaces*: the plane, the sphere, the torus. Each coloured piece of the shape on the slide is a *chart*: it flattens a part of the shape onto $\mathbb{R}^n$.

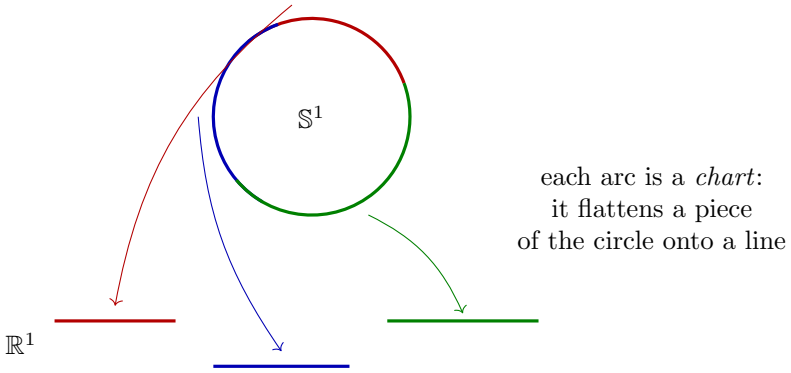


Figure 3: The circle is a 1-manifold. Three overlapping charts, each a homeomorphism from an arc to an interval of $\mathbb{R}$, cover it. (Slide 6 shows the same idea for a circle and a sphere.)

Intuition

An ant walking on a sphere sees a flat plane around itself: that local flatness is what "manifold" means. The sphere as a whole is *not* a plane, which is why one needs several overlapping charts (an atlas) to describe it, exactly as one needs several flat maps to cover the Earth without cutting it.

4 Defining a Surface *(slides slide 7)*

Definition 4.1 (Parametric and implicit forms)**.** A surface can be specified in two ways.

- **Parametric form:** a function maps a 2D domain into space, $f(\boldsymbol{u}) = \boldsymbol{x} \in \mathbb{R}^3,\ \boldsymbol{u} \in \mathcal{M}$.
- **Implicit form:** the surface is the zero level set of a scalar field, $\mathcal{S} = \{\boldsymbol{x} \mid f(\boldsymbol{x}) = 0\}$.

In the parametric form the domain point $\boldsymbol{u}$ is *not* a point in space: it belongs to a 2D manifold $\mathcal{M}$, for instance the sphere $\mathbb{S}^2$.

Remark 4.2. The symbol f denotes both functions; the argument decides which is meant: $f(\boldsymbol{u})$ returns a surface point, $f(\boldsymbol{x})$ returns a scalar.

Intuition

Parametric = "a recipe that *produces* points" (feed in a domain point, get a 3D point out). Implicit = "a rule that *judges* points" (feed in a 3D point, get a number whose sign says on which side of the surface it lies). Slide 48 will turn this into the mantra *parametric generates, implicit tests.*

5 Meanwhile in 3D-land… *(slides slides 8–9)*

"But what is the *best* representation?" The slide's answer: *it depends*, on the task: acquisition vs editing vs animation vs prediction vs processing vs optimization, and more.

The goals of the lecture (slide 9):

- understand representations *computationally* (what do they look like in code?);
- understand their limitations and benefits;
- understand conversions across representations.

Intuition

There is no free lunch: a representation that makes one operation trivial tends to make another one painful. Reading each representation below, ask three questions: how do I *store* it, what is *easy* to do with it, what is *hard*? The lecture proceeds from representations tied to sensors (depth maps, point clouds), through surfaces (meshes, parametric surfaces), to volumes (voxels, implicit fields, density fields).

6 Depth Maps *(slides slides 10–14)*

6.1 Depth maps *(slides slides 10–12)*

Depth is a *common modality across sensors* (structured-light and time-of-flight cameras such as the Kinect and industrial cameras shown on slide 10); depth is treated in detail in Lectures 04–05.

Definition 6.1 (Depth map)**.** A depth map is an image $D[\boldsymbol{x}_s] \in \mathbb{R}^+$ that records how far the scene point seen by each pixel $\boldsymbol{x}_s$ is from the camera.

Because it is just an image, *standard image-processing tools can be used* on it (slide 12): filtering, resizing, convolutional networks.

6.2 Depth Maps and Normal Maps *(slides slide 13)*

The same idea extends to other surface properties, e.g. surface normals:

Definition 6.2 (Normal map)**.** $N[\boldsymbol{x}_s] \in \mathbb{S}^2$ stores, for each pixel, the unit normal of the visible surface.

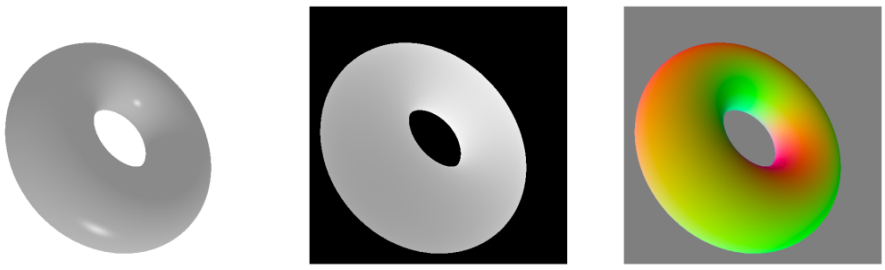


Figure 4: Depth and normal maps of a scene. The depth map is a grayscale image where brightness encodes distance; the normal map is a color image where RGB encodes the XYZ components of the unit normal.

6.3 Depth Maps — Representing the Visible *(slides slide 14)*

Remark 6.3. A depth map does not capture the *full* 3D structure: it is a *2.5D* representation, i.e. properties associated with image pixels. The slide asks: can we develop more *complete* 3D representations?

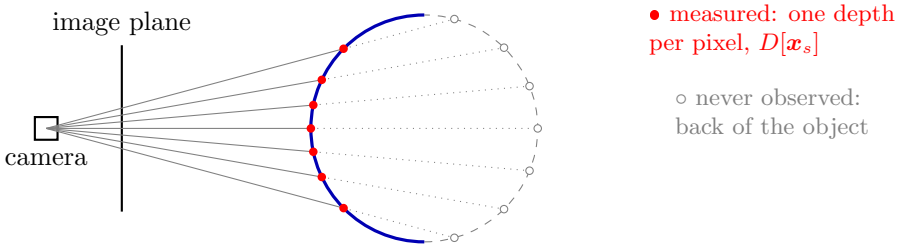


Figure 5: Side view of a depth camera. Each pixel ray returns the *first* surface hit (red); the far side of the object (dashed) and anything hidden behind the visible surface is absent from D.

Intuition

A depth map is like a relief map seen from one viewpoint: it tells the distance to whatever is *in front*, and nothing about what is behind it. Two things follow. First, cheap and fast: it is an ordinary image. Second, incomplete: to model a whole object you need several views or a representation that lives in 3D itself, which motivates point clouds next.

7 Point Clouds *(slides slides 15–21)*

7.1 Point Clouds *(slides slides 15–18)*



Figure 6: Some point clouds of scenes. Each point is a 3D position, and can be drawn as a small sphere.

Point clouds are often acquired in an image-like manner from sensors (LiDAR, depth cameras) but it can be useful to *forget* this: point clouds from multiple views can be merged into a single point cloud, which helps unify approaches independently of the sensing modality.

Definition 7.1 (Point cloud)**.** A point cloud is an *unordered set* of points $\{\boldsymbol{x}_1, \boldsymbol{x}_2, \ldots, \boldsymbol{x}_N\} \subset \mathbb{R}^3$, usually stored as an $N \times 3$ array. Reordering the rows by any permutation σ gives $\{\boldsymbol{x}_{\sigma(1)}, \ldots, \boldsymbol{x}_{\sigma(N)}\}$, the *same* point cloud.

Unlike an image, the row order carries no meaning, so processing or generation methods must be *permutation invariant*; a fully connected layer applied to the flattened array does not work.

Example 7.2 (Why a fully connected layer fails)**.** Let $X = \{\boldsymbol{x}_1 = (1,0,0), \boldsymbol{x}_2 = (0,1,0)\}$. Flattened, the two orderings give $(1,0,0,0,1,0)$ and $(0,1,0,1,0,0)$, two different vectors, so a generic linear layer returns two different outputs for one geometric object. A *symmetric* operation such as taking the per-coordinate maximum over the points returns $(1,1,0)$ in both

orders.

Intuition

Think of a bag of marbles: shaking it does not change what it contains. An operator must treat the input as a bag, not as a list. Symmetric functions (sum, mean, max over points) have this property, and network designs for point clouds are built from them.

7.2 Oriented Point Clouds *(slides slide 19)*

Definition 7.3 (Oriented point cloud, surfel)**.** An oriented point cloud stores a normal with each point: $\{(\boldsymbol{x}_1, \boldsymbol{n}_1), \ldots, (\boldsymbol{x}_N, \boldsymbol{n}_N)\}$. Points can then be treated as small flat disks rather than tiny spheres, which helps rendering under lighting. When a radius is also stored the primitives are called *surfels*.

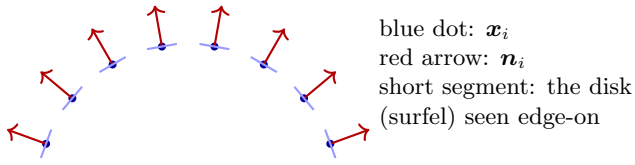


Figure 7: An oriented point cloud sampled from a curve (2D cross-section): each sample carries a position and a normal, and can be drawn as a small disk perpendicular to the normal.

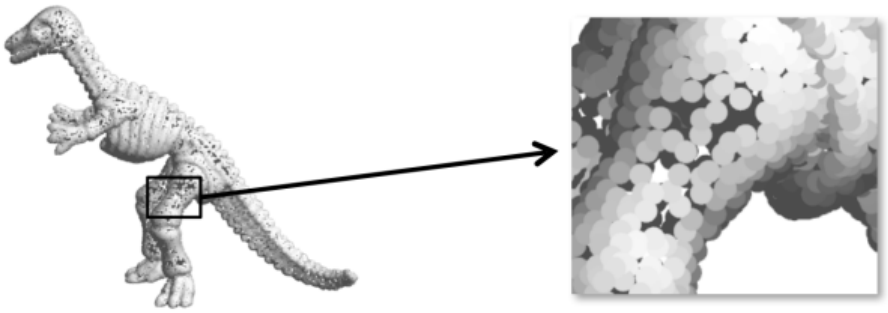


Figure 8: A surfel-based reconstruction of a dinosaur. Each surfel is a small disk with a position, a normal and a radius.

Intuition

A bare point has no facing direction, so lighting is ambiguous. Attaching a normal turns each point into a tiny mirror; adding a radius makes the mirror cover a patch of surface, so neighbours overlap and the gaps between samples close up.

7.3 No explicit connectivity *(slides slides 20–21)*

A point cloud has no explicit *connectivity* information. Two point sets that look nearly identical may come from a closed circle and from a circle with a small gap. One can always sample more points to resolve the gap, but this is super inefficient; it is more efficient to add *edges*, because connectivity does not scale like density (which grows with an area).

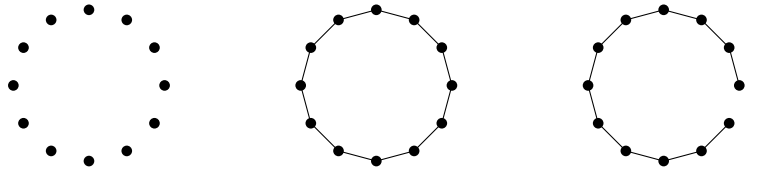


Figure 9: The same 12 points support two different shapes: only the edges (connectivity) tell them apart.

Example 7.4 (Density vs. connectivity)**.** To sample a unit sphere with spacing h one needs roughly $4\pi/h^2$ points: about 1257 for $h = 0.1$ but about 5027 for $h = 0.05$. Halving the spacing multiplies the number of points by 4, while adding one edge costs one pair of integers.

Intuition

Density is a *2D* bill (points per unit area), connectivity is a *1D* one (a few edges along features). To say "these two neighbouring points belong to the same continuous piece" it is far cheaper to draw a line than to fill the gap with hundreds of extra samples. This is exactly what meshes do.

8 Meshes *(slides slides 22–34)*

8.1 Meshes: vertices and faces *(slides slides 22–25)*

Meshes are *piecewise linear approximations* of the underlying surface; more elements allow a better approximation (slide 22 shows a torus with increasing resolution).

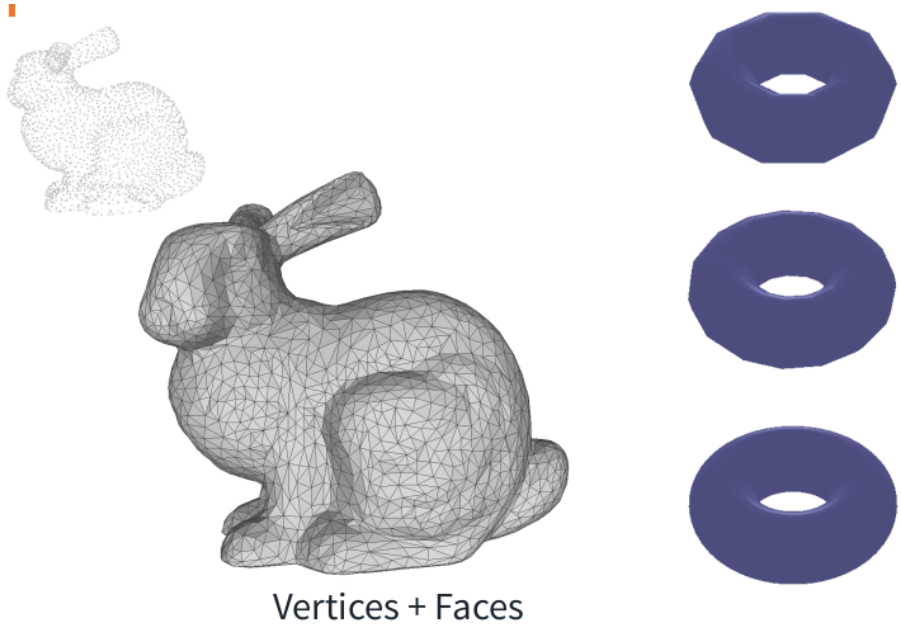
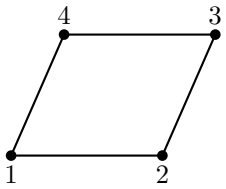
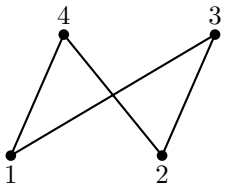


Figure 10: A torus approximated by a mesh of increasing resolution and the difference between a mesh and a pointcloud of a rabbit. The mesh is a piecewise linear approximation of the underlying surface.

Definition 8.1 (Mesh)**.** A mesh is a pair (V, F): V is an $N \times 3$ array of vertex positions (the *geometry*) and F is a list of M faces (the *topology*), each face being a list of indices of the vertices that make it up. Consecutive indices in a face are joined by an edge, so *ordering matters* in face indices.



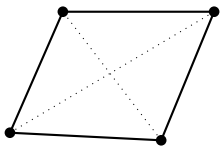
face $(1, 2, 3, 4)$: a quadrilateral



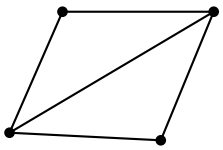
face $(1, 3, 2, 4)$: a "bow tie"

Figure 11: Same four vertices, two index orders, two different shapes: consecutive indices are connected by an edge.

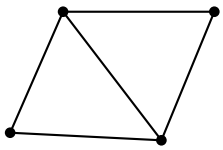
Arbitrary polygons can also be *non-planar* (four points in 3D need not lie in a plane), so one restricts to triangles: three points not on a line always define a plane.



polygon, two diagonals



triangulation A



triangulation B

Figure 12: A quad can be split into triangles in two ways. If the four points are not coplanar, A and B are two genuinely different surfaces, while each triangle on its own is always flat.

8.2 Triangular Meshes *(slides slides 26–31)*

Definition 8.2 (Triangular mesh)**.** A triangular mesh is a mesh whose faces are all triangles: $V \in \mathbb{R}^{N\times 3}$ and $F \in \{0, \ldots, N-1\}^{M\times 3}$, each row of F listing the indices of the *three* vertices of a face.

Example 8.3 (A tetrahedron in code)**.**

```
    V = np.array([[0,0,0],[1,0,0],[0,1,0],[0,0,1]], dtype=float)   # 4 x 3
F = np.array([[0,2,1],[0,1,3],[0,3,2],[1,2,3]])                  # 4 x 3, ints
```

Here $N = 4$ vertices, $M = 4$ faces and $E = 6$ edges.

A sanity check on sizes.

Proposition 8.4 (Face count of a closed genus-0 triangle mesh)**.** *Let a closed, sphere-like (genus 0) triangular mesh have V vertices, E edges and F faces. Every face has 3 edges and every edge is shared by 2 faces, so $E = 3F/2$. With Euler's formula $V - E + F = 2$ this gives*

$$F = 2V - 4.$$

Proof. Substitute $E = 3F/2$ into $V - E + F = 2$: $V - \frac{3}{2}F + F = 2$, i.e. $V - \frac{1}{2}F = 2$, hence $F = 2V - 4$. □

Intuition

You get about *twice as many triangles as vertices.* Checks: tetrahedron $V = 4 \Rightarrow F = 4$ ✓; octahedron $V = 6 \Rightarrow F = 8$ ✓; a scan with $V = 10{,}000$ vertices has $F = 19{,}996$ faces and $E = 3F/2 = 29{,}994$ edges. Useful to guess memory footprints or to spot a broken mesh.

Orientation. For a triangle, *any order gives the same triangle*, but the order determines which side is "front". The normal is $\boldsymbol{n} = (\boldsymbol{b} - \boldsymbol{a}) \times (\boldsymbol{c} - \boldsymbol{a})$.

Proposition 8.5 (Orientation and index order)**.** *A cyclic shift* $(a, b, c) \to (b, c, a)$ *leaves* $\boldsymbol{n}$ *unchanged; swapping two indices, e.g.* $(a, b, c) \to (c, b, a)$*, flips its sign.*

Proof. Expanding, $(\boldsymbol{b} - \boldsymbol{a}) \times (\boldsymbol{c} - \boldsymbol{a}) = \boldsymbol{b} \times \boldsymbol{c} - \boldsymbol{b} \times \boldsymbol{a} - \boldsymbol{a} \times \boldsymbol{c} + \boldsymbol{a} \times \boldsymbol{a} = \boldsymbol{a} \times \boldsymbol{b} + \boldsymbol{b} \times \boldsymbol{c} + \boldsymbol{c} \times \boldsymbol{a}$. This expression is unchanged by cyclic relabelling $\boldsymbol{a} \to \boldsymbol{b} \to \boldsymbol{c} \to \boldsymbol{a}$ and changes sign under the swap of any two symbols, since $\times$ is antisymmetric. □



(a, b, c): counter-clockwise, $\boldsymbol{n}$ toward the viewer (c, b, a): clockwise, $\boldsymbol{n}$ away from the viewer

Figure 13: Index order defines the winding direction and hence the side the normal points to. Swapping two indices reverses it.

Example 8.6 (Checking the tetrahedron)**.** For face $(0, 2, 1)$ of the example above, $\boldsymbol{a} = (0, 0, 0)$, $\boldsymbol{b} = (0, 1, 0)$, $\boldsymbol{c} = (1, 0, 0)$: $\boldsymbol{n} = (0, 1, 0) \times (1, 0, 0) = (0, 0, -1)$, pointing down, i.e. *outward* for the face lying in $z = 0$ (the tetrahedron sits above it). Face $(1, 2, 3)$ gives $(-1, 1, 0) \times (-1, 0, 1) = (1, 1, 1)$, also outward. All four faces are consistently oriented.

Intuition

The "classic bug": if some faces are listed clockwise and others counter-clockwise, the renderer treats some triangles as back-facing and they render black or vanish. Rule of thumb: use one winding convention for the whole mesh and check with $\boldsymbol{n}$ pointing away from the interior.

Manifold problems. Even with triangles one can still get *non-manifold* meshes. The slides identify two cases: an edge shared by more than two faces (so restrict each edge to at most two faces; one can often get away without enforcing this) and a *disconnected surface around a vertex*, which is critical for mesh-processing applications and less so for rendering.

Ray–triangle intersection and texturing.

Proposition 8.7 (Barycentric form and the ray–triangle solve)**.** *Any point of the triangle with vertices* $\boldsymbol{a}, \boldsymbol{b}, \boldsymbol{c}$ *is* $\alpha\boldsymbol{a} + \beta\boldsymbol{b} + \gamma\boldsymbol{c}$ *with* $\alpha + \beta + \gamma = 1$ *and all three weights* ≥ 0*. Setting this equal to the ray* $\boldsymbol{o} + t\boldsymbol{d}$ *gives* 3 *linear equations in the* 3 *unknowns* (t, β, γ)*, i.e. a* 3×3 *solve per triangle:*

$$\beta(\boldsymbol{b} - \boldsymbol{a}) + \gamma(\boldsymbol{c} - \boldsymbol{a}) - t\,\boldsymbol{d} = \boldsymbol{o} - \boldsymbol{a}, \qquad \alpha = 1 - \beta - \gamma.$$



an edge seen end-on, shared by 3 faces two fans touching at a single vertex

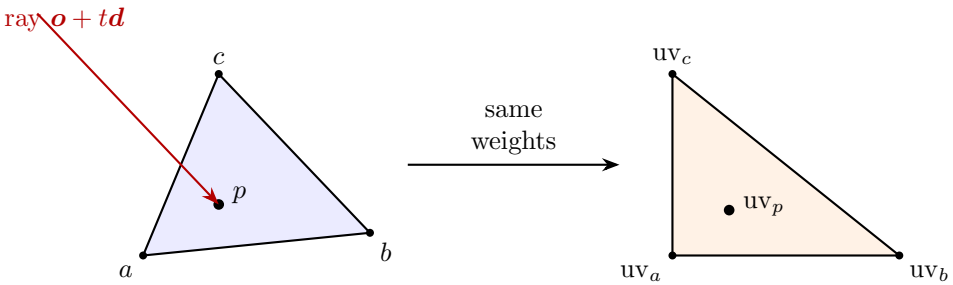
Figure 14: Two kinds of non-manifold configuration.

The ray hits the triangle iff $\beta \geq 0$, $\gamma \geq 0$, $\beta + \gamma \leq 1$ (and $t > 0$).

Proof. Substitute $\alpha = 1 - \beta - \gamma$ in $\alpha\boldsymbol{a} + \beta\boldsymbol{b} + \gamma\boldsymbol{c} = \boldsymbol{o} + t\boldsymbol{d}$ and collect terms. □

Example 8.8. Triangle $\boldsymbol{a} = (0,0,0)$, $\boldsymbol{b} = (1,0,0)$, $\boldsymbol{c} = (0,1,0)$; ray $\boldsymbol{o} = (0.25, 0.25, 1)$, $\boldsymbol{d} = (0,0,-1)$. The system reads $\beta(1,0,0)+\gamma(0,1,0)-t(0,0,-1) = (0.25, 0.25, 1)$, so $\beta = \gamma = 0.25$, $t = 1$, $\alpha = 0.5$. All weights are ≥ 0: hit at $\boldsymbol{o} + \boldsymbol{d} = (0.25, 0.25, 0)$.

Texturing is easy: each vertex also stores a coordinate into a 2D image, and inside a triangle the same weights interpolate those coordinates.



$p = \alpha a + \beta b + \gamma c$ $\mathrm{uv}_p = \alpha\,\mathrm{uv}_a + \beta\,\mathrm{uv}_b + \gamma\,\mathrm{uv}_c$

Figure 15: Left: the hit point in barycentric coordinates $(\alpha, \beta, \gamma) = (0.5, 0.25, 0.25)$. Right: the same weights interpolate the 2D texture coordinates stored at the vertices.

Intuition

Barycentric weights are "how much of each corner" a point is made of; a point near a has α close to 1. Intersecting a ray with a triangle is therefore a tiny linear-algebra problem, which is why GPUs can do billions of them, and the *same* weights carry any per-vertex quantity (texture coordinates, colours, normals) to the interior.

8.3 Triangular Meshes — Geometry vs Connectivity *(slides slides 32–34)*

The two arrays of a mesh behave very differently: V is `float`, differentiable, and gradient descent (GD) is happy; F is `int`, discrete, and GD is not happy. Modifying vertex positions can edit the shape; other changes are non-trivial (i.e. changing F).

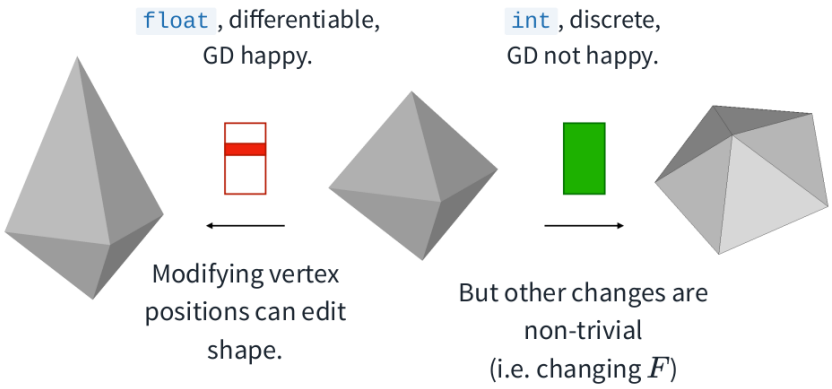


Figure 16: Editing the geometry V is easy, editing the connectivity F is hard.



edit V (`float`):
move a vertex, F untouched

edit F (`int`):
flip an edge, V untouched

Figure 17: Geometry edits are small continuous changes of real numbers; connectivity edits are combinatorial jumps.

Same counts, different meshes *(slides slide 33)*

Going from one mesh to another is non-trivial even when both have 6 vertices and 8 faces (the octahedron is one such mesh). It is even more challenging if the topology changes, e.g. torus to sphere. In general, designing methods that can predict *arbitrary* connectivity is challenging (e.g. image to mesh).

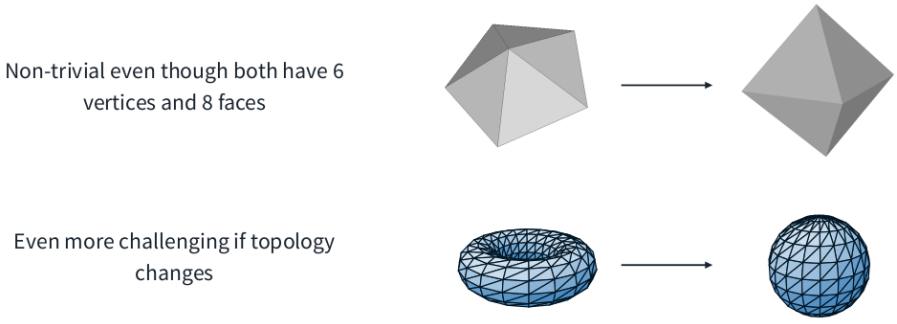


Figure 18: Two meshes with the same number of vertices and faces, but different connectivity.

Continuous deformation *(slides slide 34)*

The first change (same counts, different connectivity) is possible under continuous deformation of the shape; the torus-to-sphere change is impossible under continuous deformation (a hole cannot be closed without tearing). Yet *both* edits require similar operations for meshes: a new F and an

updated V. The slide asks: is there an alternate representation where the former (the continuous one) is easier?

Intuition

A mesh mixes a smooth part (V) with a rigid combinatorial part (F). Even a tiny, continuous change of the shape may demand a jump in F, and a network cannot easily output "an integer array of unknown length". The question hanging in the air motivates representations where shape changes are always changes of continuous numbers: parametric and, above all, implicit ones.

9 Parametric Surfaces *(slides slides 35–41)*

9.1 Parametric surfaces in 2D *(slides slide 35)*

The domain is a circle $\mathbb{S}^1$; think of it as an angle θ. A function $f(\boldsymbol{u}) = \boldsymbol{x} \in \mathbb{R}^2$, $\boldsymbol{u} \in \mathbb{S}^1$ sends each angle to a point of a curve: f_1 to a pentagon, f_2 to a square.

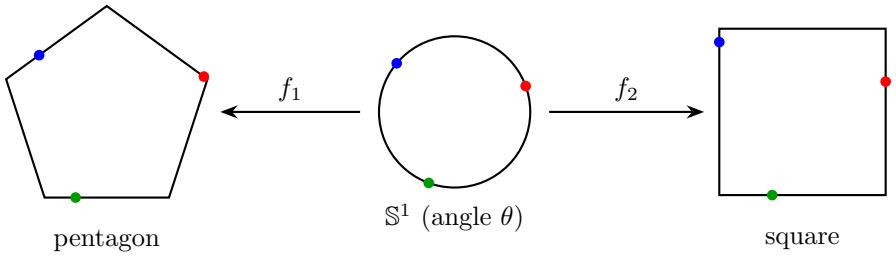


Figure 19: Two different maps f_1, f_2 from the same circle domain. Same colour = same domain point θ.

9.2 Parametric surfaces *(slides slides 36–40)*

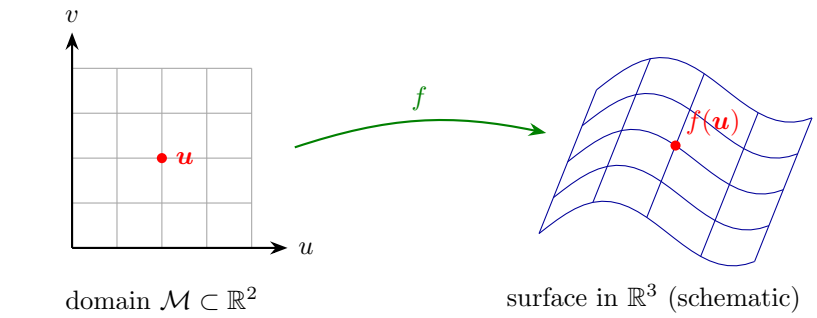
Definition 9.1 (Parametric surface)**.** A continuous function $f : \mathcal{M} \to \mathbb{R}^3$, $f(\boldsymbol{u}) = \boldsymbol{x}$, over a 2D manifold $\mathcal{M}$ parametrizes a surface. The *connectivity/topology* is constrained to be the same as $\mathcal{M}$. This disentangles discretization (vertices/faces) from shape representation. In code:

```
def f(u):
    '''Input:  u: N-dimensional vector from a 2D manifold
       Output: x: 3D point'''
```

f can be a fixed analytic function, a parametrized family, or even a neural network $f_\theta : \mathbb{S}^2 \to \mathbb{R}^3$.

Example 9.2 (Analytic examples)**.** The saddle $f((u, v)) = (u, v, v^2 - u^2)$; and the trefoil-knot tube of slide 38:

```
def trefoil(u, v):
    # r is just a float scale factor
    x = r * np.sin(3*u) / (2 + np.cos(v))
    y = r * (np.sin(u) + 2*np.sin(2*u)) / (2 + np.cos(v + np.pi*2/3))
    z = r/2 * (np.cos(u) - 2*np.cos(2*u)) * (2 + np.cos(v)) * (2 + np.cos(v + np.pi
    *2/3)) / 4
```



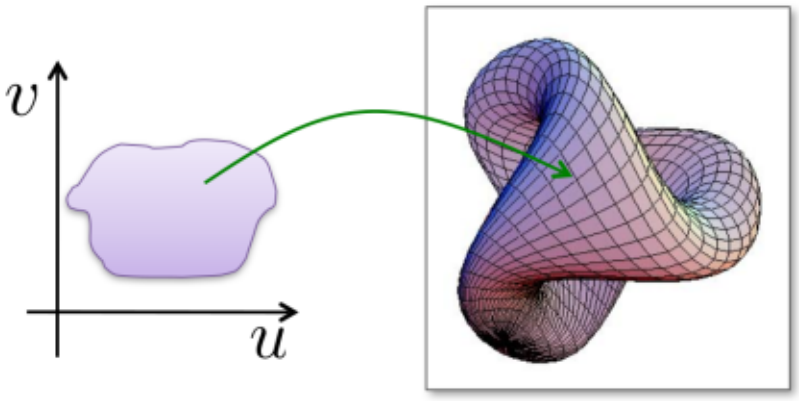


Figure 20: A parametrization warps a flat domain into a surface: grid lines of (u, v) become curves on the surface, and neighbouring parameters stay neighbouring.

Definition 9.3 (Bézier surface)**.** With $(N+1)(M+1)$ control points $\boldsymbol{k}_{i,j} \in \mathbb{R}^3$,

$$f((u,v)) = \sum_{i=0}^{N} \sum_{j=0}^{M} B_i^N(u)\, B_j^M(v)\, \boldsymbol{k}_{i,j}, \qquad B_i^N(u) = \binom{N}{i} u^i (1-u)^{N-i}.$$

Here N and M are polynomial *degrees*, not counts. The control points determine a smooth surface in 3D; Bézier patches are commonly used for designing meshes.

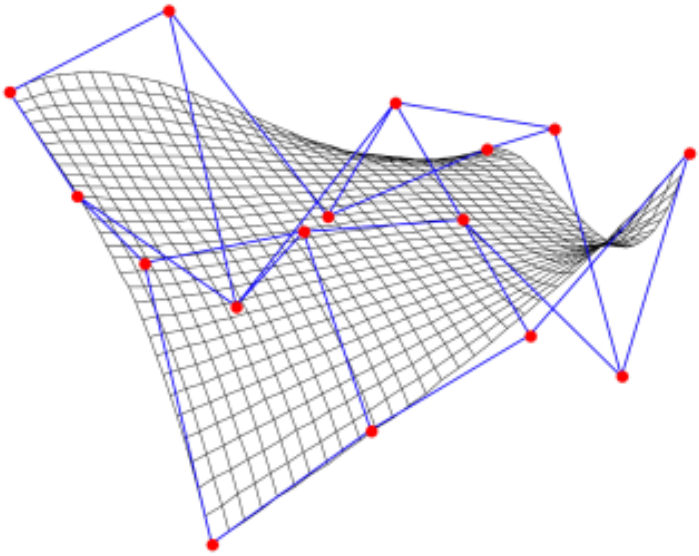


Figure 21: A Bézier surface

Example 9.4 (The bilinear patch, $N = M = 1$)**.** $B_0^1(u) = 1 - u$, $B_1^1(u) = u$, so $f = (1-u)(1-v)\boldsymbol{k}_{00} + (1-u)v\,\boldsymbol{k}_{01} + u(1-v)\boldsymbol{k}_{10} + uv\,\boldsymbol{k}_{11}$. With $\boldsymbol{k}_{00} = (0,0,0)$, $\boldsymbol{k}_{10} = (1,0,0)$, $\boldsymbol{k}_{01} = (0,1,0)$, $\boldsymbol{k}_{11} = (1,1,1)$: $f(0.5, 0.5) = \frac{1}{4}\big[(1,0,0) + (0,1,0) + (1,1,1)\big] = (0.5, 0.5, 0.25)$.

Intuition

Control points act like a coarse cage of pins: move a pin and the smooth sheet follows. The surface is a weighted average of the pins, with weights (Bernstein polynomials) that sum to one, so it stays inside the convex hull of the cage. That is why the same $(N+1)(M+1)$ numbers describe an entire smooth surface no matter how finely you later sample it.

9.3 Parametric Surfaces — Pros and Cons *(slides slide 41)*

- **Sampling is easy:** pick $\boldsymbol{u}$, evaluate f.
- **Computation in the domain:** neighbours, texture coordinates and so on are just nearby $\boldsymbol{u}$ values.
- **"Is $\boldsymbol{q}$ inside?" is hard:** you would have to search for the $\boldsymbol{u}$ whose $f(\boldsymbol{u})$ is closest to $\boldsymbol{q}$, an optimization problem.
- **Rendering is hard:** a ray hits the surface where $\boldsymbol{o} + t\boldsymbol{d} = f(\boldsymbol{u})$, a nonlinear system in $(t, \boldsymbol{u})$. In practice you convert to a mesh first.

Intuition

Generating is a *forward* computation (plug in, read off); testing or intersecting needs the *inverse* of f, which is generally not available in closed form. The price of the parametrization is also its topology: the surface has whatever topology $\mathcal{M}$ has (a sphere domain can only give sphere-like shapes).

10 Surface Representations *(slides slide 42)*

Points, meshes and parametric surfaces all describe the "skin" of an object, and none of them can turn a torus into a sphere continuously. So what can we do?

Intuition

All three fix the topology in their data (the connectivity F, the domain $\mathcal{M}$, or the absence of connectivity). To let shapes change topology smoothly, one needs a representation that assigns a value to *every point of space* rather than tracking the skin.

11 Volume Representations *(slides slides 43–46)*

Volume representations give a value to every point in space.

11.1 Voxelized 3D *(slides slides 44–46)*

Definition 11.1 (Voxel grid)**.** A voxel grid is a discretized representation of the space demarcated via cuboid endpoints $(x_{\min}, y_{\min}, z_{\min})$ and $(x_{\max}, y_{\max}, z_{\max})$: a $W \times H \times D$ array $V[i, j, k] \in [0, 1]$ storing the occupancy (or probability of occupancy) of each cell.

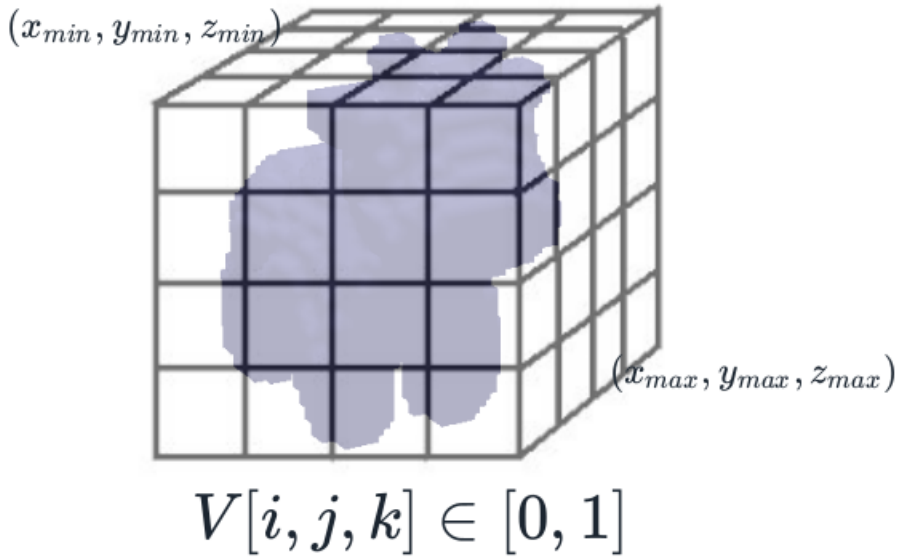


Figure 22: A $4 \times 4 \times 4$ voxel grid inside a cuboid; shaded cells are occupied.

Coding tip: disentangle *grid coordinates* from the *spatial extent in the world*. One common convention: cell size $h = (x_{\max} - x_{\min})/W$ along x, and cell i has its centre at $x_{\min} + (i + \frac{1}{2})h$; the same for y, z. For $x_{\min} = -1.5$, $x_{\max} = 1.5$, $W = 128$: $h \approx 0.0234$, and the point $x = 0.3$ falls in cell $i = \lfloor (0.3 + 1.5)/h \rfloor = 76$.

Because the representation is "image-like", *convolutions* can be used for processing and generation. It is a generic way to represent shapes *across topologies*: the sphere, the torus and the cow of slide 45 all use the same array shape, and topology is just which cells are switched on. A fixed-size output is exactly what networks like.

Resolution is expensive *(slides slide 46)*

It is computationally challenging to scale resolution ($32^3, 64^3, 128^3, 256^3$ on the slide). Worse, the surface passes through only about n^2 of the n^3 cells, so almost all the memory describes empty space or solid interior.

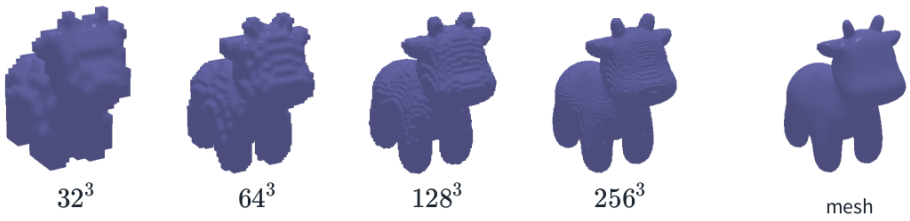


Figure 23: A rappresentation of how resolution affects surface detail. The surface passes through only about n^2 of the n^3 cells, so almost all the memory describes empty space or solid interior.

Example 11.2 (Memory of a voxel grid)**.**

n	cells n^3	surface cells $\approx n^2$	fraction $\approx 1/n$
32	32 768	1 024	3%
64	262 144	4 096	1.6%
128	2 097 152	16 384	0.8%
256	16 777 216	65 536	0.4%

At one byte per cell, 256^3 is 16 MB (64 MB with 32-bit floats) for a single object.

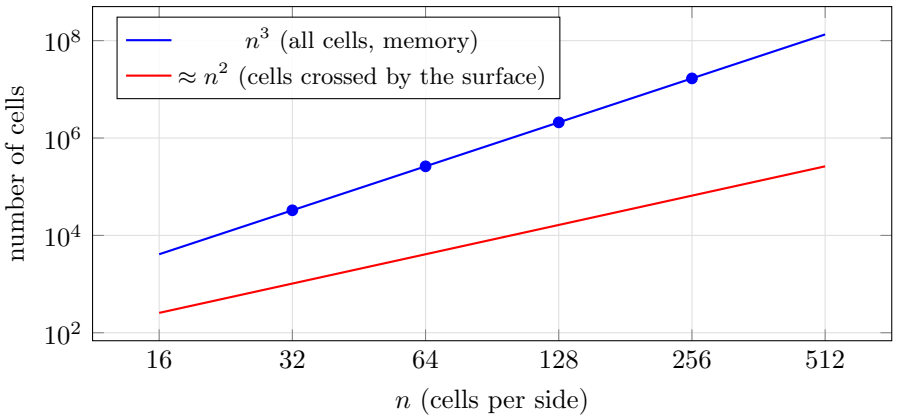


Figure 24: Doubling the resolution multiplies memory by 8 but the useful (surface) part only by 4.

Intuition

A voxel grid is a bookkeeping device: it pays the same for air as for the object. Doubling the resolution costs 8× more memory but only 4× more surface. The benefit is simplicity: any topology, fixed size, convolutions apply. The cost is that fine detail is very expensive, which motivates adaptive versions (octrees, slide 68) and continuous fields (next section).

12 Implicit Surfaces *(slides slides 47–62)*

12.1 Implicit surfaces *(slides slide 47)*

Definition 12.1 (Implicit surface)**.** A surface is the *zero level set* of a scalar field $f : \mathbb{R}^3 \to \mathbb{R}$:

$$\mathcal{S} = \{\boldsymbol{x} \in \mathbb{R}^3 \mid f(\boldsymbol{x}) = 0\}.$$

The sign tells the side: $f(\boldsymbol{x}) < 0$ inside, $f(\boldsymbol{x}) > 0$ outside (one convention, used for the rest of the lecture).

12.2 Generate vs Test *(slides slide 48)*

The unit circle, written both ways:

	Parametric	**Implicit**
Definition	$g(u) = (\cos u, \sin u),\ u \in [0, 2\pi)$	$f(x, y) = x^2 + y^2 - 1$
Give me 100 points on it	easy: evaluate g at 100 values of u	hard: you have to search for zeros
Is $(0.5, 0.5)$ inside?	hard: no direct test	easy: $f = -0.5 < 0$, so inside

Parametric representations *generate* points; implicit representations *test* points.

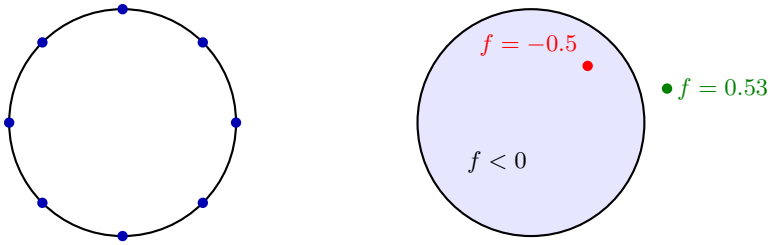


Figure 25: Left: the parametric circle produces points. Right: the implicit circle classifies any point by the sign of f.

Intuition

Parametric ≈ a machine that *prints* surface points; implicit ≈ a *bouncer* that checks tickets. Printing 100 points is trivial with the printer and awkward with the bouncer (you have to hunt for the zeros); checking a single point is trivial with the bouncer and awkward with the printer (you must invert g).

12.3 Implicit Surfaces: Examples *(slides slide 49)*

Simple functions can give concise representations of complex shapes:

$$\begin{aligned}
\text{sphere:} \quad & f(x, y, z) = x^2 + y^2 + z^2 - 1, \\
\text{torus:} \quad & f(x, y, z) = (x^2 + y^2 + z^2 + R^2 - a^2)^2 - 4R^2(x^2 + y^2), \\
\text{quartic (blob with a hole):} \quad & f(x, y, z) = x^4 + y^4 + z^4 + 2x^2y^2 + 2x^2z^2 \\
& \qquad + 2y^2z^2 + 8xyz - 10x^2 - 10y^2 - 10z^2 + 20.
\end{aligned}$$

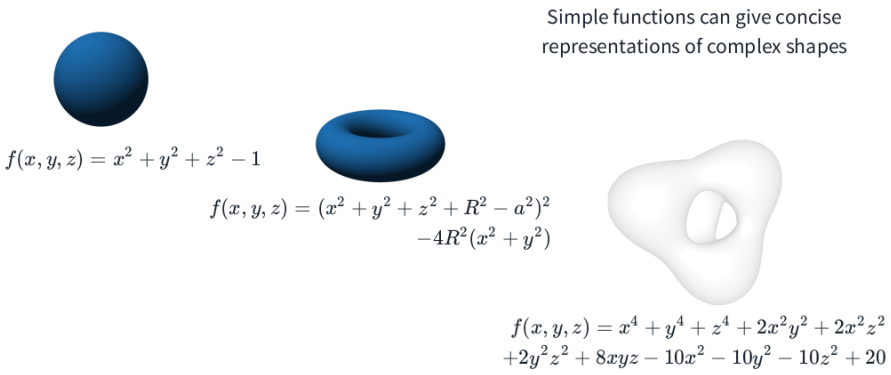


Figure 26: Implicit surfaces: sphere, torus, and a quartic with a hole. The last one is a single polynomial of degree 4, yet it has a hole in it.

12.4 When a Field Defines a Surface *(slides slide 50)*

Not every field defines a surface. Four test cases (with $r = \|\boldsymbol{x}\|$):

$f(\boldsymbol{x})$	zero set	a surface?
$\|\boldsymbol{x}\| - 1$	unit sphere	yes
$\|\boldsymbol{x}\|^2 + 1$	empty	no, there is nothing there
$\|\boldsymbol{x}\|^2$	the origin	no, a single point
$(\|\boldsymbol{x}\| - 1)^2$	unit sphere	no: $f \geq 0$ everywhere, so no inside, and $\nabla f = 0$ on it

Proposition 12.2 (When f defines a surface)**.** *f must change sign across $\mathcal{S}$, and $\nabla f \neq 0$ on $\mathcal{S}$. (If f is continuously differentiable and $\nabla f \neq 0$ at every zero of f, the implicit function theorem shows that $\mathcal{S}$ is locally the graph of a smooth function, i.e. a smooth surface.)*

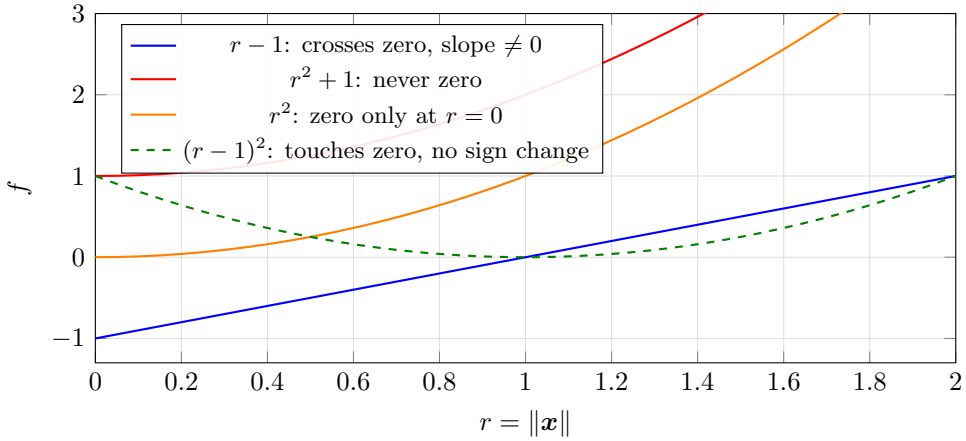


Figure 27: Radial profiles of the four candidate fields. Only the first one changes sign at the unit sphere with a non-zero slope.

Intuition

A surface is where the field *crosses* zero, like the shoreline where altitude crosses sea level. A field that only touches zero (the last row) gives a double root: no "inside", no well-defined normal. A field that never reaches zero gives nothing.

12.5 Normals for Free *(slides slide 51)*

Lemma 12.3 (Normal of an implicit surface)**.** *The gradient is perpendicular to the level sets, so on the surface*

$$\boldsymbol{n}(\boldsymbol{x}) = \frac{\nabla f(\boldsymbol{x})}{\|\nabla f(\boldsymbol{x})\|}.$$

It points toward increasing f, which by our convention is outward.

Proof. Let $\gamma(t)$ be any curve lying in $\mathcal{S}$. Then $f(\gamma(t)) = 0$ for all t; differentiating, $\nabla f(\gamma(t)) \cdot \gamma'(t) = 0$, so ∇f is orthogonal to every tangent direction of the surface. □

For the sphere, $\nabla f = 2\boldsymbol{x}$ and $\boldsymbol{n} = \boldsymbol{x}/\|\boldsymbol{x}\|$. With automatic differentiation the normals of *any* differentiable f (an MLP, say) come for free:

```
x = pts.clone().requires_grad_(True)          # (N, 3) points
v = f(x)                                      # any differentiable f, e.g. an MLP
g, = torch.autograd.grad(v.sum(), x)
n = g / g.norm(dim=-1, keepdim=True)          # (N, 3) unit normals
```

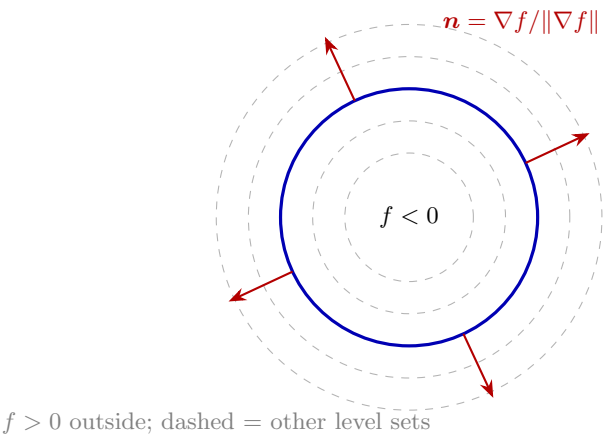


Figure 28: The gradient of f is orthogonal to the level sets and points toward larger f, i.e. out of the shape.

Intuition

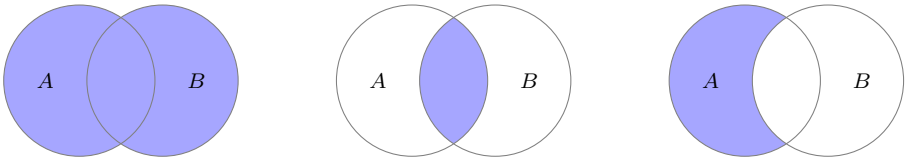
Picture f as terrain height and the surface as the zero contour line. The gradient points straight uphill, perpendicular to any contour line: that is the normal. No separate normal array has to be stored or kept consistent, unlike the per-face orientation of a mesh.

12.6 Boolean Operations *(slides slide 52)*

Proposition 12.4 (Booleans via min and max)**.** *With $f < 0$ inside,*

$$f_{A\cup B} = \min(f_A, f_B), \qquad f_{A\cap B} = \max(f_A, f_B), \qquad f_{A\setminus B} = \max(f_A, -f_B).$$

Proof. $\boldsymbol{x} \in A\cup B \iff f_A(\boldsymbol{x}) < 0$ or $f_B(\boldsymbol{x}) < 0 \iff \min(f_A, f_B) < 0$. Likewise $\boldsymbol{x} \in A\cap B \iff f_A < 0$ and $f_B < 0 \iff \max(f_A, f_B) < 0$. For $A\setminus B = A\cap B^c$, the complement of B is described (up to its boundary) by $-f_B < 0$, so $\max(f_A, -f_B) < 0$. □



$A \cup B$: $\min(f_A, f_B)$ $A \cap B$: $\max(f_A, f_B)$ $A \setminus B$: $\max(f_A, -f_B)$

Figure 29: 2D versions of the three Boolean operations (blue = result).

Intuition

"Inside the union" means "inside at least one": the most negative of the two values decides, hence min. "Inside the intersection" needs both, so the *least* negative value decides: max. Subtraction is intersection with the outside of B, and flipping the sign of f_B swaps inside and outside. The *sign* of the result is always correct; the magnitude near sharp creases is not guaranteed to remain an exact distance (see the SDF discussion next).

12.7 Many Functions, One Surface *(slides slide 53)*

The surface does not fix f: many different fields have the same zero set. So one can choose an f that is useful *away* from the surface.

12.8 Signed Distance Function *(slides slides 54–58)*

Definition 12.5 (Signed distance function (SDF))**.**

$$f(\boldsymbol{x}) = \begin{cases} -\operatorname{dist}(\boldsymbol{x}, \mathcal{S}) & \boldsymbol{x} \text{ inside} \\ +\operatorname{dist}(\boldsymbol{x}, \mathcal{S}) & \boldsymbol{x} \text{ outside} \end{cases}, \qquad \operatorname{dist}(\boldsymbol{x}, \mathcal{S}) = \min_{\boldsymbol{y} \in \mathcal{S}} \|\boldsymbol{x} - \boldsymbol{y}\|.$$

Which of the fields of slide 55 are SDFs? $x^2 + y^2 + z^2 - 1$ and the quartic of slide 49: *neither*!

Proposition 12.6 (Eikonal condition)**.** *An SDF satisfies* $\|\nabla f\| = 1$ *(wherever it is differentiable).*

Proof. Take a small step of length δ from a point. In *any* direction the distance to the surface changes by at most δ; stepping straight away from the closest surface point it changes by exactly δ. So the steepest slope of f is exactly one. □

Example 12.7. $f = \|\boldsymbol{x}\| - 1$: $\nabla f = \boldsymbol{x}/\|\boldsymbol{x}\|$, $\|\nabla f\| = 1$, an SDF. $f = \|\boldsymbol{x}\|^2 - 1$: $\nabla f = 2\boldsymbol{x}$, $\|\nabla f\| = 2\|\boldsymbol{x}\| \neq 1$, not an SDF.

Intuition

The eikonal condition says "the field grows like a ruler as you walk away from the surface". An SDF is the most *honest* field: its value literally is the distance to the surface. The other fields agree on *where* the surface is but exaggerate or shrink the distances; $r^2 - 1$ overstates by the factor $r + 1$ at radius r.

Why honesty matters: safe steps *(slides slides 57–58)*.

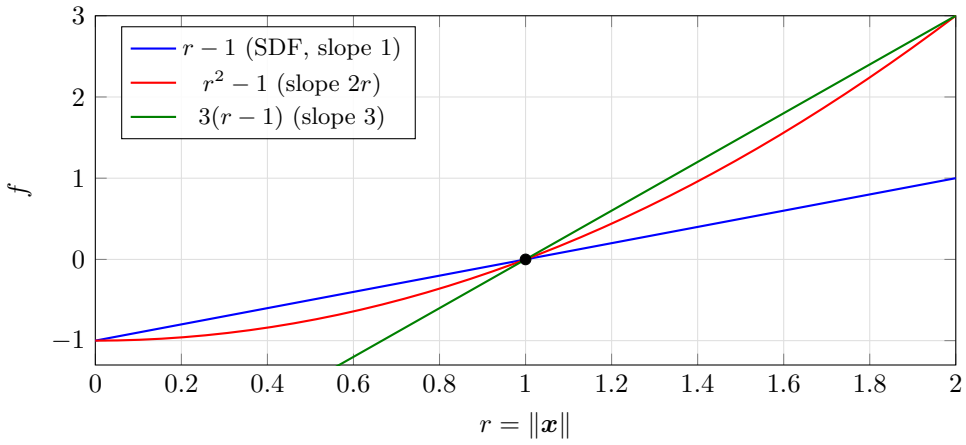


Figure 30: Three fields with the same zero set (the unit sphere, $r = 1$). Only the slope-one profile is a signed distance function.

Proposition 12.8 (Safe step)**.** *If f is an SDF, the ball of radius $|f(\boldsymbol{x})|$ around $\boldsymbol{x}$ contains no surface point in its interior. Hence a ray can safely advance by $|f(\boldsymbol{x})|$. If f overstates the distance, the ball can cross the surface and stepping by f can* jump through *it.*

Proof. By definition $|f(\boldsymbol{x})| = \operatorname{dist}(\boldsymbol{x}, \mathcal{S})$, the radius of the largest ball around $\boldsymbol{x}$ that avoids $\mathcal{S}$. For a non-SDF field the ball of radius $|f|$ can be larger than $\operatorname{dist}(\boldsymbol{x}, \mathcal{S})$ and then contains surface points. □

Example 12.9 (Slides 57–58, $\boldsymbol{x}$ at $r \approx 2.67$)**.** For $f = r^2 - 1$: $f = 6.14$, true distance 1.67, $\|\nabla f\| = 5.34$: the circle of radius $|f|$ crosses the surface. For an SDF at the same point the ball of radius $r - 1 = 1.67$ touches the surface without crossing.

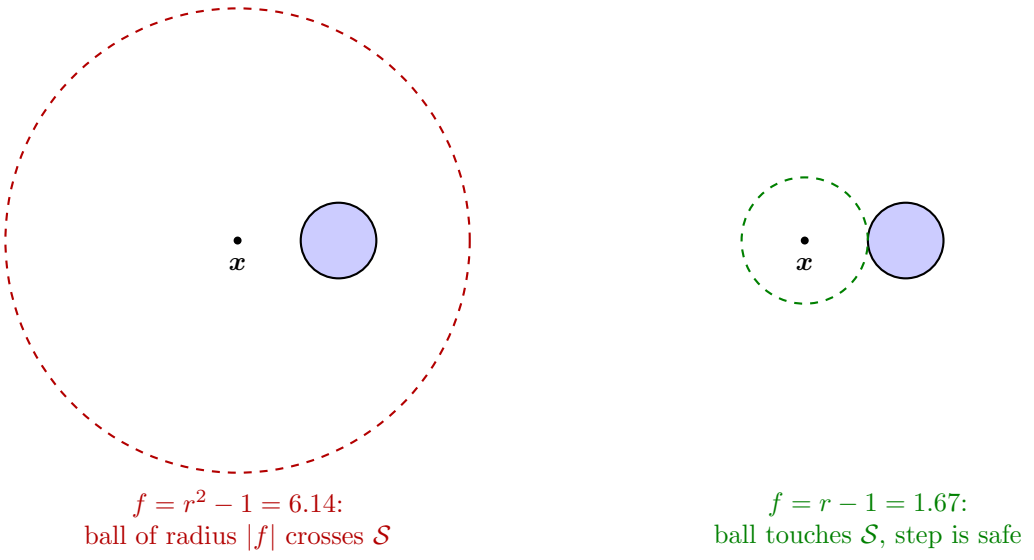


Figure 31: The same point $\boldsymbol{x}$ with two fields that define the same sphere. Left: stepping by $f = r^2 - 1$ overshoots. Right: an SDF gives the largest safe step.

Algorithm 12.10 (Sphere tracing / ray marching an SDF)**.** Given a ray $\boldsymbol{o} + t\boldsymbol{d}$ ($\|\boldsymbol{d}\| = 1$) and an SDF f: set $t \leftarrow 0$; repeat $t \leftarrow t + f(\boldsymbol{o} + t\boldsymbol{d})$ until $f < \varepsilon$ (hit) or t exceeds a maximum

(miss).

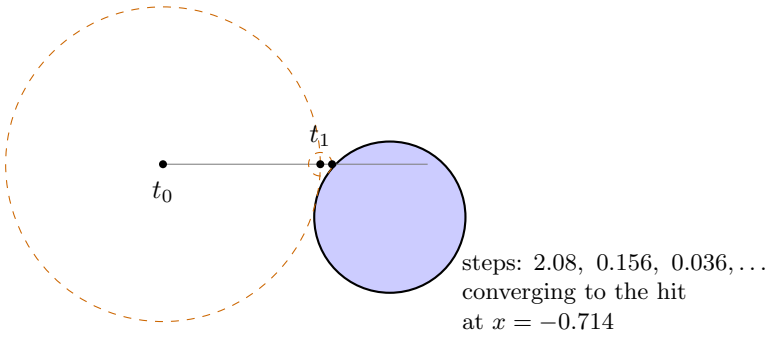


Figure 32: Sphere tracing a horizontal ray at height 0.7 against the unit sphere: each step is the SDF value, the radius of the empty ball.

Intuition

It is like walking through fog toward a wall with a distance meter: if the meter never lies (SDF) you can always walk the displayed distance without hitting the wall, and the steps shrink as you approach. A meter that overstates makes you walk into and through the wall. This is why the table on slide 66 lists "ray marching" as the way to render implicit surfaces.

12.9 Storing a Field on a Grid *(slides slide 59)*

Store samples $F[i, j, k] = f(\boldsymbol{x}_{ijk})$ at the grid points and interpolate in between: the same data structure as a voxel grid, but holding *signed distances* rather than occupancy. A *TSDF* keeps values only near the surface and clamps them to $\pm\tau$ elsewhere, $f_\tau = \text{clamp}(f, -\tau, \tau)$; it is the standard way to fuse many depth maps into one model, as in KinectFusion.

```
xs = np.linspace(-1.5, 1.5, 128)
X, Y, Z = np.meshgrid(xs, xs, xs, indexing="ij")
F = np.sqrt(X**2 + Y**2 + Z**2) - 1.0          # unit-sphere SDF on a 128^3 grid
```

Intuition

The grid now stores *how far* each cell is from the surface instead of *whether* it is occupied. Values change smoothly across the surface, so interpolation places the surface at sub-voxel accuracy, and the clamped version stores only what matters: a thin shell around the surface.

12.10 Storing a Field in a Network *(slides slide 60)*

$\{\boldsymbol{x} \mid f_\theta(\boldsymbol{x}) = 0\}$, with f_θ a small MLP (three numbers in, one out). Two requirements: $f_\theta(\boldsymbol{x}_i) \approx 0$ on the data points (the surface passes through the data), and $\|\nabla f_\theta(\boldsymbol{x})\| \approx 1$ everywhere (it behaves like an SDF). Resolution is not tied to a grid.



Figure 33: A small MLP can represent a continuous field f_θ, whose zero set is the surface.

Example 12.11 (One way to write the two requirements as a loss)**.** $\mathcal{L}(\theta) = \frac{1}{N}\sum_i |f_\theta(\boldsymbol{x}_i)| + \lambda\, \mathbb{E}_{\boldsymbol{x}}\big[(\|\nabla f_\theta(\boldsymbol{x})\| - 1)^2\big]$. The first term pulls the zero set through the samples; the second is the eikonal penalty of Proposition 12.6.

Intuition

The network is a *compressed*, resolution-free grid: query it at any point, at any density. The eikonal penalty is what stops the network from producing an arbitrary function that merely vanishes on the data.

12.11 From Implicit Surface to Mesh *(slides slide 61)*

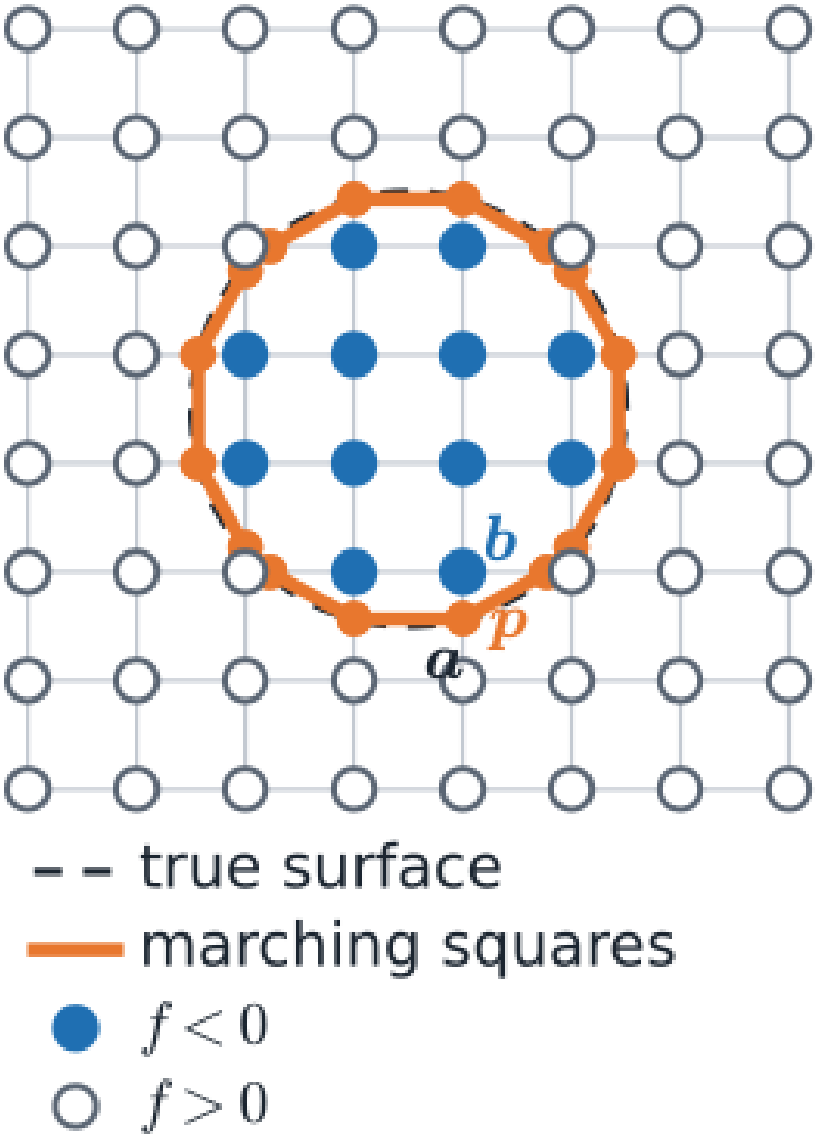


Figure 34: Marching squares (the 2D version of marching cubes) in one cell. The blue corners are inside ($f < 0$),the white corners are outside ($f > 0$). The dotted line is the true surface; the orange line is the interpolated surface from marching squares.

Algorithm 12.12 (Marching cubes)**. 1.** Evaluate f on a grid (analytic, a TSDF, or a network).
2. Find the cells whose corners differ in sign.
3. On each such edge $[\boldsymbol{a}, \boldsymbol{b}]$, place a vertex at the zero crossing $p = \boldsymbol{a} + t(\boldsymbol{b} - \boldsymbol{a})$ with $t = \dfrac{f(\boldsymbol{a})}{f(\boldsymbol{a}) - f(\boldsymbol{b})}$.
4. In 3D, join the vertices with a lookup table on the corner signs: 2^8 cases per cube.

```
from skimage.measure import marching_cubes
```

```
h = xs[1] - xs[0]                                    # voxel size, from the grid above
verts, faces, normals, _ = marching_cubes(F, level=0.0, spacing=(h, h, h))
verts += xs[0]                                       # grid -> world coordinates
```

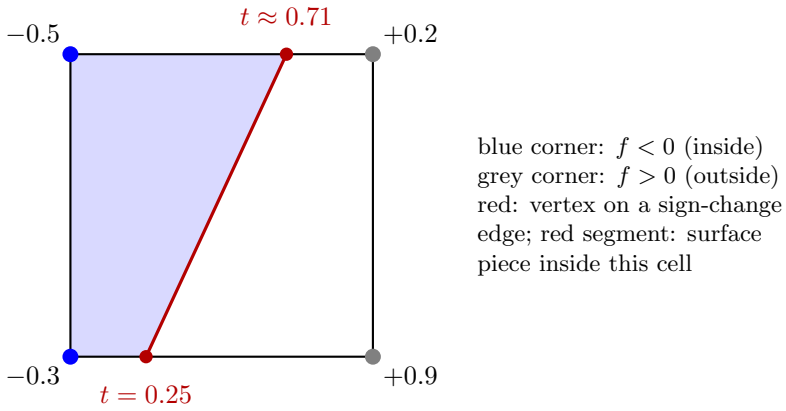


Figure 35: Marching squares (the 2D version of marching cubes) in one cell. On the bottom edge $t = \frac{-0.3}{-0.3-0.9} = 0.25$; on the top edge $t = \frac{-0.5}{-0.5-0.2} \approx 0.71$. The left and right edges have no sign change, hence no vertex.

Intuition

Marching cubes is "connect the dots along the shoreline": the corner signs say which cell edges the coastline crosses; linear interpolation of f along that edge says *where* (a corner with a small $|f|$ is close to the surface, so the vertex sits close to it). This is the bridge from implicit to mesh, which is what GPUs render.

12.12 Implicit Surfaces: Summary *(slides slide 62)*

Easy	**Hard**
inside or outside: the sign of f	getting points on the surface
any topology, changing continuously	no direct rasterization
booleans: a min or a max	an arbitrary f may define nothing
normals: ∇f	

The mirror image of parametric surfaces.

13 Boundary vs Volumetric Representations *(slides slide 63)*

Slide 63 summarizes the two families: *boundary* representations for shapes (point clouds, meshes, parametric surfaces) and *volumetric* representations for shapes (voxels, implicit fields).

14 Density Fields *(slides slides 64–65)*

14.1 Density fields *(slides slide 64)*

Occupancy asks "is this point inside?"; density asks "how opaque is this point?".

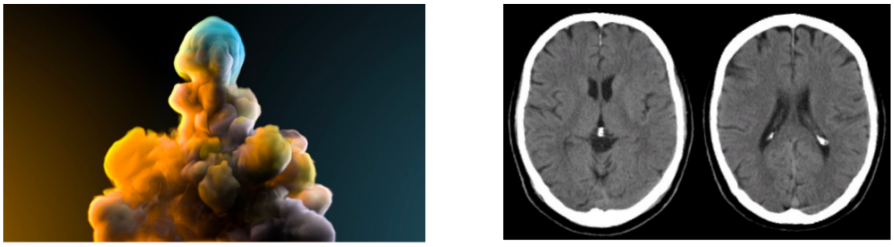


Figure 36: Two images examples

Definition 14.1 (Density field)**.** A density field assigns $\sigma(\boldsymbol{x}) \in \mathbb{R}^+$ (or $\sigma[i,j,k] \in \mathbb{R}^+$ on a grid); $\sigma(\boldsymbol{x}) \geq 0$ says how much light is absorbed per unit length at $\boldsymbol{x}$.

There is no notion of a surface: e.g. $\sigma(\boldsymbol{x}) = 0.1$ everywhere represents a semi-transparent fluid. Typical examples on the slide: smoke and CT-scan slices. The lecture will revisit this in more detail later (NeRF).

Example 14.2 (Preview: how much light survives?)**.** If a beam crosses a slab of length L of constant density σ, the fraction of light transmitted is $e^{-\sigma L}$ (absorption per unit length compounds multiplicatively). For $\sigma = 0.1$ and $L = 10$: $e^{-1} \approx 0.37$. (This is a standard consequence of the "absorbed per unit length" reading; the slides develop it later.)

14.2 Primitive-based Density Fields *(slides slide 65)*

$$\sigma(\boldsymbol{x}) = \sum_k w_k \exp\Big(-\tfrac{1}{2}(\boldsymbol{x} - \boldsymbol{\mu}_k)^\top \Sigma_k^{-1} (\boldsymbol{x} - \boldsymbol{\mu}_k) \Big).$$

These are *hybrid* representations for primitive-induced density fields (blobs/Gaussians with weights w_k, centres $\boldsymbol{\mu}_k$, covariances Σ_k); the lecture will revisit them in more detail (differentiable Gaussian rendering).

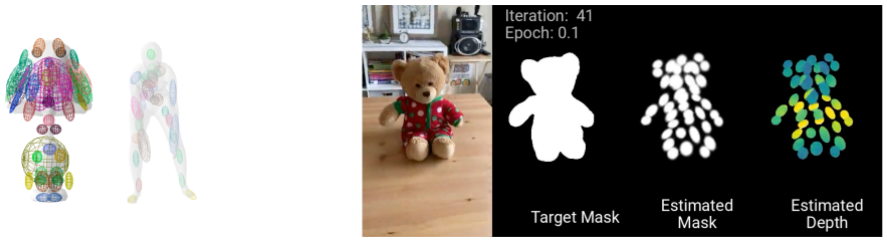


Figure 37: Images examples of primitive-based density fields

Intuition

Instead of a grid of numbers, the density is a *cloud of soft blobs*: each blob has a position, a size/orientation (via Σ_k) and a weight. The whole field can be edited by moving blobs, and only regions with blobs cost anything, so it avoids the "pay for empty space" problem of dense grids.

15 So? *(slides slide 66)*

Slide 66 compares the representations side by side (its cells are colour-coded easy / possible with effort / hard; the colours do not survive in the text of the PDF, so the qualitative ratings appear

in Section 17, based on the earlier slides):

	Points on the surface	**Inside or outside?**	**Continuous edits reach**	**Rendering**	**Typical use**
Point cloud (array $N \times 3$)	given	no surface	any point set	splats	sensors: LiDAR, RGB-D
Mesh ($V : N \times 3$, $F : M \times 3$)	sample triangles	count ray crossings	same wiring only	GPU rasterizer	graphics, games
Parametric ($f(\boldsymbol{u})$)	evaluate f	search over $\boldsymbol{u}$	same topology only	via a mesh	CAD, design
Voxels (grid $W \times H \times D$)	marching cubes	look up the cell	any topology*	blocky	3D CNNs, occupancy maps
Implicit (SDF) ($f(\boldsymbol{x})$: formula, grid, net)	marching cubes	sign of f	any topology	ray marching	reconstruction, neural fields

* with occupancy probabilities, not 0 or 1.

Intuition

No row wins every column: pick the representation for the task. Need to shoot rays fast on a GPU? Mesh. Need to change topology or run inside/outside queries? Implicit. Data straight from a LiDAR? Points. Feeding a 3D CNN? Voxels.

16 And many more... *(slides slides 67–71)*

16.1 Octrees *(slides slide 68)*

Octrees are *voxels with adaptive resolution*: surface-like efficiency for rendering and storage, volume-like efficiency for occupancy queries; but it is difficult to edit or predict the sparsity structure.

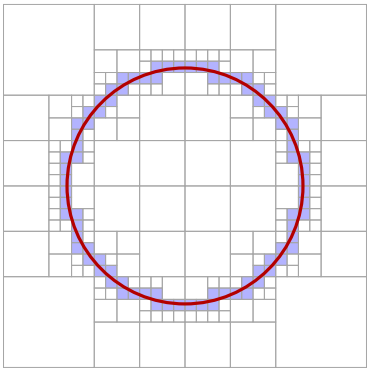


Figure 38: A quadtree (2D octree) for a disk of radius 0.65 in $[-1, 1]^2$, subdivided down to 5 levels only where the boundary passes (blue). It uses 244 leaf cells; a uniform grid of the same finest resolution (32×32) uses 1024.

Intuition

An octree spends resolution where the shape is: big cells for air and interior, tiny cells on the boundary. The trade-off is structural: the tree *shape* depends on the data, which is hard to edit or to predict with a network (the same sparse/dense tension seen with meshes' F).

16.2 Layered Depth Images *(slides slide 69)*

Definition 16.1 (Layered depth image (LDI))**.** Layer 1 is the visible content (depth and colour per pixel); layer $N{+}1$ stores the depth and colour of the *next* surface point the ray hits after layer N. This extends depth to capture "full" 3D, but the layers are often very discontinuous.

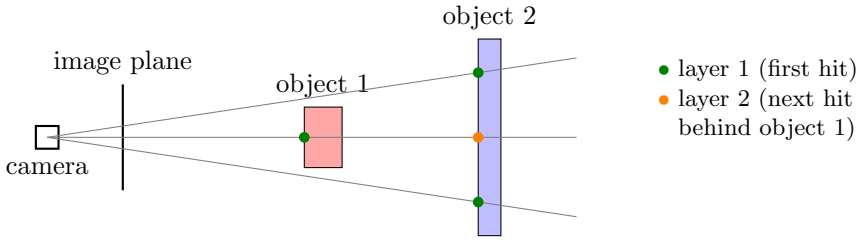


Figure 39: Three pixel rays. The upper and lower rays hit only object 2 (layer 1 only); the middle ray hits object 1 first and object 2 second, so its pixel stores two (depth, colour) entries.

Intuition

A depth map keeps only the first hit along each ray; an LDI keeps a *list* of hits. The hidden second layer (what is behind object 1) is what allows rendering from a slightly different viewpoint without holes; but the layer index jumps around from pixel to pixel, so layers are not smooth images.

16.3 Multi-plane Images *(slides slide 70)*

Multi-plane images (MPIs) are discrete volumetric representations with *non-uniform* spatial cells: a stack of fronto-parallel planes at fixed depths, each a (colour, transparency) image. They are extremely efficient to render (each layer is just a plane at fixed depth), but have limited resolution for far-away content; uniform cells may be preferable for object-centric 3D.

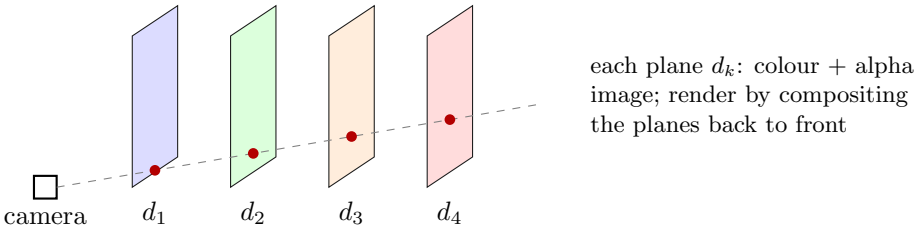


Figure 40: A multi-plane image: planes at increasing depth $d_1 < \cdots < d_4$; a pixel ray crosses all of them and the result is their composite.

Intuition

An MPI is a pile of transparent slides at fixed depths: to draw the scene from a nearby viewpoint, shift and blend the slides. Planes are spaced non-uniformly (dense near, sparse

far), which is why distant content gets little resolution.

16.4 CAD Models and CSG Trees *(slides slide 71)*

CAD models and CSG (constructive solid geometry) trees are *compositions of primitive volumes or surfaces*. They are popular across design applications but challenging for learning.

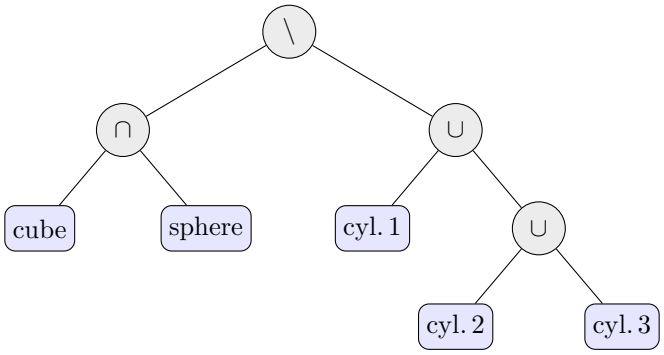


Figure 41: A CSG tree: leaves are primitives, inner nodes are Boolean operations (as on slide 71: $(\text{cube} \cap \text{sphere}) \setminus (\text{cyl.}_1 \cup (\text{cyl.}_2 \cup \text{cyl.}_3))$).

Example 16.2 (A CSG tree is a nested min / max)**.** If each primitive has an implicit function $(f_{\text{cube}}, f_{\text{sph}}, f_{c1}, f_{c2}, f_{c3})$, the tree of the figure is the field

$$f = \max\Big(\max(f_{\text{cube}}, f_{\text{sph}}),\ -\min\big(f_{c1}, \min(f_{c2}, f_{c3})\big)\Big),$$

by the Boolean rules of slide 52.

Intuition

A CSG model is a *program*: a recipe of primitives and Boolean steps. It is compact, editable and exact, but the structure (which tree, how deep) is discrete and combinatorial, so it is hard to predict with a neural network by gradient descent.

17 Summary *(slides slides 72–74)*

The summary slide (72) repeats the zoo of slide 4: mesh, point cloud, CAD model, voxels, implicit surface $\{\boldsymbol{x} \mid f(\boldsymbol{x}) = 0\}$, parametric surface $f(\boldsymbol{u}) = \boldsymbol{x} \in \mathbb{R}^3$, CSG tree, and many more. Slides 73–74 are "Questions?" and Acknowledgments: the slides are adapted from *Learning for 3D Vision*, CMU 16-825, Prof. Shubham Tulsiani.

Comparison table

	Get surface points	Inside / outside	Change topology continuously	Render directly	Memory scales with
Point cloud	given	no surface	any point set	splats	points (N)
Triangle mesh	sample triangles	count ray crossings	same wiring only	rasterizer	vertices, faces
Parametric	evaluate f	search over $\boldsymbol{u}$	same topology only	via a mesh	parameters of f
Voxels	marching cubes	look up cell	with probabilities	blocky	n^3
Implicit / SDF	marching cubes	sign of f	any topology	ray marching	grid n^3 or network

easy possible with effort hard / limited. Ratings are our reading of slides 22–62 and the table of slide 66, not a table from the slides.

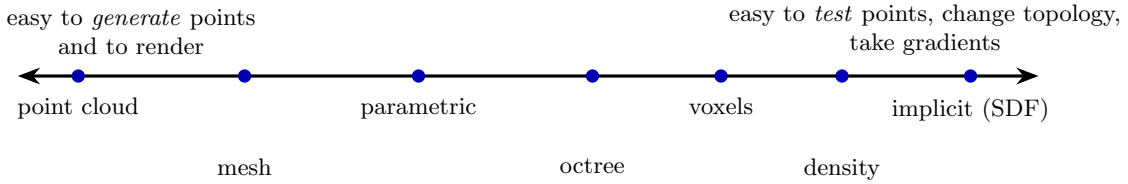


Figure 42: A qualitative map of the "generate vs. test" spectrum (our summary of slides 41, 48, 62 and 66; positions are indicative only).

Key take-aways

- **There is no universal 3D representation.** Points and depth maps come from sensors; meshes are for rendering and graphics; implicit fields are for inside/outside tests, topology changes and learning; the right choice depends on the task.
- **Parametric generates, implicit tests.** $f(\boldsymbol{u}) = \boldsymbol{x}$ makes sampling trivial and queries hard; $\{f(\boldsymbol{x}) = 0\}$ makes queries trivial (sign of f, ∇f for normals, min / max for Booleans) and sampling hard.
- **Meshes split geometry from connectivity.** V is continuous and optimizable; F is discrete, so predicting arbitrary connectivity or changing topology is hard; check winding order, and remember $F \approx 2V$ for closed genus-0 meshes.
- **Grids are simple but pay for empty space.** A n^3 voxel grid (or SDF grid) handles any topology and suits convolutions, but only $\sim n^2$ cells touch the surface; octrees and continuous fields (networks, Gaussians) fight this cost.
- **SDFs are the well-behaved implicit fields.** $\|\nabla f\| = 1$ makes $|f|$ the true distance, enabling safe ray-marching steps and TSDF fusion, and marching cubes converts any such field back into a mesh.

Part II

Lecture 3

How to read these notes. Section headers reuse the titles of the slides and are tagged with the slide range they cover. Formal statements (definitions, propositions, theorems, algorithms) are placed where the slides introduce the topic; the slides themselves state most results without proof, so the derivations are added here. Blue *Intuition* boxes explain the "why" in plain words, with worked numbers. All figures are original TikZ/pgfplots drawings; figures marked *schematic* show the qualitative trend described on the slides, not measured data.

18 From Image Formation to Measurement *(slides 1–3)*

Lecture 01 modeled how a scene becomes an image: projection sends a 3D point to a pixel and *discards the distance along the ray.* Lecture 02 covered ways to *represent* a scene: points, meshes, voxels, signed distance functions, Gaussians. Neither lecture covered how to *measure* a scene. This lecture obtains one depth value per pixel (not always – some sensors leave holes) in one of three ways: by measuring it with a laser, by stereo vision, or by *predicting* it with a learned prior.



Section	Title	Slides
3.1	From representations to measurements	4–7
3.2	Three ways to pay for depth	8–12
3.3	Inside a LiDAR	13–17
3.4	From point cloud to depth image	18–19
3.5	From recovery to prediction	20–22
3.6	Cues in closed form	23–26
3.7	Supervised single-view depth	27–35
3.8	Cues used by the network	36–39

Table 1: The agenda (slide 3) with slide ranges.

Intuition

The whole lecture is one story: an image is a *projection*, so depth is missing. Every method for getting it back must *pay* with something – time (a laser pulse), a baseline (a second viewpoint), computation (motion), or a prior (assumptions learned from data). The interesting questions are always: what did we pay, and what is therefore *guaranteed* about the answer?

19 From Representations to Measurements *(slides 4–7)*

What one image determines, and what it does not. The two preceding lectures modeled image formation and scene representation. This section separates *recovered* from *ambiguous* quantities and shows which of the two depth from one image is.

19.1 Recovered and Ambiguous Quantities *(slides 5)*

Definition 19.1 (Recovered vs. ambiguous quantity)**.** Every quantity estimated in this lecture falls into one of two cases.

- **Recovered**: the data determine it up to noise. It can be reported in meters with an error bar.
- **Ambiguous**: the data determine it only up to a *family of transformations*. Any single number reported for it comes from an added assumption.

Quantity	Free parameters	What pins it down
metric depth	0	nothing, it is already in meters
up to scale	1 (s)	one known length
up to scale and shift	2 (s, t)	two known depths
ordering only	any monotone map	a calibrated sensor

Table 2: Ambiguity ladder (slide 5). The number of free parameters says how much external information is still needed.

Intuition

Think of the free parameters as *knobs you cannot read from the data*. Metric depth has no knob. "Up to scale" has one knob: the scene may be uniformly inflated or shrunk. "Up to scale and shift" has two. "Ordering only" is the weakest: you know that A is nearer than B, but $(1, 2, 3)$ and $(1, 10, 100)$ tell exactly the same story. The slide's punchline: disagreements about depth estimation are usually disagreements about *which row of this table a quantity lives in*.

19.2 The Back-Projected Ray *(slides 6)*

Projection throws one number away per pixel; inverting it asks for that number back:

$$\boldsymbol{p}^c \;=\; z\,\boldsymbol{K}^{-1}\,\tilde{\mathbf{x}}_s. \tag{1}$$

Here $\tilde{\mathbf{x}}_s$ is the homogeneous pixel (third component 1), $\boldsymbol{K}$ the intrinsics from lecture 01, and $\boldsymbol{p}^c$ the point in the camera frame.

Proposition 19.2 (Projection is not invertible)**.** *The third component of $\boldsymbol{K}^{-1}\tilde{\mathbf{x}}_s$ is 1, so the free scalar z in (1) is the depth in meters along the optical axis. Given only $\boldsymbol{x}_s$, the set of 3D points that project to it is the whole ray*

$$\{\, z\, \boldsymbol{K}^{-1}\tilde{\mathbf{x}}_s \;:\; z > 0 \,\}.$$

Proof. Third row of $\boldsymbol{K}$ is $(0, 0, 1)$, hence $\boldsymbol{K}\boldsymbol{p}^c = (*, *, p_z^c)^\top$ and $\boldsymbol{p}^c$ projects to $\tilde{\mathbf{x}}_s$ iff $\boldsymbol{K}\boldsymbol{p}^c = \lambda\tilde{\mathbf{x}}_s$ with $\lambda = p_z^c$ (homogeneous division). Solving, $\boldsymbol{p}^c = \lambda\boldsymbol{K}^{-1}\tilde{\mathbf{x}}_s$ and $\lambda = z$. Conversely every $z\boldsymbol{K}^{-1}\tilde{\mathbf{x}}_s$ satisfies $\boldsymbol{K}(z\boldsymbol{K}^{-1}\tilde{\mathbf{x}}_s) = z\tilde{\mathbf{x}}_s \sim \tilde{\mathbf{x}}_s$. □

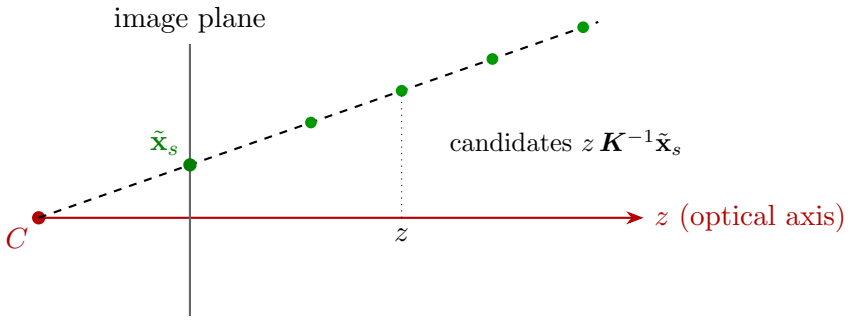


Figure 43: Back-projected ray (redrawn from slide 6; side view, y up). Every green point projects to the same pixel.

Intuition

It is like knowing the *direction* in which a laser pointer is aimed but not how far away the dot lands. The information was *destroyed at capture*, not merely hidden: no clever algorithm can un-destroy it, so every method that produces z is either measuring something extra or importing an assumption. "Producing z at every pixel is the whole of this lecture."

19.3 Scale-Depth Ambiguity *(slides 7)*

Proposition 19.3 (Global scale is unobservable from one image)**.** *Let $\{\boldsymbol{p}_j\}$ be a scene in the camera frame and $s > 0$. The scaled scene $\{s\boldsymbol{p}_j\}$ produces exactly the same image.*

Proof. $\pi(\boldsymbol{K}\, s\boldsymbol{p}) = \pi(s\, \boldsymbol{K}\boldsymbol{p}) = \pi(\boldsymbol{K}\boldsymbol{p})$, because the homogeneous division cancels s. □

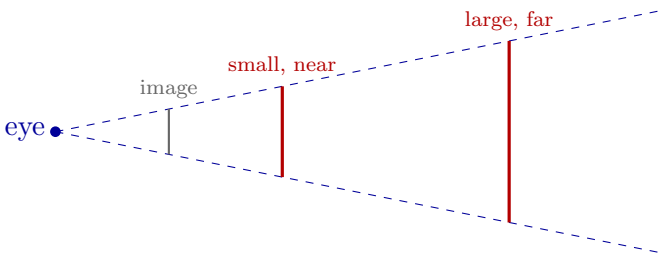


Figure 44: Scale-depth ambiguity (schematic after slide 7): a small near object and an object twice as large at twice the distance subtend the same angle.



Figure 45: two photos of a giant red bench, the second with a bicycle placed on it

Intuition

A small close object looks the same as a much larger one further away. The giant bench in the slide's photographs is a real-life example: on its own it reads as a normal bench, and only the bicycle on top reveals its true size. That bicycle is exactly a *prior about sizes* – the theme of the rest of the lecture.

20 Three Ways to Pay for Depth *(slides 8–12)*

Time, a baseline, a prior. No method returns depth for free. Each buys it with a different resource, and the resource decides what the result is good for.

20.1 Two Measurement Principles *(slides 9)*

Definition 20.1 (Time of flight and triangulation)**.** Two independent choices: the *principle* decides what is measured; the *illumination* decides only whether the sensor brings its own light (*active*) or uses ambient light (*passive*).

- **Time of flight (ToF)**: measure the delay of emitted light.
- **Triangulation**: measure the angular/pixel difference between two viewpoints separated by a known geometry.

	time of flight	triangulation
active (own light)	scanning laser, flash range camera	projected-pattern camera, active stereo
passive (ambient light)	*empty*: timing needs a known emission	stereo with a known baseline, structure from motion

Table 3: The two-by-two of depth sensing (slide 9).



Figure 46: left: speckle pattern of a projected-pattern camera with a stereo camera overlaid; right: a Hesai spinning LiDAR

Intuition

The empty cell is instructive: to time light you must *know when it left*, and ambient light has no known departure time. Also, "active" only helps with texture or timing – it does not change the physics: a projected pattern camera is still triangulation, it just paints texture onto blank walls so matching works.

20.2 Triangulation From Two Views *(slides 10)*

Two rectified cameras sit a baseline b apart. A scene point $\boldsymbol{p} = (x, y, z) \in \mathbb{R}^3$ appears at horizontal pixel offsets u_L and u_R (relative to each principal point), and similar triangles give the depth.

Theorem 20.2 (Depth from disparity)**.** *For two rectified cameras with focal length f (in pixels) and baseline b (in meters), the depth of a point with disparity $d = u_L - u_R$ is*

$$z = \frac{f\,b}{d}.$$

Proof. By the pinhole model $u_L = f\,x/z$ for the left camera, and since the right camera is displaced by b along x, $u_R = f\,(x-b)/z$. Subtracting, $d = u_L - u_R = fb/z$. (The slide draws the image plane behind the pinhole, which produces the minus signs in its two similar-triangle equations; the resulting formula is the same.) □

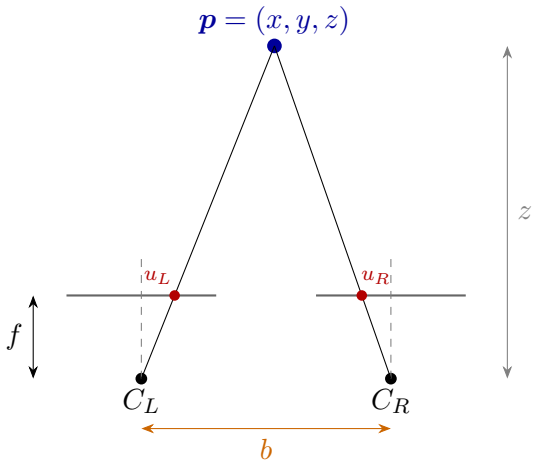


Figure 47: Rectified stereo (redrawn from slide 10, top view). With $f = 1$, $b = 3$, $z = 4$ as drawn: $u_L = 0.4$, $u_R = -0.35$, $d = 0.75 = fb/z$.

Corollary 20.3 (Unknown baseline)**.** *Disparity is inverse depth scaled by fb: $1/z = d/(fb)$. If b is unknown (a moving camera), depth is known only up to one global scale.*

Example 20.4. For $f = 700\,\text{px}$ and $b = 0.5\,\text{m}$, $fb = 350\ \text{px}\,\text{m}$ and $z = 350/d$: $d = 70\,\text{px}$ means $5\,\text{m}$, $d = 17.5\,\text{px}$ means $20\,\text{m}$, $d = 7\,\text{px}$ means $50\,\text{m}$.

Intuition

Where meters come from: b is in meters, so z is in meters – the baseline is the source of scale. Hold a finger at arm's length and blink one eye at a time: the finger jumps a lot against the background; a distant tree barely moves. That jump *is* the disparity, and it is large for near things (large $1/z$). **Matching** is the price: d requires finding the *same* point in both images, which fails on textureless or repetitive surfaces and wherever a surface is visible to one view only. Because disparity is fb times inverse depth, an unknown rig only leaves an unknown *scale* in disparity, which is why affine invariance in *disparity* suits data whose rig is unknown (used in §3.7).

20.3 The Three Currencies *(slides 11)*

20.4 The Single-Image Claim *(slides 12)*

The usual motivation for single-image depth is that it bypasses *all* these limitations. The slide's verdict: a single image does bypass the **cost**, the **sparsity** and the **hardware**, but it cannot

pay with	method	gives	costs
time	emit a pulse and time the return (LiDAR)	metric depth, directly	sparse, expensive, returns missing
a baseline	stereo, baseline known	metric depth, dense where matching works	a calibrated second camera; error growing as z^2
computation	structure from motion, baseline unknown	shape up to one global scale	a moving camera and a rigid scene
a prior	assumptions learned from data	dense, from one image, cheap	not metric, and only as good as the prior

Table 4: The three currencies (slide 11).

bypass the **physics**, because it *never measured anything*. The result is therefore an *ambiguous* quantity (same ray picture as Figure 43).

Intuition

Single-image depth is not a cheaper LiDAR; it is a different kind of object. It converts the question "how far?" into "how far, *given how scenes usually look*?" – a guess with an informed prior. Keeping this distinction in mind explains every failure mode in §3.8.

21 Inside a LiDAR *(slides 13–17)*

Paying with time. Outdoor depth labels are produced by a scanning laser, and their error sources are inherited by any network trained on them – so we need to know how a LiDAR fails.

21.1 Pulsed Time of Flight *(slides 14)*

A short pulse is emitted and a clock runs until the echo returns. The light covers the distance twice, so the interval is halved:

$$r = \frac{c\,t}{2}. \tag{2}$$

Each beam leaves in a known direction, so a return is a *direction plus a range*; the energy that comes back is reported as *intensity*. Pulse widths are a few nanoseconds (typically 5–10 ns, about 4 ns in modern full-waveform scanners). Every other part of the sensor exists to make the interval t reliable.

Example 21.1. An echo from 50 m returns after $t = 2 \cdot 50/(3 \cdot 10^8) \approx 333$ ns. A timing error of 1 ns corresponds to $c\,\delta t/2 \approx 15$ cm of range. A 5 ns pulse is $c \cdot 5\,\text{ns} \approx 1.5$ m long in space.

Intuition

It is echolocation with light: shout and time the echo. The factor $\frac{1}{2}$ is just the round trip. Because c is so large, centimeter accuracy demands sub-nanosecond timing – which is why timing electronics dominate the design.

21.2 Structured Sparsity *(slides 15)*

Azimuth resolution is set by the spin rate; elevation resolution is fixed in hardware. The two differ, so the sampling is *anisotropic*.

Definition 21.2 (Scan-line spacing)**.** For a vertical angular pitch $\Delta\theta_{\text{el}}$ (radians), the spacing between adjacent scan lines at range r is

$$s(r) \approx r\,\Delta\theta_{\text{el}}.$$

Example 21.3 (Velodyne VLP-16)**.** At 5 Hz the sensor samples every 0.1° in azimuth; at 20 Hz every 0.4° – spinning faster costs horizontal resolution. Vertically it is stuck at $\Delta\theta_{\text{el}} = 2.0° = 0.0349\,\text{rad}$, so the scan lines are 0.70 m apart at 20 m and 1.75 m apart at 50 m. A pedestrian at 50 m is crossed by one scan line or by none. (For comparison, the azimuth gap at 50 m is only 0.09 m at 5 Hz and 0.35 m at 20 Hz.)

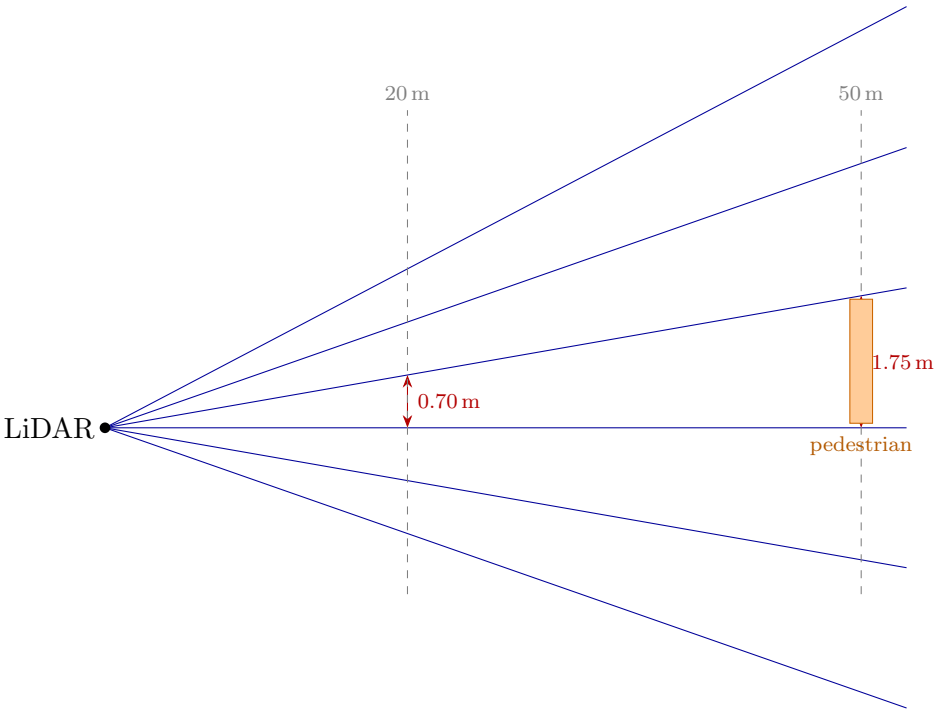


Figure 48: Original figure (not on the slides): VLP-16 elevation fan with a 2° pitch, side view. Vertical axis exaggerated: 1 cm = 1 m vertically but 1 cm = 5 m along range. At 50 m a 1.7 m pedestrian fits between two consecutive scan lines and is missed.

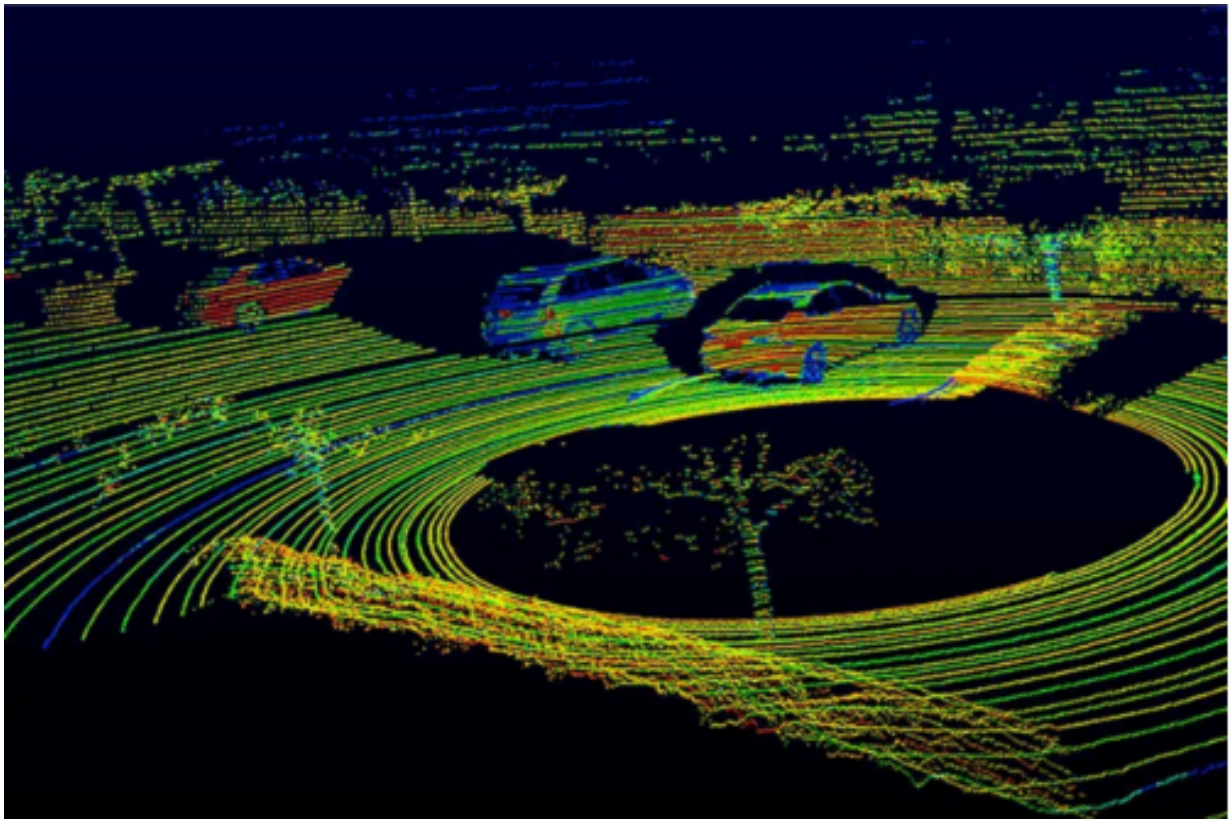


Figure 49: colored LiDAR point cloud of a parking lot: concentric ring-like scan lines with gaps between them

Intuition

Picture the grooves of a vinyl record. Samples are dense *along* a groove (azimuth) but grooves are far apart (elevation). Moving away from the sensor, the grooves spread linearly with range, while points along a groove stay comparatively tight. So a point cloud thins with distance along *one axis* much faster than the other, and the gaps form horizontal stripes.

21.3 Limitations of a LiDAR Point Cloud *(slides 16)*

- **No color and no semantics.**
- **No measurement where there was no return**, which is *distinct from a zero depth*: "I saw nothing" is not "the surface is at distance 0".
- **No sample between scan lines**, and that gap grows linearly with range.
- **Points that lie on no surface**: where one pulse straddles a silhouette, the echo mixes foreground and background into a single (wrong) range.

Intuition

The last item is the LiDAR analogue of an out-of-focus edge: half the beam hits the near object, half the far wall, and the reported range is somewhere in the empty air between them. Such "flying" points are a systematic error that a network trained on LiDAR labels will inherit at every object boundary.

21.4 Two Error Laws *(slides 17)*

The two principles fail in different ways, and the difference decides where each is worth using.

$$\text{time of flight:}\quad \delta r = \frac{c\,\delta t}{2}, \qquad \text{triangulation:}\quad \delta z \approx \frac{z^2\,\delta d}{f\,b}.$$

Proposition 21.4 (Triangulation error law)**.** *A disparity error δd produces a depth error $\delta z \approx z^2\delta d/(fb)$: quadratic in range, even for a perfect sensor.*

Proof. From Theorem 20.2, $z = fb/d$, so $\left|\frac{dz}{dd}\right| = \frac{fb}{d^2} = \frac{z^2}{fb}$. Multiply by δd. □

The timing floor $c\,\delta t/2$ is *independent of range*. What degrades with range is the *precision*: the returned power falls as $1/r^2$ or $1/r^4$ (depending on whether the target is extended or smaller than the beam – standard radar/LiDAR-equation background), and a weaker return is timed less precisely.

Example 21.5 (Numbers from the slide)**.** With $f = 700\,\text{px}$, $b = 0.5\,\text{m}$ and quarter-pixel matching ($\delta d = 0.25\,\text{px}$), $fb = 350$ and

$$\delta z(5\,\text{m}) = \tfrac{25\cdot 0.25}{350} \approx 2\,\text{cm}, \quad \delta z(20\,\text{m}) \approx 29\,\text{cm}, \quad \delta z(50\,\text{m}) \approx 1.8\,\text{m}.$$

The laser is exact where it lands, but at 50 m its scan lines are 1.75 m apart, so a pedestrian is crossed at most once.

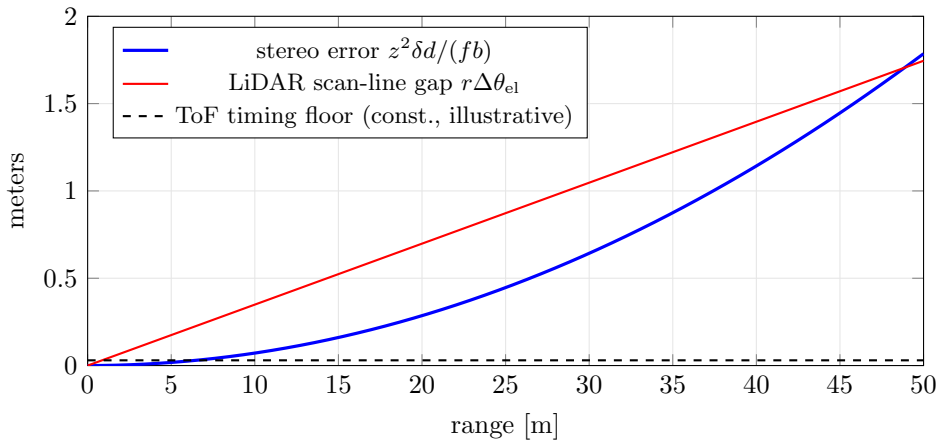


Figure 50: Original figure (not on the slides), from the slide's numbers: $f = 700\,\text{px}$, $b = 0.5\,\text{m}$, $\delta d = 0.25\,\text{px}$, $\Delta\theta_{\text{el}} = 2°$. The ToF floor level (3 cm) is only illustrative.

Intuition

The two sensors are *complementary*: LiDAR is precise where it samples but leaves holes; stereo samples everywhere but each sample degrades quadratically. This is why the remaining sections either *fuse* the two or *replace the laser with a prior*.

22 From Point Cloud to Depth Image *(slides 18–19)*

Paying with time: from cloud to depth map. A LiDAR produces a point cloud. Converting it to a depth map needs the extrinsic calibration, the intrinsics (lecture 01), and a rule for resolving points that fall in the same pixel.

22.1 Projecting a Cloud Into the Image *(slides 19)*

The extrinsics are applied, followed by the projection from lecture 01:

$$\tilde{\mathbf{x}}_s = \boldsymbol{\pi}\Big(\boldsymbol{K}\,[\,\boldsymbol{R}_{cl} \mid \boldsymbol{t}_{cl}\,]\,\tilde{\boldsymbol{p}}^{\,l}\Big)\,, \tag{3}$$

where $\boldsymbol{\pi}(\cdot)$ is the homogeneous division and $\boldsymbol{p}^l$ a point in the LiDAR frame.

Definition 22.1 (Depth map)**.** The number written into the pixel is the z component in the *camera* frame. The array of them is the depth map $\boldsymbol{D} : \Omega \to \mathbb{R}_{>0}$,

$$\boldsymbol{D}[\boldsymbol{x}_s] = \big(\boldsymbol{R}_{cl}\,\boldsymbol{p}^l + \boldsymbol{t}_{cl}\big)_z.$$

The slide stresses that this *differs from the range the LiDAR measured.*

Proposition 22.2 (Depth versus range)**.** *If θ is the angle between the ray of pixel $\boldsymbol{x}_s$ and the optical axis, then $z = r\cos\theta$, where $r = \|\boldsymbol{p}^c\|$ is the range; equivalently $r = z\,\|\boldsymbol{K}^{-1}\tilde{\mathbf{x}}_s\| \geq z$.*

Proof. By (1), $\boldsymbol{p}^c = z\,\boldsymbol{K}^{-1}\tilde{\mathbf{x}}_s$ with $\boldsymbol{K}^{-1}\tilde{\mathbf{x}}_s = (a, b, 1)^\top$; so $r = z\sqrt{a^2+b^2+1}$. The unit direction has z-component $1/\sqrt{a^2+b^2+1} = \cos\theta$. □

Example 22.3. A return with range $r = 20\,\mathrm{m}$ at $30°$ off the optical axis is written as $z = 20\cos 30° \approx 17.3\,\mathrm{m}$ in the depth map.

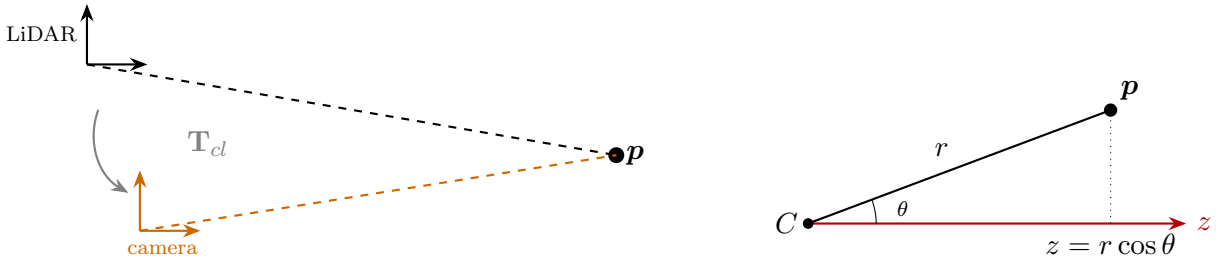


Figure 51: Left: LiDAR and camera frames related by $\mathbf{T}_{cl}$ (redrawn from slide 19). Right: range r versus depth z (original figure).

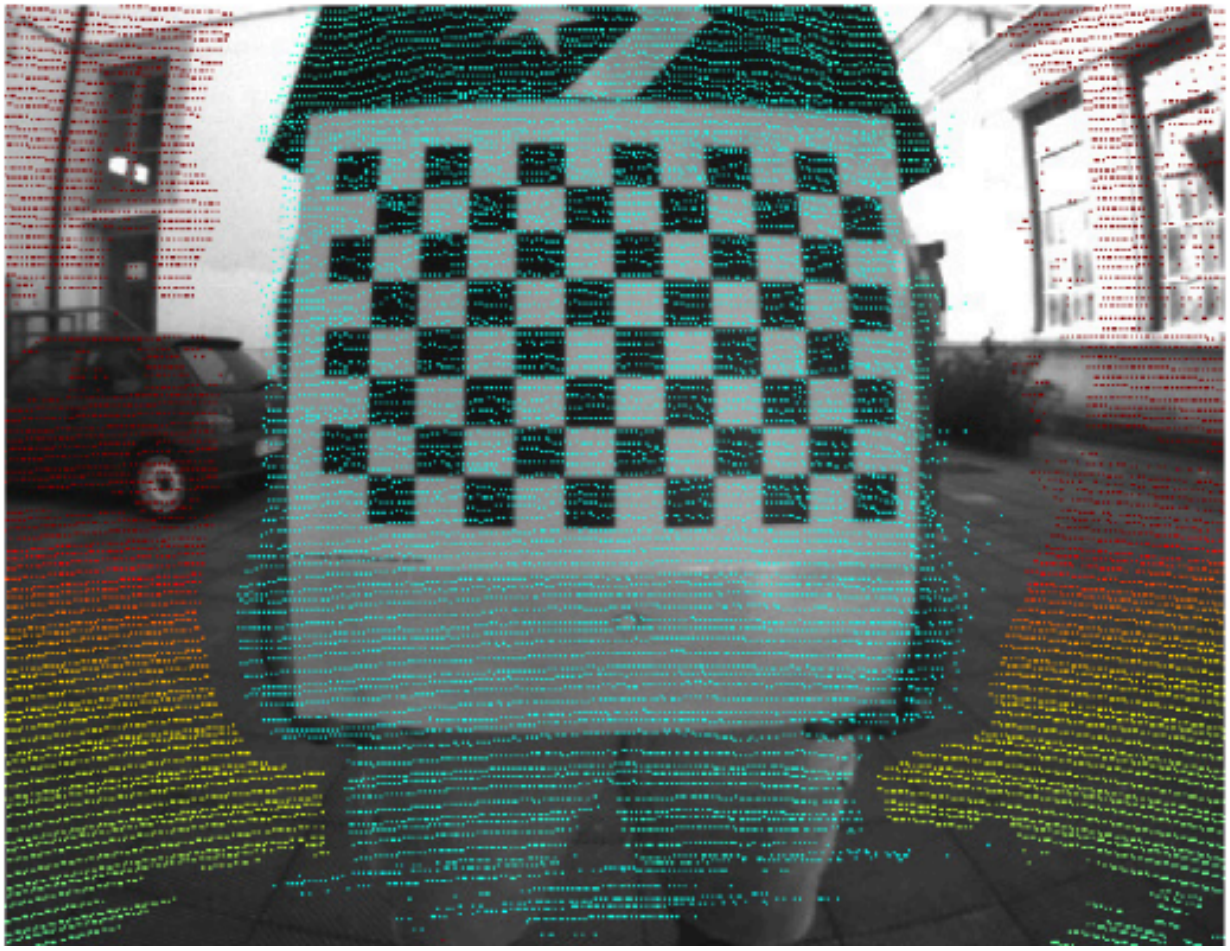


Figure 52: grayscale image of a checkerboard target with LiDAR points projected onto it, colored by depth

Algorithm 22.4 (LiDAR cloud to sparse depth map)**.** Input: points $\{\boldsymbol{p}_j^l\}$, intrinsics $\boldsymbol{K}$, extrinsics $(\boldsymbol{R}_{cl}, \boldsymbol{t}_{cl})$, image size.

1. Initialize $\boldsymbol{D} \leftarrow 0$ everywhere ("no measurement").
2. For each point: $\boldsymbol{p}^c = \boldsymbol{R}_{cl}\boldsymbol{p}^l + \boldsymbol{t}_{cl}$; skip if $p_z^c \leq 0$ (behind the camera).
3. $\tilde{\mathbf{x}}_s = \boldsymbol{\pi}(\boldsymbol{K}\boldsymbol{p}^c)$, round to a pixel; skip if outside the image.
4. If $\boldsymbol{D}[\boldsymbol{x}_s] = 0$ or $p_z^c < \boldsymbol{D}[\boldsymbol{x}_s]$, set $\boldsymbol{D}[\boldsymbol{x}_s] \leftarrow p_z^c$.

Intuition

Step 4 is one common rule for "points that fall in the same pixel" (keep the nearest, as a z-buffer does); the slide only says such a rule is needed. Valid depths are strictly positive ($\boldsymbol{D} : \Omega \to \mathbb{R}_{>0}$), so 0 can safely mark "no return" without colliding with a genuine measurement – matching the earlier warning that a missing return is not a zero depth. Also remember that the result is *sparse*: most pixels stay 0.

23 From Recovery to Prediction *(slides 20–22)*

Paying with a prior. One image does not determine depth. People judge it anyway, so they are supplying an assumption. This section turns that observation into a prior.

23.1 Monocular Cues in Human Vision *(slides 21)*

Not all 3D structures are equally likely, and human vision exploits that regularity through seven classic cues:

1. linear perspective,
2. relative size / position,
3. texture gradient,
4. occlusion (the occluded object is farther),
5. aerial perspective,
6. light and shadows,
7. blur and defocus.

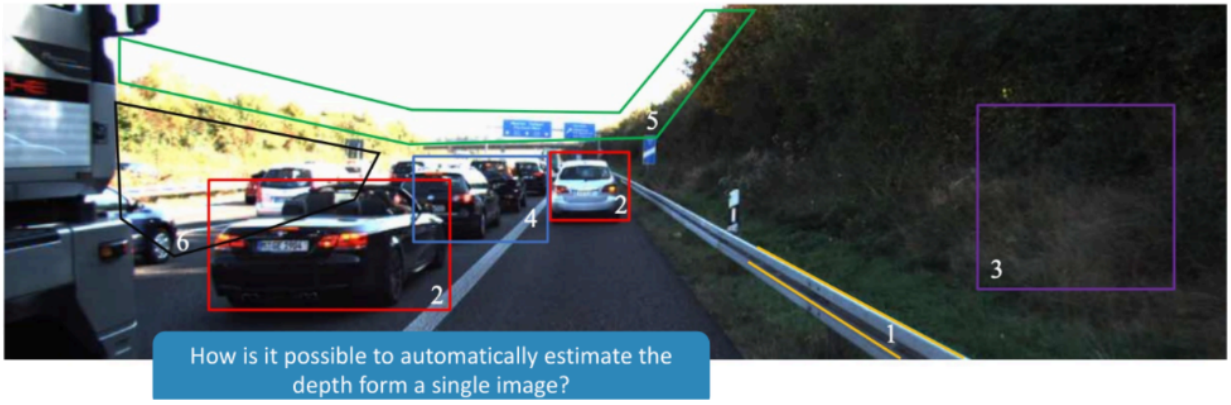


Figure 53: driving image (highway scene) with the seven cues outlined and numbered

Intuition

None of these is a measurement. Each is an *assumption about how scenes are usually arranged*: parallel lines meet, near things are big, far things are hazy. They work because the world is not a uniformly random set of 3D structures.

23.2 Depth Prediction as a Prior *(slides 22)*

Proposition 23.1 (Bayesian view of single-image depth)**.**

$$p(\boldsymbol{D} \mid \boldsymbol{I}) \;\propto\; p(\boldsymbol{I} \mid \boldsymbol{D})\, p(\boldsymbol{D}).$$

The likelihood term is nearly flat along each ray (this is the ill-posedness of §3.1), so the posterior is dominated by the prior $p(\boldsymbol{D})$. Hence single-image depth is predicted *rather than* recovered*, and a method is a claim about which scenes are likely.*

Intuition

The likelihood asks: "does this depth map explain the picture?" Along a ray, every depth explains the pixel equally well, so the picture alone cannot vote. The only thing left to break ties is what depth maps *usually look like* – and people reading depth from a single photograph is everyday evidence that such a prior exists and is learnable. Where the learned one fails is the topic of §3.8.

24 Cues in Closed Form *(slides 23–26)*

The prior, written in closed form. Vanishing points, apparent size and vertical position each follow in one line from the projection equation.

24.1 Vanishing Points *(slides 24)*

Parallel 3D lines converge in the image; the horizon is the image of the line at infinity of the ground plane. A vanishing point is the image of a *direction*, not of a location; every set of parallel lines has its own, and for a given plane they all lie on one line.

Proposition 24.1 (Vanishing point of a direction)**.** *Let $\boldsymbol{d} \in \mathbb{R}^3$ be a 3D direction. Its image $\boldsymbol{v}$ satisfies*

$$\lambda\, \boldsymbol{v} = \boldsymbol{K}\,[\boldsymbol{R} \mid \boldsymbol{t}] \begin{pmatrix} \boldsymbol{d} \\ 0 \end{pmatrix} = \boldsymbol{K}\boldsymbol{R}\,\boldsymbol{d}.$$

Proof. A direction is a point at infinity, homogeneous coordinates $(\boldsymbol{d}, 0)$. In the product $[\boldsymbol{R} \mid \boldsymbol{t}](\boldsymbol{d}, 0)^\top = \boldsymbol{R}\boldsymbol{d} + \boldsymbol{t} \cdot 0$ the translation is multiplied by zero and drops out. □

Corollary 24.2. *The horizon moves with camera **pitch and roll** only; translation does not move it.*

Example 24.3 (Horizon row for a pitched camera – extra worked case)**.** Take zero roll, image y pointing down, and pitch the camera up by θ. The level forward direction has camera-frame coordinates $\boldsymbol{R}\boldsymbol{d} = (0, \sin\theta, \cos\theta)^\top$, so $\boldsymbol{K}\boldsymbol{R}\boldsymbol{d}$ gives the horizon row $y_h = c_y + f\tan\theta$: pitching up moves the horizon *down* in the image.

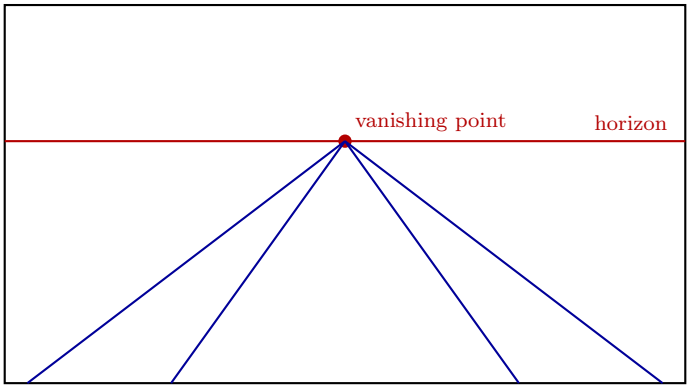


Figure 54: Original schematic (the slide shows a photograph, see below): parallel lines on the ground plane meet on the horizon.

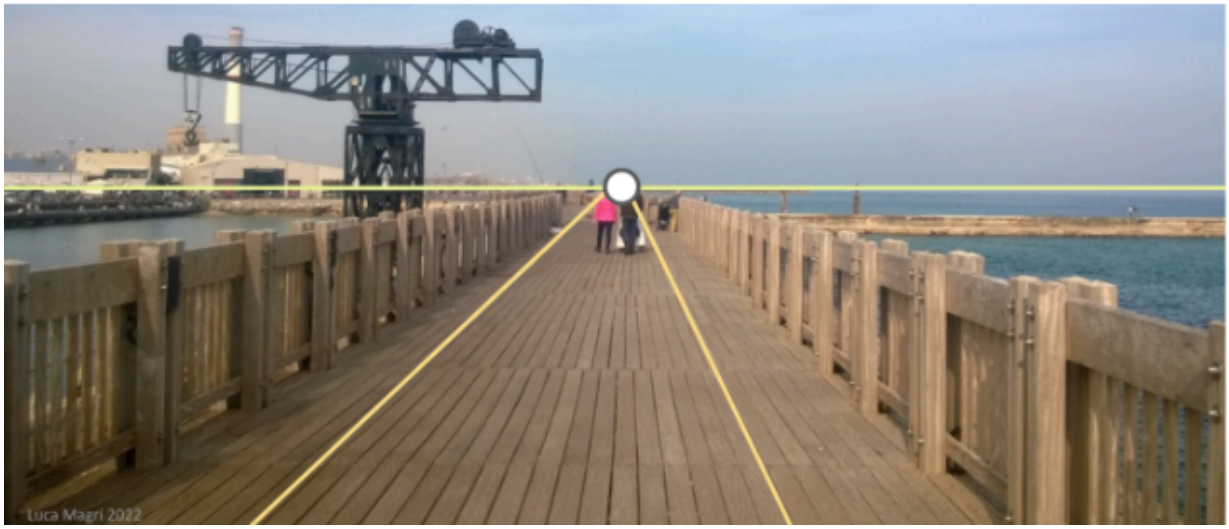


Figure 55: photo of a wooden pier with yellow lines converging to a vanishing point on the horizon

Intuition

The translation drops out because a direction has no position: walking sideways does not change where the rails converge, only tilting the camera does. The slide adds a warning to remember: *a network later on does not respect this* (§3.8).

24.2 Apparent Size *(slides 25)*

Proposition 24.4 (Depth from apparent size)**.** *If the true height H of an object is known and its image height is h (in pixels), then*

$$z = \frac{f\,H}{h}.$$

Proof. The camera center, the top and the bottom of the object form a triangle similar to the one formed by the center and the image of the object: $h/f = H/z$. □

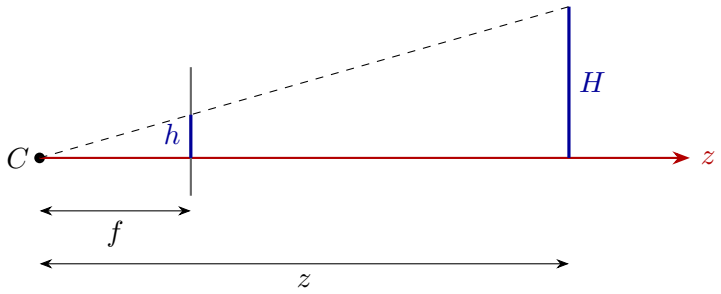


Figure 56: Apparent size (redrawn from slide 25, side view).

Example 24.5. A car $H = 1.5\,\text{m}$ tall that appears $h = 50\,\text{px}$ tall with $f = 700\,\text{px}$ is at $z = 700 \cdot 1.5/50 = 21\,\text{m}$.

Intuition

This works on driving data because almost everything in it is a car, a truck or a pedestrian, and those have known sizes: the *prior about sizes* is doing the work. It fails for objects of unknown size – the giant bench of §3.1 again.

24.3 Vertical Position *(slides 26)*

Proposition 24.6 (Depth from the ground contact point)**.** *On a flat ground plane with known camera height Y, an object touching the ground at image row y_s (horizon row y_h) has depth*

$$z = \frac{f\,Y}{y_s - y_h}.$$

Proof. For a level camera, the ground point has camera-frame vertical coordinate Y (below the camera) and depth z; projection gives $y_s - y_h = fY/z$. Solve for z. □

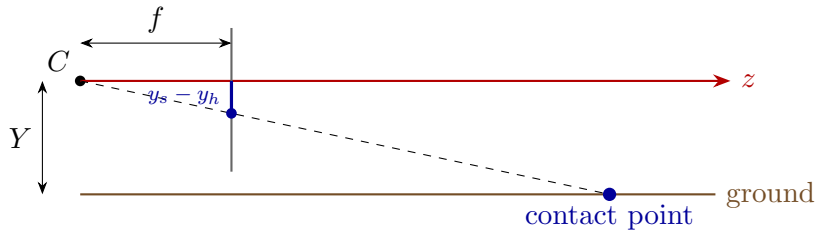


Figure 57: Vertical position (redrawn from slide 26, side view; horizon = optical axis for a level camera).

Corollary 24.7 (Sensitivity)**.** *Differentiating, $\delta z = z^2\,\delta y/(fY)$.*

Example 24.8. $f = 700\,\text{px}$, $Y = 1.65\,\text{m}$, contact point $40\,\text{px}$ below the horizon: $z = 700 \cdot 1.65/40 \approx 28.9\,\text{m}$. A one-pixel error moves this by $z^2/(fY) \approx 0.73\,\text{m}$.

Intuition

No knowledge of the object is required, only its contact point with the ground; but *the assumptions carry the estimate*: flat ground, known camera height, known horizon. Note the resemblance to Proposition 21.4: the error also grows as z^2, with the camera height Y playing the role of a baseline (a remark of these notes, not on the slide). Cues near the horizon are extremely sensitive.

25 Supervised Single-View Depth *(slides 27–35)*

Paying with a prior, part two. Fitting the prior requires an architecture, a loss that respects the scale ambiguity, and measured depth data.

25.1 The Supervised Setup *(slides 28)*

Definition 25.1 (Supervised depth learning)**.** A training set of image / depth-map pairs, and a network from $3 \times H \times W$ to $1 \times H \times W$ trained with a per-pixel loss:

$$\text{TR} = \{(\boldsymbol{I}_i, \boldsymbol{D}_i)\}_{i=1}^{N}, \qquad \hat{\boldsymbol{D}} = f_\theta(\boldsymbol{I}).$$

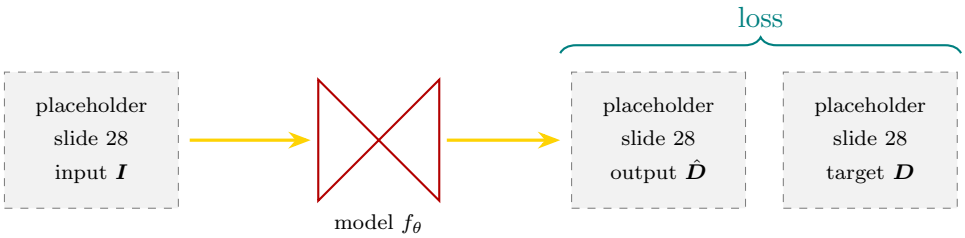


Figure 58: The supervised setup (redrawn from slide 28; the two depth maps are photographs to copy from the slide).

Intuition

Data, network and per-pixel loss are the whole recipe; the slide's summary is that *every remaining design decision is inside the loss* (*what* counts as a correct prediction when scale is unobservable).

25.2 Three Datasets *(slides 29)*

Each of the three sits in a different cell of the principles table: a projected-pattern camera, a scanning laser, and triangulation with no known baseline.

- **NYU Depth v2**: indoor, from a structured-light depth camera, with its raw depth *inpainted* to give a dense label.
- **KITTI**: driving, LiDAR projected into the camera as in §3.4.
- **MegaDepth**: internet photo collections through structure from motion and multi-view stereo. Dense, varied, and *scale-ambiguous by construction.*

These are the $\boldsymbol{D}_i$ of the previous slide. Each inherits the error sources of the sensor that produced it, and the last carries no metric scale at all – which is what the next two methods are designed around.

Intuition

Every dataset is a *sensor's opinion* of depth. NYU inherits the holes and inpainting of a structured-light camera, KITTI inherits LiDAR sparsity and flying points, MegaDepth inherits SfM's arbitrary scale. Datasets with "no scale & shift" or "ordinal" labels cannot be trained on with a plain L2 loss: the loss has to be told which ambiguity to ignore.

25.3 The Scale-Invariant Loss *(slides 30)*

The direct choice, $\mathcal{D}(\boldsymbol{D}, \hat{\boldsymbol{D}}) = \|\boldsymbol{D} - \hat{\boldsymbol{D}}\|_2^2$, penalizes the network for a global scale the image does not determine. What is required is a loss invariant to that scale:

$$\mathcal{D}(\boldsymbol{y}, \boldsymbol{y}^*) = \mathcal{D}(s\boldsymbol{y}, \boldsymbol{y}^*) \quad \forall s > 0.$$

Dataset	Annotation	Depth type	Accuracy	Diversity	# images
DIML Indoor	RGB-D	metric	medium	medium	220K
MegaDepth	SfM	no scale	medium	medium	130K
ReDWeb	stereo	no scale & shift	medium	high	3600
WSVD	stereo	no scale & shift	medium	high	1.5M
3D Movies	stereo	no scale & shift	medium	high	75K
DIW	user clicks	ordinal pair	low	high	496K
ETH3D	laser	metric	high	low	454
Sintel	synthetic	(metric)	high	medium	1064
KITTI	laser/stereo	metric	medium	low	93K
NYUDv2	RGB-D	metric	medium	low	407K
TUM-RGBD	RGB-D	metric	medium	low	80K

Table 5: Trimmed transcription of the dataset table on slide 29 (the indoor/outdoor/dynamic/video/dense check-mark columns are omitted; copy the full table from the slide if needed).

Definition 25.2 (Eigen–Puhrsch–Fergus scale-invariant loss (NIPS 2014))**.** Mean squared error in log space, plus a per-image offset:

$$\mathcal{D}(\boldsymbol{y},\boldsymbol{y}^*) = \frac{1}{n}\sum_{i=1}^{n}\big(\log y_i - \log y_i^* + \alpha\big)^2, \qquad \alpha = \frac{1}{n}\sum_{i=1}^{n}\big(\log y_i^* - \log y_i\big).$$

Lemma 25.3 (Closed form and invariance)**.** *Write* $d_i = \log y_i - \log y_i^*$*. Then* $\alpha = -\bar{d}$ *minimizes the expression over all offsets, and*

$$\mathcal{D}(\boldsymbol{y},\boldsymbol{y}^*) = \frac{1}{n}\sum_i d_i^2 - \frac{1}{n^2}\Big(\sum_i d_i\Big)^2 = \operatorname{Var}(d_i).$$

Moreover $\mathcal{D}(s\boldsymbol{y},\boldsymbol{y}^*) = \mathcal{D}(\boldsymbol{y},\boldsymbol{y}^*)$ *for every* $s > 0$*.*

Proof. Differentiating $\sum_i (d_i+\alpha)^2$ in α and setting to zero gives $\alpha = -\bar{d}$; substituting yields $\frac{1}{n}\sum (d_i - \bar{d})^2$, which is the stated variance. Replacing $\boldsymbol{y}$ by $s\boldsymbol{y}$ adds the constant $\log s$ to every d_i; a variance does not change under a constant shift. □

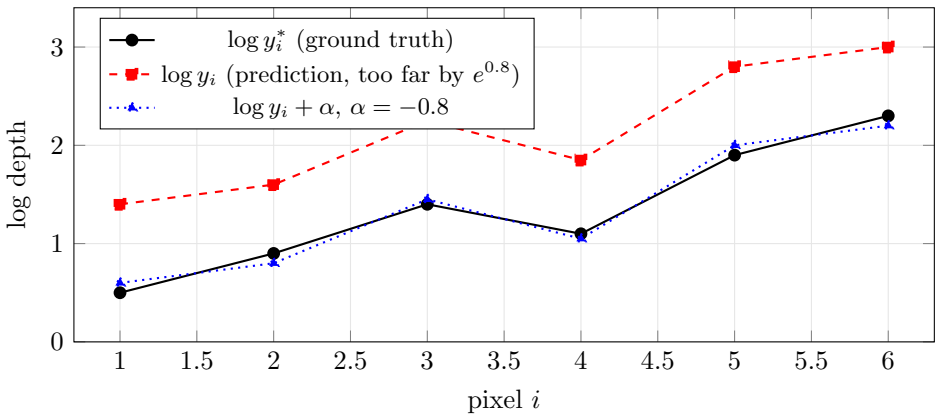


Figure 59: Original toy example (not on the slides). The prediction has the right shape but a wrong global scale. Naive log-MSE would be ≈ 0.646; the scale-invariant loss, after aligning by e^{α}, is only ≈ 0.0058.

Intuition

In log space a wrong *scale* becomes a constant *offset*, and the loss subtracts the best constant offset before looking at the errors. So only the *shape* of the depth map is judged. Two side remarks from the slide: log space stops far pixels (numerically large) from dominating, and $e^{\alpha}\boldsymbol{y}$ is the scalar multiple that best aligns the prediction to the ground truth – every scalar multiple of $\boldsymbol{y}$ scores identically.

25.4 Output Spaces *(slides 31)*

Five quantities a depth network can predict differ in how many unknowns are left free, and in *what the free parameters act on.*

	output	free param.	aligned by	trained this way to
1	metric depth	0	nothing	plan and measure
2	scale-invariant depth	1 (s)	one scale	share one scale per scene
3	affine-invariant disparity	2 (s,t)	scale and shift	mix datasets with no common zero
4	affine-invariant depth	2 (s,t)	scale and shift	mix datasets in depth (3.11)
5	ordinal	any monotone map	nothing	rank only

Table 6: Output spaces (slide 31). Scale-invariant: $\hat{y} \approx s\, y^*$; affine-invariant: $\hat{q} \approx s\, q^* + t$ with $q = 1/y$ for disparity.

These are routinely conflated: Eigen's log-space is *one* parameter, affine invariance *two*. The affine row appears twice because the same pair (s,t) can act on disparity or on depth, and the two differ in 3D (next subsection). Benchmarks align the free parameters against the ground truth first, then score with δ and AbsRel, so a model is scored on *shape, not on meters.*

Example 25.4 (Standard metrics, background)**.** AbsRel $= \frac{1}{n}\sum_i |\hat{y}_i - y_i^*|/y_i^*$, and δ_1 is the fraction of pixels with $\max(\hat{y}_i/y_i^*,\, y_i^*/\hat{y}_i) < 1.25$; both are computed after the alignment above.

25.5 Affine Invariance in Disparity and in Depth *(slides 32)*

Proposition 25.5 (Affine map in disparity = projective map in space)**.** *An affine map in inverse depth,*

$$\frac{1}{z} \longmapsto \frac{s}{z} + t \quad\Longleftrightarrow\quad z \longmapsto \frac{z}{s + t\, z},$$

is a nonlinear map in depth. Back-projecting the transformed depth gives $\hat{\boldsymbol{p}} = \boldsymbol{p}/(s + t\, p_z)$*, which is a* projective *map of space: planes remain planes.*

Proof. $s/z + t = (s + tz)/z$, so $z \mapsto z/(s + tz)$. Back-projection by (1): $\hat{\boldsymbol{p}} = \frac{z}{s+tz}\boldsymbol{K}^{-1}\tilde{\mathbf{x}}_s = \boldsymbol{p}/(s + tp_z)$. To see planes stay planes, let $\boldsymbol{a}^\top \boldsymbol{p} = c$ and $w = s + tp_z$. From $\hat{\boldsymbol{p}} = \boldsymbol{p}/w$ we get $\hat{p}_z = p_z/w$, hence $p_z = s\hat{p}_z/(1 - t\hat{p}_z)$ and $w = s/(1 - t\hat{p}_z)$, so $\boldsymbol{p} = w\hat{\boldsymbol{p}}$. Substituting, $s\, \boldsymbol{a}^\top \hat{\boldsymbol{p}} = c\,(1 - t\hat{p}_z)$, which is linear in $\hat{\boldsymbol{p}}$. □

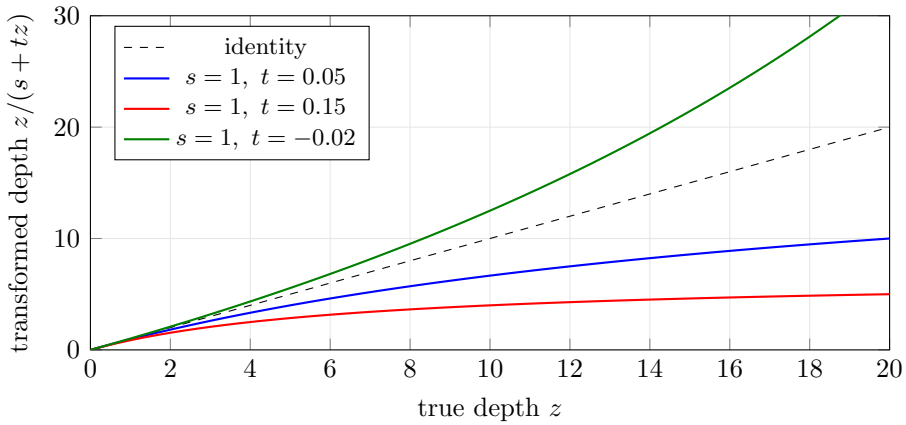


Figure 60: Original figure (not on the slides): an affine map in disparity acts on *depth* as $z \mapsto z/(s+tz)$, a compressive or expansive warp, not a rescaling.

Intuition

A ground plane is therefore *tilted, not bent*: you can decide a scale and shift wrongly, and flat things stay flat but the world may look sheared toward or away from the camera. An affine map in depth itself would instead be $z \mapsto sz + t$, a rigid stretch and translation along the ray. The two choices give the same 2-parameter count but *different 3D geometry* – the reason the table lists the affine row twice.

25.6 MiDaS: Mixing the Datasets *(slides 33)*

No single dataset covers the visual world, so MiDaS trains on several. The obstacle is that they disagree about what they measure (metric, no scale, no scale and shift, ordinal, ...). Its recipe:

- predict in **disparity space** (inverse depth up to scale and shift);
- train with a **scale-and-shift-invariant loss**, where the alignment (s, t) is a least-squares fit solved *in closed form per image*;
- add a **multi-scale gradient matching** term, which is the gradient loss again, at four resolutions.

The output space is the affine-invariant *disparity* row of the table, not the affine-invariant *depth* row under it.

Lemma 25.6 (Closed-form alignment)**.** *For predictions $\hat{q}_i$ and targets q_i^*, the pair*

$$(s, t) = \arg\min_{s,t} \sum_i \left(s\,\hat{q}_i + t - q_i^*\right)^2$$

is $s = \dfrac{\mathrm{Cov}(\hat{q}, q^*)}{\mathrm{Var}(\hat{q})}$, $t = \bar{q}^* - s\,\bar{\hat{q}}$ *(assuming* $\mathrm{Var}(\hat{q}) > 0$*).*

Proof. Setting $\partial/\partial t = 0$ gives $t = \bar{q}^* - s\bar{\hat{q}}$. Setting $\partial/\partial s = 0$: $s\sum \hat{q}_i^2 + t\sum \hat{q}_i = \sum \hat{q}_i q_i^*$. Substituting t and dividing by n: $s(\overline{\hat{q}^2} - \bar{\hat{q}}^2) = \overline{\hat{q}q^*} - \bar{\hat{q}}\bar{q}^*$. □

The loss is then evaluated on the residual *after* alignment. The gradient matching term, in its standard form, penalizes gradients of the aligned residual R^k at each of the $K = 4$ scales k:

$\frac{1}{M}\sum_{k=1}^{K}\sum_i(|\nabla_x R_i^k| + |\nabla_y R_i^k|)$ (form from the original MiDaS paper; the slide only says "the gradient loss again, at four resolutions").

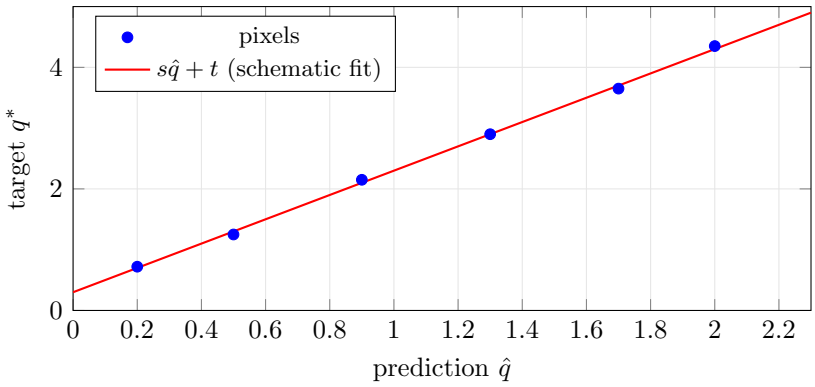


Figure 61: Original schematic (not on the slides): per-image least-squares alignment of a prediction to targets that use a different scale and zero.

Intuition

Each training image gets its *own* best-fit (s, t) before the network is judged, like a teacher who marks a student's drawing after allowing it to be enlarged and moved on the page. That is what lets one network learn from a metric laser dataset, an up-to-scale SfM dataset and a stereo-movie dataset at the same time. Working in *disparity* matters because an unknown rig only contributes an unknown scale there (Corollary 20.3).

25.7 Depth Anything V2 *(slides 34)*

Labeled depth data is the bottleneck, so the second version removes the requirement for it (the first version also trained on labeled real data).

Algorithm 25.7 (Teacher–student training)**.**
1. Train a **teacher** on precise *synthetic* images only, where depth is exact and complete.
2. Use the teacher to **pseudo-label** a very large set of unlabeled real photographs.
3. Train the **student** on those pseudo-labels.

Architecture and losses: encoder from self-supervised pretraining, decoder from dense prediction, and the *same* scale-and-shift-invariant plus gradient-matching losses as MiDaS. The output space is affine-invariant inverse depth; metric models are a separate fine-tuning step on top.

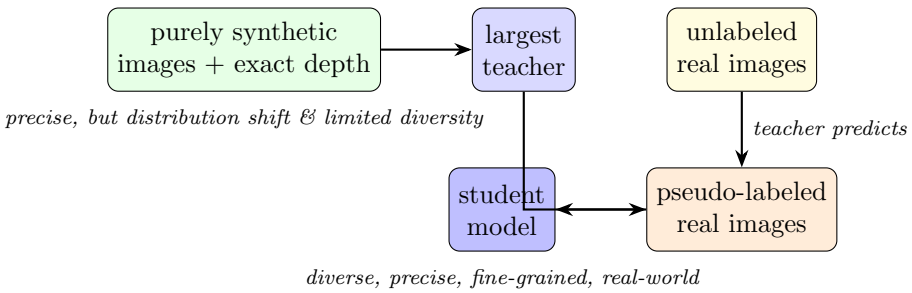


Figure 62: Depth Anything V2 pipeline (redrawn from slide 34).

Intuition

Synthetic images give *perfect* labels but look unlike the real world; real photos look right but have no labels. The teacher transfers its precision to real photos through pseudo-labels, so the student sees real-world diversity with clean, fine-grained targets. The price: the student inherits what the teacher believes – its prior – rather than anything measured.

25.8 Failure on Anamorphic Art *(slides 35)*

Anamorphic street art is a flat pavement painted to look like a hole. The predicted depth map is wrong throughout, with no indication of uncertainty. Nothing has malfunctioned: the prior behaves as trained, on an input constructed to exploit it. *The prior is both the method and the failure mode.*



Figure 63: left: photo of a street painting of a clock tower on pavement; right: predicted depth map from MegaDepth demo

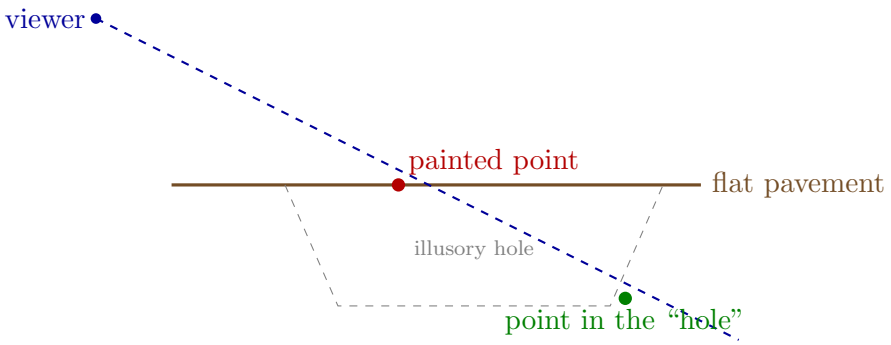


Figure 64: Original figure (not on the slides): from the painting's viewpoint the painted point on the flat ground and the point at the bottom of the imagined hole lie on the same ray (Proposition 19.2). One image cannot distinguish them, so the network chooses the "likely" one.

Intuition

The picture is a concrete instance of the back-projected ray: the artist chooses a flat surface whose appearance is *exactly* what a deep hole would look like from one viewpoint. Depth-from-one-image cannot know; it picks the answer its training data make most probable. Note also the "no indication of uncertainty": the network is confidently wrong.

26 Cues Used by the Network *(slides 36–39)*

What the prior turns out to be. A monocular depth network is a prior fitted to a dataset. Three probes examine which image evidence that prior relies on.

26.1 The Pasted-Car Experiment *(slides 37)*

A car is cropped from the image and pasted back at a chosen ground contact point and a chosen scale, placing the two cues of §3.6 in conflict: both correct (position & size), *position only*, or *size only*. The finding (paraphrased): across four networks, the vertical position in the image wins over the apparent size of known obstacles.

- Position only tracks the true distance; size only does not.
- The result holds across four networks.
- Vertical position needs no object model and generalizes to objects absent from the training set.

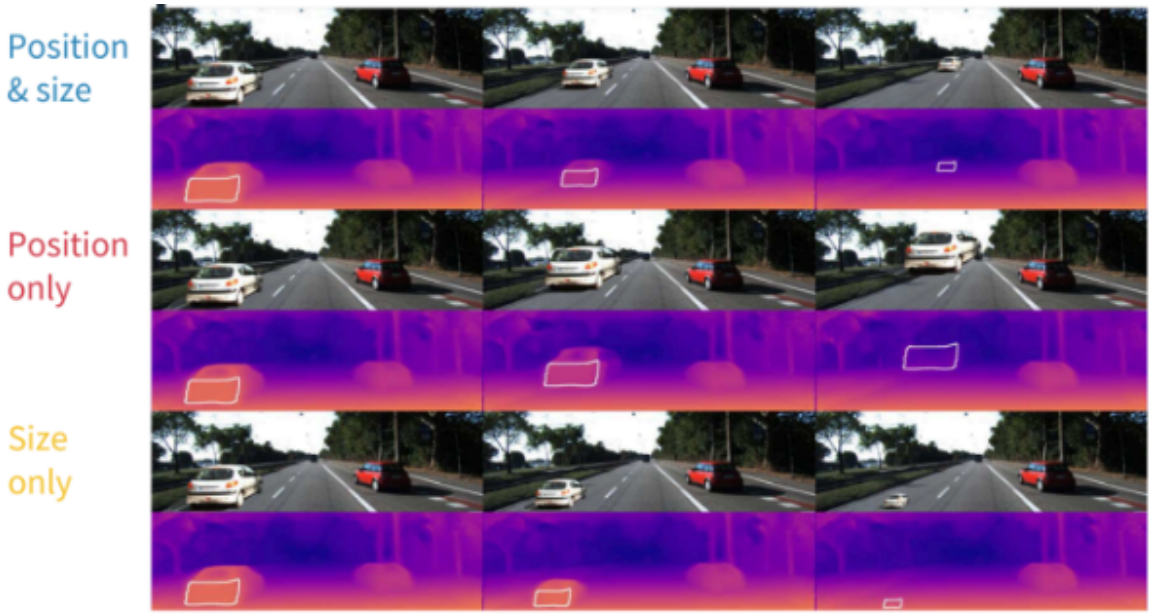


Figure 65: grid of pasted-car examples (position & size / position only / size only) with predicted depth maps

Intuition

The networks behave like the vertical-position cue of Proposition 24.6, not like Proposition 24.4. Sensible: vertical position works for *anything* touching the ground, whereas size needs a size prior per object class. But it also means a car pasted "floating" higher in the frame is judged farther, even if it appears huge.

26.2 Pitch and Roll *(slides 38)*

A pitch change is simulated with a vertical crop and a roll with a rotated crop; both are recovered from the predicted depth. Findings: the networks **only partially** track camera orientation, and small pitch changes measurably disturb the estimated distance to obstacles.

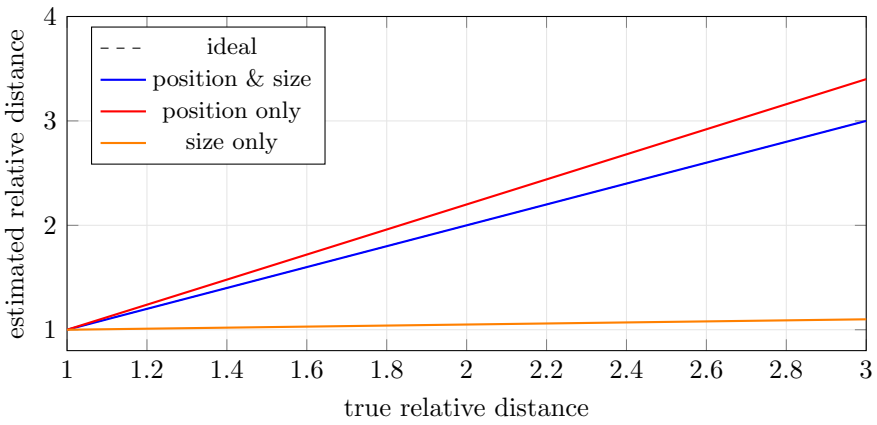


Figure 66: Qualitative schematic after the plot on slide 37, not measured data: positional cues track the ideal line (with some overestimate and larger spread), size-only stays almost flat.

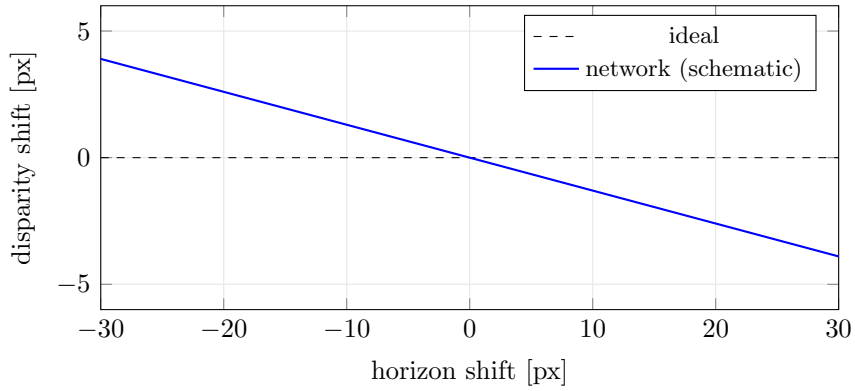


Figure 67: Qualitative schematic after the plot on slide 38, not measured data: an ideal system would ignore the crop (zero shift), the networks' output drifts roughly linearly and opposite to the horizon shift.

Intuition

A crop does *not* change an object's distance to the horizon, $y_s - y_h$, so by Proposition 24.6 the true depth cannot change. But it does change the object's *absolute* vertical position, and the predictions change with it. The network reads vertical image position, rather than distance to the horizon – exactly the shortcut warned about in §3.6, where the horizon depends on pitch and roll only.

26.3 Dependence on the Training Camera *(slides 39)*

A third probe: what happens when only the *camera* changes. A network trained at one focal length predicts the wrong depth at another, with the scene held identical. The apparent-size relation (Proposition 24.4) contains f, and the network has absorbed the training value as a constant. Supplying the intrinsics to the network as extra input channels removes most of the error. The slide's summary: the prior is fitted to the training *camera* as much as to the training *scenes*.

Proposition 26.1 (Idealized focal-length bias – worked consequence, not on the slide)**.** *Suppose a network has absorbed $z = f_0 H/h$ with the training focal length f_0. At test focal length f the image height is $h' = fH/z$, so the predicted depth is*

$$\hat{z} = \frac{f_0 H}{h'} = \frac{f_0}{f}\, z.$$

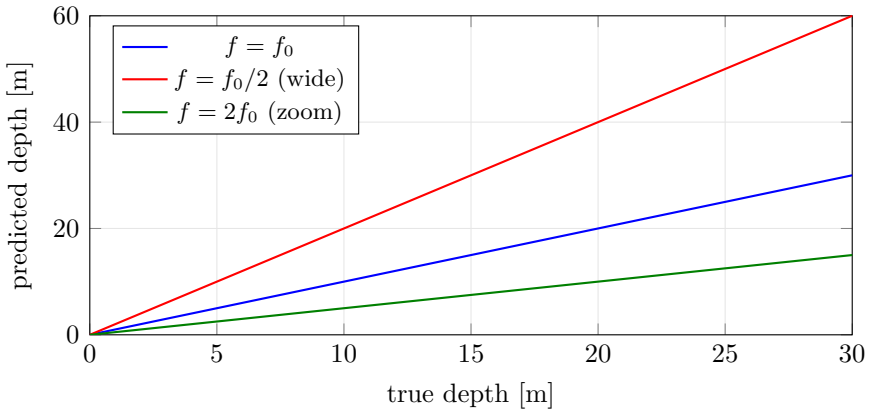


Figure 68: Original figure (not on the slides): predicted depth under the idealization of Proposition 26.1.

Intuition

Zooming in makes everything look bigger, and a network that silently assumes the training focal length reads "bigger" as "closer". This is a scale error, so it is precisely the kind of ambiguity metric prediction must resolve – giving the network the intrinsics removes the hidden constant.

Next: Self-Supervision and a Generative Prior *(slides 40)*

Part I closes owing two debts: the prior was fitted to labels that a sensor had to measure, and mixing those labels cost two free parameters. Part II removes the labels, then pays back the two numbers.

Part I in One Slide *(slides 1–2)*

Part I obtained depth in two ways and closed *owing two debts*:

- **Time.** A laser measures range directly: metric and exact where it lands, but sparse, and it produces the labels everything else trains on.
- **A prior.** A network fitted to those labels is dense and cheap, but it *predicts* rather than recovers depth. The prior was fitted to labels that a sensor had to measure.

Mixing labels of different origin moved the output down a table, at a cost of free parameters.

Definition 26.2 (Output types and their free parameters)**.** A depth-like output is defined only up to a transformation, and each transformation leaves free parameters that something else must later fix.

output	free parameters	aligned by
metric depth	0	nothing
scale-invariant depth	1	one scale s
affine-invariant disparity or depth	2	scale and shift (s, t)
ordinal	any monotone map	nothing

Intuition

Each free parameter is a *number a sensor must still supply.* If a network says "this pixel is twice as far as that one", it has told you shape but not meters. Example: the truth is $D = 2.0\,\hat{D} + 0.3$ m. A metric network outputs D itself; an affine-invariant one outputs $\hat{D}$ and you need the pair (2.0, 0.3) to recover meters. The whole lecture is about who supplies those numbers: the baseline of a stereo rig, a large generative prior, or a handful of LiDAR returns.

27 Supervision without Ground Truth *(slides 3–6)*

This is section 3.9 of the lecture. Its thesis: *a stereo pair supplies its own supervision, provided the warp connecting the two images is differentiable.*

27.1 View Synthesis and the Backward Warp *(slides 4)*

Training uses stereo pairs $\{(\boldsymbol{I}_L, \boldsymbol{I}_R)\}$ only; there is no depth anywhere in the training set. The objective is to reproduce one image of the pair from the other. Depth is the only internal quantity that makes this task solvable, so if the network solves it, depth is what it must have learned. This holds only where a pixel is **textured** and visible in **both** views; elsewhere the objective says nothing and a hand-written prior fills the gap.

Definition 27.1 (Backward warp)**.** Given a disparity map $d(u, v)$ defined on the *target* image, the backward warp synthesizes the target from the source by

$$\tilde{\boldsymbol{I}}_L(u, v) = \boldsymbol{I}_R\big(u - d(u, v),\, v\big). \tag{4}$$

Every *target* pixel is mapped to exactly one *source* location.

Proposition 27.2 (Why the warp must be backward)**.** *Forward mapping (pushing each source pixel to a target location) lets several sources land on one target pixel and leaves other*

target pixels empty: collisions and holes. Backward mapping assigns each target pixel exactly one value, which makes the warp a differentiable function *of d rather than a scatter operation.*

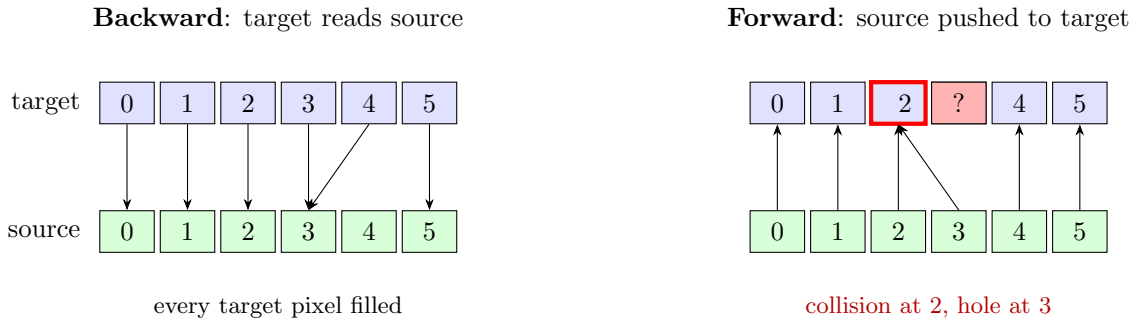


Figure 69: Original 1D illustration of backward versus forward mapping (slide 4 shows the same effect on small coloured grids).

Intuition

Think of filling in a form versus mailing letters. Backward warping is the form: each target pixel asks *"where in the other image should I look?"* and gets exactly one answer. Forward warping is the post office: every source pixel is mailed to an address; some mailboxes get two letters (collision), others none (hole), and "two letters or none" has no useful gradient. Note the sign convention: slide 4 writes $u - d$ for the left image as target, slide 5 writes $+d$ for a target/source pair set up the other way round; only the consistent use matters.

27.2 Image Reconstruction as Supervision *(slides 5)*

The pipeline of slide 5 has three steps: (1) a deep CNN predicts an inverse depth from the left image; (2) the right image is warped with it into a synthetic left image; (3) the reconstruction error against the real left image is the loss.

Definition 27.3 (Disparity and photometric loss)**.** Under a rectified stereo rig with focal length f and baseline b, the predicted inverse depth is the disparity

$$d(\boldsymbol{p}) = \frac{f\,b}{z(\boldsymbol{p})}. \tag{5}$$

The self-supervised loss compares the warped and real image,

$$\mathcal{L}_{\text{photo}} = \sum_{\boldsymbol{p}} \big\| \tilde{\boldsymbol{I}}(\boldsymbol{p}) - \boldsymbol{I}_L(\boldsymbol{p}) \big\|, \qquad \tilde{\boldsymbol{I}}(\boldsymbol{p}) = \boldsymbol{I}_R\big(\boldsymbol{p} + d(\boldsymbol{p})\big). \tag{6}$$

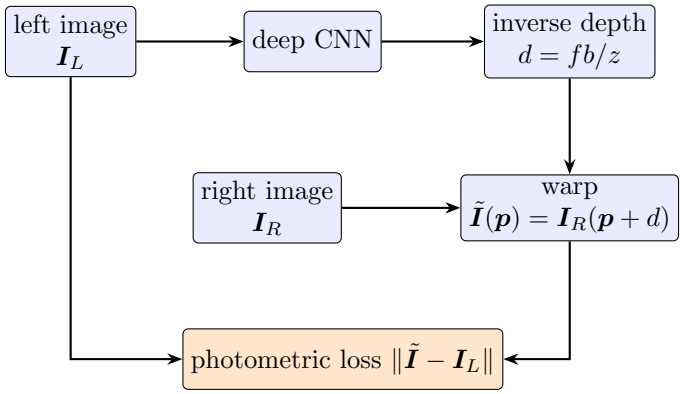


Figure 70: Self-supervised stereo training (redrawn from slide 5). No depth label appears anywhere.

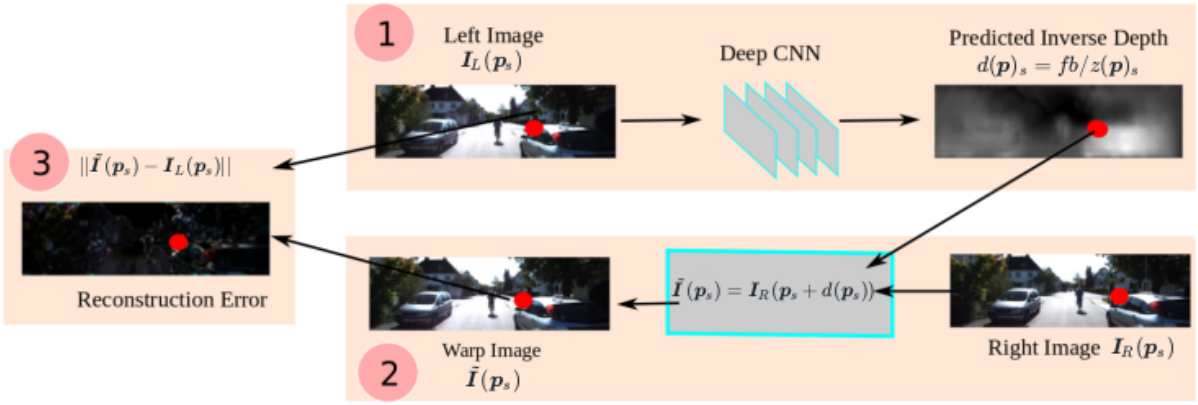


Figure 71: the KITTI street scene: left image, predicted inverse-depth map, warped image and reconstruction-error map (steps 1–3, with the red marker point)

Intuition

Depth is the *only* unknown the network can vary to make the warped image match the real one, so a network that solves the task has learned depth. Worked numbers (illustrative): with $f = 700$ px and $b = 0.5$ m, a car at $z = 10$ m has $d = 35$ px, whereas at $z = 70$ m it has only $d = 5$ px. Disparity is *inverse* depth: distant objects move the image little, so the training signal is weak at range.

27.3 Where the Loss Is Blind *(slides 6)*

Proposition 27.4 (Blind spots of the photometric loss)**.** *(a)* Textureless regions. *Writing the residual as* $r = \boldsymbol{I}_R(u - d) - \boldsymbol{I}_L(u)$ *and the squared loss as* $\frac{1}{2}r^2$*,*

$$\frac{\partial \mathcal{L}}{\partial d} = -\, r \cdot \boldsymbol{I}_R'(u - d), \tag{7}$$

which vanishes wherever $\boldsymbol{I}(u_0 + 1) \approx \boldsymbol{I}(u_0)$ *(flat image, zero derivative): such pixels receive* no training signal at all.
(b) Occlusions. *A pixel visible to one camera only has no match in the other view, so the residual there is meaningless.*

Both gaps are filled by hand-written priors: a smoothness term, left-right consistency, a minimum over the two reprojections, and masking of pixels that do not move between frames.

Proposition 27.5 (The baseline decides what the trained model is worth)**.** *Trained on a stereo pair, the model inherits the known baseline b and is metric* for that rig. *Trained on video, the baseline is unknown, so the output is shape up to one global scale and every image needs its own alignment.*

Intuition

The photometric loss is a detective that only works with clues. Flat white wall: no clue, no gradient, so the smoothness prior guesses. Something hidden from one eye: no partner to compare with. The baseline is the ruler: with stereo the ruler length b is known, so meters come free; with a monocular video the ruler length is not known, so you get a beautiful shape in unknown units (one free parameter, as in the table of Section 1).

28 Diffusion, From Scratch *(slides 7–22)*

This is section 3.10. Its thesis: *a model trained to remove noise is a prior: it points toward the data from anywhere in the space, and it can be sampled, conditioned or steered.* (Adapted on the slides from MIT 6.S183, Lecture 1, by D. Pfrommer.)

28.1 Supervised Learning *(slides 8)*

Supervised learning fits **one output per input**. At a depth edge that output is the average of the near surface and the far one: a distance that belongs to neither.

Proposition 28.1 (MSE regression returns the conditional mean)**.** *Let Y be the target and $\boldsymbol{x}$ the input. Among all functions f, the minimizer of $\mathbb{E}\|f(\boldsymbol{x}) - Y\|^2$ is $f^\star(\boldsymbol{x}) = \mathbb{E}[Y \mid \boldsymbol{x}]$.*

Proof. For fixed $\boldsymbol{x}$ with $m = \mathbb{E}[Y \mid \boldsymbol{x}]$, $\mathbb{E}[\|f - Y\|^2 \mid \boldsymbol{x}] = \|f - m\|^2 + \mathbb{E}[\|Y - m\|^2 \mid \boldsymbol{x}]$; the second term does not depend on f, and the first is minimized by $f = m$. □

Intuition

At a depth edge half the training examples say 2 m (near surface) and half say 10 m (far surface). The best single answer under squared error is 6 m, which is the depth of *nothing*. That is the blurred, flying-pixel edge of regression networks.

28.2 Generative Learning *(slides 9)*

A generative model fits a **distribution** over outputs, not one value. "The band is what a diffusion model represents."

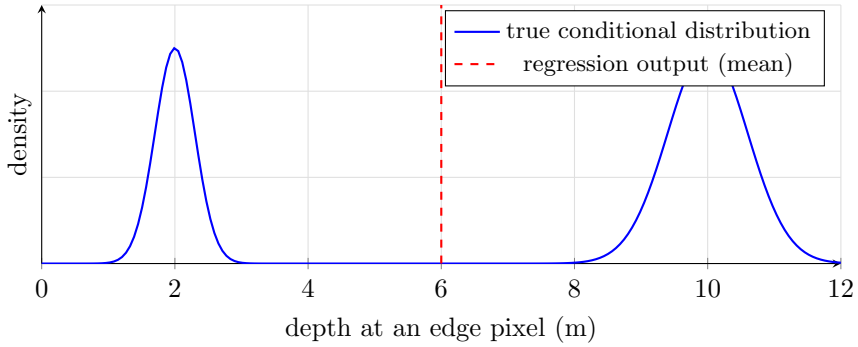


Figure 72: Original toy figure: a bimodal conditional distribution at a depth edge. The regressor answers the mean, which lies in a gap where the density is zero; a generative model can return a sample from one of the two modes.

28.3 Many Outputs for One Input – Text to Image *(slides 10)*

One conditioning input has many valid completions. A regressor averages them; a diffusion model picks one.

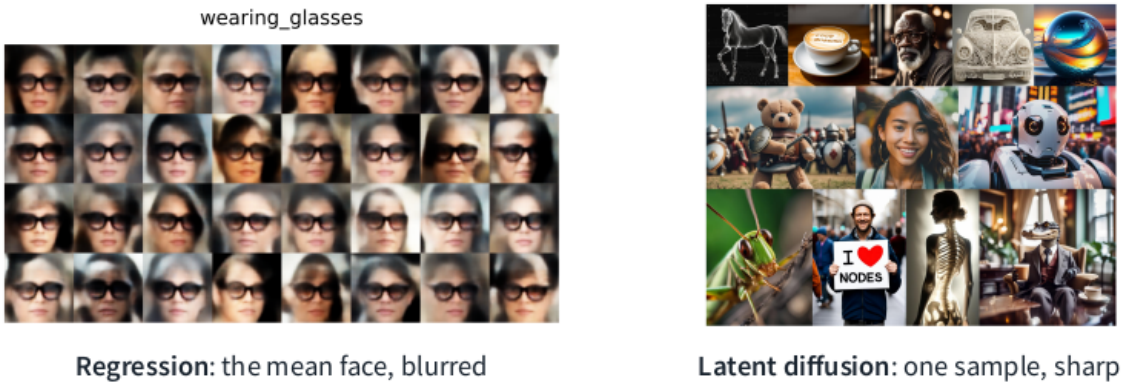


Figure 73: left: grid of "wearing_glasses" faces from regression (the mean face, blurred); right: grid of latent-diffusion samples (one sample, sharp)

Intuition

Blur is not a training failure. It is the correct answer to the wrong question: the conditional mean of a set of sharp images is not a sharp image. The same averaging is what flattens a depth edge.

28.4 The Forward and Reverse Process *(slides 11)*

The **forward process** adds noise to data one step at a time until nothing of the data is left. It is fixed and has no parameters. It is inspired by Brownian motion (many small independent displacements), and after enough steps the sample is indistinguishable from Gaussian noise. A diffusion model learns the **reverse process**, which recovers data from noise; that reverse process *is the prior*.

Definition 28.2 (Forward and reverse processes)**.** The forward chain $q(\boldsymbol{x}_t \mid \boldsymbol{x}_{t-1})$ maps $\boldsymbol{x}_0$ (data) to $\boldsymbol{x}_T$ (noise) and is fixed; the model $p_\theta(\boldsymbol{x}_{t-1} \mid \boldsymbol{x}_t)$ learns the way back.

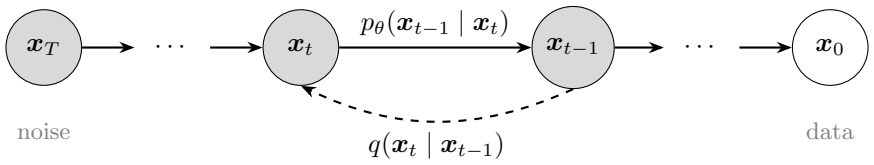


Figure 74: Forward (dashed, fixed) and reverse (solid, learned) chains, redrawn from slide 11.

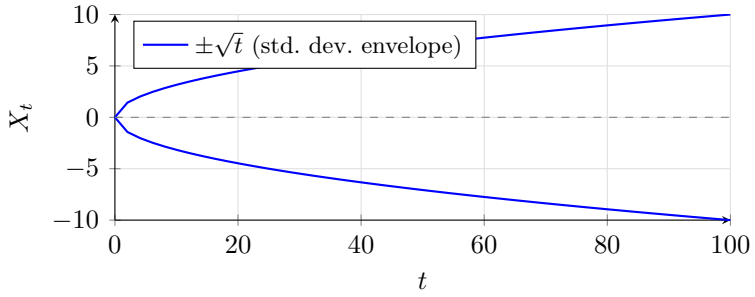


Figure 75: Original figure: the spread of a random walk grows like $\sqrt{t}$, the mechanism that buries the data in noise.

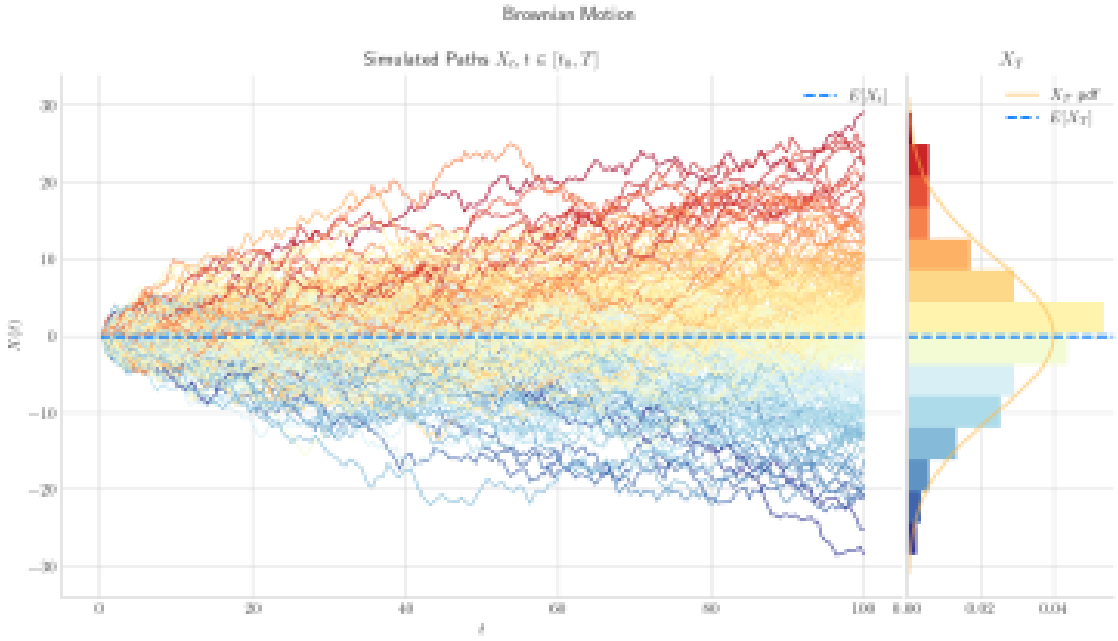


Figure 76: Brownian-motion simulated paths (coloured bundle of trajectories) with the histogram of X_T on the right, and the two small face images (noisy, clean) below the chain

28.5 Training by Noise Prediction *(slides 12)*

A denoising diffusion model estimates the noise vector $\boldsymbol{\epsilon}$ from a noise level $\sigma > 0$ and a noisy input $\boldsymbol{x}_\sigma \approx \boldsymbol{x}_0 + \sigma\boldsymbol{\epsilon}$, where $\boldsymbol{x}_0$ lies on the data manifold $\mathcal{K} \subset \mathbb{R}^n$.

Definition 28.3 (Denoiser and training loss)**.** A denoiser $\boldsymbol{\epsilon}_\theta : \mathbb{R}^n \times \mathbb{R}_+ \to \mathbb{R}^n$ is learned by

minimizing

$$L_{\text{simple}}(\theta) := \mathbb{E}_{\boldsymbol{x}_0,\sigma,\boldsymbol{\epsilon}}\Big[\big\|\boldsymbol{\epsilon}_\theta(\boldsymbol{x}_0 + \sigma\boldsymbol{\epsilon}, \sigma) - \boldsymbol{\epsilon}\big\|^2\Big], \tag{8}$$

with $\boldsymbol{x}_0$ sampled from the training data, σ from a training noise schedule, and $\boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, \boldsymbol{I})$.

Intuition

The recipe is three lines: take a clean example, add noise of a random strength you remember, and ask the network "what noise did I add?". Subtracting the guessed noise gives the guessed clean point, $\hat{\boldsymbol{x}}_0 = \boldsymbol{x}_\sigma - \sigma\boldsymbol{\epsilon}_\theta$. No labels beyond the data itself are needed.

28.6 The Spiral Toy Model *(slides 13)*

Everything that follows is shown on a 2D data set, where the prior is visible: *it is where the points are*. The data are points of a 2D spiral $\boldsymbol{x}_0$. The network is a fully connected MLP fed with $\boldsymbol{x}_0 + \sigma\boldsymbol{\epsilon}$ and a 2D noise-level embedding of σ, and it predicts the noise.

```
def get_sigma_embeds(sigma):
    sigma = sigma.unsqueeze(1)
    return torch.cat([torch.sin(torch.log(sigma)/2),
                      torch.cos(torch.log(sigma)/2)], dim=1)

class TimeInputMLP(nn.Module):
    def __init__(self, dim, hidden_dims):
        super().__init__()
        layers = []
        for in_dim, out_dim in pairwise((dim + 2,) + hidden_dims):
            layers.extend([nn.Linear(in_dim, out_dim), nn.GELU()])
        layers.append(nn.Linear(hidden_dims[-1], dim))
        self.net = nn.Sequential(*layers)
        self.input_dims = (dim,)

    def forward(self, x, sigma):
        sigma_embeds = get_sigma_embeds(sigma)
        nn_input = torch.cat([x, sigma_embeds], dim=1)
        return self.net(nn_input)

model = TimeInputMLP(dim=2, hidden_dims=(16,128,128,128,128,16))
```

Intuition

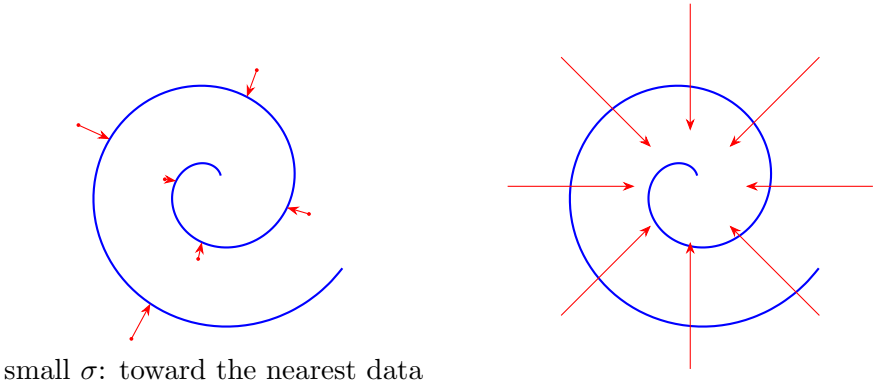
The extra two inputs $(\sin, \cos)$ of $\log\sigma/2$ tell the network *how noisy* its input is. This matters because the right answer depends on σ: at small σ the noise is a small nudge off the nearest data point, at large σ the input is almost pure noise. Note that the first layer has width $\text{dim} + 2 = 4$: two coordinates plus the two embedding numbers.

28.7 Training the Toy Model *(slides 14)*

```
def training_loop(loader, model, schedule, epochs=10000):
    optimizer = torch.optim.Adam(model.parameters())
    for _ in range(epochs):
        for x0 in loader:
            optimizer.zero_grad()
            sigma, eps = generate_train_sample(x0, schedule)
            eps_hat = model(x0 + sigma * eps, sigma)
```

```
            loss = nn.MSELoss()(eps_hat, eps)
            loss.backward()
            optimizer.step()
```

The learned denoiser can be drawn as a **vector field** by plotting $\boldsymbol{x} - \sigma\boldsymbol{\epsilon}_\theta(\boldsymbol{x}, \sigma)$. *At small noise the arrows point at the nearest data. At large noise they point at the middle of the whole data set.* The slide's training curve falls from about 0.70 to about 0.41 over 15 000 epochs.



small σ: toward the nearest data

large σ: toward the middle

Figure 77: Original sketch of the denoiser field on the spiral (compare slide 14).

Remark 28.4 (Why the loss does not reach zero). The curve plateaus near 0.41, not 0. This is expected: even the best possible denoiser cannot recover $\boldsymbol{\epsilon}$ exactly, because many $(\boldsymbol{x}_0, \boldsymbol{\epsilon})$ pairs produce the same $\boldsymbol{x}_\sigma$; the floor equals the average posterior variance of $\boldsymbol{\epsilon}$ (Appendix A, item A.2).

28.8 The Ideal Denoiser *(slides 15)*

The **ideal denoiser** minimizes the training loss; it is a function of the data set $\mathcal{K}$ and the noise level, and nothing else:

$$\boldsymbol{\epsilon}^* := \arg\min_{\boldsymbol{\epsilon}_\theta} \mathbb{E}_{\boldsymbol{x}_0,\sigma,\boldsymbol{\epsilon}} \big\| \boldsymbol{\epsilon}_\theta(\boldsymbol{x}_0 + \sigma\boldsymbol{\epsilon}, \sigma) - \boldsymbol{\epsilon} \big\|^2. \tag{9}$$

Theorem 28.5 (Ideal denoiser for a finite data set)**.** *For a finite $\mathcal{K}$ (data uniformly distributed on $\mathcal{K}$, Gaussian noise),*

$$\boldsymbol{\epsilon}^*(\boldsymbol{x}_\sigma, \sigma) = \frac{\sum_{\boldsymbol{x}_0 \in \mathcal{K}} (\boldsymbol{x}_\sigma - \boldsymbol{x}_0) \exp\!\big(-\|\boldsymbol{x}_\sigma - \boldsymbol{x}_0\|^2 / 2\sigma^2\big)}{\sigma \sum_{\boldsymbol{x}_0 \in \mathcal{K}} \exp\!\big(-\|\boldsymbol{x}_\sigma - \boldsymbol{x}_0\|^2 / 2\sigma^2\big)}. \tag{10}$$

It is a weighted average of all data points: nearer points count more.

Proof. By the conditional-mean proposition, $\boldsymbol{\epsilon}^*(\boldsymbol{x}_\sigma, \sigma) = \mathbb{E}[\boldsymbol{\epsilon} \mid \boldsymbol{x}_\sigma]$ with $\boldsymbol{\epsilon} = (\boldsymbol{x}_\sigma - \boldsymbol{x}_0)/\sigma$. By Bayes, $p(\boldsymbol{x}_0 \mid \boldsymbol{x}_\sigma) \propto \exp(-\|\boldsymbol{x}_\sigma - \boldsymbol{x}_0\|^2/2\sigma^2)$ on $\mathcal{K}$. Taking the expectation of $(\boldsymbol{x}_\sigma - \boldsymbol{x}_0)/\sigma$ under these weights gives the formula. □

Intuition

Worked 1D example with $\mathcal{K} = \{-1, +1\}$, $\boldsymbol{x}_\sigma = 0.2$. For $\sigma = 0.5$ the weights are $e^{-1.44/0.5} \approx 0.056$ (for -1) and $e^{-0.64/0.5} \approx 0.278$ (for $+1$), giving $\boldsymbol{\epsilon}^* \approx -0.93$ and clean estimate $\hat{\boldsymbol{x}}_0 = 0.2 - 0.5 \cdot (-0.93) \approx 0.66$: pulled toward the nearer point $+1$. For a large $\sigma = 5$ the two

weights are almost equal (0.972 vs 0.987), $\boldsymbol{\epsilon}^* \approx 0.04$ and $\hat{\boldsymbol{x}}_0 \approx 0.01$: the arrow points at the middle of the data set, exactly as in the slide's three panels ($\sigma = 0.01, 0.1, 0.5$).

28.9 The Smoothed Distance Function *(slides 16)*

The distance to a set, $\operatorname{dist}_{\mathcal{K}}(\boldsymbol{x}) := \inf\{\|\boldsymbol{x} - \boldsymbol{x}_0\| : \boldsymbol{x}_0 \in \mathcal{K}\}$, is not differentiable everywhere, so its gradient is hard to learn. Smoothing it by σ fixes that.

Definition 28.6 (Smoothed distance)**.**

$$D_\sigma(\boldsymbol{x}) := -\sigma^2 \log\Big(\sum_{\boldsymbol{x}_0 \in \mathcal{K}} \exp\big(-\|\boldsymbol{x}_0 - \boldsymbol{x}\|^2 / 2\sigma^2\big) \Big). \tag{11}$$

(The slide denotes the smoothed squared distance by $\operatorname{dist}^2_{\mathcal{K}}(\boldsymbol{x}, \sigma)$ and writes the identity below with a factor $\frac{1}{2}$; with the definition as printed, $\frac{1}{2}\operatorname{dist}^2_{\mathcal{K}}(\boldsymbol{x}, \sigma)$ corresponds to D_σ.)

Proposition 28.7 (Denoiser = gradient of the smoothed distance)**.** $\frac{1}{2}\nabla_{\boldsymbol{x}} \operatorname{dist}^2_{\mathcal{K}}(\boldsymbol{x}, \sigma) = \nabla_{\boldsymbol{x}} D_\sigma(\boldsymbol{x}) = \sigma\, \boldsymbol{\epsilon}^*(\boldsymbol{x}, \sigma)$, *and* $\min_{\boldsymbol{x}_0} \frac{1}{2}\|\boldsymbol{x} - \boldsymbol{x}_0\|^2 - \sigma^2 \log|\mathcal{K}| \le D_\sigma(\boldsymbol{x}) \le \min_{\boldsymbol{x}_0} \frac{1}{2}\|\boldsymbol{x} - \boldsymbol{x}_0\|^2$.

Proof. Differentiating, $\nabla D_\sigma = -\sigma^2 \cdot \frac{\sum e^{-\|\boldsymbol{x}_0 - \boldsymbol{x}\|^2/2\sigma^2}\,(-(\boldsymbol{x} - \boldsymbol{x}_0)/\sigma^2)}{\sum e^{-\|\boldsymbol{x}_0 - \boldsymbol{x}\|^2/2\sigma^2}} = \frac{\sum (\boldsymbol{x} - \boldsymbol{x}_0) e^{\cdots}}{\sum e^{\cdots}} = \sigma \boldsymbol{\epsilon}^*$ by Theorem 28.5. The bounds follow from $e^{-a_{\min}/\sigma^2} \le \sum_i e^{-a_i/\sigma^2} \le |\mathcal{K}|\, e^{-a_{\min}/\sigma^2}$ with $a_i = \frac{1}{2}\|\boldsymbol{x} - \boldsymbol{x}_{0,i}\|^2$. □

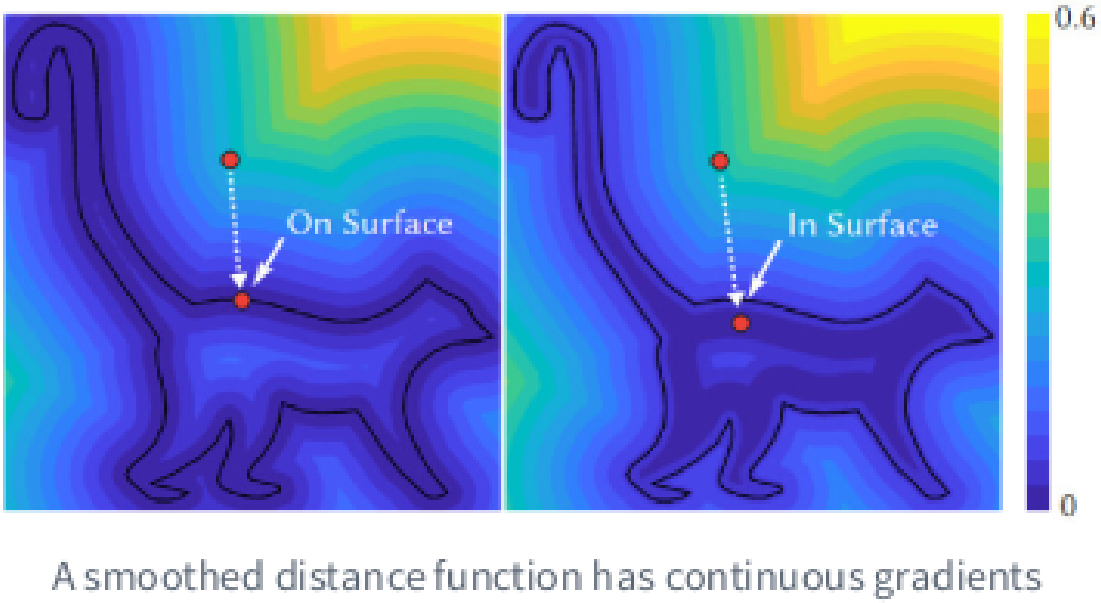


Figure 78: the two coloured distance-field images of the cat silhouette ("On Surface" and "In Surface" with the dotted arrows), with the colour bar 0–0.6

Intuition

min has kinks where two data points are equally near; a *soft*-min has none, and its slope is the vector from the weighted mean of the data to $\boldsymbol{x}$. So the denoiser is a compass that is defined *everywhere* and always points toward the data: that is what makes it a prior. Equivalently $\hat{\boldsymbol{x}}_0 = \boldsymbol{x}_\sigma - \sigma\boldsymbol{\epsilon}^*$ is simply the weighted mean of the data points.

28.10 Sampling From a Diffusion Model – Why iterative? *(slides 17)*

Given a noisy $\boldsymbol{x}_\sigma$ and its noise level, the denoiser estimates the clean point directly,

$$\hat{\boldsymbol{x}}_0(\boldsymbol{x}_\sigma, \sigma) := \boldsymbol{x}_\sigma - \sigma\boldsymbol{\epsilon}_\theta(\boldsymbol{x}_\sigma, \sigma). \tag{12}$$

A sampler walks a decreasing sequence of noise levels, taking one such estimate at each step. **DDIM** is the update

$$\boldsymbol{x}_{t-1} = \boldsymbol{x}_t + (\sigma_{t-1} - \sigma_t)\,\boldsymbol{\epsilon}_\theta(\boldsymbol{x}_t, \sigma_t). \tag{13}$$

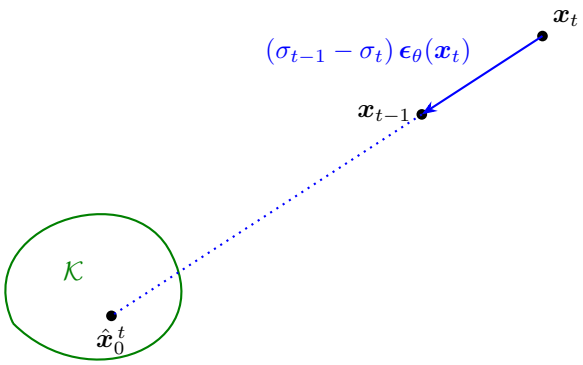


Figure 79: DDIM step (redrawn from slide 17): a straight line toward the current estimate $\hat{\boldsymbol{x}}_0^t$ of the data.

Intuition

One step goes only *part* of the way to the current best guess, because the guess is poor while the input is very noisy; after moving, you re-evaluate the compass. The step is a straight line toward the estimate. This is the equation that depth completion reaches inside and steers (Section 31). Algebraically the step re-noises the clean estimate to the lower level: $\boldsymbol{x}_{t-1} = \hat{\boldsymbol{x}}_0^t + \sigma_{t-1}\boldsymbol{\epsilon}_\theta$ (Appendix A).

28.11 The DDIM Sampler *(slides 18)*

Algorithm 28.8 (DDIM sampler). `for t = N, ..., 1:` $\boldsymbol{x}_{t-1} \leftarrow \boldsymbol{x}_t + (\sigma_{t-1} - \sigma_t)\,\boldsymbol{\epsilon}_\theta(\boldsymbol{x}_t, \sigma_t)$; `return` $\boldsymbol{x}_0$.
Start from $\boldsymbol{x}_N = \sigma_N \cdot \mathcal{N}(\mathbf{0}, \boldsymbol{I})$.

```
@torch.no_grad()
def samples_ddim(model, sigmas, batchsize=1):
    xt = model.rand_input(batchsize) * sigmas[0]
    for sig, sig_prev in pairwise(sigmas):
        xt = xt - (sig - sig_prev) * model.predict_eps(xt, sig)
    return xt
```

On the toy model: *fifty steps, no retraining, and the trajectory follows the field.*

28.12 Probabilistic Sampling *(slides 19)*

DDIM is deterministic: the same starting noise always gives the same sample. Adding a noise term back makes the sampler stochastic:

$$\boldsymbol{x}_{t-1} = \boldsymbol{x}_t + (\sigma_{t-1} - \sigma_t)\,\boldsymbol{\epsilon}_\theta(\boldsymbol{x}_t, \sigma_t) + \mu\,\boldsymbol{w}_t, \qquad \boldsymbol{w}_t \sim \mathcal{N}(\mathbf{0}, \boldsymbol{I}). \tag{14}$$

(The slide writes $\sigma_{t'}$ for the next, lower level.) With $\mu = 0$ this is DDIM again; as μ grows the trajectories spread, and the same input gives different samples. The slide shows $\mu = 0.001, 0.01, 0.1$.

Intuition

μ is a "temperature": $\mu = 0$ rolls a ball down a fixed groove, $\mu > 0$ shakes the table on the way. This is the reason different seeds give different depth maps in Marigold (Section 29).

28.13 Three Uses of a Prior *(slides 20)*

A prior that exists as a **separate object** can be used in three ways, each a later section of the lecture.

use	meaning	section
Sample	draw from the distribution the model represents	3.10
Condition	train it with a side input $\boldsymbol{c}$, so it samples from $\mathrm{p}(\boldsymbol{x} \mid \boldsymbol{c})$	3.11
Guide	modify the state inside the sampling loop at test time, with no retraining	3.13

The third is the one that turns a depth prior into a metric depth map.

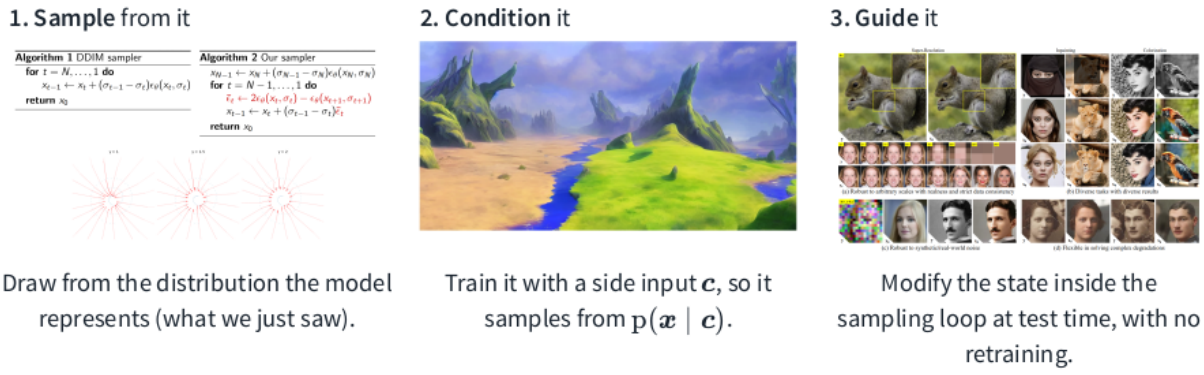


Figure 80: the three panels: sampler comparison plots (Algorithm 1 vs Algorithm 2), the generated landscape (conditioning), and the super-resolution / inpainting / colorization examples (guidance)

28.14 The Autoencoder and Latent Diffusion *(slides 21)*

Images are big, so an autoencoder compresses them first: an encoder $\mathcal{E}$ maps $\boldsymbol{x}$ to a latent $\boldsymbol{z}$, and a decoder $\mathcal{D}$ maps back, $\boldsymbol{z} = \mathcal{E}(\boldsymbol{x})$, $\tilde{\boldsymbol{x}} = \mathcal{D}(\boldsymbol{z})$. The autoencoder is trained **first**, then **frozen**; the diffusion model never sees pixels and you decode once, at the end. Diffusion runs on $\boldsymbol{z}$ with the same loss:

$$L = \mathbb{E}_{\sigma, \boldsymbol{z}_0, \boldsymbol{\epsilon}}\Big[\big\|\boldsymbol{\epsilon} - \boldsymbol{\epsilon}_\theta(\boldsymbol{z}_0 + \sigma\boldsymbol{\epsilon}, \sigma)\big\|^2\Big]. \tag{15}$$

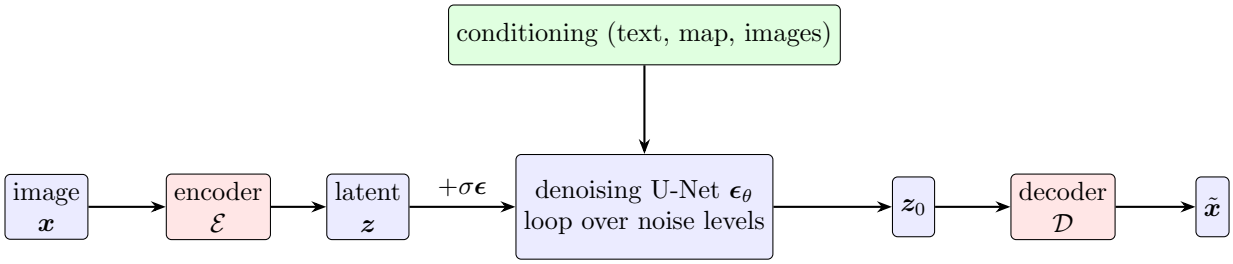


Figure 81: Latent diffusion in outline (original, inspired by slide 21). Pink boxes are frozen autoencoder parts.

Intuition

Worked numbers: $768 \times 768 \times 3 \to 96 \times 96 \times 4$, i.e. 1 769 472 numbers become 36 864: eight times smaller per side, four channels, **48 times fewer numbers**. Each of the 50 denoising steps is therefore 48 times cheaper than it would be in pixel space.

28.15 Conditioning and Guidance *(slides 22)*

Conditioning is a side input the network was *trained with*; the model responds to it because it was trained to. **Guidance** modifies the sampling trajectory *at test time*, using a criterion the network was never trained on. Sampling is a loop, and its state can be modified between steps.

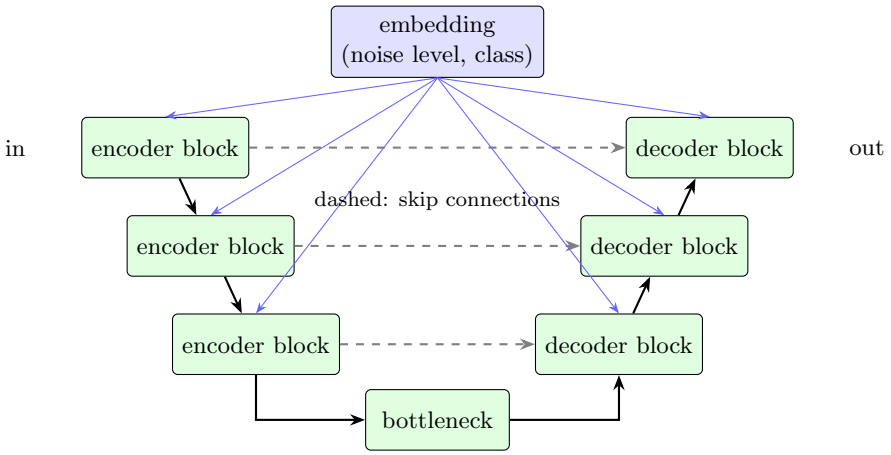


Figure 82: Simplified U-Net redrawn from slide 22. Each block is GroupNorm–SiLU–Conv3×3 with the embedding injected through a Linear layer (scale and shift); attention is optional.

29 Marigold *(slides 23–31)*

Section 3.11: a depth estimator from an image diffusion model. Stable Diffusion's autoencoder is kept, only its denoiser is fine-tuned, and the output is a depth map rather than an image.

29.1 The Marigold Method *(slides 24)*

The argument: a monocular estimator's knowledge of the visual world is **bounded by its training set**, while a text-to-image model trained on billions of captioned photographs has already seen it. Hence: if the cornerstone of monocular depth is a comprehensive representation of the visual world, one should be able to derive a depth estimator from a pretrained image diffusion model.

Definition 29.1 (Marigold)**.** Take a pretrained text-to-image latent diffusion model, **freeze the VAE**, and fine-tune only the denoising U-Net so that what it denoises is a *depth* latent rather than an image latent. Properties from the slide: affine-invariant monocular depth; trained on **synthetic data only**; about 2.5 days on a single consumer GPU.

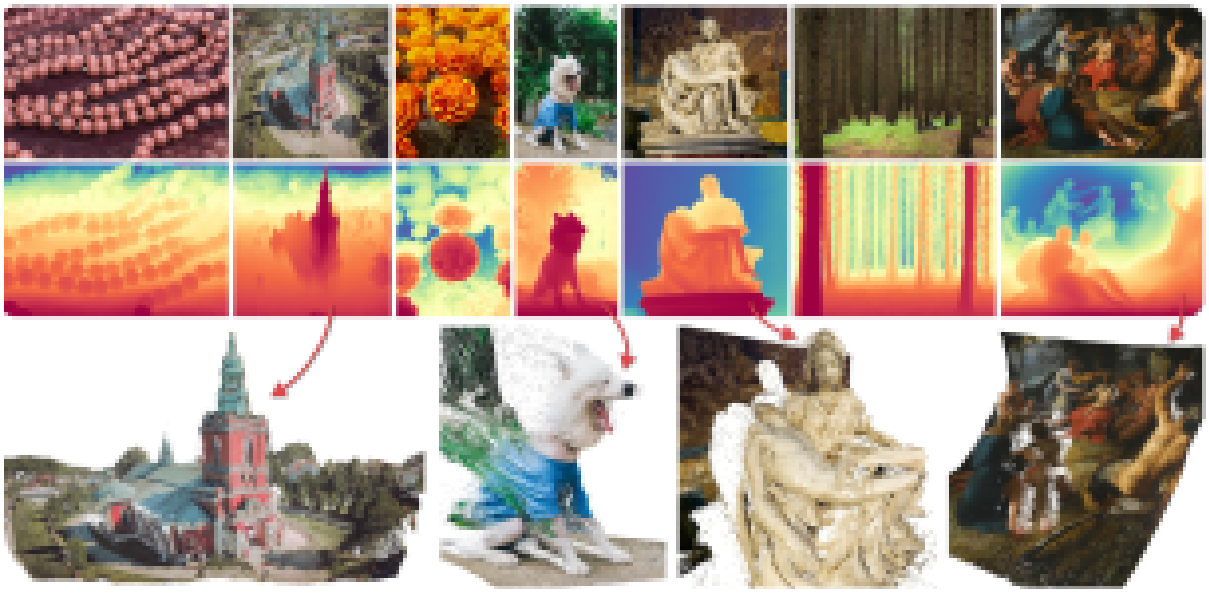


Figure 83: the strip of example results: photographs (beads, church, marigolds, dog, sculpture, forest, painting) with their depth maps and back-projected 3D views

29.2 From the Spiral to Marigold *(slides 25)*

Marigold is the spiral model with one extra input; nothing in the algorithm changes.

	toy model	Marigold
starts from	random weights	Stable Diffusion's weights
a data point	a 2D point	a depth latent
the denoiser	a small MLP	Stable Diffusion's U-Net
the condition	none	the image latent
trained on	spiral points	74K rendered image–depth pairs
the sampler	DDIM	the same DDIM
a sample is	a point on the spiral	a depth map the RGB allows

The first row is the whole bet: *the visual prior is already in those weights*, and fine-tuning moves what they denoise from images to depth. (Stable Diffusion indexes noise levels by a timestep t and scales its input; both are fixed functions of σ, so the process is the same.)

29.3 The Source of Affine Invariance *(slides 26)*

The normalization defines the output space: the affine-invariant *depth* row of the table in Part I, not the affine-invariant *disparity* row that MiDaS uses.

Definition 29.2 (Percentile normalization)**.** With $\boldsymbol{D}_2$ and $\boldsymbol{D}_{98}$ the 2nd and 98th percentiles

of the depth map $\boldsymbol{D}$, the training target is

$$\tilde{\boldsymbol{D}} = \left(\frac{\boldsymbol{D} - \boldsymbol{D}_2}{\boldsymbol{D}_{98} - \boldsymbol{D}_2} - 0.5\right) \times 2, \qquad \text{and metric depth is recovered as} \qquad \tilde{\boldsymbol{D}}_{\text{metric}} = s\tilde{\boldsymbol{D}} + t. \tag{16}$$

Proposition 29.3 (Normalization removes scale and shift)**.** *If* $\boldsymbol{D}' = a\boldsymbol{D} + b$ *with* $a > 0$*, then* $\tilde{\boldsymbol{D}}' = \tilde{\boldsymbol{D}}$*.*

Proof. Percentiles commute with increasing affine maps: $\boldsymbol{D}'_2 = a\boldsymbol{D}_2 + b$, $\boldsymbol{D}'_{98} = a\boldsymbol{D}_{98} + b$. Hence $(\boldsymbol{D}' - \boldsymbol{D}'_2)/(\boldsymbol{D}'_{98} - \boldsymbol{D}'_2) = a(\boldsymbol{D} - \boldsymbol{D}_2)/(a(\boldsymbol{D}_{98} - \boldsymbol{D}_2))$, the same ratio. □

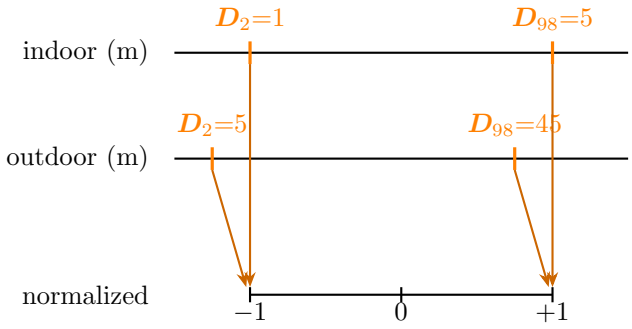


Figure 84: Original figure: an indoor map (few meters) and an outdoor map (tens of meters) land on the same $[-1, +1]$ range; each map is normalized by its *own* percentiles (slide 26). The numbers are illustrative.

Intuition

Every training map is stretched to fill $[-1, +1]$, so a room and a highway look identical to the network in terms of range. **The model never sees meters and never sees a common zero, so it cannot learn them; it is constrained to predict shape.** Example: in a room with $\boldsymbol{D}_2 = 1$ m and $\boldsymbol{D}_{98} = 5$ m, a pixel at 3 m maps to 0; on a street with $\boldsymbol{D}_2 = 5$ m and $\boldsymbol{D}_{98} = 45$ m, a pixel at 25 m also maps to 0.

29.4 Conditioning on the Image *(slides 27)*

The denoiser takes the noisy depth latent $\boldsymbol{z}_\sigma^{(\boldsymbol{D})}$ and the image latent $\boldsymbol{z}^{(\boldsymbol{I})}$:

$$\boldsymbol{\epsilon}_\theta\big(\boldsymbol{z}_\sigma^{(\boldsymbol{D})},\, \boldsymbol{z}^{(\boldsymbol{I})},\, \sigma\big), \qquad \boldsymbol{z}_\sigma = \operatorname{cat}\big(\boldsymbol{z}_\sigma^{(\boldsymbol{D})}, \boldsymbol{z}^{(\boldsymbol{I})}\big). \tag{17}$$

The depth map is replicated to three channels and encoded by the **frozen RGB autoencoder**, whose reconstruction error on depth is negligible. **Concatenation** suits a condition that is spatially aligned with the output, pixel for pixel; cross-attention is for conditions that are not.

Definition 29.4 (Four changes to the network)**.** (1) Concatenate the image latent and the noisy depth latent along the channels; (2) double the U-Net's input channels; (3) duplicate the first layer's weights and **halve** them, so activation magnitudes do not inflate; (4) **disable text conditioning** (original Stable Diffusion).

Intuition

Why halve? Let the original first layer be $W\boldsymbol{z}$. The new layer sees $[\boldsymbol{z};\boldsymbol{z}]$ and uses $[W/2,\ W/2]$, so its output is $W\boldsymbol{z}/2 + W\boldsymbol{z}/2 = W\boldsymbol{z}$: at the start of fine-tuning the pretrained network behaves as before. Without halving, the same input would produce $2W\boldsymbol{z}$, twice the activations the pretrained weights expect. Since the latent has 4 channels, the new input has 8. Concatenation works here because the depth map and the image are aligned pixel by pixel.

29.5 Training Marigold *(slides 28)*

```
def training_loop(loader, unet, vae, schedule, epochs):
    optimizer = torch.optim.Adam(unet.parameters())
    for _ in range(epochs):
        for image, depth in loader:                    # pairs
            optimizer.zero_grad()
            # start lock
            x0 = vae.encode(normalize(depth))          # a depth latent
            cond = vae.encode(image)                   # the condition
            # end lock
            sigma, eps = generate_train_sample(x0, schedule)
            eps_hat = unet(x0 + sigma*eps, cond, sigma)   # sees both
            loss = nn.MSELoss()(eps_hat, eps)
            loss.backward()
            optimizer.step()
```

The objective is the one from the spiral, unchanged. Only the U-Net trains; the VAE is frozen at both ends. Data: 74K rendered pairs, about 2.5 days on one consumer GPU.

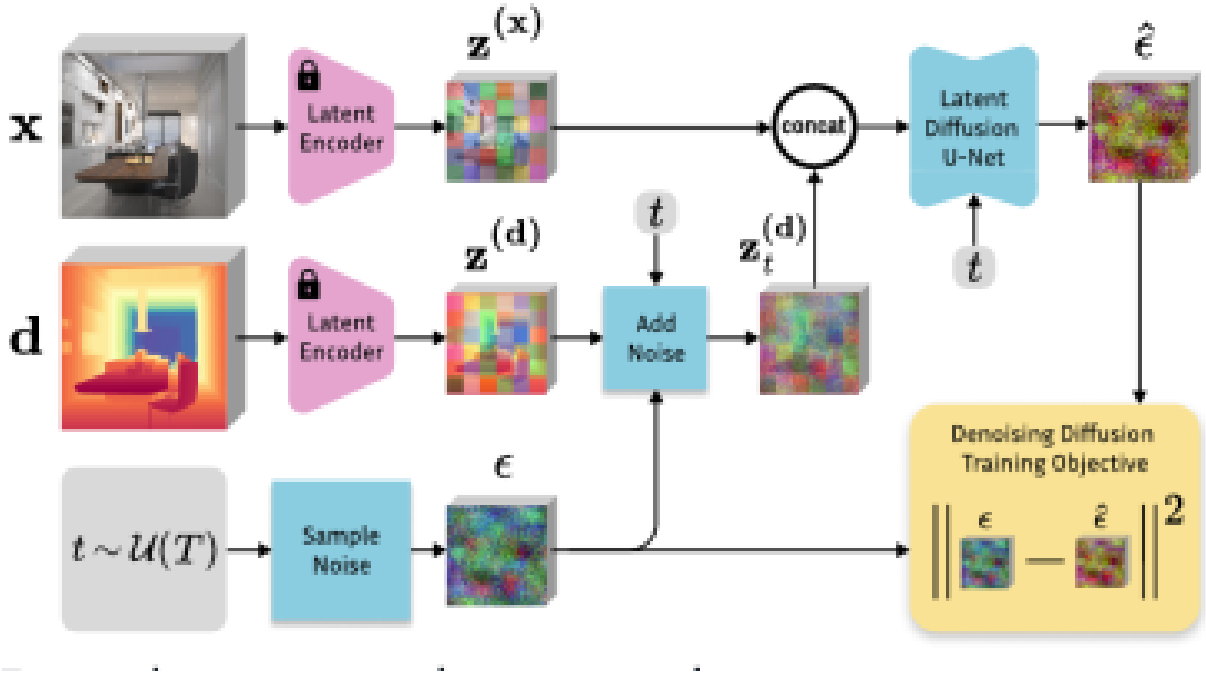


Figure 85: Marigold training (redrawn from slides 27–28). Pink: frozen. The noise $\boldsymbol{\epsilon}$ is also fed to the loss.

29.6 Inference *(slides 29)*

Algorithm 29.5 (Marigold inference, one run)**.** (1) Encode the image **once**; (2) initialize the depth latent $\boldsymbol{z}_T^{(D)}$ as pure Gaussian noise; (3) run the denoising loop, concatenating the image latent at **every** step; (4) decode $\boldsymbol{z}_0^{(D)}$ and average the three channels. From the paper: 50 DDIM steps.

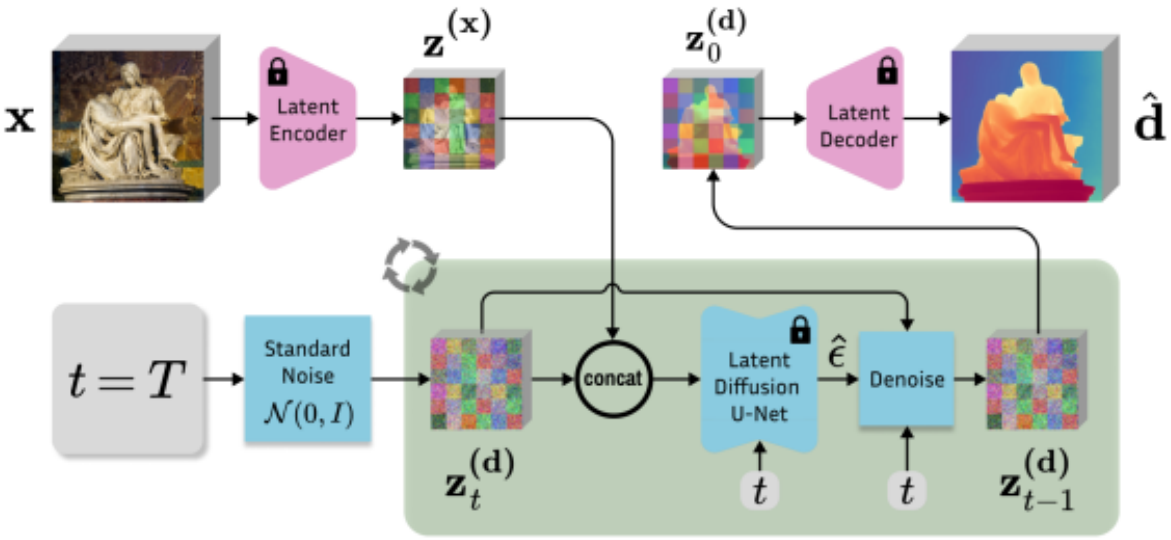


Figure 86: Marigold inference

29.7 Ensembling Over Samples *(slides 30)*

Probabilistic sampling is why different seeds give different depth maps; Marigold runs the loop N times and combines the results.

Algorithm 29.6 (Ensembling)**.** (1) Each run is affine-invariant on its own scale, so first fit a scale and a shift per run so that the maps agree with each other (minimize the pairwise distance between the aligned maps, with a penalty that keeps the range on $[0, 1]$); (2) take the pixel-wise **median**. Default $N = 10$; **no ground truth is used**.

Intuition

Ten drafts of the same drawing, each in its own unknown units. First rescale and shift each draft to line up with the others (they only agree up to (s_i, t_i)), then let the median vote at each pixel, which ignores outlier drafts.

29.8 Limitations of Marigold *(slides 31)*

- **No metric scale.**
- **No zero**: not even a common origin between two images of the same room.
- **Slow**: 50 denoising steps $\times$ 10 ensemble members $= 500$ network evaluations per image.
- **It returns a sample** (non-deterministic).

30 Back to 3D *(slides 32–34)*

Section 3.12: back-projection turns a depth map into points and shows which of the unknown parameters distort the result.

30.1 Effect of Scale and Shift Errors *(slides 33)*

A depth map is a 2.5D surface: one depth per pixel, with nothing behind what is visible.

Definition 30.1 (Back-projection)**.** A pixel with intrinsics-normalized ray $\boldsymbol{r} = \boldsymbol{K}^{-1}(u, v, 1)^\top$ and depth z back-projects to $\boldsymbol{p} = z\,\boldsymbol{r}$.

Proposition 30.2 (Affine depth error)**.** *If $\hat{z} = sz+t$, the reconstructed point is $\hat{\boldsymbol{p}} = (s+t/z)\,\boldsymbol{p}$. In particular: the **scale** s multiplies the whole cloud (a similarity, so shape is preserved); the **shift** t drags every point a constant depth along its own ray, a different displacement in every direction (the shape can distort).*

Proof. $\hat{\boldsymbol{p}} = \hat{z}\,\boldsymbol{r} = (sz + t)\boldsymbol{r} = (s + t/z)\,z\boldsymbol{r}$. □

Corollary 30.3 (Affine error in disparity)**.** *If disparity is $\hat{d} = sd + t$ then $\hat{z} = 1/\hat{d}$ and $\hat{\boldsymbol{p}} = \boldsymbol{p}/(s + tz)$: a* projective *map of space.*

Proof. $\hat{z} = \dfrac{1}{s/z + t} = \dfrac{z}{s + tz}$, so $\hat{\boldsymbol{p}} = \hat{z}\boldsymbol{r} = \boldsymbol{p}/(s + tz)$. □

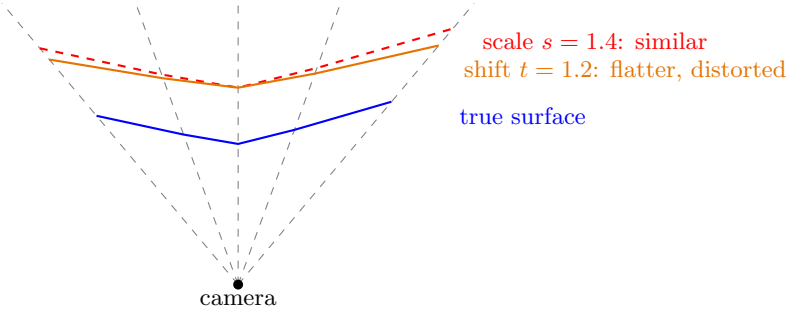


Figure 87: Original side view. A bowl-shaped surface seen along five rays. Scaling by s keeps the shape (red); adding a constant depth t along each ray (orange) keeps the depth differences but widens the lateral spread, so the bowl becomes flatter.

Intuition

Scale is a photocopier enlargement: bigger, same shape. Shift is pushing every point straight away from the camera by the same distance: because rays fan out, far-off directions move sideways more than the centre, so straight things bend. Example: $s = 1$, $t = 1$ m: a point at $z = 2$ is scaled by $1 + 1/2 = 1.5$, one at $z = 10$ by only 1.1; near points are magnified more, so a wall receding from the camera is bent.

30.2 Sources of Scale and Shift *(slides 34)*

Two numbers are required, and a single image cannot supply them. Possible sources:

- a **metric-depth model**; camera-aware models narrow the generalization gap by taking the intrinsics as input (3.8);
- a **known baseline** at training time (3.9);
- a **known object size**: the apparent-size cue, which needs recognition;
- **known camera height plus a flat-ground assumption**: the vertical-position cue, which needs flat ground;
- or a **handful of real depth measurements**.

31 Marigold-DC *(slides 35–38)*

Section 3.13: guidance from sparse depth measurements. Nothing is retrained: the denoising loop is modified at test time to move the sample toward the available measurements.

31.1 The Depth Completion Task *(slides 36)*

Definition 31.1 (Depth completion)**.** Depth completion upgrades sparse depth measurements into dense depth maps guided by a conventional image. The sparse points come from the LiDAR returns of Part I, structure-from-motion landmarks, or a phone's depth sensor.

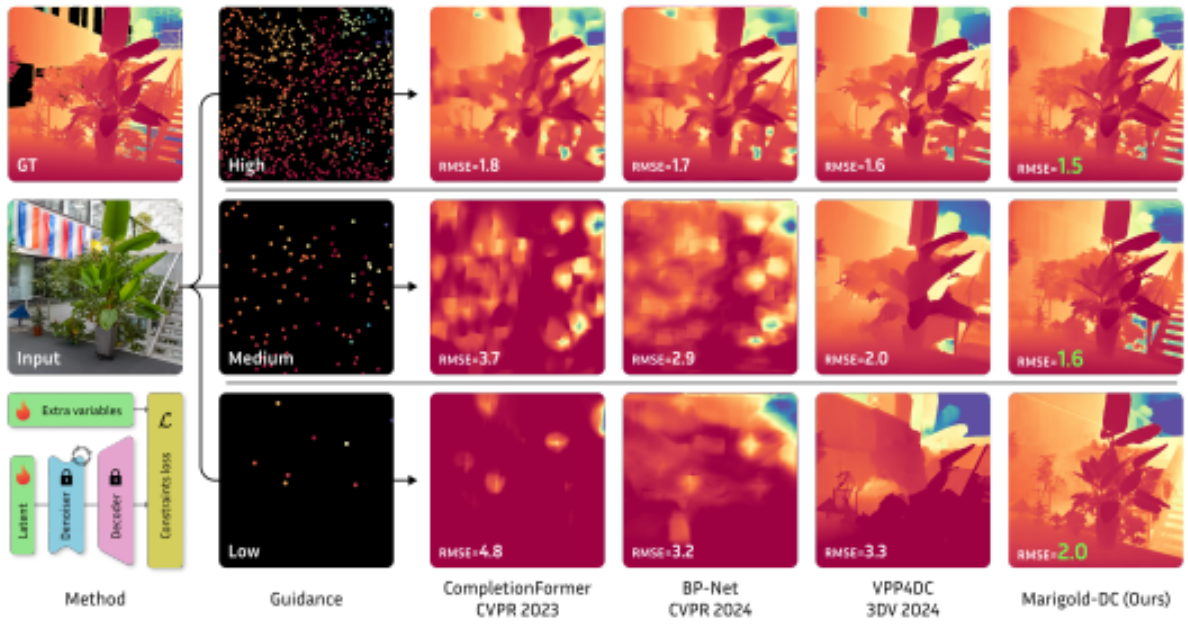


Figure 88: the comparison grid: ground truth, input image and the sparse guidance at high / medium / low density, with the outputs of CompletionFormer, BP-Net, VPP4DC and Marigold-DC and their RMSE labels

guidance density	CompletionFormer	BP-Net	VPP4DC	Marigold-DC
high	1.8	1.7	1.6	**1.5**
medium	3.7	2.9	2.0	**1.6**
low	4.8	3.2	3.3	**2.0**

RMSE values printed on slide 36 (units not stated on the slide).

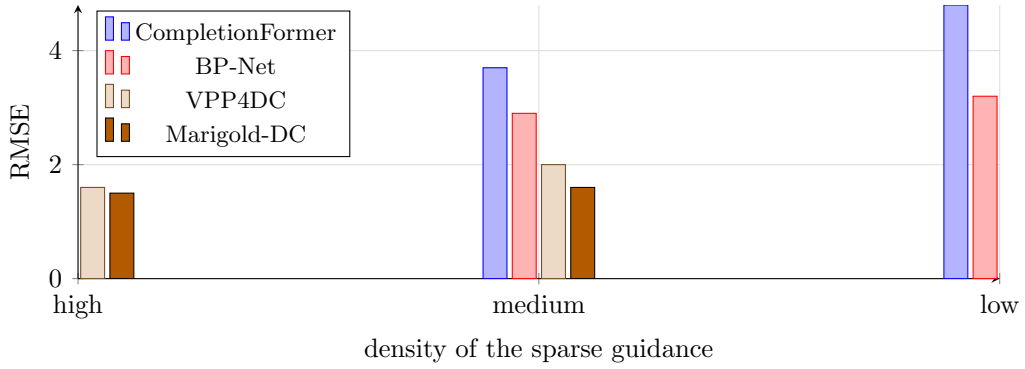


Figure 89: Plot of the RMSE numbers on slide 36. Baselines degrade quickly as the guidance gets sparser; Marigold-DC degrades slowly because the prior fills what the sensor did not sample.

31.2 Steering the Loop *(slides 37)*

Depth completion is reframed as *image-conditional depth generation guided by sparse measurements*. It is **generation rather than regression**: the output comes from a prior. The image **conditions** the network, the sparse depth **guides** the loop, and the measurements were never part of training. Every step already walks toward the model's current estimate; a second pull is added, toward the measurements.

Algorithm 31.2 (Guided step)**.** At every step of the loop: (1) read the *preview*, the current guess at the finished depth map, $\hat{\boldsymbol{z}}_0^t = \boldsymbol{z}_t - \sigma_t \boldsymbol{\epsilon}_\theta$ (Tweedie formula); (2) decode it and turn it into meters with the scale and the shift, $\hat{\boldsymbol{d}}_t^{(\mathrm{m})} = \hat{a}\,\mathcal{D}(\hat{\boldsymbol{z}}_0^t) + \hat{b}$; (3) compare it with the few pixels Ω that have a measurement $\boldsymbol{c}$, $\mathcal{L} = \left\| \hat{\boldsymbol{d}}_t^{(\mathrm{m})} - \boldsymbol{c} \right\|_\Omega$; (4) nudge the latent and the two scalars $(\hat{a}, \hat{b})$ toward the measurements by gradient descent on $\mathcal{L}$; (5) take the ordinary DDIM step.

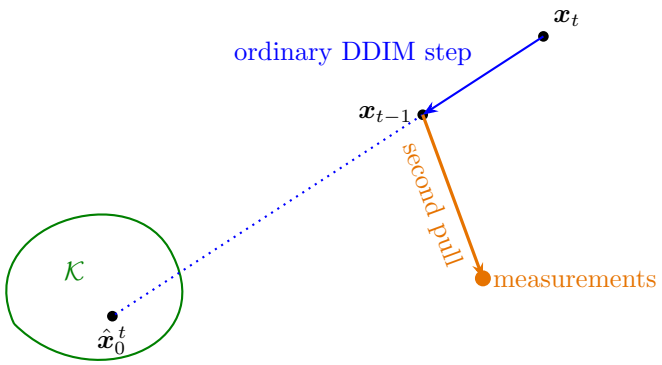


Figure 90: The guided step (redrawn from slide 37): DDIM step plus a second pull toward the sparse measurements.

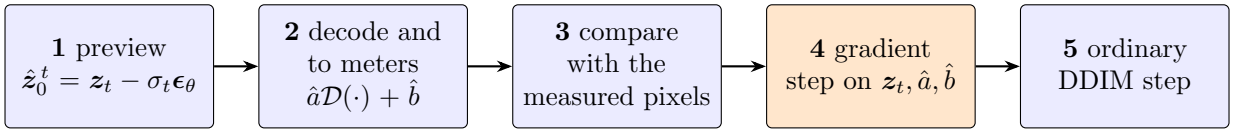


Figure 91: The five steps as a pipeline.

31.3 Nothing Retrained *(slides 38)*

Marigold gives affine-invariant depth; the sparse map gives *metric* depth at a few pixels. The unknowns are the **latent**, which decides the shape, and the **scale and shift**, which decide the meters. All three are optimized, for one image, over fifty steps, and then discarded. Gradients are taken with respect to the latent and the two scalars, so **no weights change**: the U-Net and the VAE are frozen throughout.

Theorem 31.3 (Zero-shot property, as claimed on the slide)**.** *There is no depth-completion dataset, no architecture change and no new checkpoint. The method is therefore zero-shot across indoor and outdoor scenes, where the baselines ship one checkpoint per domain.*

Intuition

The frozen network is a very good but unit-less sketch artist; the LiDAR points are a few tape-measure readings. At each step the sketch is bent slightly (through the latent) so that it does not contradict the readings, and the ruler (scale and shift) is adjusted until the sketch reads in meters. Because the readings are used only at test time, the same model handles a room and a street: the debts of Section 1 are paid at once, the prior supplying what the sensor cannot sample and the sensor the two numbers the prior cannot know.

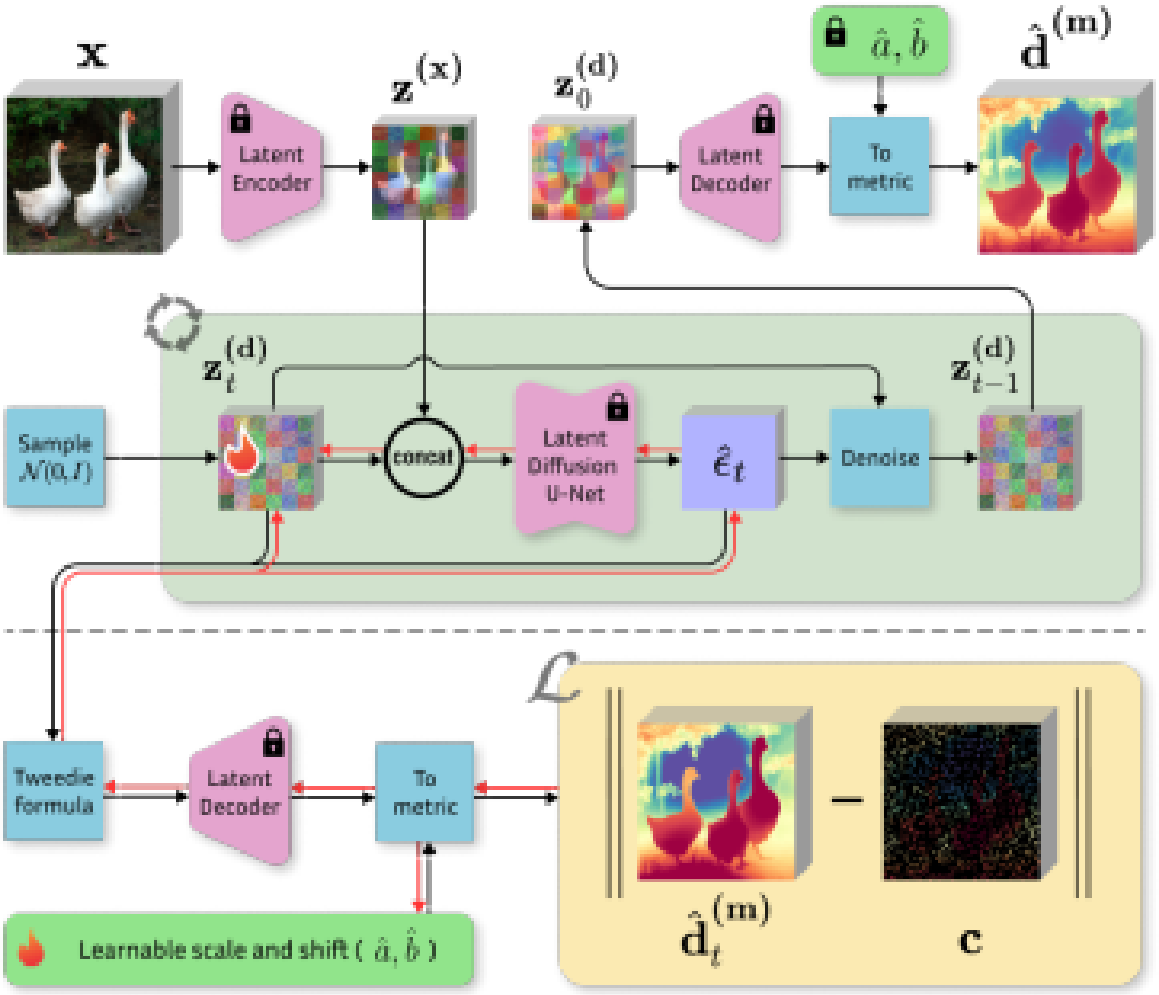


Figure 92: the Marigold-DC diagram: goose photograph, encoder/decoder, frozen U-Net loop with the red gradient arrows, sparse map **c** and the resulting metric depth (the block structure is redrawn in the figures above)

Summary *(slides 39–40)*

Comparison of the methods covered

method	pays with	output	strength	main limitation
Pulsed LiDAR	time	metric, sparse	exact where it lands; error nearly range-independent	stripes growing $\propto r$; no return $\neq 0$; mixed points at edges
Rectified stereo	baseline	metric, dense where matched	dense; metric from b	error $\propto z^2/(fb)$; fails on texture-poor, repeated, occluded regions
Structure from motion	computation	shape up to scale	no second camera needed	unknown global scale; rigid scene
Structured light (NYU)	projected pattern	metric, dense indoors	dense labels at short range	inpainted holes; indoor only
Geometric cues (size, vertical position)	assumptions	metric if assumptions hold	closed form, interpretable	need known H, flat ground, known Y and horizon
Eigen scale-invariant loss	prior	depth up to 1 scale	loss respects scale ambiguity	one scale per scene; dataset-specific sensor errors
MiDaS	prior, mixed data	affine-invariant disparity	mixes datasets with no common zero	not metric; shape only
Depth Anything V2	prior, pseudo-labels	affine-invariant inverse depth	scales with unlabeled real images	not metric without fine-tuning; inherits teacher's prior

Every method in this lecture bought depth with a different resource. The last section bought it with two at once.

method	paid with	when	free parameters left
A laser (3.3, 3.4)	time	test	0, but sparse
Stereo	a known baseline	test	0, error grows as z^2
Structure from motion	viewpoints, no baseline	test	1, one global scale
Supervised (3.7)	a prior, on sensor labels	train	0, 1 or 2, set by the labels
Self-supervision (3.9)	a baseline	train	0, for that rig
Marigold (3.11)	a generative prior	train	2
Marigold-DC (3.13)	a prior and time	both	0, dense, nothing retrained

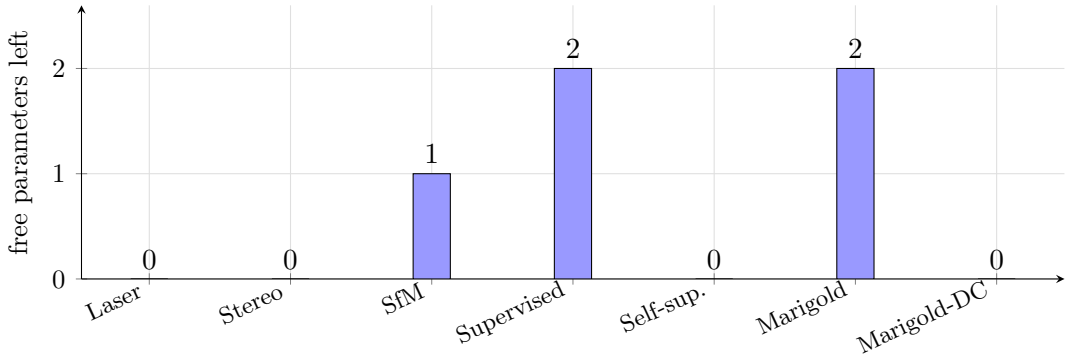


Figure 93: Free parameters left after each method (last column of the table). "Supervised" is drawn at its worst case: it ranges from 0 to 2 depending on the labels.

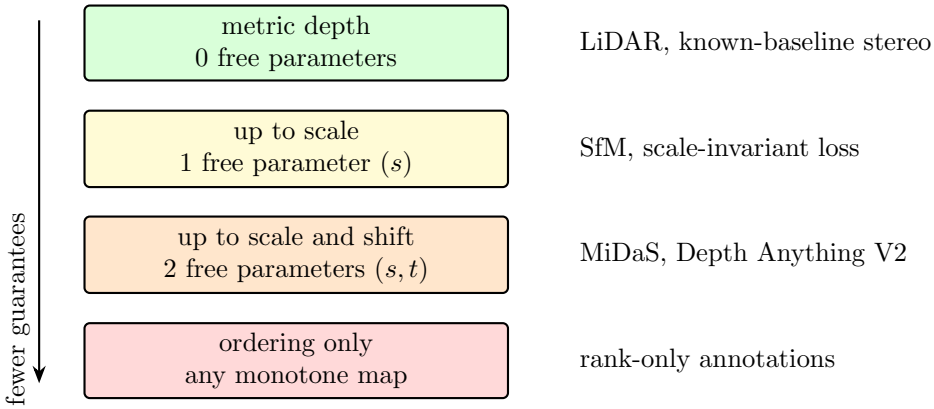


Figure 94: Output spaces ordered by how much of the depth is actually determined. Moving down the ladder, the claim weakens but the data requirement does too.

What each method lets you claim

Key take-aways

- **Depth from one pixel is undetermined.** Back-projection returns a ray; metric depth always needs a *currency*: time (LiDAR), a baseline (stereo) or a prior (learning). A single image bypasses cost, sparsity and hardware, but not the physics.
- **The two physical principles fail differently.** Time of flight has a range-independent timing floor but sparse, structured samples; triangulation is dense but its error grows as $z^2/(fb)$. LiDAR labels must be converted with the camera-frame z, a z-buffer and a validity mask.
- **A learned depth network is a prior, and its output is ambiguous.** The likelihood is flat along each ray, so scale-invariant and affine-invariant losses (in log space or in disparity) ask the network only for shape; disparity is preferred because planes stay planes under an affine map in inverse depth.
- **The prior is both the method and the failure mode.** Networks read vertical image position more than apparent size, only partially respect pitch and roll, and absorb the training focal length; anamorphic art defeats them with no sign of uncertainty.
- **Always ask "recovered or ambiguous?"** Whether a reported depth is in meters, up to scale, or up to scale and shift determines what it can be used for (planning versus shape) and is the key to reading benchmark numbers (aligned first, then scored with AbsRel and δ).
- **The prior supplies what the sensor cannot sample, and the sensor supplies the two numbers the prior cannot know.**
- Self-supervision from stereo needs no depth labels, but the photometric loss is blind on flat and occluded pixels (hand-written priors fill the gap) and is metric only for the baseline it was trained with.
- A diffusion denoiser is a prior: it learns the gradient of a smoothed distance to the data, is defined everywhere, and can be sampled, conditioned, or guided; regression by contrast returns the conditional mean, which blurs depth edges.
- Marigold reuses Stable Diffusion's weights: frozen VAE, U-Net fine-tuned with the image latent concatenated; percentile normalization makes the output affine-invariant, so it gives shape but no meters, no common zero, and costs $50 \times 10 = 500$ evaluations.

- An affine error $\hat{z} = sz + t$ maps to $\hat{\boldsymbol{p}} = (s + t/z)\boldsymbol{p}$: scale is harmless to shape, shift distorts it. Marigold-DC fixes both by optimizing the latent, scale and shift at test time against a few metric points, with no retraining.

Acknowledgments and sources *(slides 41–42)*

The diffusion section (3.10) is adapted from *Introduction to Diffusion*, MIT 6.S183, IAP 2026, Lecture 1, by D. Pfrommer; parts of both halves are adapted from *Depth Estimation* (Politecnico di Milano, 2024) by L. Magri. The reference list on slide 42 (Pfrommer, Yuan, Ke et al., Rombach et al., Yang et al., Ranftl et al., Godard et al., Ho–Jain–Abbeel, Song–Meng–Ermon, Karras et al., and others) and further reading (Yuan's smalldiffusion) are given there.

A Supplementary derivations (not on the slides)

The following results are *not* on the slides; they are added to justify remarks made in the main text.

Proposition A.1 (DDIM step = re-noising the clean estimate)**.** *Equation* (13) *is equivalent to* $\boldsymbol{x}_{t-1} = \hat{\boldsymbol{x}}_0^t + \sigma_{t-1}\,\boldsymbol{\epsilon}_\theta(\boldsymbol{x}_t, \sigma_t)$.

Proof. $\hat{\boldsymbol{x}}_0^t = \boldsymbol{x}_t - \sigma_t \boldsymbol{\epsilon}_\theta$, hence $\boldsymbol{x}_t = \hat{\boldsymbol{x}}_0^t + \sigma_t \boldsymbol{\epsilon}_\theta$ and $\boldsymbol{x}_{t-1} = \boldsymbol{x}_t + (\sigma_{t-1} - \sigma_t)\boldsymbol{\epsilon}_\theta = \hat{\boldsymbol{x}}_0^t + \sigma_{t-1}\boldsymbol{\epsilon}_\theta$. □

Reading: keep the current clean estimate and re-attach the *same* predicted noise, only with the smaller amplitude σ_{t-1}.

Proposition A.2 (Irreducible floor of the training loss)**.** *For the ideal denoiser,* $L_{simple}(\boldsymbol{\epsilon}^*) = \mathbb{E}_{\boldsymbol{x}_\sigma, \sigma} \operatorname{tr} \operatorname{Cov}(\boldsymbol{\epsilon} \mid \boldsymbol{x}_\sigma)$*, which is strictly positive whenever the posterior of* $\boldsymbol{x}_0$ *given* $\boldsymbol{x}_\sigma$ *is not a point mass.*

Proof. Applying the conditional-mean decomposition of Section 3.1 with $Y = \boldsymbol{\epsilon}$ and $f = \mathbb{E}[\boldsymbol{\epsilon} \mid \boldsymbol{x}_\sigma]$ gives $\mathbb{E}\|\boldsymbol{\epsilon} - \boldsymbol{\epsilon}^*\|^2 = \mathbb{E} \operatorname{tr} \operatorname{Cov}(\boldsymbol{\epsilon} \mid \boldsymbol{x}_\sigma)$; and $\boldsymbol{\epsilon} = (\boldsymbol{x}_\sigma - \boldsymbol{x}_0)/\sigma$, so $\operatorname{Cov}(\boldsymbol{\epsilon} \mid \boldsymbol{x}_\sigma) = \operatorname{Cov}(\boldsymbol{x}_0 \mid \boldsymbol{x}_\sigma)/\sigma^2$. □

Reading: this explains why the slide-14 curve stops around 0.41 rather than 0: the remaining loss is genuine ambiguity about which data point the noisy input came from, not an optimization failure.

Part III

Lecture 4

Abstract

These notes follow the slide deck *04 Stereo Depth Estimation* (60 slides) section by section; every header carries its slide range. Formal statements (definitions, lemmas, proofs) are merged in where the slides introduce a topic, and blue *Intuition* boxes explain the "why" in plain words. **Source note:** only the slide deck was uploaded, no companion chapter. The formal content therefore comes from standard references that the slides themselves cite (Hartley–Zisserman / Szeliski for geometry) and from the papers named on the slides (Zbontar–LeCun, Mayer et al., Kendall et al., Wang et al., Leroy et al.). Where a statement goes beyond what the slides say, it is marked as such. Diagrams are redrawn in TikZ; photographs are left as dashed placeholders that name the slide to copy from.

Agenda and Learning Objectives *(slides 1–2)*

The lecture has five parts: 4.1 Two-View Geometry, 4.2 Block Matching, 4.3 Siamese Networks, 4.4 End-to-End Learning, 4.5 Stereo Without Calibration. After it you should be able to:

1. explain why calibration and rectification reduce stereo matching to a 1D search, and derive $z = fb/d$ together with its error;
2. explain block matching, its failure modes and the left-right consistency test;
3. describe how learned matching costs and end-to-end networks entered stereo;
4. explain how matching in 3D removes calibration and rectification from the matching step, and what it gives up.

B Two-View Geometry *(slides 3–13)*

B.1 Depth Cues and Binocular Stereopsis *(slides 4)*

Recovering depth from a single image is *ill-posed* (previous lecture). Three ways around it:

- **Time-of-flight (ToF)** sensors measure the round-trip time to an obstacle ⇒ metric depth.
- **Stereo vision** triangulates using two views; because the baseline is known, depth is metric.
- **A depth prior** (object assumptions or a neural network) usually gives depth only *up to an affine* transformation.

[PHOTO PLACEHOLDER]
Insert the image from **slide 4**:
3.2cm

Historical and natural binocular systems: Cyclops (cube illusion), stalk-eyed fly, stereoscopic rangefinder (four small photos on the slide).

Intuition

Your two eyes are a metric ruler *because* the distance between them is fixed. A single eye (or a monocular network) can say "this is nearer than that" but not "this is 12 m away". Stereo trades a second camera for a physical scale.

B.2 Two-View Stereo Matching *(slides 5)*

Goal: recover the *disparity* (proportional to inverse depth) for every pixel of a reference image from two input images.

[PHOTO PLACEHOLDER]
Insert the image from **slide 5**:
3.6cm

Left image, right image and the colour-coded disparity map (Geiger et al., ACCV 2010).

B.3 The Classical Stereo Pipeline *(slides 6)*

Task: build a dense 3D model of a static scene from two images. Pipeline:

1. calibrate the cameras intrinsically and extrinsically;
2. rectify the images using the calibration;
3. compute a disparity map for the reference image;
4. remove outliers with a consistency/occlusion test;
5. obtain depth from disparity using the calibration;
6. build a 3D model, e.g. by volumetric fusion and meshing.

Steps 1–2 (highlighted on the slide) are exactly what Section F will remove; we revisit this list at the end (*The Pipeline Revisited*).

B.4 Epipolar Geometry *(slides 7)*

Definition B.1 (Essential matrix)**.** Let a 3D point $\boldsymbol{x}$ project to $_1 \propto \mathbf{K}_1\boldsymbol{x}$ in camera 1 and to $_2 \propto \mathbf{K}_2(\mathbb{R}\boldsymbol{x}+)$ in camera 2 (are homogeneous pixel coordinates). The *normalized coordinates* are $\tilde{\mathbf{x}}_i = \mathbf{K}_i^{-1}{}_i$. The *essential matrix* is $\mathbb{E} = []_\times\mathbb{R}$, where $[]_\times$ is the skew-symmetric matrix with $[]_\times\boldsymbol{v} = \times\boldsymbol{v}$.

Proposition B.2 (Epipolar constraint)**.** *Corresponding points satisfy* $\tilde{\mathbf{x}}_2^\top\mathbb{E}\,\tilde{\mathbf{x}}_1 = 0$. *Hence the match of* $_1$ *lies on the* epipolar line $\tilde{\boldsymbol{l}}_2 = \mathbb{E}\tilde{\mathbf{x}}_1$ *in image 2, and the match of* $_2$ *lies on* $\tilde{\boldsymbol{l}}_1 = \mathbb{E}^\top\tilde{\mathbf{x}}_2$ *in image 1. All epipolar lines of an image pass through its* epipole.

Proof. Up to positive scale, $\tilde{\mathbf{x}}_1 \propto \boldsymbol{x}$ and $\tilde{\mathbf{x}}_2 \propto \mathbb{R}\boldsymbol{x}+$. Then $(\mathbb{R}\boldsymbol{x}+)^\top[]_\times\mathbb{R}\boldsymbol{x} = (\mathbb{R}\boldsymbol{x})^\top[]_\times(\mathbb{R}\boldsymbol{x}) +^\top []_\times\mathbb{R}\boldsymbol{x} = 0+0$, because $\boldsymbol{v}^\top[]_\times\boldsymbol{v} = 0$ and $^\top(\times\boldsymbol{u}) = 0$ for every $\boldsymbol{v}, \boldsymbol{u}$. Scaling does not change a zero. The statement on lines is the observation that $\tilde{\mathbf{x}}_2^\top\tilde{\boldsymbol{l}}_2 = 0$ is the equation of a line in image 2. □

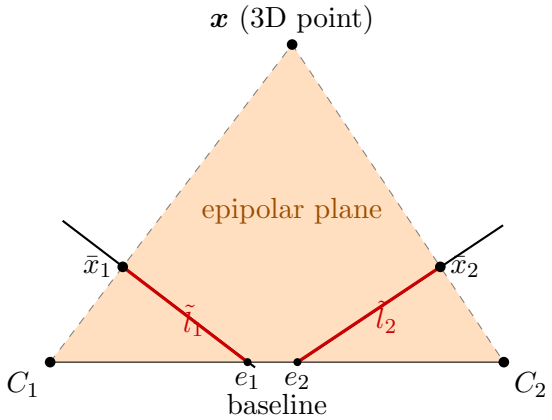


Figure 95: Epipolar geometry (redrawn from slide 7). The camera centres and the 3D point span the epipolar plane; it cuts each image plane in an epipolar line, and the baseline pierces the image planes at the epipoles.

Example B.3 (Why the 1D search matters)**.** For a VGA image ($640 \times 480 \approx 307{,}000$ pixels), an unconstrained correspondence search tests about 3×10^5 candidates per pixel; searching along one epipolar line tests about 640. The saving is $480\times$, i.e. the image height.

Intuition

Given where a pixel sits in image 1, the 3D point can be anywhere along its viewing ray. Seen from camera 2, that ray is a straight line, not a blob: the matching pixel must lie on it. So we ask "which point *on this line* looks the same?" instead of "which point *in the image*?".

B.5 Image Rectification *(slides 8)*

If both cameras face exactly the same direction (image planes co-planar), the epipoles are at infinity and the epipolar lines are parallel: the correspondence of a pixel lies on the *same horizontal scanline*. Real rigs are never perfectly aligned, but we can re-warp both images by a *rotation about the camera centres*, mapping them to a common plane parallel to the baseline; this is *rectification*. Because the rotation is about the camera centre, the 3D structure need not be known.

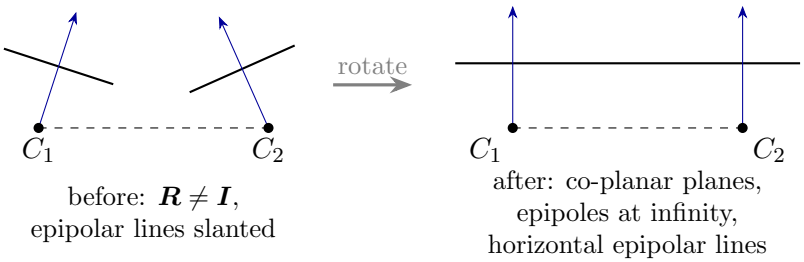


Figure 96: Rectification as a pure rotation of each camera (top view; blue arrows are optical axes). After the rotation both image planes are the same plane, parallel to the baseline.

[PHOTO PLACEHOLDER]
Insert the image from **slide 8**:
3.4cm

Example pair of a house before and after rectification, with red horizontal lines (slide 8).

B.6 Rectified Epipolar Constraint *(slides 9)*

Proposition B.4 (Rectified setup)**.** *Let* $\mathbf{K}_1 = \mathbf{K}_2 = \mathbb{R} = \boldsymbol{I}$ *and* $= (t, 0, 0)^\top$*. Then*

$$\mathbb{E} = []_\times = \begin{pmatrix} 0 & 0 & 0 \\ 0 & 0 & -t \\ 0 & t & 0 \end{pmatrix}, \qquad {}_2^\top \mathbb{E}\, {}_1 = t\, y_1 - t\, y_2 = 0 \implies y_1 = y_2.$$

Proof. With ${}_1 = (x_1, y_1, 1)^\top$ we get $\mathbb{E}_1 = (0, -t, t\, y_1)^\top$, and ${}_2^\top (0, -t, t\, y_1)^\top = -t\, y_2 + t\, y_1$. For $t \neq 0$ this vanishes iff $y_1 = y_2$. □

Intuition

The only thing separating the two cameras is a sideways slide. A sideways slide moves every point horizontally and never vertically, so the match is on the same row. This is why stereo code can loop along scanlines.

B.7 Calculating the Rectifying Rotation Matrix *(slides 10)*

Lemma B.5 (Rectifying rotation)**.** *Let* $\mathbb{R}$*, be the relative pose (*$\boldsymbol{x}_2 = \mathbb{R}\boldsymbol{x}_1 +$*). Define*

$$\boldsymbol{r}_1 = \frac{-\mathbb{R}^\top}{\|\mathbb{R}^\top\|_2}, \quad \boldsymbol{r}_2 = \frac{[(0,0,1)^\top]_\times \boldsymbol{r}_1}{\|[(0,0,1)^\top]_\times \boldsymbol{r}_1\|_2}, \quad \boldsymbol{r}_3 = \boldsymbol{r}_1 \times \boldsymbol{r}_2, \quad = (\boldsymbol{r}_1, \boldsymbol{r}_2, \boldsymbol{r}_3)^\top.$$

Then is a rotation and $\boldsymbol{r}_1 = (1, 0, 0)^\top$*: the epipole of camera 1 becomes an ideal point in the* x*-direction.*

Proof. The centre of camera 2 in the frame of camera 1 solves $\mathbb{R}\boldsymbol{x} + = \mathbf{0}$, i.e. $\boldsymbol{x} = -\mathbb{R}^\top$; the epipole of image 1 is this direction, so it is $\propto \boldsymbol{r}_1$. By construction $\boldsymbol{r}_2 \perp \boldsymbol{r}_1$ (a cross product is orthogonal to its factors), both are unit vectors, and $\boldsymbol{r}_3 = \boldsymbol{r}_1 \times \boldsymbol{r}_2$ completes a right-handed orthonormal basis. Hence the rows of are orthonormal and $\boldsymbol{r}_1 = (\boldsymbol{r}_1^\top \boldsymbol{r}_1, \boldsymbol{r}_2^\top \boldsymbol{r}_1, \boldsymbol{r}_3^\top \boldsymbol{r}_1)^\top = (1, 0, 0)^\top$. □

Algorithm B.6 (Rectification). *Input:* two images, intrinsics $\mathbf{K}_1, \mathbf{K}_2$ and relative pose $(\mathbb{R},)$.

1. Estimate $\mathbb{E}$, decompose it into and $\mathbb{R}$, and build as in Lemma B.5.
2. Warp image 1: $\tilde{\mathbf{x}}'_1 = \mathbf{K}\, \mathbf{K}_1^{-1}\, {}_1$.
3. Warp image 2: $\tilde{\mathbf{x}}'_2 = \mathbf{K}\, \mathbb{R}^\top\, \mathbf{K}_2^{-1}\, {}_2$.

(**K** is the common intrinsic matrix of the rectified pair.)

Intuition

Think of $\boldsymbol{r}_1$ as "the direction pointing from camera 1 to camera 2". Rectification spins both cameras so that this direction becomes the image x-axis. Camera 2 first needs undoing its own rotation $\mathbb{R}$ (hence $\mathbb{R}^\top$) so that both cameras are spun in the same world frame. The recipe for $\boldsymbol{r}_2$ also explains a limit stated later (slide 34): if the baseline points along the optical axis (forward motion), $\boldsymbol{r}_1 \parallel (0,0,1)^\top$, the cross product vanishes and $\boldsymbol{r}_2$ is undefined – planar rectification breaks down.

B.8 Rectified Pair and Disparity *(slides 11)*

In a rectified pair the correspondence of a pixel lies in the same image row; the horizontal displacement is the *disparity* $d = x_1 - x_2$ (on the slide 0–160 px, with the match to the left). Disparity is anti-proportional to depth: far points move less than near points.

[PHOTO PLACEHOLDER]
Insert the image from **slide 11**:
4.2cm

Rectified pair (two lab scenes and a street scene) with colour-coded disparity maps and the 0–160 px colour bar (slide 11).

B.9 Disparity to Depth *(slides 12)*

Theorem B.7 (Triangulation in a rectified pair)**.** *Let the cameras have focal length f (in pixels) and baseline b, and let $d = x_1 - x_2$ be the disparity of a point at depth z. Then*

$$z = \frac{f\,b}{d}.$$

Proof (slide version). The left and right rays lie in the epipolar plane and intersect at the 3D point; similar triangles give $\frac{z-f}{b-d} = \frac{z}{b}$. Hence $zb - fb = zb - zd$, so $zd = fb$. □

Proof (direct). Place camera 1 at the origin and camera 2 at $(b, 0, 0)$. A point (X, Y, z) projects to $x_1 = fX/z$ and $x_2 = f(X-b)/z$ (principal point at the origin). Subtracting, $d = x_1 - x_2 = fb/z$. □

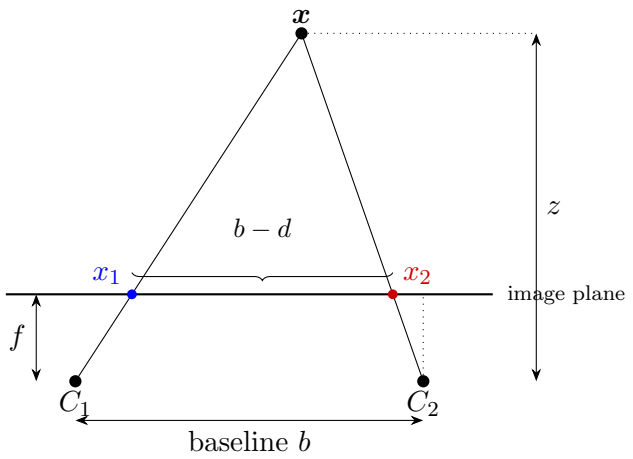


Figure 97: Triangulation (redrawn from slide 12). With $f = 1$, $b = 4$, $z = 4$ in drawing units, each camera measures its own image coordinate ($x_1 = 0.65$ from C_1, $x_2 = -0.35$ from C_2), so $d = x_1 - x_2 = 1 = fb/z$; the similar triangles $(f,\ b-d)$ and $(z,\ b)$ give the formula.

Intuition

Close one eye and point at your thumb, then switch eyes: the thumb jumps a lot. A distant tree barely moves. Disparity *is* that jump, and $z = fb/d$ converts "how much it jumped" into "how far it is". Doubling the baseline doubles every jump, so a given pixel precision resolves finer depth steps.

B.10 Worked Example: Baseline and Depth Error *(slides 13)*

Theorem B.8 (Depth error)**.** *A disparity error δd causes, to first order, a depth error*

$$\delta z \approx \left|\frac{\partial z}{\partial d}\right| \delta d = \frac{fb}{d^2}\,\delta d = \frac{z^2}{fb}\,\delta d.$$

Proof. Differentiate $z = fb\,d^{-1}$: $\partial z/\partial d = -fb/d^2$. Substitute $d = fb/z$ to get $fb/d^2 = z^2/(fb)$. □

Corollary B.9. *The relative error $\delta z/z = z\,\delta d/(fb)$ grows linearly with z; doubling b halves δz at every depth; if the smallest measurable disparity is $d_{\min}$, the usable range is $z_{\max} = fb/d_{\min}$.*

Example B.10 (Car-like rig, $f \approx 721$ px, $b \approx 0.54$ m, $fb \approx 389$ px·m, $\delta d = 1$ px)**.**

depth z	disparity d	depth error δz
10 m	38.9 px	0.26 m
20 m	19.5 px	1.03 m
40 m	9.7 px	4.11 m

(Beyond the slide: the linearisation is first order; with $d-1$ at $z = 40$ m the exact value is $389/8.73 - 40 \approx 4.6$ m.)

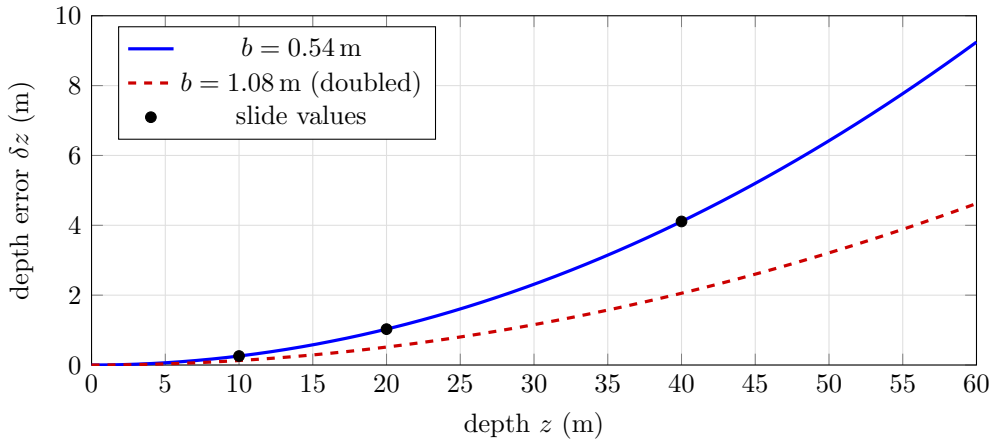


Figure 98: Depth error for a one-pixel disparity error, $\delta z = z^2 \delta d/(fb)$ with $f = 721\,\mathrm{px}$. The error is quadratic in depth; doubling the baseline halves it everywhere (new figure, not on the slides).

Intuition

At 40 m the disparity is only 9.7 px. Being off by one pixel means being off by about 10 % of the measurement, and that 10 % turns into 4 m. Near the camera the same one-pixel slip costs a few centimetres. A wider baseline helps the arithmetic, but the two views then differ more and matching gets harder – the central design tension of stereo.

C Block Matching *(slides 14–21)*

C.1 Correspondence Ambiguity *(slides 15)*

How do we decide that two image points correspond? Assume they look "similar". But a single pixel does not reveal local structure (ambiguity), so we compare a small patch – and even then the task is difficult (repetitive or textureless regions, changing appearance).

[PHOTO PLACEHOLDER]
Insert the image from **slide 15**:
3.6cm

Three grayscale views of an animal with red boxes marking a tail/patch whose appearance changes (slide 15).

C.2 Similarity Metrics *(slides 16)*

Take two $K \times K$ windows flattened to vectors $, \in \mathbb{R}^{K^2}$.

Definition C.1 (SSD and ZNCC)**.**

$$(x, y, d) = \|(x, y) - (x - d, y)\|_2^2,$$
$$(x, y, d) = \frac{(-\mu_L)^\top (-\mu_R)}{\|-\mu_L\|_2 \, \|-\mu_R\|_2},$$

with μ_L, μ_R the means of the two windows (arguments omitted on the right). SSD is a *cost* (minimise); ZNCC is a *similarity* in $[-1, 1]$ (maximise).

Lemma C.2 (Gain/bias invariance)**.** *For $a > 0$ and $b \in \mathbb{R}$, $(a + b\mathbf{1},) = (,)$. Moreover, for the normalised windows $\hat{\boldsymbol{w}} = (\boldsymbol{w} - \mu)/\|\boldsymbol{w} - \mu\|_2$ one has $(\hat{\boldsymbol{w}}_L, \hat{\boldsymbol{w}}_R) = 2 - 2\,(,)$.*

Proof. The mean of $a + b\mathbf{1}$ is $a\mu_L + b$, so the centred vector is $a(-\mu_L)$ and its norm $a\|-\mu_L\|$ (as $a > 0$); a cancels in the ratio. For unit vectors $\boldsymbol{u}, \boldsymbol{v}$: $\|\boldsymbol{u} - \boldsymbol{v}\|^2 = 2 - 2\boldsymbol{u}^\top \boldsymbol{v}$, and $\hat{\boldsymbol{w}}_L^\top \hat{\boldsymbol{w}}_R =$. □

Example C.3. Let $= (10, 20, 30, 40)$ and $= +100 = (110, 120, 130, 140)$ (same texture, brighter exposure). Then $= 4 \cdot 100^2 = 40{,}000$ – a terrible score – but $= 1$, a perfect match. For the reversed window $(40, 30, 20, 10)$, $= -1$.

Intuition

SSD asks "are the numbers equal?", ZNCC asks "do the two patches have the same *pattern of ups and downs*?". When the two cameras have different gain or exposure, only the second question is the right one. Other metrics exist (Szeliski, Ch. 12.3; Hirschmüller and Scharstein, TPAMI 2009).

C.3 Block Matching *(slides 17)*

Algorithm C.4 (Block matching with winner-takes-all). *Input:* a rectified image pair.

1. Choose a disparity range $[0, d_{\max}]$.
2. For every pixel (x, y) and every d, compute the matching cost $C(x, y, d)$ (e.g. SSD), giving a *cost volume*.
3. Winner-takes-all: $\hat{d}(x, y) = \arg\min_{d \in [0, d_{\max}]} C(x, y, d)$.
4. Repeat with the roles of the images exchanged and apply the left-right consistency check to remove outliers.

Naively this costs $O(W H D K^2)$ per image.

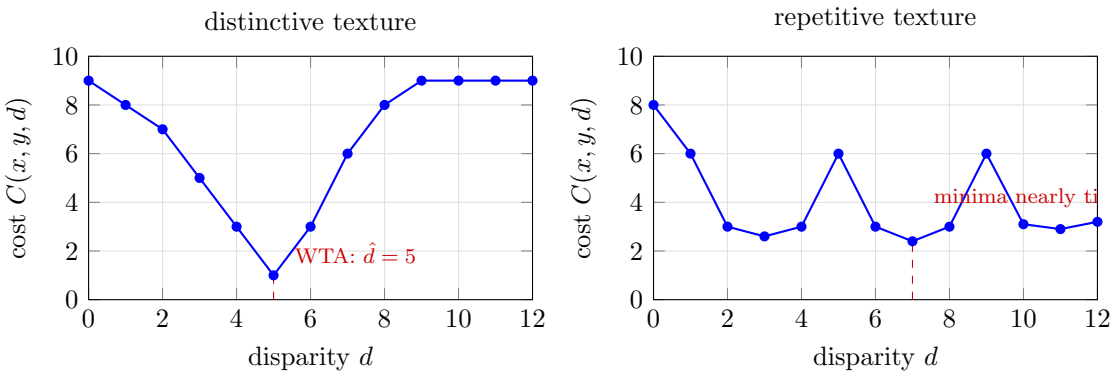


Figure 99: Illustrative cost curves along one scanline (values invented for illustration, not measured). Left: a clear unique minimum. Right: repeated texture produces several near-equal minima, so noise can make winner-takes-all pick the wrong one.

[PHOTO PLACEHOLDER]
Insert the image from **slide 17**:
3.6cm

Left image, computed disparities (noisy, with holes) and ground truth (slide 17).

Intuition

Block matching is a "find-my-twin" game on one row: slide a small window of the left image along the row of the right image and keep the position with the lowest cost. Each pixel decides alone – nobody checks whether neighbours agree – which is why the output is noisy and why later sections add regularisation and learning.

C.4 Block Matching: Half Occlusions *(slides 18)*

A *half-occlusion* is a region visible in the left image but not in the right one (red region on the slide).

Proposition C.5 (Width of the half-occluded strip)**.** *Let a fronto-parallel foreground with disparity d_f stand in front of a background with disparity $d_b < d_f$. The strip of background that is visible in the left image but hidden in the right image lies immediately left of the foreground's left edge in the left image and has width $d_f - d_b$ pixels.*

Proof. Let the foreground occupy $[a, e]$ in the left image; it appears at $[a - d_f, e - d_f]$ in the right image. A background pixel x of the left image appears at $x - d_b$ in the right image. It is hidden in the right image iff its right-image position $x - d_b$ falls inside the foreground's footprint $[a - d_f, e - d_f]$ while $x < a$ (it is not itself foreground). The condition $x - d_b \geq a - d_f$ reads $x \geq a - (d_f - d_b)$, so the hidden background pixels are exactly $x \in [a - (d_f - d_b), a)$, an interval of length $d_f - d_b$. □

Example C.6. With the rig of the previous section, a car at 10 m ($d_f = 38.9$ px) in front of a wall at 40 m ($d_b = 9.7$ px) hides a strip 29.2 px wide next to it. No matching cost can find a twin there, because the twin does not exist.

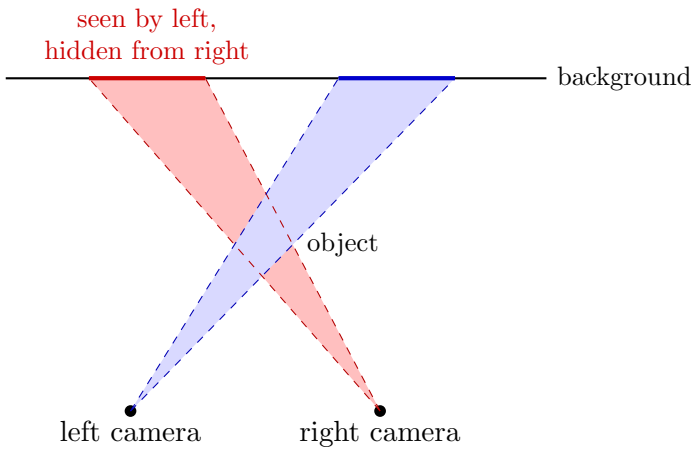


Figure 100: Half-occlusion (top view, redrawn from slide 18). Red: background strip hidden from the right camera but visible to the left one; blue: strip hidden from the left camera.

[PHOTO PLACEHOLDER]
Insert the image from **slide 18**:
3.3cm

Left image with the half-occluded region marked in red, zoomed left/right crops, and the occlusion sketch (slide 18).

C.5 Block Matching: Assumption Violations *(slides 19)*

Block matching assumes that *all pixels inside the window are displaced by the same d*: the *fronto-parallel assumption*. It is often wrong: slanted surfaces deform perspectively between views, and at disparity discontinuities the window content changes differently on the two sides.

Proposition C.7 (Disparity on a plane is affine)**.** *For a plane $\boldsymbol{n}^\top \boldsymbol{X} = h$ ($h \neq 0$) in the rectified camera frame, $1/z$ is an affine function of the pixel (x, y); hence so is the disparity $d = fb/z$.*

Proof. A pixel sees $\boldsymbol{X} = z\,\mathbf{K}^{-1}$. Substituting in the plane equation, $z\,\boldsymbol{n}^\top \mathbf{K}^{-1} = h$, so $1/z = \boldsymbol{n}^\top \mathbf{K}^{-1}/h$, which is linear in $= (x, y, 1)^\top$. Multiply by fb. □

Intuition

A fronto-parallel wall has constant disparity; a floor or a slanted wall has a disparity that *ramps* across the window. If the ramp is 0.1 px per pixel and the window is 11 pixels wide, the left and right windows disagree by about 1.1 px at the edges, enough to hurt the match.

Larger windows suffer more.

[PHOTO PLACEHOLDER]
Insert the image from **slide 19**:
3.2cm

Street scene and Teddy-like scene crops showing perspective deformation of windows on slanted surfaces (slide 19).

C.6 Effect of Window Size *(slides 20)*

Trade-off. Small windows give matching ambiguities and noisy disparity maps. Larger windows give smoother results but lose detail and suffer *border bleeding*: when a window straddles a strong edge, left and right patches get the same disparity because of the dominant edge; the weakly textured background (red on the slide) lacks gradients to overcome it.

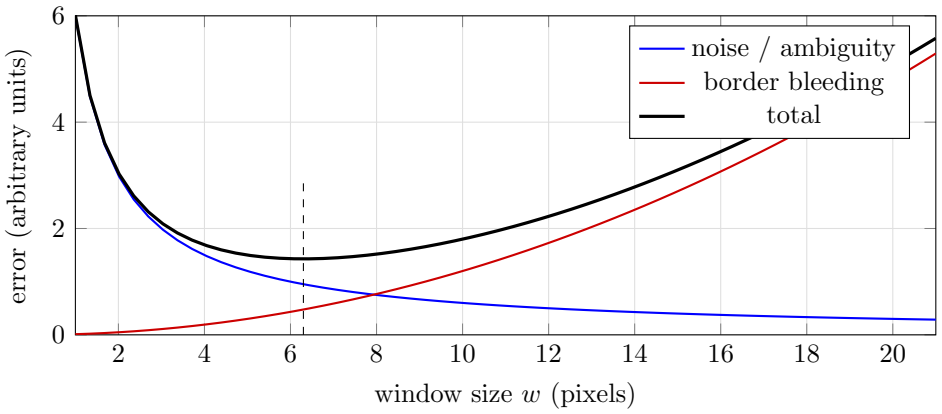


Figure 101: Conceptual bias–variance-style trade-off in window size (illustrative curves, not measured; the slides show it with 5×5 vs 11×11 results).

[PHOTO PLACEHOLDER]
Insert the image from **slide 20**:
3.3cm

Disparity results with 5×5 and 11×11 windows, with “border bleeding” regions boxed (slide 20).

C.7 Left-Right Consistency Test *(slides 21)*

Definition C.8 (Left-right consistency)**.** Let $d_L(x, y)$ be the disparity from matching with the left image as reference, and $d_R(x', y)$ the one with the right image as reference. A pixel is *consistent* (kept) if

$$\big|\, d_L(x, y) - d_R\big(x - d_L(x, y),\, y\big)\big| \le \tau,$$

typically with a small threshold τ (about one pixel); otherwise it is marked invalid. In words: follow the match from left to right and back; you must land where you started (a cycle).

Example C.9. Left disparities at $x = 2, \ldots, 6$ are $d_L = (2, 2, 2, 4, 2)$, so these pixels map to right pixels $x' = x - d_L = (0, 1, 2, 1, 4)$. Suppose $d_R(0) = d_R(1) = d_R(2) = d_R(4) = 2$. Then the differences $|d_L - d_R(x')|$ are $(0, 0, 0, 2, 0)$: with $\tau = 1$ the pixel $x = 5$ is rejected. Note that $x = 3$ and $x = 5$ both claim right pixel 1, which is impossible for a surface that is visible in both views.

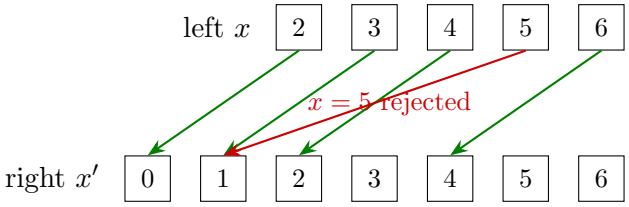


Figure 102: Toy left-to-right matches from the example (new figure). Two left pixels landing on the same right pixel flags an inconsistency.

[PHOTO PLACEHOLDER]
Insert the image from **slide 21**:
3.3cm

Left and right disparity maps used for the consistency check (slide 21).

Intuition

Two witnesses must tell the same story: if "left says my twin is 4 px away" but "that twin says its twin is 2 px away", at least one of them is lying. Half-occluded pixels fail this test automatically, since the supposed twin belongs to something else.

D Siamese Networks *(slides 22–28)*

D.1 Siamese Networks for Stereo Matching *(slides 23)*

Hand-crafted features and similarity metrics do not capture all relevant geometric and radiometric invariances or occlusion patterns; the world is too complex to specify them by hand. Zbontar and LeCun (JMLR 2016) therefore treat *matching cost computation as an image classification problem*:

- train a CNN patch-wise on image pairs with ground-truth disparity (e.g. KITTI Stereo 2015);
- compute features for each of the two images with the learned model;
- correlate features between the two images (dot product);
- take the maximum per pixel (winner-takes-all) or run a global optimisation.

[PHOTO PLACEHOLDER]
Insert the image from **slide 23**:
2.8cm

Small table of left patch / right patch / label (good match, bad match) with traffic-sign crops (slide 23).

D.2 Siamese Network Architecture *(slides 24)*

Two variants are compared on the slide.

Learned similarity	**Cosine similarity**
learn features *and* the similarity metric; potentially more expressive; slow ($W \times H \times D$ MLP evaluations)	learn features and apply a dot product; the features must do the heavy lifting; fast matching (no network evaluation)

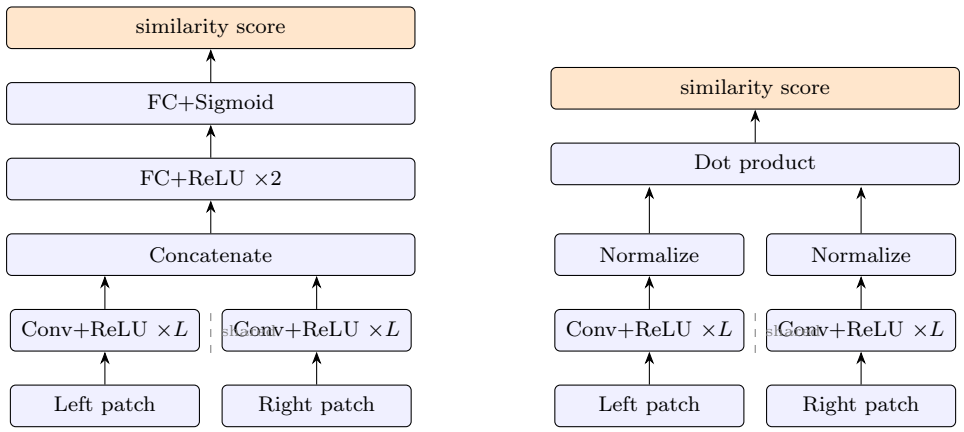


Figure 103: The two Siamese stereo architectures of Zbontar and LeCun (redrawn from slide 24; layer counts are schematic). Both towers share weights.

Example D.1 (Why the cosine head is fast)**.** For a KITTI-sized image (1242×375) and $D = 228$ candidate disparities, the learned-similarity head needs $WHD \approx 1.06 \times 10^8$ network evaluations. The cosine head runs each tower once per image, then each cost is just a dot product of two stored feature vectors.

Intuition

Learned similarity is like hiring an expert who inspects every candidate pair. Cosine similarity is like giving every patch a fingerprint once and then comparing fingerprints: much cheaper, but the fingerprint must be good enough that a plain dot product suffices.

D.3 Training Triplets *(slides 25)*

Definition D.2 (Training triplet)**.** A triplet is $\big((\boldsymbol{x}_L^{\text{ref}}),\ (\boldsymbol{x}_R^{\text{neg}}),\ (\boldsymbol{x}_R^{\text{pos}})\big)$ with

$$\boldsymbol{x}_R^{\text{neg}} = (x_L^{\text{ref}} - d + o_{\text{neg}},\, y_L^{\text{ref}}), \qquad \boldsymbol{x}_R^{\text{pos}} = (x_L^{\text{ref}} - d + o_{\text{pos}},\, y_L^{\text{ref}}),$$

where d is the ground-truth disparity, $o_{\text{neg}} \sim \mathcal{U}(\{-N_{\text{hi}}, \dots, -N_{\text{lo}}, N_{\text{lo}}, \dots, N_{\text{hi}}\})$ and $o_{\text{pos}} \sim \mathcal{U}(\{-P_{\text{hi}}, \dots, P_{\text{hi}}\})$. Typical values: $P_{\text{hi}} = 1$, $N_{\text{lo}} = 3$, $N_{\text{hi}} = 6$.

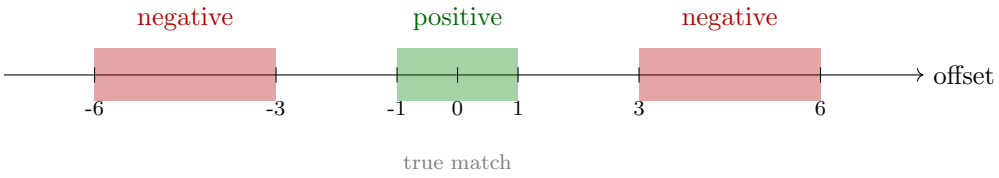


Figure 104: Where positive and negative right patches are sampled relative to the true match (new figure; ranges $P_{\text{hi}} = 1$, $N_{\text{lo}} = 3$, $N_{\text{hi}} = 6$ from the slide).

[PHOTO PLACEHOLDER]
Insert the image from **slide 25**:
3.3cm

Zoomed right image with a window and rows of green "positive" crops and red "negative" crops (slide 25).

Intuition

Negatives are not random patches: they come from just a few pixels away from the truth (3 to 6 px), so they look similar and the network must learn the fine cues. Positives are allowed ± 1 px of slack, because a match is never pixel-perfect. Negatives closer than N_{lo} are excluded since they would be nearly indistinguishable from the positives. This is a mild form of *hard negative mining*. With these values there are 3 positive and 8 negative offsets to draw from.

D.4 Hinge Loss *(slides 26)*

Definition D.3 (Hinge (margin) loss)**.** With s_+ and s_- the network scores of the positive and negative example and margin $m > 0$,

$$\mathcal{L} = \max(0,\, m + s_- - s_+).$$

Lemma D.4. *$\mathcal{L} = 0$ iff $s_+ \geq s_- + m$. When $\mathcal{L} > 0$, $\partial\mathcal{L}/\partial s_+ = -1$ and $\partial\mathcal{L}/\partial s_- = +1$: gradient descent raises s_+ and lowers s_- at the same rate.*

Example D.5 ($m = 0.2$, the value used in the paper).

s_+	s_-	$m + s_- - s_+$	loss	meaning
0.9	0.3	-0.4	0	clearly separated, nothing to learn
0.6	0.5	0.1	0.1	right order but too close: push apart
0.4	0.7	0.5	0.5	wrong order: large correction

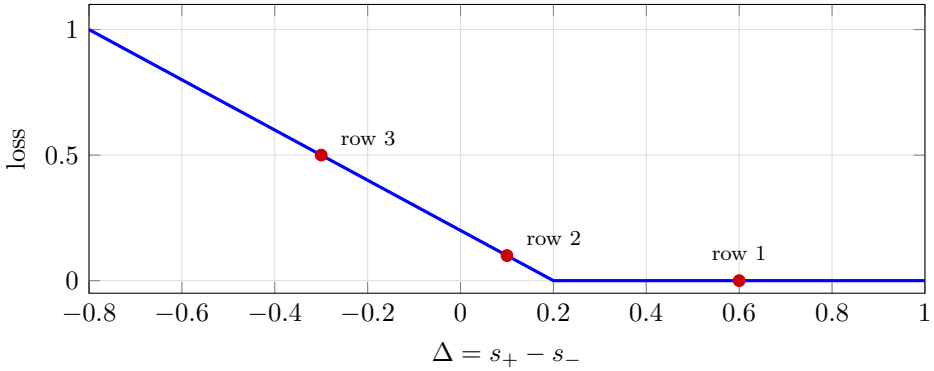


Figure 105: Hinge loss with $m = 0.2$ as a function of the score gap; red dots are the three rows of the table (new figure).

Intuition

The loss does not demand that the positive scores high in absolute terms, only that it beats the negative by a safety margin. Once it does, the example is "solved" and stops contributing – training focuses its effort on the pairs that are still confusable.

D.5 Comparisons *(slides 27)*

The slide compares, on a KITTI street scene, the left input, standard block matching, the Siamese-network matching cost, a semiglobal-matching (SGM, "optimize") variant, the left and right disparity maps, and the left-right consistency test. The qualitative message: learned costs are far less noisy than SAD/SSD-style block matching, and adding a global optimisation (SGM) on top improves them further.

[PHOTO PLACEHOLDER]
Insert the image from **slide 27**:
5.0cm

Grid of six images: left input, Siamese network, standard block matching, left disparity (SGM), right disparity, left-right consistency test (slide 27).

D.6 Similarity Constraint: Failure Cases *(slides 28)*

The underlying assumption is that *corresponding regions look similar and non-corresponding regions look different*. It fails for: **repetitions** (several patches look alike), **textureless surfaces** (everything looks alike), **occlusions** (the twin does not exist) and **non-Lambertian surfaces** (the same point looks different from two views, e.g. reflections). The fix suggested on the slide is spatial regularisation (smoothness priors, cf. Huang, Lee and Mumford on range-image statistics).

[PHOTO PLACEHOLDER]
Insert the image from **slide 28**:
4.0cm

Street photo annotated with "Repetitions", "Textureless surfaces", "Occlusions" and "Non-Lambertian surfaces" crops (slide 28).

Intuition

A matching cost only looks at one pixel (or patch) at a time. When the local evidence is ambiguous or absent, it needs a prior about how the world behaves – "neighbouring pixels usually have similar depth" – to choose between candidates.

E End-to-End Learning *(slides 29–31)*

E.1 DispNet and Synthetic Data *(slides 30)*

- DispNet (Mayer et al., CVPR 2016) was the first end-to-end trained deep network for stereo.
- It uses a U-Net-like architecture with skip connections to retain details.
- A *correlation layer* compares left and right feature maps (the slide notes: 40 px displacement at feature resolution corresponds to 160 px in the input image, i.e. a stride of 4).
- Training uses a multi-scale loss (disparity error in pixels) and *curriculum learning* (easy to hard).
- Pre-train on large *synthetic* datasets (cheap annotations: FlyingThings3D, Monkaa), then fine-tune on a little real data (expensive annotations).

Definition E.1 (1D correlation layer *(general form, not written on the slide)*)**.** For feature maps f_L, f_R and maximum displacement D, a correlation layer outputs the volume $c(x, y, d) = \langle f_L(x, y), f_R(x - d, y) \rangle$ for $d = 0, \ldots, D$.

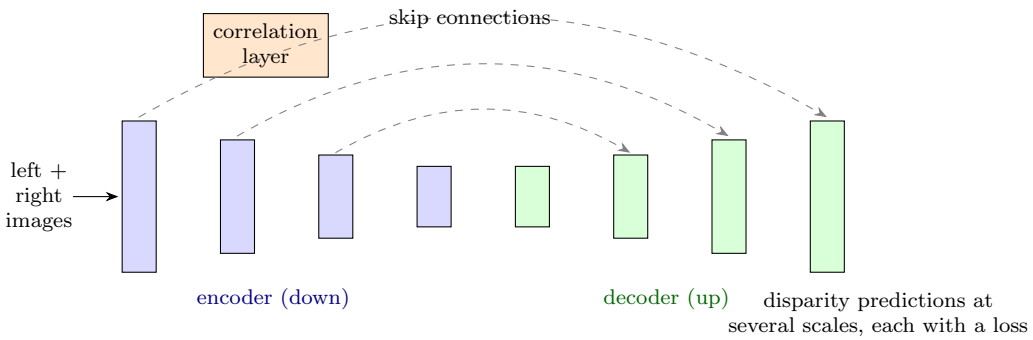


Figure 106: Schematic of a DispNet-style encoder–decoder with skip connections (new schematic, not a reproduction of the slide's network figure).

[PHOTO PLACEHOLDER]
Insert the image from **slide 30**:
3.3cm

DispNet network diagram plus FlyingThings3D and Monkaa sample images (slide 30).

Intuition

Hand-designed block matching has no way to "learn" that a textureless wall continues smoothly. A network trained on millions of synthetic examples sees every ambiguity, and the U-Net structure lets coarse levels settle the global layout while fine levels sharpen the edges. The synthetic→real recipe exists because perfect ground truth is trivial for a renderer and very costly for real scenes.

E.2 GC-Net *(slides 31)*

Key idea (Kendall et al., ICCV 2017): build a *disparity cost volume* and apply *3D convolutions* to it. The learned matching cost $c_\theta(d)$ is converted to a disparity with a differentiable expectation.

Definition E.2 (Soft argmin)**.**

$$\hat{d} = \mathbb{E}[d] = \sum_{d=0}^{d_{\max}} p(d)\, d, \qquad p(d) =_d \big(-c_\theta(d)\big) = \frac{e^{-c_\theta(d)}}{\sum_k e^{-c_\theta(k)}}.$$

Proposition E.3. *(i)* $\hat{d} \in [0, d_{\max}]$ *and* $\hat{d}$ *may be non-integer (sub-pixel). (ii)* $\hat{d}$ *is differentiable in the costs with* $\dfrac{\partial \hat{d}}{\partial c(k)} = -p(k)\,\big(k - \hat{d}\big)$.

Proof. (i) p is a probability vector, so $\hat{d}$ is a convex combination of the integers $0, \dots, d_{\max}$. (ii) $\partial p(d)/\partial c(k) = -p(d)(\delta_{dk} - p(k))$, hence $\partial \hat{d}/\partial c(k) = \sum_d d\,[-p(d)(\delta_{dk} - p(k))] = -p(k)k + p(k)\hat{d}$. □

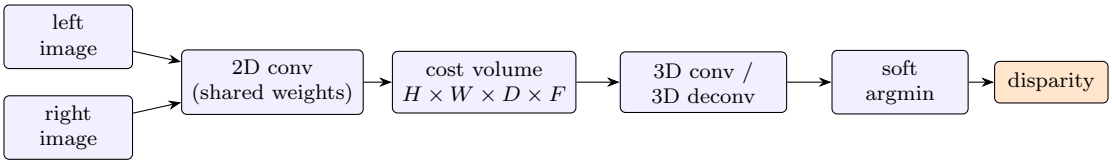


Figure 107: GC-Net pipeline (redrawn from slide 31).

Example E.4 (Soft argmin on two cost profiles, $d = 0, \dots, 4$)**.**

	costs $c(d)$	$p(d)$	$\hat{d}$
unimodal	$(3, 1, 0, 1, 3)$	$(0.027, 0.200, 0.545, 0.200, 0.027)$	2.0
bimodal	$(0, 4, 4, 4, 0)$	$(0.487, 0.009, 0.009, 0.009, 0.487)$	2.0

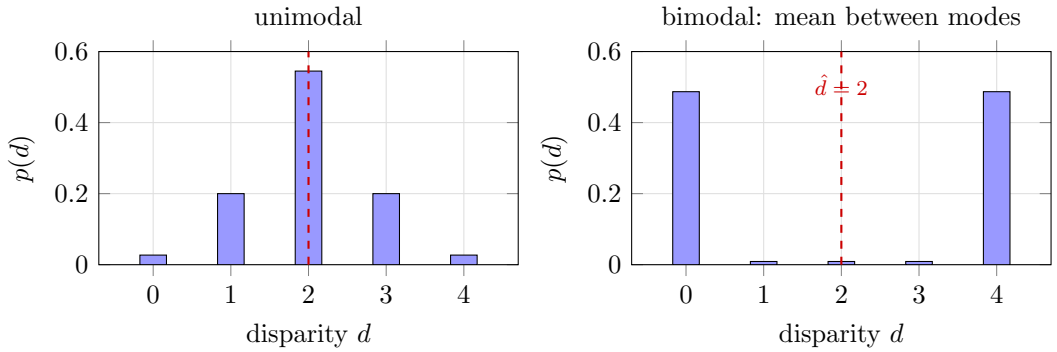


Figure 108: Soft-argmin outputs (red dashed line) for the two profiles of the example. In the bimodal case the expectation is a disparity that neither mode supports (new figure).

Example E.5 (Memory of a 3D volume (illustrative sizes))**.** A volume with $H = 256$, $W = 512$, $D = 48$ (disparities at quarter resolution) and $F = 32$ float32 channels takes $256 \cdot 512 \cdot 48 \cdot 32 \cdot 4 \approx 0.8\,\text{GB}$, before any activations of the 3D convolutions are stored for back-propagation.

Intuition

An ordinary argmin is a hard decision with no gradient. The soft argmin "votes" with weights given by the softmax, which makes the whole network trainable end to end and allows sub-pixel answers. Its weakness is visible in the bimodal row: at an object boundary the cost has two valleys (foreground and background); the expectation averages them and invents a surface in between – the *smoothness bias* that SMD-Nets later address (backup slides). 3D convolutions add context along the disparity axis at the price of memory ("slightly better performance but large memory requirements").

F Stereo Without Calibration *(slides 32–45)*

F.1 The Cost of Calibration *(slides 33)*

Calibration is measured *offline*, with a target, before any image pair is matched. It drifts with temperature, vibration and refocusing; a small rotation error leaves corresponding points on

different rows after rectification. Two arbitrary photographs come with no calibration at all and cannot be rectified. The learned stereo networks of Sec. 4.4 inherit all of this: rectified input and a fixed disparity range. (Calibration is the topic of the next lecture.)

F.2 Limits of Rectification and Local Matching *(slides 34)*

- Forward motion puts the epipole *inside* the image, where planar rectification breaks down (compare the remark after Lemma B.5).
- Large rotations between the views distort the rectified images heavily.
- A wide baseline lowers the depth error (Theorem B.8) but changes the appearance of every patch, so local matching fails.

[PHOTO PLACEHOLDER]
Insert the image from **slide 34**:
4.2cm

Pair of images with dense colour-coded correspondences under strong camera motion, from MASt3R (slide 34). Caption: "Dense correspondences where camera motion degrades visual similarity".

F.3 Pointmaps *(slides 35)*

Definition F.1 (Pointmap)**.** A pointmap $\boldsymbol{X}^{a,1} \in \mathbb{R}^{H\times W\times 3}$ assigns to each pixel $\boldsymbol{x}_s$ of image I_a, $a \in \{1,2\}$, its 3D point $\boldsymbol{X}^{a,1}[\boldsymbol{x}_s] \in \mathbb{R}^3$ expressed in the frame of camera 1 (the second index names the frame). For image 1 with depth z, intrinsics $\mathbf{K}_1$ and augmented pixel ${}_s$:

$$\boldsymbol{X}^{1,1}[\boldsymbol{x}_s] = z\,\mathbf{K}_1^{-1}{}_s.$$

DUSt3R regresses ^{1,1} and ^{2,1} from two images – both in the frame of camera 1 (the hat marks a prediction). Two pixels correspond when their 3D points coincide: *matching becomes a 3D problem.*

Proposition F.2 (Disparity map as a special case)**.** *For a rectified pair with focal length f, principal point (c_x, c_y) and baseline b, a pixel (x, y) with disparity d has pointmap entry* $\boldsymbol{X}^{1,1} = \dfrac{b}{d}\,(x - c_x,\ y - c_y,\ f)^\top$.

Proof. $z = fb/d$ by Theorem B.7, and $\mathbf{K}_1^{-1} = ((x-c_x)/f, (y-c_y)/f, 1)^\top$; multiply. □

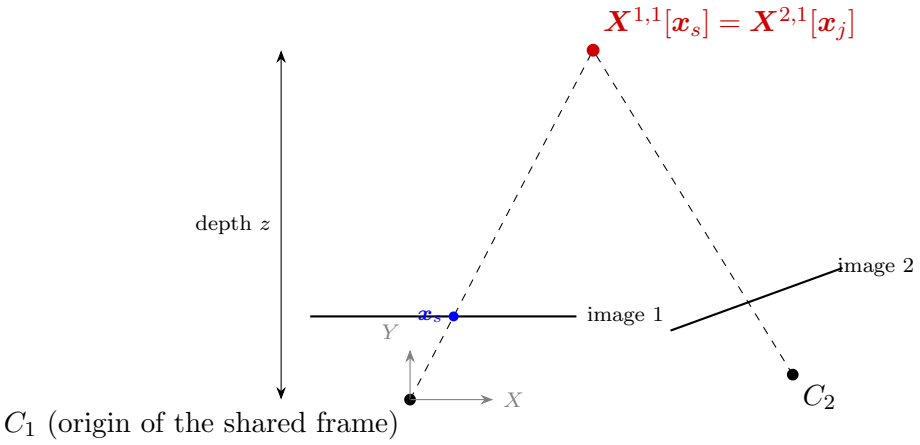


Figure 109: Pointmaps (new figure). Pixel $\boldsymbol{x}_s$ in image 1 and pixel $\boldsymbol{x}_j$ in image 2 are *the same 3D point* expressed in camera 1's frame; this is the correspondence criterion.

Intuition

A disparity map stores one number per pixel and only makes sense along a rectified row. A pointmap stores the whole 3D position in a frame shared by both images, so the notion "these two pixels see the same thing" becomes simply "their 3D points are equal" – no rows, no calibration of the pair needed to state it.

F.4 The DUSt3R Architecture *(slides 36)*

$$\boldsymbol{H}^a = \text{Encoder}(I_a), \quad \boldsymbol{H}'^1, \boldsymbol{H}'^2 = \text{Decoder}(\boldsymbol{H}^1, \boldsymbol{H}^2), \quad {}^{a,1}, C^a = \text{Head}_{3\text{D}}\big([\boldsymbol{H}^a, \boldsymbol{H}'^a]\big).$$

- **Siamese encoder:** one ViT-Large (a vision transformer over image patches) encodes both images with *shared weights* into tokens $\boldsymbol{H}^a$.
- **Two ViT-Base decoders** exchange information through *cross-attention* and output tokens $\boldsymbol{H}'^a$.
- **Heads** regress a pointmap ^{a,1} and a per-pixel confidence C^a from the concatenated tokens.
- **MASt3R** (blue on the slide) extends DUSt3R with local descriptors and fast reciprocal matching.

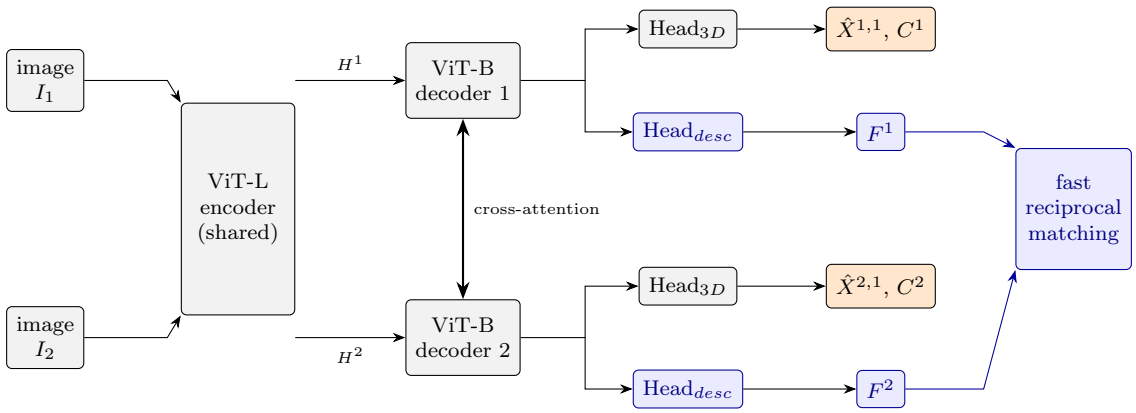


Figure 110: DUSt3R/MASt3R architecture (redrawn from slides 36 and 39; blue = MASt3R additions).

[PHOTO PLACEHOLDER]
Insert the image from **slide 36**:
3.2cm

Photos at the left (input images) and right (resulting geometric matching and feature-based fast matching visuals) of the architecture figure (slide 36).

Intuition

The encoder turns each image into a bag of patch tokens independently; the decoders then let the two images "talk" through cross-attention so that each token can relate to the other view. The heads translate tokens into per-pixel 3D coordinates. Crucially, both outputs are in the *same* frame, which is what makes 3D matching possible.

F.5 MASt3R Network Output *(slides 37)*

Per image, the network outputs three dense maps: a pointmap $\in \mathbb{R}^{H\times W\times 3}$ (with $[i] \in \mathbb{R}^3$), a confidence $C \in \mathbb{R}^{H\times W}$ (with $C[i] \in \mathbb{R}$) and a local descriptor map $F \in \mathbb{R}^{H\times W\times 24}$ (with $F[i] \in \mathbb{R}^{24}$).

[PHOTO PLACEHOLDER]
Insert the image from **slide 37**:
3.0cm

Three 3D-slab sketches: pointmap $\hat{X}$, confidence C and local feature F with a highlighted pixel (slide 37).

F.6 Training Objective and Metric Scale *(slides 38)*

Definition F.3 (Confidence-weighted regression loss)**.** With $\rho, \hat{\rho}$ the mean distances of the valid ground-truth and predicted points to the origin,

$$\ell_{\text{regr}}(a,i) = \left\| \tfrac{1}{\hat{\rho}} \hat{\boldsymbol{X}}^{a,1}[i] - \tfrac{1}{\rho} \boldsymbol{X}^{a,1}[i] \right\|, \qquad \mathcal{L}_{\text{conf}} = \sum_{a\in\{1,2\}} \sum_{i\in\mathcal{V}^a} C^a[i]\, \ell_{\text{regr}}(a,i) - \alpha \log C^a[i],$$

$\mathcal{V}^a$ the pixels of I_a with valid ground truth, $\alpha = 0.2$. Dividing by $\rho, \hat{\rho}$ removes the unknown scale.

Proposition F.4 (Optimal confidence)**.** *For a fixed error $\ell > 0$, the function $C \mapsto C\ell - \alpha \log C$ on $C > 0$ is minimised at $C^* = \alpha/\ell$, with minimum value $\alpha\big(1+\log(\ell/\alpha)\big)$.*

Proof. The derivative $\ell - \alpha/C$ vanishes at $C = \alpha/\ell$, and the second derivative $\alpha/C^2 > 0$ shows it is a minimum. □

Example F.5. With $\alpha = 0.2$: a pixel with error $\ell = 0.1$ gets $C^* = 2$; one with $\ell = 1$ gets $C^* = 0.2$. (The paper's parametrisation of C may constrain its range; the proposition is the unconstrained picture.)

Intuition

Without the $-\alpha \log C$ term the network could "switch off" every hard pixel by shrinking its confidence towards zero and the loss would vanish. The log term charges a price for low confidence, so the best compromise is a confidence *inversely proportional to the expected error*: it learns to say "I am unsure here" only where it really is (sky, reflections, textureless areas).

Note (from the original paper, not from the slides): to my knowledge, for datasets with metric ground truth MASt3R does not apply this normalisation, which is how it learns metric scale; the slides only state the normalisation and later say the metric scale "comes from a learned prior".

F.7 The Matching Head *(slides 39)*

Regressed 3D points are robust to viewpoint changes but imprecise. MASt3R therefore adds a second head per image producing dense local descriptors from the same tokens:

$$F^a = \mathrm{Head}^a_{\mathrm{desc}}\big([\boldsymbol{H}^a, \boldsymbol{H}'^a]\big) \in \mathbb{R}^{H\times W\times 24}, \quad a \in \{1,2\}.$$

It is a two-layer MLP with a GELU activation; the descriptor $F^a[i] \in \mathbb{R}^{24}$ of each pixel is normalised to unit length. It is trained jointly with the 3D head, so the descriptors stay *grounded* in the pointmaps.

F.8 InfoNCE and the Hinge Loss *(slides 40)*

Definition F.6 (Matching loss)**.** With temperature $\tau = 0.07$ and $s_\tau(i,j) = \exp\big(F^1[i]^\top F^2[j]/\tau\big)$,

$$\mathcal{L}_{\mathrm{match}} = -\sum_{(i,j)\in\mathcal{M}} \left(\log \frac{s_\tau(i,j)}{\sum_{i'\in\mathcal{P}^1} s_\tau(i',j)} + \log \frac{s_\tau(i,j)}{\sum_{j'\in\mathcal{P}^2} s_\tau(i,j')} \right).$$

$\mathcal{M}$: ground-truth correspondences (pixel pairs with the same ground-truth 3D point); $\mathcal{P}^1, \mathcal{P}^2$: their pixels in each image. The total loss is $\mathcal{L}_{\mathrm{total}} = \mathcal{L}_{\mathrm{conf}} + \beta\mathcal{L}_{\mathrm{match}}$ with $\beta = 1$.

Proposition F.7 (InfoNCE contains a smooth hinge)**.** *If only one positive and one negative candidate are present, each term equals* $\log\big(1 + e^{-(u_+ - u_-)/\tau}\big)$ *with* $u_\pm$ *the descriptor dot products. This is a smooth version of the hinge loss of Sec. 4.3 (compare* $\tau\log(1 + e^{-\Delta/\tau}) \to \max(0, -\Delta)$ *as* $\tau \to 0$*).*

Proof. $-\log \dfrac{e^{u_+/\tau}}{e^{u_+/\tau} + e^{u_-/\tau}} = \log\big(1 + e^{-(u_+ - u_-)/\tau}\big).$ □

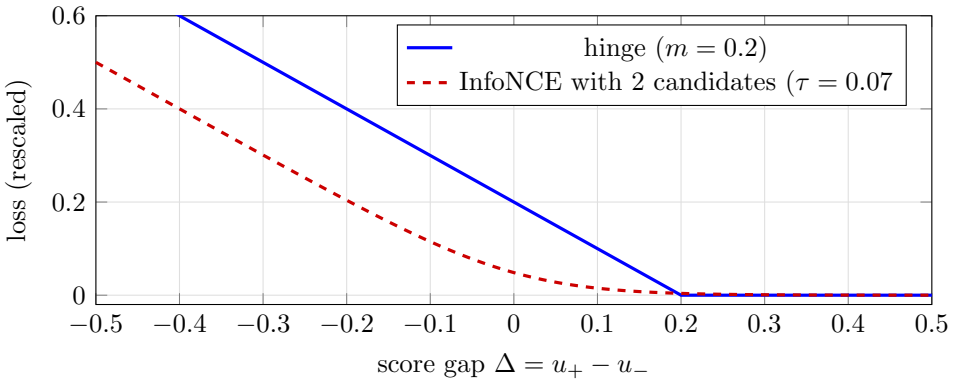


Figure 111: Hinge versus a two-candidate InfoNCE term (new figure; the second is multiplied by τ so the slopes are comparable). InfoNCE never becomes exactly zero: it keeps pushing already-separated pairs apart.

Intuition

The hinge loss compares the match with *one* negative and a fixed margin. InfoNCE is a cross-entropy over *all* candidates: every other matched pixel acts as a negative and only the exact pixel is rewarded. That gives a much richer learning signal per correspondence, and the low temperature makes the descriptor space sharply peaked.

F.9 Effect of 3D Grounding *(slides 41)*

Ablation on the Map-free relocalization benchmark (validation set; estimate the metric pose of a query from a single reference image). All rows take depth from the same monocular depth network.

	model, matching on	VCRE AUC ↑	median error ↓	pose AUC ↑
I	DUSt3R, 3D points	0.704	1.10 m, 9.4°	0.344
II	MASt3R, 3D points	0.732	0.94 m, 3.6°	0.409
III	MASt3R, matching loss only, descriptors	0.744	1.10 m, 10.8°	0.382
IV	MASt3R, descriptors	**0.752**	**0.93 m, 3.0°**	**0.435**

VCRE AUC: area under the curve of the reprojection error of virtual 3D points under the estimated pose, up to 90 px. Pose AUC: area under the curve of pose accuracy.

- Descriptors beat matching on 3D points (II vs. IV: 0.732 → 0.752).
- Without the 3D regression loss, the median rotation error grows from 3.0° to 10.8° (III vs. IV).

Intuition

Descriptors give *precise* matches; the 3D loss gives *geometric sanity*. Row III shows that a descriptor trained alone has decent reprojection scores but poor rotation – the matches are locally sharp yet not globally consistent. Training both together gives the best of each.

F.10 Reciprocal Matching *(slides 42)*

Definition F.8 (Reciprocal (mutual nearest-neighbour) matches)**.**

$$\hat{\mathcal{M}} = \{(i,j) \mid j =_2 (F^1[i]) \text{ and } i =_1 (F^2[j])\}, \qquad {}_2(F^1[i]) = \arg\min_j \|F^2[j] - F^1[i]\|,$$

and ${}_1$ is the same from image 2 to image 1.

Lemma F.9. *$\hat{\mathcal{M}}$ is a partial one-to-one matching: no pixel of either image appears in more than one pair.*

Proof. If $(i,j),(i,j') \in \hat{\mathcal{M}}$ then $j =_2 (F^1[i]) = j'$ (nearest neighbour is a function, ignoring ties); symmetrically for a pixel j appearing with two i's. □

Example F.10 (1D toy descriptors)**.** $F^1 = (0, 1, 5)$ and $F^2 = (0.9, 5.2, 6)$. Then ${}_2$ sends $0 \to 0.9$, $1 \to 0.9$, $5 \to 5.2$, while ${}_1$ sends $0.9 \to 1$, $5.2 \to 5$, $6 \to 5$. Mutual pairs: $(1, 0.9)$ and $(5, 5.2)$. The pair $(0, 0.9)$ is rejected because 0.9 prefers 1.

- It is the left-right consistency test of Sec. 4.2, applied over the whole image instead of one row.
- Without rectification there is no epipolar line: every pixel of image 2 is a candidate, so a naive implementation compares all pixel pairs, $O(W^2H^2)$. For a 512×384 image this is $\approx 3.9 \times 10^{10}$ pairs (illustrative size).

Intuition

It is the same "two witnesses must agree" idea as the left-right test, but without a row to search along. Mutual agreement is a cheap, strong outlier filter; the price is a huge candidate set, which is why MASt3R uses a *fast* matching scheme (the slides name it but do not detail it).

F.11 MASt3R Training Details *(slides 43)*

- **Data:** 14 datasets (indoor, outdoor, synthetic, real, object-centric), 10 with metric ground truth (Habitat, ARKitScenes, BlendedMVS, MegaDepth, Static Scenes 3D, ScanNet++, CO3D-v2, Waymo, Map-free, WildRGB-D, Virtual KITTI 2, Unreal4K, TartanAir and an internal dataset).
- **Amount:** 650k pairs per epoch, equally from all datasets, 35 epochs: 650k × 35 = 22.75 million pairs, batch size 64.
- **Images:** 512 px on the long side, five aspect ratios, random crops and colour jitter.
- **Model:** DUSt3R network (ViT-L encoder, ViT-B decoder), initialised from the public DUSt3R checkpoint.

- **Optimisation:** AdamW, learning rate 10^{-4} with 7 warm-up epochs and cosine decay, weight decay 0.05; 4096 ground-truth correspondences per pair for the matching loss.
- Hardware and training time: not reported in the paper.

F.12 Results *(slides 44)*

Map-free relocalization, test set	
method	VCRE AUC ↑
LoFTR, monocular depth	0.634
DUSt3R	0.697
MASt3R	**0.933**

DTU multi-view stereo, zero-shot (mm)			
method	Acc. ↓	Comp. ↓	Overall ↓
DUSt3R	2.677	0.805	1.741
MASt3R	**0.403**	**0.344**	**0.374**

DTU: Acc. is the mean distance from the reconstruction to the ground truth (structured light), Comp. the reverse, Overall their mean.

[PHOTO PLACEHOLDER]
Insert the image from **slide 44**:
3.4cm

Grid of object photos/reconstructions from DTU (slide 44, right of the tables).

F.13 The Pipeline Revisited *(slides 45)*

Step	**Calibrated stereo**	**MASt3R**
1. Calibration	measured offline, required	not needed to match; focal estimated; known intrinsics for the best poses
2. Rectification	required	not used
3. Disparity	1D search along a row	reciprocal nearest neighbours over the whole image
4. Outliers	left-right consistency test	reciprocity, the same test in 2D
5. Depth	$z = fb/d$, metric from the measured baseline	pointmap, metric scale from a learned prior
6. 3D model	fusion of depth maps	pointmaps, or triangulation of the matches

Given up: a measured metric scale, full resolution (512-pixel windows), low compute, and more than two images per inference. A calibrated rig still wins for real-time and embedded systems and whenever metric scale must be *measured*.

Intuition

Each step of the classical recipe has a counterpart, but the machinery moves from "geometry we measure" to "geometry the network has learned". You gain freedom (arbitrary cameras, wide baselines, forward motion) and lose guarantees (scale, speed, resolution).

Summary

Comparison of the methods covered

Method	Matching signal	Needs calibration + rectification?	Search / output	Main strength / weakness
Block matching (SSD/ZNCC)	hand-crafted patch cost, WTA	yes	1D along row; disparity	simple; noisy, fronto-parallel assumption, border bleeding
Siamese CNN (Zbontar–LeCun)	learned cost (MLP) or cosine of learned features	yes	1D along row; cost volume + WTA/SGM	far less noise; slow if MLP head; ambiguity remains
DispNet	correlation layer, end-to-end regression	yes (fixed disparity range)	disparity at several scales	fast, uses synthetic pre-training; rectified input only
GC-Net	3D conv on cost volume, soft argmin	yes (fixed disparity range)	sub-pixel disparity	better accuracy; large memory (3D volume), smoothness bias
DUSt3R / MASt3R	pointmaps + 24-d descriptors, reciprocal NN	no (focal estimated)	whole image; pointmap, matches, pose	arbitrary views and wide baselines, scale from a prior; no measured scale, lower resolution, compute heavy

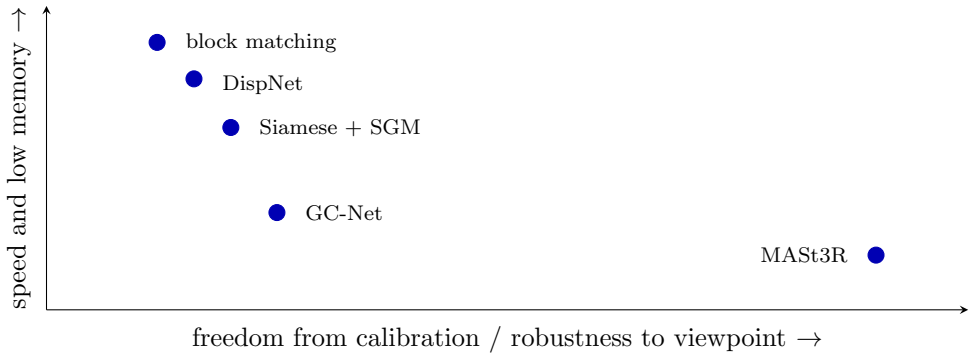


Figure 112: Qualitative placement of the methods on two axes, drawn from the discussion on the slides (calibration/rectification dependence, memory and compute); positions are *not* measured values.

Key take-aways

- **Geometry buys you a 1D search.** With calibration and rectification, matching is along a scanline (a 480× smaller search for VGA) and depth follows from $z = fb/d$.
- **Stereo error is quadratic in depth:** $\delta z \approx z^2 \delta d/(fb)$. A wider baseline helps the arithmetic but makes matching harder.
- **Block matching fails where local similarity fails** (occlusions, slanted surfaces, repetitions, textureless areas); window size trades noise against border bleeding, and the left-right consistency test removes many outliers.
- **Learning replaced the hand-made parts in two steps:** learned matching costs (Siamese nets, hinge loss on triplets), then end-to-end networks (DispNet, GC-Net with cost volumes and soft argmin) – still tied to rectified input and a fixed disparity range.
- **Matching in 3D drops calibration and rectification from the matching step** (DUSt3R/MASt3R: pointmaps, descriptors, reciprocal matching) at the price of measured

metric scale, resolution, compute and the number of views; calibrated rigs still win for real-time and embedded, metrically accurate use.

Part IV

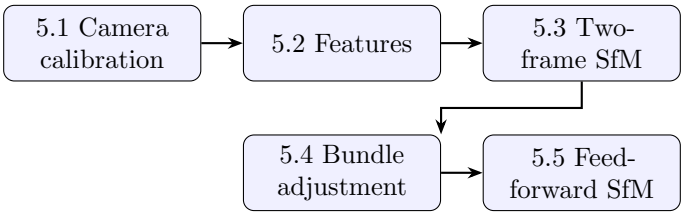
Lecture 5

Abstract

These notes follow the slide deck section by section (slide ranges are given in every heading). Formal material – definitions, lemmas, theorems and proofs – is merged in at the point where the slides introduce the topic; it is taken from the standard references cited on the slides (Zhang 2000, Lowe 1999/2004, Hartley–Zisserman, Triggs et al. 2000, Schönberger–Frahm 2016, the SuperPoint and VGGT papers) and re-derived here. Every formal result is followed by a blue *Intuition* box. Boxes titled *Beyond the slides* mark facts that are not on the slides. Dashed boxes mark photos that should be copied in from the slide deck.

Agenda and Learning Objectives *(slides 1–2)*

The lecture takes us from a single calibrated camera to a full reconstruction pipeline and finally to networks that perform the whole reconstruction in one forward pass.



Learning objectives (slide 2).

1. Derive the epipolar constraint, estimate the essential matrix from correspondences, recover the relative pose up to scale, and triangulate a point from two views.
2. Formulate bundle adjustment as least squares over poses in and 3D points, solve it with Gauss–Newton and the Schur complement, and describe the stages of incremental SfM.
3. Explain how a feed-forward transformer predicts cameras, depth, point maps and tracks, and how bundle adjustment refines them.

Notation used throughout.

Symbol	Meaning
$= (x, y, 1)$	homogeneous pixel coordinates
$\tilde{\mathbf{x}} = \boldsymbol{K}^{-1}$	local ray direction of a pixel (calibrated camera)
$\boldsymbol{R}, \boldsymbol{t}$, $\boldsymbol{T} = [\boldsymbol{R}\|\boldsymbol{t}] \in$	rotation, translation, pose (world $\to$ camera)
$\boldsymbol{t}$	skew-symmetric matrix with $\boldsymbol{tv} = \boldsymbol{t} \times \boldsymbol{v}$
$\mathbb{E} = \boldsymbol{tR}$, $= \boldsymbol{K}_2^{-\mathsf{T}} \mathbb{E} \boldsymbol{K}_1^{-1}$	essential / fundamental matrix
$\boldsymbol{x}_p^w$, $\boldsymbol{x}_{ip}^s$	3D world point p, its observation in image i
$\propto$	equality up to a nonzero scale factor

G Camera Calibration *(slides 3–7)*

G.1 What calibration is and how it is done *(slides 4–7)*

Camera calibration is the process of finding the intrinsic and extrinsic parameters of a camera. Most commonly a *known calibration target* (an image or a checkerboard) is used.

Definition G.1. Pinhole camera with intrinsics and extrinsicspinhole A world point $\boldsymbol{x}^w \in \mathbb{R}^3$ is mapped to a pixel by

$$\lambda = \boldsymbol{K}\,[\boldsymbol{R}\,|\,\boldsymbol{t}] \begin{pmatrix} \boldsymbol{x}^w \\ 1 \end{pmatrix}, \qquad \boldsymbol{K} = \begin{pmatrix} f_x & s & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{pmatrix},$$

where $\boldsymbol{K}$ holds the *intrinsics* (focal lengths f_x, f_y in pixels, skew s, principal point (c_x, c_y)) and $[\boldsymbol{R}|\boldsymbol{t}] \in$ are the *extrinsics*, i.e. the camera pose. Lens *distortion* (e.g. radial, $\boldsymbol{x}_d = \boldsymbol{x}\,(1 + k_1 r^2 + k_2 r^4)$ with $r = \|\boldsymbol{x}\|$ in normalized coordinates) is modelled by extra parameters applied before $\boldsymbol{K}$.

Intuition

$\boldsymbol{K}$ says how the metric ray through the pinhole is converted to pixels; $[\boldsymbol{R}|\boldsymbol{t}]$ says where the camera sits in the world. Calibration is an inverse problem: we look at something whose geometry we know perfectly (the target) and ask which camera would have produced those pixels.

The procedure on slides 4–7 has four stages (Fig. 113):

1. the known target is **captured in different poses** (slide 5);
2. **features** (e.g. corners) on the target are detected in the images (slide 6);
3. a **closed-form solution** initializes all parameters except the distortion parameters;
4. a **non-linear optimization** of all parameters minimizes the reprojection errors (slide 7), jointly over intrinsics and extrinsics (=poses).

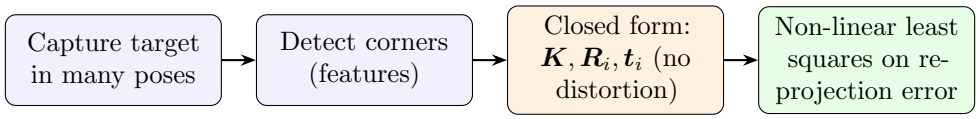


Figure 113: The calibration pipeline of Zhang (2000), redrawn from the description on slides 4–7.

[3.2cm]4A known calibration target held in front of several cameras (multi-camera calibration setup). [3.2cm]5The calibration checkerboard captured in 20 different poses. [3.2cm]6Corners detected on the target (red crosses, blue squares: extracted corners). [3.2cm]7Recovered extrinsic parameters: the target planes in the camera frame after joint optimization (Zhang, TPAMI 2000).

Theorem G.2. *Zhang's constraints from a planezhang Let the target lie in the plane $Z = 0$ of the world, so a target point is $(X, Y, 0)$. The homography $\boldsymbol{H} = [\boldsymbol{h}_1\ \boldsymbol{h}_2\ \boldsymbol{h}_3]$ mapping target coordinates $(X, Y, 1)$ to pixels satisfies $\boldsymbol{H} = \lambda \boldsymbol{K}[\boldsymbol{r}_1\ \boldsymbol{r}_2\ \boldsymbol{t}]$. With $\boldsymbol{B} = \boldsymbol{K}^{-\mathsf{T}}\boldsymbol{K}^{-1}$ (symmetric, positive definite) every view yields the two linear equations in $\boldsymbol{B}$*

$$\boldsymbol{h}_1 \boldsymbol{B}\, \boldsymbol{h}_2 = 0, \qquad \boldsymbol{h}_1 \boldsymbol{B}\, \boldsymbol{h}_1 = \boldsymbol{h}_2 \boldsymbol{B}\, \boldsymbol{h}_2.$$

Proof. For $Z = 0$ the column $\boldsymbol{r}_3$ is multiplied by zero, so $\lambda = \boldsymbol{K}[\boldsymbol{r}_1\ \boldsymbol{r}_2\ \boldsymbol{t}](X, Y, 1)$, which gives $\boldsymbol{H}$. From $\boldsymbol{h}_i = \lambda \boldsymbol{K} \boldsymbol{r}_i$ we get $\boldsymbol{r}_i = \lambda^{-1} \boldsymbol{K}^{-1} \boldsymbol{h}_i$ for $i = 1, 2$. Rotation columns are orthogonal and unit norm: $\boldsymbol{r}_1 \boldsymbol{r}_2 = 0$ gives $\boldsymbol{h}_1 \boldsymbol{K}^{-\mathsf{T}} \boldsymbol{K}^{-1} \boldsymbol{h}_2 = 0$ and $\boldsymbol{r}_1 \boldsymbol{r}_1 = \boldsymbol{r}_2 \boldsymbol{r}_2$ gives the second equation. □

Intuition

A rotation matrix has two "rules": its columns are perpendicular and equally long. A single view of a flat board gives a homography, and those two rules, pushed through the unknown $\boldsymbol{K}$, become two equations on $\boldsymbol{B}$ (which encodes $\boldsymbol{K}$). The board in one pose cannot tell you everything; a tilted board in a different pose tells you something new. That is why the target is captured in different poses.

Counting. $\boldsymbol{B}$ is symmetric, so it has 6 entries; it is defined up to scale, so 5 unknowns (matching the 5 intrinsics of $\boldsymbol{K}$). Each view gives 2 equations, hence ≥ 3 views for a general $\boldsymbol{K}$ (or 2 views if the skew $s = 0$ is assumed).

Algorithm G.3. Zhang's calibrationzhangalg

1. For each view j estimate the homography $\boldsymbol{H}_j$ from the detected corners (DLT, as for homography estimation in lecture 01).
2. Stack the two equations of Thm. **??** for all views as $\boldsymbol{V}\boldsymbol{b} = 0$ and take the right singular vector of the smallest singular value; recover $\boldsymbol{K}$ from $\boldsymbol{b}$.
3. For every view: $\lambda = 1/\|\boldsymbol{K}^{-1}\boldsymbol{h}_1\|$, $\boldsymbol{r}_1 = \lambda\boldsymbol{K}^{-1}\boldsymbol{h}_1$, $\boldsymbol{r}_2 = \lambda\boldsymbol{K}^{-1}\boldsymbol{h}_2$, $\boldsymbol{r}_3 = \boldsymbol{r}_1 \times \boldsymbol{r}_2$, $\boldsymbol{t} = \lambda\boldsymbol{K}^{-1}\boldsymbol{h}_3$. (Project $[\boldsymbol{r}_1\boldsymbol{r}_2\boldsymbol{r}_3]$ to the closest rotation with an SVD.) Distortion is initialized to zero.
4. Refine all parameters jointly by minimizing the reprojection error $\sum_{j,m}\|_{jm} - \hat{\pi}(\boldsymbol{K}, \boldsymbol{k}, \boldsymbol{R}_j, \boldsymbol{t}_j, \boldsymbol{X}_m)\|^2$ with a non-linear least-squares solver (Levenberg–Marquardt).

Intuition

Steps 1–3 are cheap and linear but ignore noise statistics and distortion; they only have to be *good enough to start*. Step 4 is the honest estimator. This "closed form to initialize, non-linear least squares to polish" pattern returns again in bundle adjustment (Section 5.4).

Example G.4. Size of a calibration problem (illustrative numbers)calibsize A 9×6 checkerboard (54 inner corners) seen in 20 views gives $2 \cdot 54 \cdot 20 = 2160$ scalar residuals. The unknowns are 5 intrinsics, 2 distortion coefficients and $6 \cdot 20 = 120$ pose parameters: 127 in total. The problem is hugely over-determined, which is what makes it accurate.

H Feature Detection and Description *(slides 8–20)*

H.1 Point Features *(slides 9–11)*

[3.4cm]9Two views of the same mountain scene; red boxes mark patches that are flat sky (uninformative), an edge, and a distinctive corner-like region (image credit: R. Szeliski).

Definition H.1. Point featurepointfeat A point feature is a pair $(\boldsymbol{k}, \boldsymbol{f})$: a *keypoint* $\boldsymbol{k} = (x, y) \in \mathbb{R}^2$, the pixel location of a salient region, and a *descriptor* $\boldsymbol{f} \in \mathbb{R}^D$ that describes the appearance of the image around $\boldsymbol{k}$. An image yields a set of M features $\{(\boldsymbol{k}_j, \boldsymbol{f}_j)\}_{j=1}^{M}$. Two features $(\boldsymbol{k}, \boldsymbol{f})$ and $(\boldsymbol{k}', \boldsymbol{f}')$ in two images *match* when their descriptors are close, i.e. $\|\boldsymbol{f} - \boldsymbol{f}'\|$ is small.

Intuition

Think of a descriptor as a "fingerprint" of a patch. A flat piece of sky has a fingerprint that matches thousands of other patches (useless), whereas the tip of a mountain has a unique one. Point features describe local salient regions, they can be used to describe and match images from different viewpoints, and they form the basis of the sparse 3D reconstruction methods of this lecture.

The requirements (slide 11): features should be **invariant** to perspective effects and illumination, and the same 3D point should have **similar vectors independent of pose/viewpoint**. Plain RGB/intensity patches do not have this property, so we need something better.

[3.4cm]11Object recognition with local scale-invariant features: SIFT patches on three known products are found in a cluttered scene (Lowe, ICCV 1999).

H.2 Scale Invariant Feature Transform (SIFT) *(slides 12–15)*

Detection (slide 12). SIFT builds a *scale space* by iteratively filtering the image with a Gaussian. Adjacent scales are subtracted, yielding *Difference of Gaussian* (DoG) images; interest points (=blobs) are detected as extrema in the resulting scale space (Fig. 114).

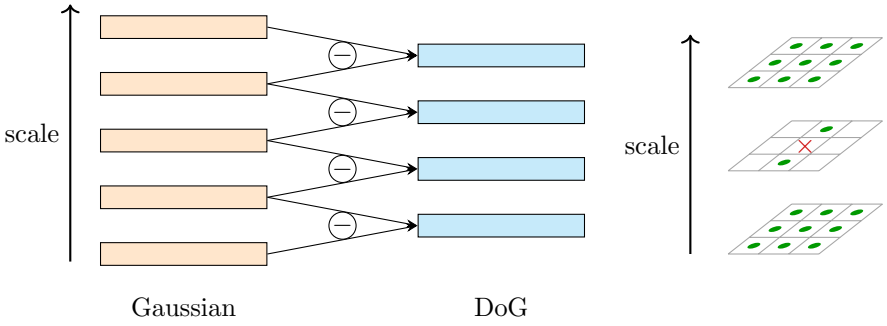


Figure 114: Left: first octave of the Gaussian scale space and the DoG images obtained by subtracting neighbouring scales. Right: a candidate (red cross) is kept only if it is larger or smaller than all its 26 neighbours in space and scale (green).

Proposition H.2. *DoG approximates the scale-normalized Laplaciandog Let* $G(\boldsymbol{x}, \sigma) = \frac{1}{2\pi\sigma^2} e^{-\|\boldsymbol{x}\|^2/2\sigma^2}$. *Then* $\partial G/\partial\sigma = \sigma\nabla^2 G$ *and, for a scale ratio* k *close to 1,*

$$G(\boldsymbol{x}, k\sigma) - G(\boldsymbol{x}, \sigma) \;\approx\; (k-1)\,\sigma^2\nabla^2 G(\boldsymbol{x}, \sigma).$$

Proof. G solves the heat equation $\partial G/\partial(\sigma^2) = \frac{1}{2}\nabla^2 G$, i.e. $\partial G/\partial\sigma = \sigma\nabla^2 G$. A finite difference in σ gives $G(k\sigma) - G(\sigma) \approx (k\sigma - \sigma)\,\partial G/\partial\sigma = (k-1)\sigma^2\nabla^2 G$. □

Intuition

Subtracting two slightly different blurs is a cheap way to compute a blob detector (the Laplacian). Multiplying by σ^2 makes its response comparable across scales: a big blob at a coarse scale gives the same peak value as a small blob at a fine scale, so extrema over (position, scale) find blobs together with their size. That is where scale invariance comes from.

Description (slide 13). SIFT **rotates** the descriptor to **align with the dominant gradient orientation**; **gradient histograms** are computed for local sub-regions; all histograms are concatenated and normalized to form a **128D feature vector**.

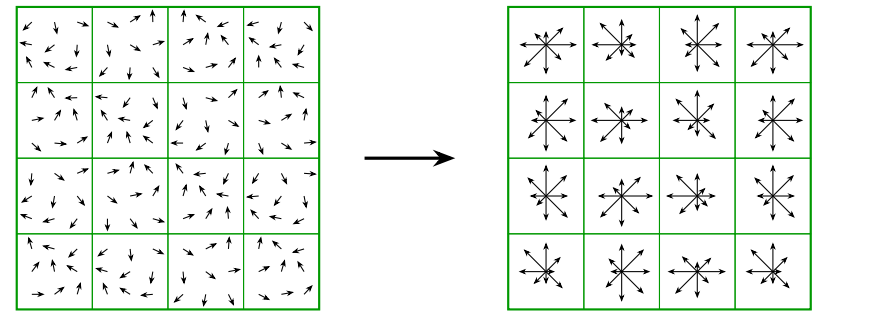


Figure 115: Schematic of the SIFT descriptor: the gradients around the keypoint (after rotating to the dominant orientation) are pooled into an orientation histogram with 8 bins per cell; 4×4 cells give a 128D vector.

Intuition

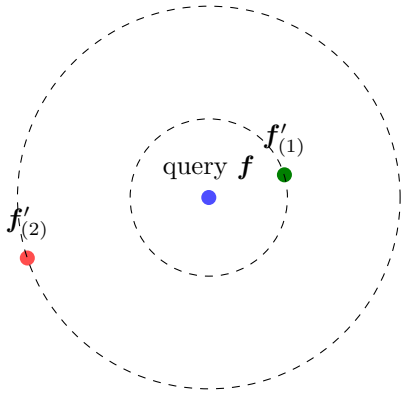
Histograms of gradient *directions* throw away exact pixel positions inside each cell (robust to small shifts) but keep the coarse layout (distinctive). Rotating to the dominant orientation removes the camera's in-plane rotation; normalizing the vector removes affine illumination changes (gain).

In Lowe's implementation, the vector is normalized, entries are clipped at 0.2, and it is re-normalized, which reduces the influence of non-linear illumination effects. The slide shows a 2×2 grid only for drawing convenience.

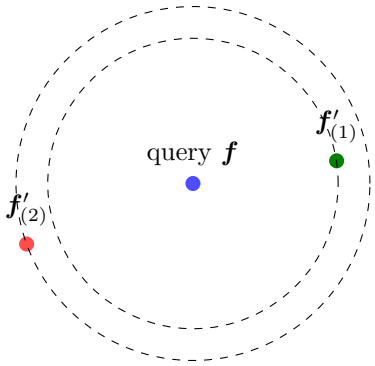
H.3 Matching features *(slides 14–15)*

Many algorithms exist (SIFT, SURF, U-SURF, BRISK, ORB, FAST and learned ones such as SuperPoint). SIFT was a seminal work thanks to its invariance and robustness and enabled large-scale SfM; despite being 20 years old it is still used, e.g. in COLMAP. Correspondences are retrieved by efficient nearest-neighbour search. Ambiguous matches are filtered with the **ratio test**: compute the distance to the closest neighbour divided by the distance to the second closest; a large ratio (> 0.8) indicates that the match might not be correct.

Definition H.3. Lowe's ratio testratio For a query descriptor $\boldsymbol{f}$ with nearest neighbours $\boldsymbol{f}'_{(1)}, \boldsymbol{f}'_{(2)}$ in the other image, accept the match $\boldsymbol{f} \leftrightarrow \boldsymbol{f}'_{(1)}$ iff $\dfrac{\|\boldsymbol{f} - \boldsymbol{f}'_{(1)}\|}{\|\boldsymbol{f} - \boldsymbol{f}'_{(2)}\|} < \tau$, with typically $\tau = 0.8$.



$d_1 = 1.04,\ d_2 = 2.53$, ratio 0.41: **accept**



$d_1 = 1.92,\ d_2 = 2.34$, ratio 0.82: **reject**

Figure 116: Toy example in a 2D descriptor space. If the best match is not clearly better than the runner-up, the match is ambiguous.

Intuition

If I ask "who is my best friend?" and the answer is "person A, and person B is almost as close", I should not trust the answer. The ratio test measures that gap. With ratio 0.41 the closest descriptor is more than twice as close as the next; with ratio 0.82 it is basically a coin flip between two candidates (e.g. a repeated window on a facade).

[3.8cm]15Two images of Notre-Dame with SIFT correspondences (green: correct, red: wrong) (image credits: J. Hays).

H.4 SuperPoint: Learned Interest Points and Descriptors *(slides 16–20)*

Idea (slide 16). A fully convolutional network computes interest points and their descriptors in one forward pass over the full image. Training is **self-supervised** in three stages (Fig. 117): (a) a base detector, MagicPoint, is trained on rendered shapes whose corners are known; (b) it labels real, unlabeled images by Homographic Adaptation; (c) detector and descriptor are trained jointly on image pairs related by a known homography. Runtime: about 13 ms for a 480×640 image on one Titan X GPU (about 70 fps).

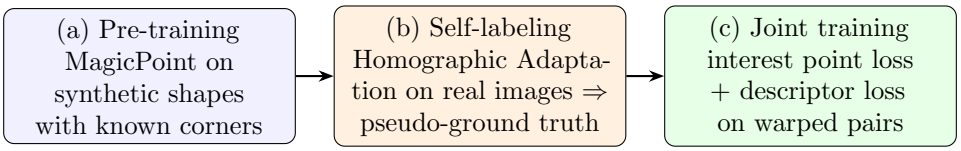


Figure 117: The three SuperPoint training stages (slide 16).

Base detector and training details (slide 17). MagicPoint is trained for 200,000 iterations on *Synthetic Shapes* (quads, triangles, cubes, stars, lines, checkerboards, quad grids), rendered on the fly so that no image is seen twice. Pseudo-ground truth: 80,000 MS-COCO images, grayscale, 240×320, Homographic Adaptation with $N_h = 100$, run twice. Joint training: one random homography per image (milder than in adaptation), cross-entropy for interest points, a hinge loss for descriptors. Optimization: batch size 32, Adam with learning rate 10^{-3}, augmentation (Gaussian noise, motion blur, brightness), descriptor length $D = 256$.

[3.2cm]17Left: Synthetic Shapes with known corners. Right: MagicPoint against FAST, Harris and Shi–Tomasi corners on noisy images.

Homographic Adaptation (slide 18).

Definition H.4. Covariant detector and Homographic Adaptationhomadapt Let f_θ map an image to a heatmap of interest points and let $\tilde{\boldsymbol{H}}(\cdot)$ denote warping an image or heatmap by the homography $\tilde{\boldsymbol{H}}$. f_θ is *covariant* if $f_\theta(\tilde{\boldsymbol{H}}(\boldsymbol{I})) = \tilde{\boldsymbol{H}}(f_\theta(\boldsymbol{I}))$. Homographic Adaptation computes the label heatmap

$$\hat{F}(\boldsymbol{I}) = \frac{1}{N_h} \sum_{i=1}^{N_h} \tilde{\boldsymbol{H}}_i^{-1}\big(f_\theta(\tilde{\boldsymbol{H}}_i(\boldsymbol{I}))\big),$$

with random homographies $\tilde{\boldsymbol{H}}_i$ and $N_h = 100$ warps.

Lemma H.5. *Averaging over warpsavgwarp If f_θ is exactly covariant, then $\hat{F}(\boldsymbol{I}) = f_\theta(\boldsymbol{I})$ for every N_h.*

Proof. Each term equals $\tilde{\boldsymbol{H}}_i^{-1}\big(\tilde{\boldsymbol{H}}_i(f_\theta(\boldsymbol{I}))\big) = f_\theta(\boldsymbol{I})$, and the mean of N_h equal terms is that term. □

Intuition

A real network is only approximately covariant: it finds a corner from one viewpoint and misses it from another. Looking at the image through 100 different warps and voting (after un-warping the heatmaps) keeps the points the detector finds *consistently* and averages out accidental misses, so the labels impose covariance on the training data. The labels are generated on everyday photographs with no annotation of interest points: that is what "self-supervised" means here.

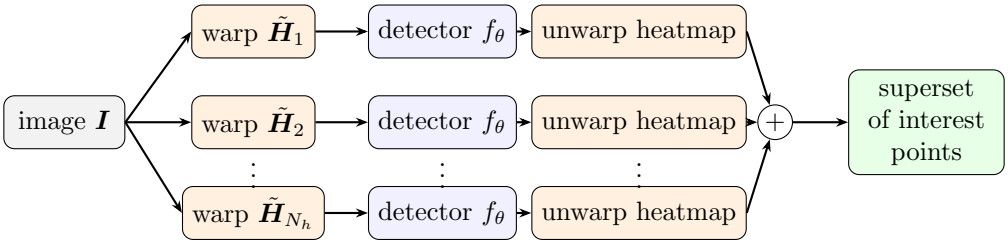


Figure 118: Homographic Adaptation (redrawn from slide 18): detect on warped copies, un-warp, average, and extract the pseudo-ground-truth points.

Architecture (slide 19). The encoder consists of eight 3×3 convolutions and three 2×2 max pools, hence one cell c per 8×8 pixels. Two heads share it (Fig. 119).

- **Keypoint head:** $\boldsymbol{S} \in \mathbb{R}^{H/8 \times W/8 \times 65}$. Each cell is classified into 65 classes: one per pixel and one for "no keypoint" (the 65th class). The loss is the cross-entropy against the pseudo-ground-truth label y_c, averaged over the $HW/64$ cells:

$$\mathcal{L}_p = -\frac{64}{HW} \sum_c \log \frac{\exp \boldsymbol{S}[c, y_c]}{\sum_{j=1}^{65} \exp \boldsymbol{S}[c, j]}.$$

- **Descriptor head:** $\boldsymbol{F} \in \mathbb{R}^{H/8 \times W/8 \times 256}$, one unit-length descriptor $\boldsymbol{f} \in \mathbb{R}^{256}$ per cell, interpolated at each keypoint $\boldsymbol{k}$ (bi-cubic interpolation, L2 normalization).

- **Descriptor loss:** over all pairs of cells of the two images, with $s = 1$ if the homography maps one onto the other,

$$s \max(0,\, 1 - \boldsymbol{f}\boldsymbol{f}') + (1 - s) \max(0,\, \boldsymbol{f}\boldsymbol{f}' - 0.2).$$

Self-supervision through known homography.

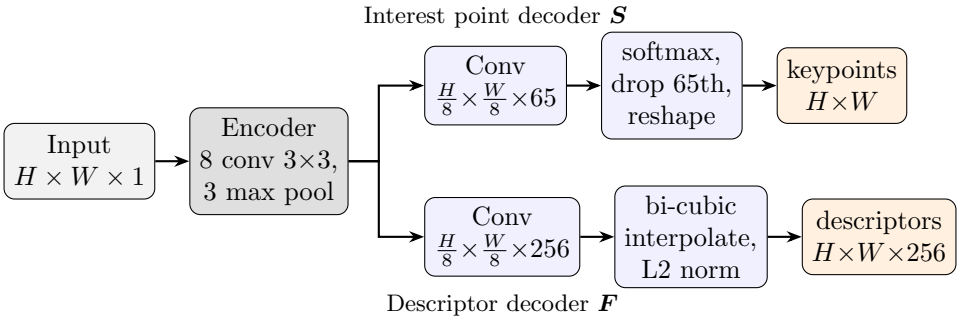


Figure 119: SuperPoint architecture (redrawn from slide 19).

Example H.6. Tensor sizes for a 480×640 imagespsize There are $60 \times 80 = 4800$ cells. The keypoint head outputs a $60 \times 80 \times 65$ tensor; after the softmax the dustbin channel is dropped and the remaining 64 channels are reshaped to 8×8 pixels, giving a full-resolution 480×640 heatmap. The descriptor head outputs $60 \times 80 \times 256$, i.e. a coarse semi-dense grid of descriptors that is interpolated at the detected keypoints. At 13 ms per image this is ≈ 77 frames per second (the slide quotes 70).

Intuition

Instead of asking "is this pixel a corner?" 480×640 times, the network asks each 8×8 cell "which of your 64 pixels is the corner, or is there none?". That is a 65-way classification, solved with an ordinary softmax and cross-entropy. The descriptor loss pulls descriptors of cells that the homography matches towards the same direction (dot product close to 1, margin 1) and pushes non-matching ones apart (dot product below 0.2).

Results on HPatches (slide 20). HPatches has 116 scenes, 57 with large illumination changes and 59 with large viewpoint changes; at most 1000 points per method at 480×640.

Method	ε=1	ε=3	ε=5	Rep.	MLE↓	NN mAP	M. Score
SuperPoint	0.310	**0.684**	**0.829**	0.581	1.158	**0.821**	**0.470**
LIFT	0.284	0.598	0.717	0.449	1.102	0.664	0.315
SIFT	**0.424**	0.676	0.759	0.495	**0.833**	0.694	0.313
ORB	0.150	0.395	0.538	**0.641**	1.157	0.735	0.266

Table 7: HPatches. ε=1, 3, 5: share of image pairs whose estimated homography is correct within ε pixels; Rep.: detector repeatability; MLE: mean localization error in pixels (lower is better); NN mAP: mean average precision of nearest-neighbour descriptor matching; M. Score: matching score.

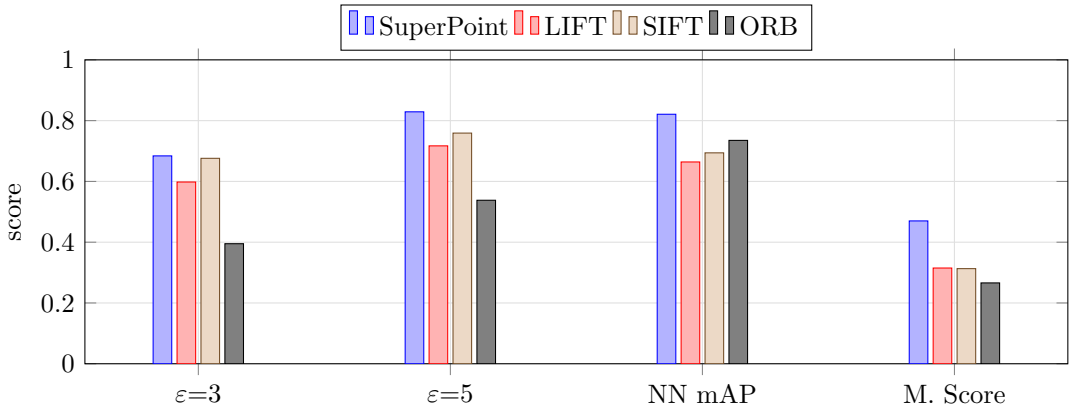


Figure 120: Selected HPatches metrics from the table (higher is better).

Intuition

SuperPoint wins on the metrics that depend on *descriptors* and on how well matches are spread across the image (homography accuracy, NN mAP, matching score), while SIFT is still best at sub-pixel *localization* (lowest MLE) and at ε=1. On slide 20 the fourth row shows a failure of SuperPoint and LIFT under an extreme in-plane rotation, a case absent from their training data: a learned method is only as invariant as its training augmentations.

[3.8cm]20Qualitative matches of SuperPoint, LIFT, SIFT and ORB on six HPatches pairs (green: matches; the fourth row shows an extreme in-plane rotation).

I Two-Frame Structure-from-Motion *(slides 21–35)*

I.1 Epipolar Geometry *(slides 22–23)*

Goal: recovery of camera pose (and 3D structure) from image correspondences. The required relationships are described by the two-view *epipolar geometry*.

- Let $\boldsymbol{R}$ and $\boldsymbol{t}$ denote the relative pose between two perspective cameras.
- A 3D point $\boldsymbol{x}$ is projected to pixel $_1$ in image 1 and to pixel $_2$ in image 2.
- The 3D point $\boldsymbol{x}$ and the two camera centers span the **epipolar plane**.
- The correspondence of $_1$ in image 2 must lie on the **epipolar line** $\tilde{\boldsymbol{l}}_2$ in image 2.
- All epipolar lines pass through the **epipole**.

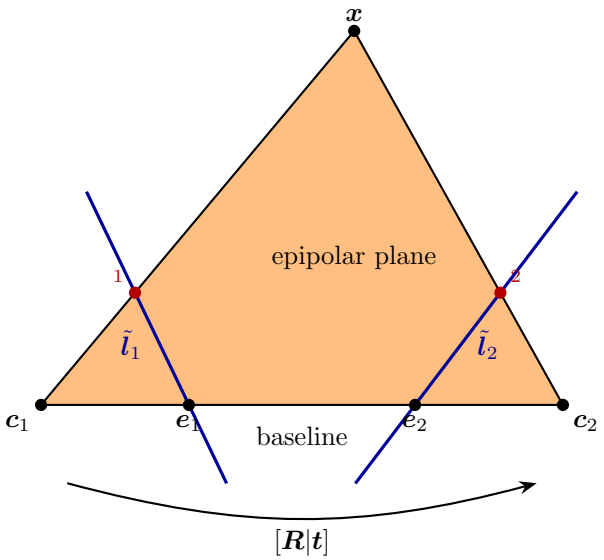


Figure 121: Epipolar geometry (redrawn from slide 22). The image planes are drawn edge-on: their intersection with the epipolar plane is the epipolar line.

Definition I.1. Epipolar plane, lines and epipolesepidef For a world point $\boldsymbol{x}$ and camera centers $\boldsymbol{c}_1, \boldsymbol{c}_2$, the *epipolar plane* is the plane through $\boldsymbol{x}, \boldsymbol{c}_1, \boldsymbol{c}_2$. The *baseline* is the line $\boldsymbol{c}_1\boldsymbol{c}_2$. The *epipole* $\boldsymbol{e}_i$ in image i is the image of the other camera's center. The *epipolar line* $\tilde{\boldsymbol{l}}_2$ of a pixel $_1$ is the intersection of the epipolar plane with image 2.

Intuition

If I see a point in camera 1, I know it lies somewhere on the ray through $\boldsymbol{c}_1$ and $_1$, but not how far. Camera 2 sees that whole ray as a *line*. So the 2D search for a match shrinks to a 1D search along the epipolar line (slide 30 shows this on real images). Because every ray passes through $\boldsymbol{c}_1$, all the epipolar lines in image 2 pass through the projection of $\boldsymbol{c}_1$: the epipole.

Derivation of the epipolar constraint (slide 23). Let $\boldsymbol{K}_i$ be the calibration matrix of camera i, and $\tilde{\mathbf{x}}_i = \boldsymbol{K}_i^{-1}{}_i$ the local ray direction of pixel $_i$ in camera i.

Theorem I.2. *Epipolar constraintepi Let* $\boldsymbol{x}_2 = \boldsymbol{R}\boldsymbol{x}_1 + \boldsymbol{t}$ *relate the coordinates of a 3D point in the two camera frames, and let* $\mathbb{E} = \boldsymbol{t}\boldsymbol{R}$ *be the* essential matrix. *Then*

$$\tilde{\mathbf{x}}_2 \, \mathbb{E} \, \tilde{\mathbf{x}}_1 = 0.$$

Proof. A point's camera coordinates are proportional to its ray direction: $\boldsymbol{x}_i \propto \tilde{\mathbf{x}}_i$, say $\boldsymbol{x}_1 = z_1\tilde{\mathbf{x}}_1$, $\boldsymbol{x}_2 = z_2\tilde{\mathbf{x}}_2$. Then $z_2\tilde{\mathbf{x}}_2 = z_1\boldsymbol{R}\tilde{\mathbf{x}}_1 + \boldsymbol{t}$, so $\tilde{\mathbf{x}}_2 \propto \boldsymbol{R}\tilde{\mathbf{x}}_1 + s\,\boldsymbol{t}$ for some scalar s (slide: $\boldsymbol{x}_2 \propto \boldsymbol{R}\tilde{\mathbf{x}}_1 + s\boldsymbol{t}$). Taking the cross product with $\boldsymbol{t}$ (i.e. multiplying by $\boldsymbol{t}$, which kills $\boldsymbol{t}$) gives $\boldsymbol{t}\tilde{\mathbf{x}}_2 \propto \boldsymbol{t}\boldsymbol{R}\tilde{\mathbf{x}}_1$. Taking the dot product with $\tilde{\mathbf{x}}_2$: the left side is $\tilde{\mathbf{x}}_2\boldsymbol{t}\tilde{\mathbf{x}}_2 = 0$ because $\boldsymbol{t}$ is skew-symmetric ($\boldsymbol{v}\boldsymbol{t}\boldsymbol{v} = \boldsymbol{v}\cdot(\boldsymbol{t}\times\boldsymbol{v}) = 0$ by the triple product). Hence $\tilde{\mathbf{x}}_2\boldsymbol{t}\boldsymbol{R}\tilde{\mathbf{x}}_1 = 0$. □

Intuition

The three vectors $\tilde{\mathbf{x}}_2$, $\boldsymbol{R}\tilde{\mathbf{x}}_1$ and $\boldsymbol{t}$ all lie in the epipolar plane, so they are linearly dependent; their scalar triple product, $\tilde{\mathbf{x}}_2 \cdot (\boldsymbol{t} \times \boldsymbol{R}\tilde{\mathbf{x}}_1)$, vanishes. That is the entire constraint, written as a bilinear form $\tilde{\mathbf{x}}_2\mathbb{E}\,\tilde{\mathbf{x}}_1 = 0$. It does not involve depth: we can test correspondences before knowing any 3D structure.

Example I.3. The rectified stereo pairrectified Take $\boldsymbol{R} = \boldsymbol{I}$ and $\boldsymbol{t} = (1, 0, 0)$ (a pure sideways shift of one unit). Then $\mathbb{E} = \boldsymbol{t} = \left(\begin{smallmatrix} 0 & 0 & 0 \\ 0 & 0 & -1 \\ 0 & 1 & 0 \end{smallmatrix}\right)$ and for $\tilde{\mathbf{x}}_i = (u_i, v_i, 1)$: $\mathbb{E}\tilde{\mathbf{x}}_1 = (0, -1, v_1)$, so $\tilde{\mathbf{x}}_2\mathbb{E}\tilde{\mathbf{x}}_1 = v_1 - v_2 = 0$: corresponding points have the **same row**, i.e. epipolar lines are horizontal. This is the stereo setting of lecture 04. With $\boldsymbol{X} = (X, Y, Z)$ in camera 1, $u_1 = X/Z$ and $u_2 = (X + 1)/Z$, so $u_2 - u_1 = 1/Z$. If $u_1 = 0.1$ and $u_2 = 0.3$, then $Z = 5$ (in units of the baseline).

I.2 Structure of the Essential Matrix *(slides 24)*

Proposition I.4. *Properties of* $\mathbb{E}$*eprops Let* $\mathbb{E} = \boldsymbol{t}\boldsymbol{R}$ *with* $\boldsymbol{R} \in$ *and* $\boldsymbol{t} \neq 0$. *Then*

(i) $\mathbb{E}\mathbb{E} = \boldsymbol{t}\boldsymbol{t} = \|\boldsymbol{t}\|^2\boldsymbol{I} - \boldsymbol{t}\boldsymbol{t}$*, hence the singular values of* $\mathbb{E}$ *are* $(\|\boldsymbol{t}\|, \|\boldsymbol{t}\|, 0)$ *and* $\mathbb{E}$ *has rank 2;*

(ii) $\boldsymbol{t}\mathbb{E} = 0$ *(left null vector* $\boldsymbol{t} \propto \boldsymbol{e}_2$*) and* $\mathbb{E}\,\boldsymbol{R}\boldsymbol{t} = 0$ *(right null vector* $\boldsymbol{R}\boldsymbol{t} \propto \boldsymbol{e}_1$*): each epipole is the image of the other camera's center;*

(iii) $\mathbb{E}$ *has 5 degrees of freedom: 3 for* $\boldsymbol{R}$ *and 3 for* $\boldsymbol{t}$*, minus one for the scale;*

(iv) (scale ambiguity) scaling the scene and $\boldsymbol{t}$ *by the same* α *leaves every image point unchanged, since* $\alpha\boldsymbol{x}_2 = \boldsymbol{R}(\alpha\boldsymbol{x}_1) + \alpha\boldsymbol{t}$*. Hence* $\|\boldsymbol{t}\|$ *cannot be observed and* $\mathbb{E}$ *is defined only up to scale, sign included.*

Proof. (i) $\mathbb{E}\mathbb{E} = \boldsymbol{t}\boldsymbol{R}\boldsymbol{R}\boldsymbol{t} = \boldsymbol{t}\boldsymbol{t} = \|\boldsymbol{t}\|^2\boldsymbol{I} - \boldsymbol{t}\boldsymbol{t}$, which has eigenvalues $\|\boldsymbol{t}\|^2$ (twice, on $\boldsymbol{t}^\perp$) and 0 (along $\boldsymbol{t}$). (ii) $\boldsymbol{t}\boldsymbol{t} = -(\boldsymbol{t}\boldsymbol{t}) = 0$, and $\mathbb{E}\boldsymbol{R}\boldsymbol{t} = \boldsymbol{t}\boldsymbol{t} = 0$. The camera-1 center in camera-2 coordinates is $\boldsymbol{R}\cdot\boldsymbol{0} + \boldsymbol{t} = \boldsymbol{t}$, so $\boldsymbol{e}_2 \propto \boldsymbol{t}$; the camera-2 center in camera-1 coordinates is $-\boldsymbol{R}\boldsymbol{t}$, so $\boldsymbol{e}_1 \propto \boldsymbol{R}\boldsymbol{t}$. (iii) and (iv) follow from counting and from the displayed identity. □

Intuition

Rank 2 is the algebraic signature of "all epipolar lines pass through one point": the epipole is the null direction. The scale ambiguity is the same effect that makes a photo of a model train indistinguishable from a photo of a real train: a single moving camera can recover the *shape* of the scene and its *motion direction*, never the absolute size, unless something metric (a known length, an IMU, a stereo baseline) is available.

Theorem I.5. *Characterization of essential matricesechar A real 3×3 matrix is an essential matrix (up to scale) iff its SVD has the form $\boldsymbol{U}(\sigma,\sigma,0)\boldsymbol{V}$ with $\sigma>0$.*

Proof. ($\Rightarrow$) is Prop. **??**(i). ($\Leftarrow$) Normalize $\sigma=1$ and choose $\boldsymbol{U},\boldsymbol{V}$ with $\det=+1$ (flip the sign of the third column if needed: this does not change the matrix because $\sigma_3=0$). Theorem **??** below shows $\boldsymbol{U}(1,1,0)\boldsymbol{V}=\boldsymbol{u}_3\boldsymbol{R}$ with $\boldsymbol{R}=\boldsymbol{U}\boldsymbol{W}\boldsymbol{V}\in$. □

I.3 Estimating the Essential Matrix: the 8-point algorithm *(slides 25)*

Each correspondence $\tilde{\mathbf{x}}_1=(x_1,y_1,1)$, $\tilde{\mathbf{x}}_2=(x_2,y_2,1)$ gives one *linear* equation in the nine entries of $\mathbb{E}$. Stacking the rows of a matrix into a vector,

$$\tilde{\mathbf{x}}_2\mathbb{E}\tilde{\mathbf{x}}_1=\big(\tilde{\mathbf{x}}_2\mathbb{E}\tilde{\mathbf{x}}_1\big)=\big(\tilde{\mathbf{x}}_1\otimes\tilde{\mathbf{x}}_2\big)(\mathbb{E})=\boldsymbol{a}(\mathbb{E})=0,$$

$$\boldsymbol{a}=(x_2x_1,\ x_2y_1,\ x_2,\ y_2x_1,\ y_2y_1,\ y_2,\ x_1,\ y_1,\ 1),\qquad (\mathbb{E})=(e_{11},e_{12},\dots,e_{33}).$$

Algorithm I.6. Normalized 8-point algorithmeightpt

1. (Normalization) Whiten the observations to zero mean and unit variance, because products of two measurements amplify noise unevenly (Hartley 1997).
2. Stack the rows $\boldsymbol{a}_i$ of $N\geq8$ correspondences into $\boldsymbol{A}\in\mathbb{R}^{N\times9}$ and solve $\min\|\boldsymbol{A}(\mathbb{E})\|$ s.t. $\|(\mathbb{E})\|=1$: the solution is the last right singular vector of $\boldsymbol{A}$.
3. (Projection) The estimate is not an essential matrix when there is noise. Take its SVD $\boldsymbol{U}(\sigma_1,\sigma_2,\sigma_3)\boldsymbol{V}$; the closest essential matrix in the Frobenius norm has singular values $(\bar\sigma,\bar\sigma,0)$ with $\bar\sigma=(\sigma_1+\sigma_2)/2$. As the scale is free, take $\mathbb{E}=\boldsymbol{U}(1,1,0)\boldsymbol{V}$.
4. (Undo the normalization.)

Intuition

Eight equations for nine unknowns defined up to scale: that is the minimal linear count. The $\boldsymbol{A}$-matrix is built so that $\boldsymbol{A}(\mathbb{E})$ lists the "epipolar errors" of all correspondences, and the last singular vector is the $\mathbb{E}$ that makes their sum of squares smallest. Step 3 then repairs the answer: the noisy $\mathbb{E}$ has three unequal singular values, so we average the top two and zero the third (which restores rank 2 and equal non-zero singular values).

Lemma I.7. *Projection onto essential matriceseproj Let $\boldsymbol{M}=\boldsymbol{U}(\sigma_1,\sigma_2,\sigma_3)\boldsymbol{V}$ with $\sigma_1\geq\sigma_2\geq\sigma_3\geq0$. Among matrices of the form $\boldsymbol{U}'(\sigma,\sigma,0)\boldsymbol{V}'$ the one closest to $\boldsymbol{M}$ in the Frobenius norm is $\boldsymbol{U}(\bar\sigma,\bar\sigma,0)\boldsymbol{V}$ with $\bar\sigma=(\sigma_1+\sigma_2)/2$.*

Sketch. The Frobenius distance between matrices with singular values $\boldsymbol{\sigma}$ and $\boldsymbol{\sigma}'$ (sorted) is minimized by sharing the singular vectors and is then $\|\boldsymbol{\sigma}-\boldsymbol{\sigma}'\|_2$ (Mirsky/von Neumann). Minimizing $(\sigma_1-s)^2+(\sigma_2-s)^2+\sigma_3^2$ over s gives $s=\bar\sigma$. □

Intuition

It is just "the average is the best single number to replace two numbers in the least-squares sense".

5-point solver. The essential matrix has 5 degrees of freedom, hence 5 correspondences suffice (Nistér 2004). This minimal solver is what runs inside RANSAC in COLMAP and OpenCV.

Minimal solvers are used inside RANSAC because the number of samples needed to hit an all-inlier set grows quickly with the sample size: with an inlier ratio $w=0.5$, the probability that

a random sample of 8 is all inliers is $w^8 \approx 0.4\%$ while for 5 it is $w^5 \approx 3.1\%$, so about $8\times$ fewer iterations are needed.

I.4 Decomposing the Essential Matrix *(slides 26)*

Write $\mathbb{E} = \boldsymbol{U}(1,1,0)\boldsymbol{V}$, where $\boldsymbol{u}_k, \boldsymbol{v}_k$ denote the k-th columns of $\boldsymbol{U}, \boldsymbol{V}$ with $\det \boldsymbol{U} = \det \boldsymbol{V} = +1$. Let

$$\boldsymbol{W} = \begin{pmatrix} 0 & -1 & 0 \\ 1 & 0 & 0 \\ 0 & 0 & 1 \end{pmatrix}.$$

Theorem I.8. *Four-fold ambiguitydecomp* $\mathbb{E} = \boldsymbol{t}\boldsymbol{R}$ *has exactly the four solutions (all up to scale)*

$$\boldsymbol{R} \in \{\boldsymbol{U}\boldsymbol{W}\boldsymbol{V},\ \boldsymbol{U}\boldsymbol{W}\boldsymbol{V}\}, \qquad \boldsymbol{t} \in \{+\boldsymbol{u}_3,\ -\boldsymbol{u}_3\}, \qquad \|\boldsymbol{t}\| = 1.$$

Proof. For a unit vector $\boldsymbol{u}_3 = \boldsymbol{U}\boldsymbol{e}_3$ and $\det \boldsymbol{U} = 1$ we have $\boldsymbol{u}_3 = \boldsymbol{U}\boldsymbol{e}_3\boldsymbol{U}$, and $\boldsymbol{e}_3 = (1,1,0)\boldsymbol{W}$ (direct multiplication). Hence $\boldsymbol{u}_3 = \boldsymbol{U}(1,1,0)\boldsymbol{W}\boldsymbol{U}$. Then

$$\boldsymbol{u}_3\,(\boldsymbol{U}\boldsymbol{W}\boldsymbol{V}) = \boldsymbol{U}(1,1,0)\boldsymbol{W}\boldsymbol{W}\boldsymbol{V} = \mathbb{E}.$$

Moreover, $\boldsymbol{W}^2 = (-1,-1,1)$ gives $\boldsymbol{u}_3\,(\boldsymbol{U}\boldsymbol{W}\boldsymbol{V}) = \boldsymbol{U}(1,1,0)\boldsymbol{W}^2\boldsymbol{V} = -\mathbb{E}$, i.e. $-\boldsymbol{u}_3(\boldsymbol{U}\boldsymbol{W}\boldsymbol{V}) = \mathbb{E}$. So $(\boldsymbol{U}\boldsymbol{W}\boldsymbol{V}, +\boldsymbol{u}_3)$ and $(\boldsymbol{U}\boldsymbol{W}\boldsymbol{V}, -\boldsymbol{u}_3)$ solve $\mathbb{E} = \boldsymbol{t}\boldsymbol{R}$ exactly. Since $\mathbb{E}$ and $-\mathbb{E}$ are the same essential matrix (it is only defined up to scale, sign included), the pairs $(\boldsymbol{U}\boldsymbol{W}\boldsymbol{V}, -\boldsymbol{u}_3)$ and $(\boldsymbol{U}\boldsymbol{W}\boldsymbol{V}, +\boldsymbol{u}_3)$ are equally valid. The null space of $\boldsymbol{t}$ is $\operatorname{span}(\boldsymbol{t})$, which forces $\boldsymbol{t} \propto \boldsymbol{u}_3$, so there are no other solutions. Finally $\boldsymbol{R}' = \boldsymbol{U}\boldsymbol{W}\boldsymbol{V} = \big(\boldsymbol{U}(-1,-1,1)\boldsymbol{U}\big)\boldsymbol{R}$ with $\boldsymbol{R} = \boldsymbol{U}\boldsymbol{W}\boldsymbol{V}$: it differs from $\boldsymbol{R}$ by a half turn about $\boldsymbol{u}_3 \propto \boldsymbol{t}$. □

Intuition

Two independent ambiguities hide in $\mathbb{E}$. (1) The *sign* of $\boldsymbol{t}$: $\mathbb{E}$ and $-\mathbb{E}$ are the same constraint, so we cannot tell if the second camera is left or right of the first from the epipolar constraint alone. (2) The *twisted pair*: rotating camera 2 by 180° about the baseline maps every epipolar plane to itself, so it also satisfies the constraint. $2 \times 2 = 4$ candidates; the geometry of the next slide tells us which one is real.

I.5 Geometry of the Four Solutions *(slides 27)*

With the true pose $(\boldsymbol{R}, \boldsymbol{t})$ and the other rotation $\boldsymbol{R}'$, the four candidates place a scene point as drawn in Fig. 122.

- **Baseline reversal** ($\boldsymbol{t} \to -\boldsymbol{t}$, left to right in the slide): moves the center of camera 2, $-\boldsymbol{R}\boldsymbol{t}$, to the other side of camera 1. Every point lands behind both cameras.
- **Twisted pair** ($\boldsymbol{R} \to \boldsymbol{R}'$, top to bottom): $\boldsymbol{R}'^{\mathsf{T}}\boldsymbol{R} = \boldsymbol{U}(-1,-1,1)\boldsymbol{U}$, a half turn about $\boldsymbol{u}_3 \propto \boldsymbol{t}$: camera 2 turns by 180° about the baseline. Each point lands in front of one camera only.

Only $(\boldsymbol{R}, \boldsymbol{t})$ puts the scene points in front of both cameras.

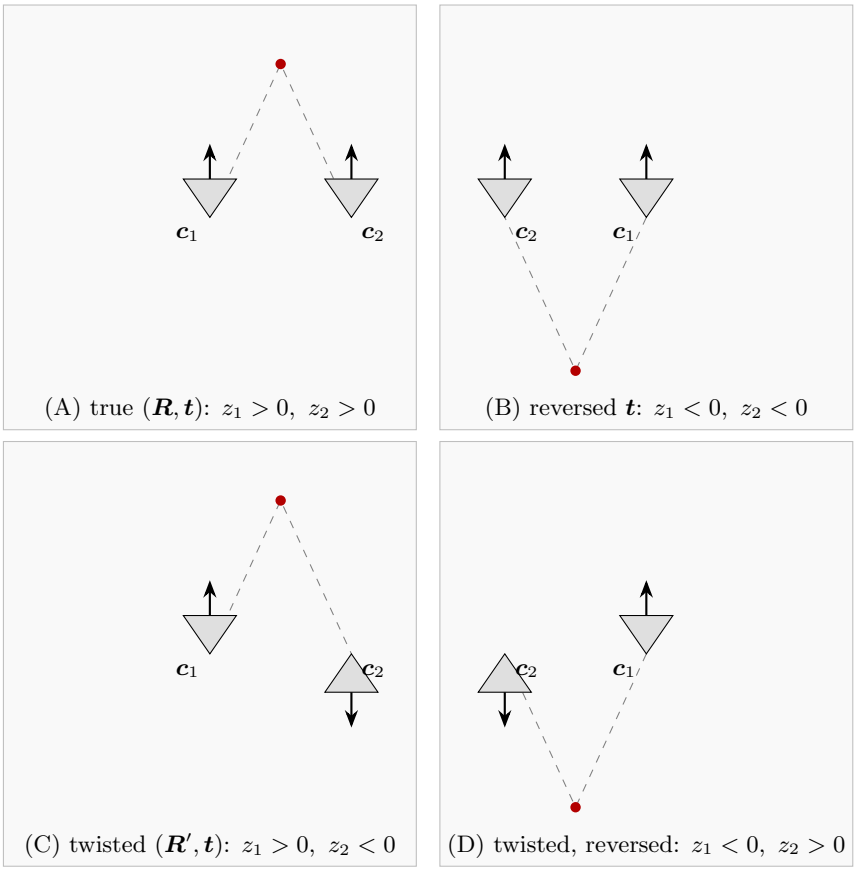


Figure 122: Schematic of the four candidate poses (after slide 27; in 2D, cameras drawn as triangles with an arrow along the viewing direction, red dot: reconstructed point, dashed: reconstructed rays). Only panel A has the point in front of both cameras.

Intuition

The point is the same in all four panels; what changes is where the two cameras "look". If I flip the baseline, I must also look in the opposite direction to see the same image, so the point appears behind me. If I twist camera 2 around the baseline, camera 2 now looks away from the scene: the point is behind it but still in front of camera 1.

I.6 Disambiguation by Cheirality *(slides 28)*

Cheirality: a reconstructed point must lie in front of both cameras. For each correspondence and each candidate $(\boldsymbol{R}_k, \boldsymbol{t}_k)$, the depths z_1, z_2 along the two rays follow from $\boldsymbol{x}_2 = \boldsymbol{R}\boldsymbol{x}_1 + \boldsymbol{t}$ with $\boldsymbol{x}_i = z_i \tilde{\mathbf{x}}_i$:

$$z_2 \tilde{\mathbf{x}}_2 = z_1 \boldsymbol{R}_k \tilde{\mathbf{x}}_1 + \boldsymbol{t}_k \iff \begin{bmatrix} -\boldsymbol{R}_k \tilde{\mathbf{x}}_1 & \tilde{\mathbf{x}}_2 \end{bmatrix} \begin{pmatrix} z_1 \\ z_2 \end{pmatrix} = \boldsymbol{t}_k.$$

Three equations, two unknowns: solve by least squares.

Algorithm I.9. Pose from the essential matrixcheir

1. Decompose $\mathbb{E}$ into the four candidates (Thm. **??**).
2. For each candidate and each correspondence solve the least-squares system above.
3. Keep the candidate with the most correspondences with $z_1 > 0$ and $z_2 > 0$. Count over *all* points: noise and points near infinity can flip individual signs.

Result: $[\boldsymbol{R}|\boldsymbol{t}]$, metric up to one global scale, fixed here by the convention $\|\boldsymbol{t}\| = 1$; later by the gauge of bundle adjustment (5.4) or by a normalization (5.5). In OpenCV: `findEssentialMat`

(5-point solver inside RANSAC), then `recoverPose` (decomposition and this test).

Example I.10. Cheirality in numberscheirnum Use the rectified example: $u_1 = 0.1$, $u_2 = 0.3$, $\boldsymbol{R} = \boldsymbol{I}$. For $\boldsymbol{t}_k = (1, 0, 0)$ the system $\left[-\tilde{\mathbf{x}}_1\ \tilde{\mathbf{x}}_2\right](z_1, z_2) = \boldsymbol{t}_k$ with $\tilde{\mathbf{x}}_1 = (0.1, v, 1)$, $\tilde{\mathbf{x}}_2 = (0.3, v, 1)$ reads $-0.1z_1 + 0.3z_2 = 1$, $-vz_1 + vz_2 = 0$, $-z_1 + z_2 = 0$, so $z_1 = z_2 = z$ and $0.2z = 1$, $z = 5 > 0$: accepted. For $\boldsymbol{t}_k = (-1, 0, 0)$ we get $0.2z = -1$, $z = -5 < 0$: the point would be behind both cameras: rejected.

I.7 Estimating the Epipolar Geometry with Unknown Intrinsics *(slides 29–30)*

If the calibration is unknown, we cannot use the local ray directions $\tilde{\mathbf{x}}_i = \boldsymbol{K}_i^{-1}{}_i$ and only know the pixel coordinates ${}_i$.

Definition I.11. Fundamental matrixfund Substituting $\tilde{\mathbf{x}}_i = \boldsymbol{K}_i^{-1}{}_i$ in the epipolar constraint gives

$$\tilde{\mathbf{x}}_2 \mathbb{E} \tilde{\mathbf{x}}_1 =_2 \underbrace{\boldsymbol{K}_2^{-\mathsf{T}} \mathbb{E} \boldsymbol{K}_1^{-1}}_{} {}_1 = 0.$$

is the *fundamental matrix*. Like $\mathbb{E}$ it is (without noise) of rank two and the epipoles can be recovered in the same way (as null vectors). The epipolar line of ${}_1$ in image 2 is $\tilde{\boldsymbol{l}}_2 =_1$.

The intrinsic parameters cannot be directly determined from alone (a learned model that estimates the field of view directly from the images appears in 5.5).

Intuition

is just $\mathbb{E}$ "dressed" in the unknown camera matrices: the same geometry, but expressed in pixels. It has 7 degrees of freedom (rank 2 and defined up to scale, $9 - 1 - 1$) instead of 5, because two extra parameters are not tied to a rigid motion. It is enough to draw epipolar lines and reject outliers, but you need $\boldsymbol{K}_i$ to convert it into a metric pose.

[4.0cm]30Epipolar lines for two real images based on the estimated epipolar geometry; corresponding points lie on the corresponding epipolar lines.

I.8 Triangulation *(slides 31–35)*

Problem (slide 32). Given the camera intrinsics and extrinsics, how can we recover 3D geometry? With noisy 2D observations ${}_1, {}_2$ the two rays might not intersect; we want the 3D point $\boldsymbol{x}$ that is, in some sense, closest to the two rays (Fig. 123).

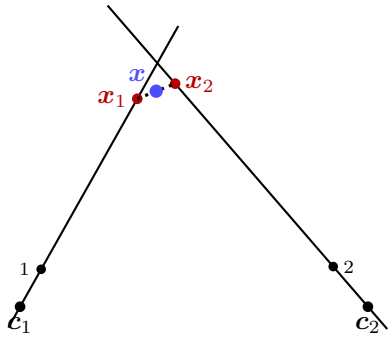


Figure 123: Two noisy rays do not intersect; $\boldsymbol{x}_1, \boldsymbol{x}_2$ are their closest points and $\boldsymbol{x}$ their midpoint (slide 32).

Proposition I.12. *Midpoint triangulationmidpoint Let the rays be $\boldsymbol{c}_i + s_i \boldsymbol{d}_i$ ($\|\boldsymbol{d}_i\| = 1$) and $\boldsymbol{b} = \boldsymbol{c}_2 - \boldsymbol{c}_1$. The closest points have parameters solving*

$$\begin{pmatrix} 1 & -\boldsymbol{d}_1\boldsymbol{d}_2 \\ \boldsymbol{d}_1\boldsymbol{d}_2 & -1 \end{pmatrix} \begin{pmatrix} s_1 \\ s_2 \end{pmatrix} = \begin{pmatrix} \boldsymbol{d}_1\boldsymbol{b} \\ \boldsymbol{d}_2\boldsymbol{b} \end{pmatrix},$$

and the midpoint $\boldsymbol{x} = \frac{1}{2}(\boldsymbol{x}_1 + \boldsymbol{x}_2)$ minimizes the sum of squared distances to the two rays. The system degenerates exactly when $\boldsymbol{d}_1 \parallel \boldsymbol{d}_2$.

Proof. Minimize $\|\boldsymbol{c}_1 + s_1\boldsymbol{d}_1 - \boldsymbol{c}_2 - s_2\boldsymbol{d}_2\|^2$. Setting the derivatives in s_1, s_2 to zero gives $\boldsymbol{d}_1(s_1\boldsymbol{d}_1 - s_2\boldsymbol{d}_2 - \boldsymbol{b}) = 0$ and $\boldsymbol{d}_2(s_1\boldsymbol{d}_1 - s_2\boldsymbol{d}_2 - \boldsymbol{b}) = 0$, which is the stated system. Its determinant is $-1 + (\boldsymbol{d}_1\boldsymbol{d}_2)^2 = -\sin^2\gamma$, where γ is the angle between the rays: zero iff the rays are parallel. □

Intuition

Two rays in 3D generically miss each other by a small gap; the segment joining their closest points is perpendicular to both. Taking its midpoint is the natural compromise. The determinant $\sin^2\gamma$ already hints at the next topic: when the rays are nearly parallel the problem becomes ill-conditioned.

Linear triangulation (DLT, slide 33). Let $\tilde{\boldsymbol{x}}_i^s = \tilde{\boldsymbol{P}}_i \tilde{\boldsymbol{x}}^w$ denote the projection of a 3D world point onto the image of the i-th camera. Both sides are homogeneous: they have the same direction but may differ in magnitude, so we consider the cross product $\bar{\boldsymbol{x}}_i^s \times \tilde{\boldsymbol{P}}_i \tilde{\boldsymbol{x}}^w = 0$. With $\tilde{\boldsymbol{p}}_{ik}$ the k-th row of $\tilde{\boldsymbol{P}}_i$ this gives

$$\underbrace{\begin{pmatrix} x_i^s\,\tilde{\boldsymbol{p}}_{i3} - \tilde{\boldsymbol{p}}_{i1} \\ y_i^s\,\tilde{\boldsymbol{p}}_{i3} - \tilde{\boldsymbol{p}}_{i2} \end{pmatrix}}_{\boldsymbol{A}_i} \tilde{\boldsymbol{x}}^w = 0.$$

Stacking $N \geq 2$ observations yields $\boldsymbol{A}\tilde{\boldsymbol{x}}^w = 0$. As $\tilde{\boldsymbol{x}}^w$ is homogeneous this is a constrained least-squares problem; the solution is the right singular vector of $\boldsymbol{A}$ with the smallest singular value (the Direct Linear Transformation, as for homography estimation in lecture 01).

Intuition

Each view contributes two independent linear equations (the third row of the cross product is a combination of the other two), and the unknown 3D point has 3 degrees of freedom, so two views already give $4 \geq 3$ equations. A single image of a point removes two degrees of freedom (it constrains the point to a line); the second one removes the rest and adds one redundant equation to absorb noise.

Reprojection error minimization (slide 34). While DLT often works well, it is not invariant to projective transformations. The gold standard is to minimize the reprojection error numerically,

$$\bar{\boldsymbol{x}}_w^* = \arg\min_{\bar{\boldsymbol{x}}_w} \sum_{i=1}^{N} \|\bar{\boldsymbol{x}}_i^s(\bar{\boldsymbol{x}}_w) - \bar{\boldsymbol{x}}_i^o\|_2^2,$$

with observations $\bar{\boldsymbol{x}}_i^o$. This allows taking measurement noise appropriately into account. For two views the minimum can also be obtained in closed form as the solution of a sixth degree polynomial (Hartley and Zisserman, Section 12.5).

Triangulation uncertainty (slide 35). Triangulation works differently well depending on the relative camera pose: uncertainty (shaded region) increases as the rays become more parallel. There is a **trade-off**: feature matching is easier for nearby views, but triangulation is harder. (A learned prior can also reconstruct views with little or no overlap, see 5.5.)

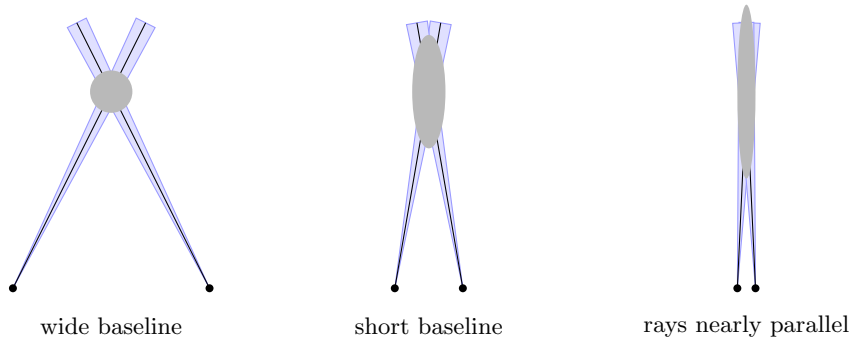


Figure 124: Schematic of triangulation uncertainty (after slide 35): the grey region is where the two noisy rays are consistent with the observations. It grows as the rays become parallel. The ellipses are drawn by hand, not computed.

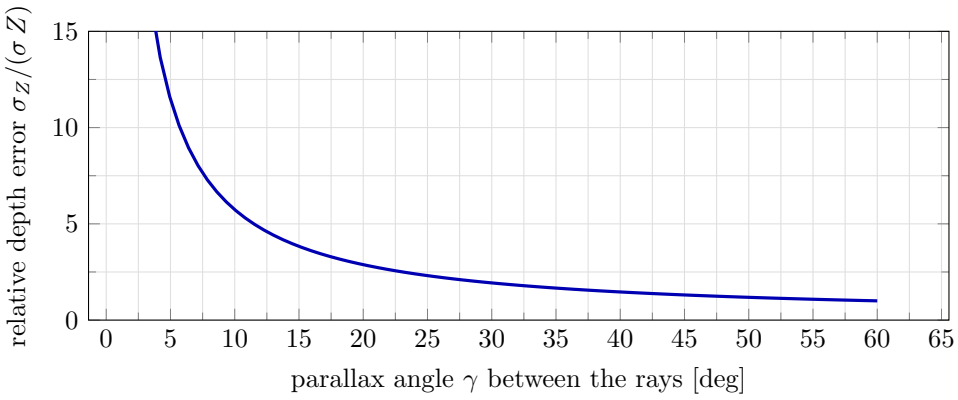


Figure 125: Original illustration of the trade-off. For a point at distance Z seen under angular noise σ per ray, the depth error is, to first order, $\sigma_Z \approx \sigma Z/(2\sin(\gamma/2)) \approx \sigma Z/\gamma$ (small γ). Below about 5° the error explodes; above about 30° it is flat, but matching becomes hard.

Intuition

Hold a finger at arm's length and look with one eye at a time: it jumps against the background. Do the same with a mountain: nothing moves. The jump is the parallax. Parallax is the signal that triangulation measures; if it is tiny, noise swamps it. Meanwhile large viewpoint changes make appearance very different, so matching fails. SfM pipelines therefore choose their image pairs carefully ('enough baseline for triangulation', slide 48).

J Bundle Adjustment *(slides 36–54)*

J.1 Bundle Adjustment *(slides 37–39)*

[3.8cm]37Photo collection of Notre-Dame (left) and the resulting sparse reconstruction with camera frusta (right) (Snavely, Seitz and Szeliski, SIGGRAPH 2006).

Goal: optimize the reprojection errors (distance between an observed feature and the projected 3D point in the image plane) with respect to the camera parameters and the 3D point cloud.

Definition J.1. Bundle adjustmentba Let $\mathcal{T} = \{\boldsymbol{T}_i\}$ with $\boldsymbol{T}_i \in (i = 1..N)$ denote the camera poses; the calibrations $\boldsymbol{K}_i$ are known and fixed. Let $\mathcal{X}_w = \{\boldsymbol{x}_p^w\}$, $\boldsymbol{x}_p^w \in \mathbb{R}^3$ $(p = 1..P)$ be the 3D points in world coordinates and $\mathcal{X}_s = \{\boldsymbol{x}_{ip}^s\}$, $\boldsymbol{x}_{ip}^s \in \mathbb{R}^2$ the image observations in all cameras. Bundle adjustment minimizes the reprojection error of all observations:

$$\mathcal{T}^*, \mathcal{X}_w^* = \arg\min_{\mathcal{T}, \mathcal{X}_w} \sum_{i=1}^{N} \sum_{p=1}^{P} w_{ip} \left\| \boldsymbol{x}_{ip}^s - \pi_i(\boldsymbol{x}_p^w) \right\|_2^2,$$

where w_{ip} indicates whether point p is observed in image i and $\pi_i(\boldsymbol{x}_p^w)$ is the projection of the 3D world point into camera i with its pose $\boldsymbol{T}_i = [\boldsymbol{R}_i|\boldsymbol{t}_i]$ and calibration $\boldsymbol{K}_i$:

$$\pi_i(\boldsymbol{x}_p^w) = \begin{pmatrix} \tilde{x}_{ip}^s / \tilde{w}_{ip}^s \\ \tilde{y}_{ip}^s / \tilde{w}_{ip}^s \end{pmatrix}, \qquad \tilde{\boldsymbol{x}}_{ip}^s = \boldsymbol{K}_i(\boldsymbol{R}_i \boldsymbol{x}_p^w + \boldsymbol{t}_i).$$

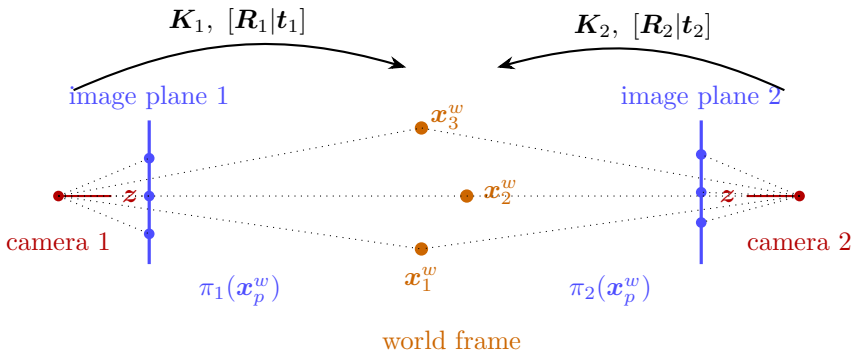


Figure 126: The bundle adjustment setup (slide 39): the 3D points are projected into both cameras by π_i. With the $\boldsymbol{K}_i$ fixed, bundle adjustment optimizes the poses $\mathcal{T}$ and the points $\mathcal{X}_w$ jointly.

Intuition

Imagine each 3D point and each camera attached to the others by elastic bands: every observation is a band pulling the *predicted* pixel towards the *measured* pixel. The name "bundle" refers to the bundles of light rays between each camera center and its points. Adjusting them all at once (poses and points) finds the configuration where the total tension (squared reprojection error) is lowest. This is the maximum-likelihood estimate under independent isotropic Gaussian pixel noise.

J.2 Challenges of Bundle Adjustment *(slides 40)*

- **Initialization.** The energy landscape is highly non-convex: a good initialization is crucial to avoid bad local minima. As initializing all 3D points and cameras jointly is difficult (occlusion, viewpoint, matching outliers), *incremental* bundle adjustment starts from a

carefully selected two-view reconstruction and iteratively adds new images/cameras.

- **Optimization.** With millions of features and thousands of cameras, large-scale bundle adjustment is computationally demanding (cubic complexity in the number of unknowns). Luckily the problem is *sparse* (not every 3D point is observed in every camera), and efficient sparse implementations (e.g. g2o) can be exploited.
- Here the calibrations $\boldsymbol{K}_i$ are fixed; bundle adjustment can also estimate one $\boldsymbol{K}_i$ per camera.
- Predicting cameras and points with a network removes the incremental initialization (5.5).

J.3 Bundle Adjustment as Nonlinear Least Squares *(slides 41)*

Collect the unknowns in $\mathcal{S} = (\boldsymbol{T}_1, \dots, \boldsymbol{T}_N, \boldsymbol{x}_1^w, \dots, \boldsymbol{x}_P^w)$ with $\boldsymbol{T}_i = \left(\begin{smallmatrix} \boldsymbol{R}_i & \boldsymbol{t}_i \\ \boldsymbol{0} & 1 \end{smallmatrix}\right) \in$ and $\boldsymbol{x}_p^w \in \mathbb{R}^3$. Each residual compares the observation with the projection of the point, moved into camera i by its pose:

$$E(\mathcal{S}) = \sum_{i,p} w_{ip} \|\boldsymbol{r}_{ip}\|_2^2, \qquad \boldsymbol{r}_{ip} = \boldsymbol{x}_{ip}^s - \pi\big(\boldsymbol{K}_i(\boldsymbol{R}_i \boldsymbol{x}_p^w + \boldsymbol{t}_i)\big),$$

with the perspective division $\pi(\tilde{x}, \tilde{y}, \tilde{w}) = (\tilde{x}/\tilde{w}, \tilde{y}/\tilde{w})$. Stacking all residuals into $\boldsymbol{r}(\mathcal{S})$ gives $E(\mathcal{S}) = \|\boldsymbol{r}(\mathcal{S})\|_2^2$.

- π_i contains a rotation and a perspective division: there is no closed form, and the minimization iterates from an initialization.
- The poses do not form a vector space, so each iteration must update them on .
- E does not change under a global rotation, translation and scale of all poses and points: these 7 degrees of freedom are fixed by holding one pose and one distance.

Proposition J.2. *Gauge freedomgauge Let $(\boldsymbol{R}_i, \boldsymbol{t}_i, \boldsymbol{x}_p^w)$ be any solution. For any rotation $\boldsymbol{Q}$, translation $\boldsymbol{a}$ and scale $\alpha > 0$, define $\boldsymbol{x}_p'^w = \alpha \boldsymbol{Q} \boldsymbol{x}_p^w + \boldsymbol{a}$, $\boldsymbol{R}_i' = \boldsymbol{R}_i \boldsymbol{Q}$, $\boldsymbol{t}_i' = \alpha \boldsymbol{t}_i - \boldsymbol{R}_i \boldsymbol{Q} \boldsymbol{a}$. Then every projection, and hence E, is unchanged. Consequently E is constant along a 7-dimensional family of solutions (the Hessian has a 7-dimensional null space).*

Proof. $\boldsymbol{R}_i' \boldsymbol{x}_p'^w + \boldsymbol{t}_i' = \alpha \boldsymbol{R}_i \boldsymbol{x}_p^w + \boldsymbol{R}_i \boldsymbol{Q} \boldsymbol{a} + \alpha \boldsymbol{t}_i - \boldsymbol{R}_i \boldsymbol{Q} \boldsymbol{a} = \alpha(\boldsymbol{R}_i \boldsymbol{x}_p^w + \boldsymbol{t}_i)$, so each camera-frame point is only multiplied by α. Since a pixel is a ratio of coordinates, π_i is invariant to scaling its argument, and all residuals are unchanged. The family has $3 + 3 + 1 = 7$ parameters. □

Intuition

The images do not know where the world origin is, which way is "north", or whether the scene is a toy or a real building (the scale ambiguity of Section 5.3). Fixing *one pose* (6 dof: $\boldsymbol{T}_1 = \boldsymbol{I}$) plus *one distance* (1 dof: e.g. $\|\boldsymbol{t}_2\| = 1$) removes the 7 free directions; this is precisely the "gauge" chosen in the initialization of incremental SfM (slide 48).

J.4 Updates in the Tangent Space *(slides 42)*

A pose cannot be updated by addition: $\boldsymbol{T}_i + \Delta \boldsymbol{T}$ is in general not a rigid transformation, since $\boldsymbol{R}\boldsymbol{R} = \boldsymbol{I}$ breaks, and its 12 entries carry only 6 degrees of freedom. The update is instead a 6-vector in the Lie algebra $\cong \mathbb{R}^6$, the tangent space of , mapped onto the group by the exponential map. Points live in $\mathbb{R}^3$ and are added:

$$\boldsymbol{T}_i \leftarrow \mathrm{Exp}(\Delta \boldsymbol{\xi}_i)\, \boldsymbol{T}_i, \qquad \boldsymbol{x}_p^w \leftarrow \boldsymbol{x}_p^w + \Delta \boldsymbol{x}_p, \qquad \mathrm{Exp}(\Delta \boldsymbol{\xi}) = \exp \begin{pmatrix} \Delta \boldsymbol{\omega} & \Delta \boldsymbol{\rho} \\ \boldsymbol{0} & 0 \end{pmatrix}, \quad \Delta \boldsymbol{\xi} = (\Delta \boldsymbol{\rho}, \Delta \boldsymbol{\omega}),$$

where $\Delta\boldsymbol{\omega}$ is an axis-angle rotation, $\Delta\boldsymbol{\rho}$ the translational part and exp the matrix exponential. The update is *minimal and unconstrained*: 6 numbers per pose, and every update stays in .

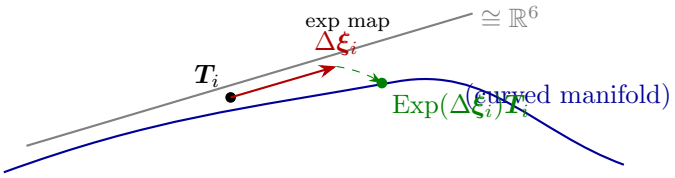


Figure 127: Local update on a curved space: take a step $\Delta\boldsymbol{\xi}_i$ in the flat tangent space, then map it back onto the group with the exponential map (schematic).

For $\theta = \|\Delta\boldsymbol{\omega}\|$ the exponential has a closed form (Rodrigues): $\exp\Delta\boldsymbol{\omega} = \boldsymbol{I} + \frac{\sin\theta}{\theta}\Delta\boldsymbol{\omega} + \frac{1-\cos\theta}{\theta^2}\Delta\boldsymbol{\omega}^2$, and $\mathrm{Exp}(\Delta\boldsymbol{\xi}) = \left(\begin{smallmatrix} \exp\Delta\boldsymbol{\omega} & \boldsymbol{V}\Delta\boldsymbol{\rho} \\ \boldsymbol{0} & 1 \end{smallmatrix}\right)$ with $\boldsymbol{V} = \boldsymbol{I} + \frac{1-\cos\theta}{\theta^2}\Delta\boldsymbol{\omega} + \frac{\theta-\sin\theta}{\theta^3}\Delta\boldsymbol{\omega}^2$.

Intuition

Think of walking on the surface of the Earth: you plan a step on a flat map (the tangent plane) and then "wrap" it back onto the sphere. If you added numbers straight into the 12 matrix entries you would step *off* the surface (the matrix stops being a rotation).

J.5 The Gauss–Newton Algorithm *(slides 43)*

Each residual depends on one pose and one point. Around the current estimate it is linearized in their increments:

$$\boldsymbol{r}_{ip}\big(\mathrm{Exp}(\Delta\boldsymbol{\xi}_i)\boldsymbol{T}_i,\ \boldsymbol{x}_p^w + \Delta\boldsymbol{x}_p\big) \approx \boldsymbol{r}_{ip} + \boldsymbol{J}_{ip}^{\xi}\Delta\boldsymbol{\xi}_i + \boldsymbol{J}_{ip}^{x}\Delta\boldsymbol{x}_p.$$

Lemma J.3. *Jacobians of the reprojection residualjac With $\boldsymbol{x}_{ip}^c = \boldsymbol{R}_i\boldsymbol{x}_p^w + \boldsymbol{t}_i$ and $\boldsymbol{J}_\pi \in \mathbb{R}^{2\times 3}$ the Jacobian of the projection $\boldsymbol{x}^c \mapsto \pi(\boldsymbol{K}_i\boldsymbol{x}^c)$ (so $\boldsymbol{J}_\pi = \big(\partial\pi/\partial\tilde{\boldsymbol{x}}\big)\boldsymbol{K}_i$), both blocks at zero increment are*

$$\boldsymbol{J}_{ip}^{\xi} = -\boldsymbol{J}_\pi\begin{bmatrix}\boldsymbol{I} & -\boldsymbol{x}_{ip}^c\end{bmatrix} \in \mathbb{R}^{2\times 6}, \qquad \boldsymbol{J}_{ip}^{x} = -\boldsymbol{J}_\pi\boldsymbol{R}_i \in \mathbb{R}^{2\times 3}.$$

Proof. For a small $\Delta\boldsymbol{\xi} = (\Delta\boldsymbol{\rho}, \Delta\boldsymbol{\omega})$, $\mathrm{Exp}(\Delta\boldsymbol{\xi}) \approx \boldsymbol{I} + \left(\begin{smallmatrix}\Delta\boldsymbol{\omega} & \Delta\boldsymbol{\rho}\\ \boldsymbol{0} & 0\end{smallmatrix}\right)$, so the camera-frame point becomes $\boldsymbol{x}^c + \Delta\boldsymbol{\omega}\times\boldsymbol{x}^c + \Delta\boldsymbol{\rho} = \boldsymbol{x}^c + \Delta\boldsymbol{\rho} - \boldsymbol{x}^c\Delta\boldsymbol{\omega}$. The residual is $\boldsymbol{x}^s - \pi(\boldsymbol{K}\boldsymbol{x}^c)$, so by the chain rule $\partial\boldsymbol{r}/\partial\Delta\boldsymbol{\xi} = -\boldsymbol{J}_\pi[\boldsymbol{I}\ \ -\boldsymbol{x}^c]$. A change $\Delta\boldsymbol{x}$ of the world point changes $\boldsymbol{x}^c$ by $\boldsymbol{R}_i\Delta\boldsymbol{x}$, giving $-\boldsymbol{J}_\pi\boldsymbol{R}_i$. □

Intuition

The first block says: moving the camera by $\Delta\boldsymbol{\rho}$ is the same as moving the point by $\Delta\boldsymbol{\rho}$ in the camera frame, while rotating by $\Delta\boldsymbol{\omega}$ moves the point sideways by an amount proportional to its distance ($\Delta\boldsymbol{\omega}\times\boldsymbol{x}^c$): far points are sensitive to rotation, near points to translation. The leading minus sign comes from $\boldsymbol{r}$ = observed − predicted.

With all increments stacked into $\Delta\boldsymbol{s} = (\Delta\boldsymbol{\xi}_1, \ldots, \Delta\boldsymbol{\xi}_N, \Delta\boldsymbol{x}_1, \ldots, \Delta\boldsymbol{x}_P) \in \mathbb{R}^{6N+3P}$, minimizing $\|\boldsymbol{r} + \boldsymbol{J}\Delta\boldsymbol{s}\|_2^2$ gives the normal equations; the update is repeated until convergence:

$$\boldsymbol{H}\Delta\boldsymbol{s} = -\boldsymbol{b}, \quad \boldsymbol{H} = \boldsymbol{J}\boldsymbol{J}, \quad \boldsymbol{b} = \boldsymbol{J}\boldsymbol{r}, \qquad \boldsymbol{T}_i \leftarrow \mathrm{Exp}(\Delta\boldsymbol{\xi}_i)\boldsymbol{T}_i, \quad \boldsymbol{x}_p^w \leftarrow \boldsymbol{x}_p^w + \Delta\boldsymbol{x}_p.$$

Algorithm J.4. Gauss–Newton for bundle adjustmentgn

1. Start from an initialization $\mathcal{S}_0$ (e.g. two-view reconstruction).
2. Compute all residuals $\boldsymbol{r}_{ip}$ and Jacobians (Lemma **??**).
3. Build $\boldsymbol{H} = \boldsymbol{J}\boldsymbol{J}$ and $\boldsymbol{b} = \boldsymbol{J}\boldsymbol{r}$ and solve $\boldsymbol{H}\Delta\boldsymbol{s} = -\boldsymbol{b}$ (exploiting sparsity, next subsection).
4. Update poses on and points in $\mathbb{R}^3$; repeat steps 2–4 until $\|\Delta\boldsymbol{s}\|$ or the decrease of E is small.

In practice, Levenberg–Marquardt is used: $(\boldsymbol{H}+\lambda\boldsymbol{D})\Delta\boldsymbol{s} = -\boldsymbol{b}$ with a damping λ that interpolates between Gauss–Newton ($\lambda \to 0$, fast near the minimum) and gradient descent (λ large, safe far away). Robust loss functions (Huber, Cauchy) are commonly applied to reduce the effect of outliers. COLMAP uses the solver Ceres (slide 51).

J.6 The Schur Complement *(slides 44)*

With the pose increments $\Delta\boldsymbol{\xi} \in \mathbb{R}^{6N}$ ordered before the point increments $\Delta\boldsymbol{x} \in \mathbb{R}^{3P}$, the normal equations read

$$\begin{pmatrix} \boldsymbol{H}_{\xi\xi} & \boldsymbol{H}_{\xi x} \\ \boldsymbol{H}_{x\xi} & \boldsymbol{H}_{xx} \end{pmatrix} \begin{pmatrix} \Delta\boldsymbol{\xi} \\ \Delta\boldsymbol{x} \end{pmatrix} = - \begin{pmatrix} \boldsymbol{b}_{\xi} \\ \boldsymbol{b}_{x} \end{pmatrix}.$$

$\boldsymbol{H}_{xx}$ is block diagonal, one 3×3 block per point, so its inverse is cheap. Eliminating the points leaves a system in the $6N$ pose increments alone, and back substitution recovers the points.

Theorem J.5. *Schur complement reductionschur If $\boldsymbol{H}_{xx}$ is invertible, the pose increments solve*

$$\big(\boldsymbol{H}_{\xi\xi} - \boldsymbol{H}_{\xi x}\boldsymbol{H}_{xx}^{-1}\boldsymbol{H}_{x\xi}\big)\Delta\boldsymbol{\xi} = -\big(\boldsymbol{b}_{\xi} - \boldsymbol{H}_{\xi x}\boldsymbol{H}_{xx}^{-1}\boldsymbol{b}_{x}\big), \qquad \Delta\boldsymbol{x} = -\boldsymbol{H}_{xx}^{-1}\big(\boldsymbol{b}_{x} + \boldsymbol{H}_{x\xi}\Delta\boldsymbol{\xi}\big),$$

and these give the same pose update as the full system.

Proof. The second block row reads $\boldsymbol{H}_{x\xi}\Delta\boldsymbol{\xi} + \boldsymbol{H}_{xx}\Delta\boldsymbol{x} = -\boldsymbol{b}_{x}$, which gives $\Delta\boldsymbol{x}$ as stated. Substituting it into the first block row, $\boldsymbol{H}_{\xi\xi}\Delta\boldsymbol{\xi} - \boldsymbol{H}_{\xi x}\boldsymbol{H}_{xx}^{-1}(\boldsymbol{b}_{x} + \boldsymbol{H}_{x\xi}\Delta\boldsymbol{\xi}) = -\boldsymbol{b}_{\xi}$, and rearranging gives the reduced system. □

The matrix on the left, $\boldsymbol{S} = \boldsymbol{H}_{\xi\xi} - \boldsymbol{H}_{\xi x}\boldsymbol{H}_{xx}^{-1}\boldsymbol{H}_{x\xi}$, is the Schur complement of $\boldsymbol{H}_{xx}$ in $\boldsymbol{H}$. It yields the same pose update from $6N$ unknowns instead of $6N + 3P$: thousands instead of millions. Each point is then updated independently.

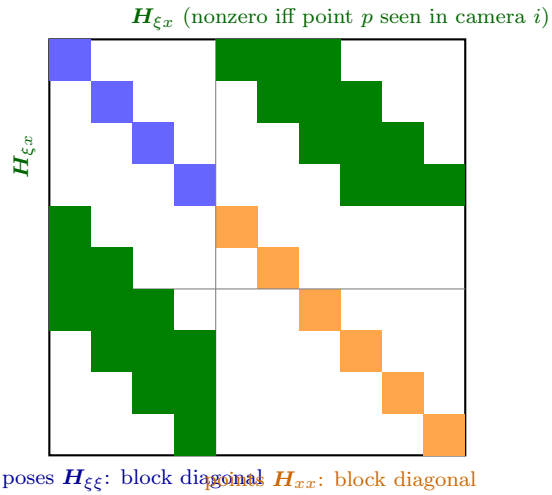


Figure 128: Arrowhead sparsity of the BA Hessian for $N = 4$ cameras and $P = 6$ points (original illustration). Cameras and points interact only if they share an observation. Eliminating the points (orange) leaves a small 4×4-block reduced camera system.

Example J.6. Why the Schur complement mattersschurnum Take $N = 100$ cameras and $P = 100{,}000$ points (a modest dataset). The full system has $6N + 3P = 300{,}600$ unknowns, a dense solve would cost $\sim (3 \cdot 10^5)^3 \approx 2.7 \cdot 10^{16}$ operations. The reduced system has $6N = 600$ unknowns, costing $\sim 600^3 \approx 2.2 \cdot 10^8$ operations; the 10^5 small 3×3 inversions are trivial. That is a factor of about 10^8.

Intuition

The points never talk directly to each other, only through the cameras that see them. So we can "integrate out" each point (a tiny 3×3 problem), which replaces the point-to-camera coupling by an additional camera-to-camera coupling for every pair of cameras that share a point. After solving for the cameras, each point just reads the new camera positions and updates itself. (This is the standard trick behind *reduced camera systems* in Triggs et al. 2000.)

J.7 Incremental Structure-from-Motion *(slides 45–51)*

The feature matching and reconstruction pipeline (COLMAP, Schönberger and Frahm 2016) has two big stages (Fig. 129): **correspondence search** (find and match robust 2D features across images) and **incremental reconstruction** (start with two views and incrementally add cameras).

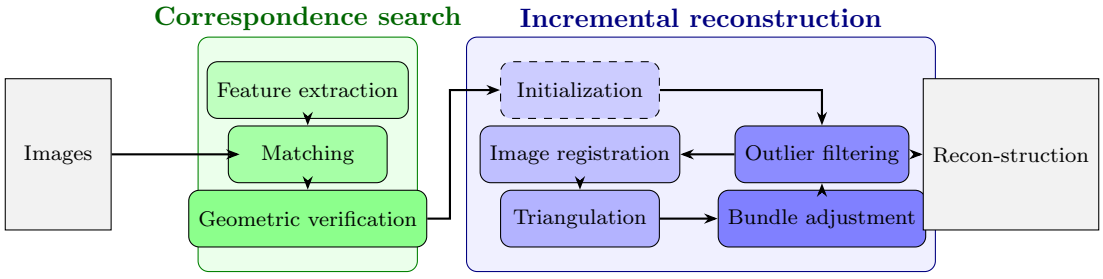


Figure 129: Incremental SfM pipeline of COLMAP (redrawn from slide 45). The registration–triangulation–filtering–bundle-adjustment loop runs once per added image.

Feature extraction (slide 46). Detect features (e.g. SIFT, SURF, BRISK) in all input images. [3.4cm]46Features detected on a photograph of a monument: boxes show location, scale and orientation of each feature (Schönberger and Frahm, CVPR 2016).

Feature matching and geometric verification (slide 47). Find overlapping image pairs and the associated feature correspondences. Candidate matches are verified geometrically, i.e. they are kept only if they are consistent with an epipolar geometry or homography (RANSAC). [3.8cm]47Graph of images (nodes) connected when they overlap; example photos of the Trevi fountain at different scales and lighting (Schönberger and Frahm, CVPR 2016).

Initialization (slide 48).

Algorithm J.7. SfM initialization from two viewsinit

1. Select **two views** with many correspondences ${}_1 \leftrightarrow {}_2$ and enough baseline for triangulation.
2. Estimate $\mathbb{E}$ from $\tilde{\mathbf{x}}_2 \mathbb{E} \tilde{\mathbf{x}}_1 = 0$, with $\tilde{\mathbf{x}}_i = \boldsymbol{K}_i^{-1}{}_i$, by the 5-point solver inside RANSAC.
3. Decompose $\mathbb{E}$ into $[\boldsymbol{R}|\boldsymbol{t}]$ and keep the candidate that puts the points in front of both cameras (Alg. **??**).
4. Fix the gauge: $\boldsymbol{T}_1 = \boldsymbol{I}$ and $\boldsymbol{T}_2 = [\boldsymbol{R}|\boldsymbol{t}]$ with $\|\boldsymbol{t}\| = 1$ (this removes the 7 gauge degrees of freedom, Prop. **??**).
5. **Triangulate** the inliers to obtain the first points $\boldsymbol{x}_p^w$.

Image registration (slide 49). Find a new image with correspondences to the current set and estimate its camera pose. Let $\mathcal{X} = \{\bar{\boldsymbol{x}}_i^s, \bar{\boldsymbol{x}}_i^w\}_{i=1}^N$ be a set of 3D-to-2D correspondences related by $\bar{\boldsymbol{x}}_i^s = \boldsymbol{P}\bar{\boldsymbol{x}}_i^w$. As the vectors are homogeneous, they have the same direction but differ in magnitude, so the equation can be expressed as $\bar{\boldsymbol{x}}_i^s \times \boldsymbol{P}\bar{\boldsymbol{x}}_i^w = 0$. Using the Direct Linear Transform this is a linear equation in the entries of $\boldsymbol{P}$; the solution of the constrained system (fixing $\boldsymbol{P}$'s scale) is given by SVD. Given that $\boldsymbol{P} = \boldsymbol{K}[\boldsymbol{R}|\boldsymbol{t}]$ and $\boldsymbol{K}$ is upper-triangular, both $\boldsymbol{K}$ and $\boldsymbol{R}$ can be obtained from the 3×3 front submatrix of $\boldsymbol{P}$ by standard RQ factorization. If $\boldsymbol{K}$ is known, we can even estimate $\boldsymbol{P}$ from only three points (P3P). In practice, RANSAC is used to remove outliers.

Intuition

Registration is the "PnP" problem: given a few points whose 3D position is already known from earlier triangulation, find where the new camera stands. $\boldsymbol{P}$ has 11 degrees of freedom (a 3×4 matrix up to scale) and every 3D–2D pair provides two independent equations, so DLT needs at least 6 points ($11/2 = 5.5$, rounded up). If $\boldsymbol{K}$ is known only 6 unknowns remain, and 3 points (6 equations) suffice, at the price of up to four solutions (P3P). Triangulation (known cameras $\rightarrow$ points) and registration (known points $\rightarrow$ camera) alternate: this is how the reconstruction "crawls" outwards.

Triangulation (slide 50). Given the newly registered image, new correspondences can be triangulated. In COLMAP, a robust triangulation method handles outliers.

Bundle adjustment and outlier filtering (slide 51). Minimize the reprojection error of all observations with respect to all poses and 3D points (Def. **??**). Since incremental SfM only affects the model locally, COLMAP performs *local* BA on the locally connected images, and *global* BA only once in a while for efficiency. For the sparse large-scale optimization it uses Ceres. After each BA, observations with large reprojection errors and cameras with abnormal fields of view or large distortion coefficients are removed.

Algorithm J.8. Incremental SfM (COLMAP-style)incsfm

1. Extract features in all images, match overlapping pairs, verify geometrically.
2. Initialize with a two-view reconstruction (Alg. **??**).
3. **Repeat** until no image can be added: register the best new image (DLT/P3P inside RANSAC); triangulate new points; run local BA; filter outliers (large reprojection error, abnormal cameras); occasionally run global BA.

Intuition

Because the cost is highly non-convex, you "grow a crystal": start from a tiny well-conditioned seed (two views) and add only what is consistent with it, re-optimizing after each addition so errors never accumulate far from where the optimizer can fix them. The price: images are registered one at a time, so an early mistake propagates (a limit discussed in 5.5).

J.8 Building Rome in a Day, COLMAP SfM and COLMAP MVS *(slides 52–54)*

- **Building Rome in a Day** (Agarwal et al., ICCV 2009): first large-scale SfM from unstructured images (slide 52).
- **COLMAP** significantly improves accuracy and robustness compared to prior work (slide 53).
- COLMAP features a second **multi-view stereo** stage to obtain dense geometry (slide 54; Schönberger et al., ECCV 2016). A feed-forward network predicts dense depth without known cameras (5.5), whereas MVS needs SfM first.

[3.6cm]52Sparse point cloud and camera frusta of the Colosseum reconstructed from internet photos (Agarwal, Snavely, Simon, Seitz and Szeliski, "Building Rome in a Day", ICCV 2009). [3.6cm]53COLMAP sparse reconstruction of a city scene; red marks the camera positions (Schönberger and Frahm, CVPR 2016). [3.6cm]54Dense COLMAP multi-view stereo reconstruction of a building (Schönberger, Zheng, Frahm and Pollefeys, ECCV 2016).

K Feed-Forward Structure-from-Motion *(slides 55–68)*

K.1 Limits of Incremental Structure-from-Motion *(slides 56)*

Looking back at Fig. 129:

- every box is a separately engineered stage, with its own thresholds and failure cases;
- images are registered one at a time: a poor initial pair, outliers or drift propagate to every later image;
- runtime grows to hours for large photo collections; few views, low overlap or little texture leave too few correspondences;
- **Global SfM** estimates all cameras jointly without learning: rotation averaging, then camera positions, then one bundle adjustment. GLOMAP (Pan, Baráth, Pollefeys and Schönberger, ECCV 2024), a recent global pipeline, is on par with COLMAP in accuracy and orders of magnitude faster.

Intuition

Incremental SfM is like building a house brick by brick with one mason: reliable but slow, and a crooked first brick tilts the whole house. Global SfM hires many masons who work simultaneously and then reconcile. Feed-forward SfM, the topic of this section, replaces the masons by a single network that has seen millions of houses and sketches the whole building at once.

K.2 From Learned Components to Learned Reconstruction *(slides 57)*

Year	Method	What is learned
2018	SuperPoint	keypoints and their descriptors, from one network (5.2)
2020	SuperGlue	the matching of two keypoint sets, with a graph neural network
2021	LoFTR	dense matches between two images, with a transformer and no detector
2024	VGGSfM	a complete SfM pipeline, differentiable, trained end to end
2024	DUSt3R, MASt3R	pointmaps from two images (lecture 04); more views need a global alignment
2025	VGGT	cameras, depth, point maps and tracks for many views, in one forward pass

Table 8: Progressive replacement of the classical pipeline by learned components (slide 57).

Definition K.1. Pointmappointmap For an $H \times W$ image $\boldsymbol{I}_a$, the pointmap $\hat{\boldsymbol{X}}^{a,1} \in \mathbb{R}^{H \times W \times 3}$ stores *one 3D point per pixel*, for both images $a \in \{1, 2\}$, expressed in the frame of camera 1.

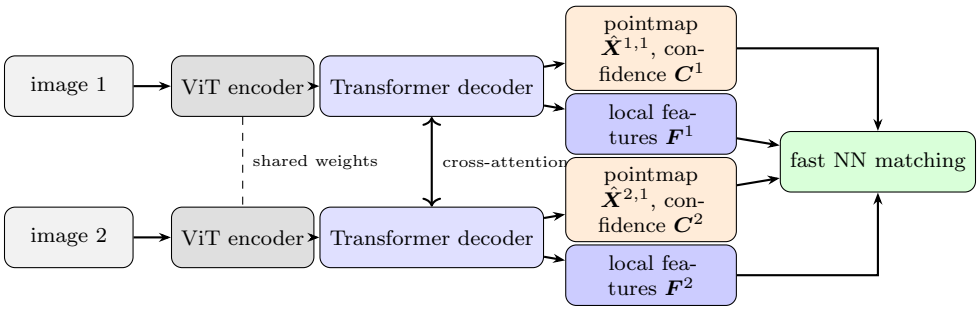


Figure 130: MASt3R-style two-view network (simplified from slide 57): tokens $\boldsymbol{H}^a$ from a ViT encoder are decoded with cross-attention into pointmaps with confidence (grey path) and local features (blue path), used for geometrical or feature-based matching.

Intuition

A pointmap is a depth map that already lives in a common world frame: once pixel (u, v) of image 2 and pixel (u', v') of image 1 carry (almost) the same 3D point, they are a correspondence *by construction*. DUSt3R and MASt3R use this to skip the detect-describe-match-verify chain, but for more than two images they still need a post-hoc global alignment; VGGT removes that last step.

K.3 Visual Geometry Grounded Transformer (VGGT) *(slides 58)*

- **Input:** one to hundreds of images of a scene; the first image defines the world frame.
- **Output,** in one forward pass of under a second: camera parameters, depth maps, point maps and point tracks for every image.
- Without post-processing, it often outperforms methods that refine their output by geometric optimization.

[4.2cm]58VGGT takes a set of images of a scene (left) and predicts, in one forward pass, camera poses, a dense point cloud, depth maps and point tracks (Wang et al., CVPR 2025).

K.4 Problem Definition *(slides 59)*

Definition K.2. VGGT as a functionvggtdef VGGT is one network with learned weights θ. It takes N images $\boldsymbol{I}_1, \ldots, \boldsymbol{I}_N$, each of $H \times W$ pixels, and predicts four outputs per image i (the hat marks a prediction):

$$f_\theta(\boldsymbol{I}_1, \ldots, \boldsymbol{I}_N) = \big(\hat{\boldsymbol{g}}_i,\ \hat{\boldsymbol{D}}_i,\ \hat{\boldsymbol{X}}_i,\ \boldsymbol{F}_i\big)_{i=1}^{N}.$$

- **Camera** $\hat{\boldsymbol{g}}_i \in \mathbb{R}^9$: rotation quaternion $\hat{\boldsymbol{q}}_i \in \mathbb{R}^4$, translation $\hat{\boldsymbol{t}}_i \in \mathbb{R}^3$ and field of view $\hat{\boldsymbol{\phi}}_i = (\hat{\phi}_x, \hat{\phi}_y)$. They give the pose $\boldsymbol{T}_i$ and the calibration $\boldsymbol{K}_i$ with focal lengths $f_x = W/(2\tan(\phi_x/2))$, $f_y = H/(2\tan(\phi_y/2))$ and the principal point at the image center.
- **Depth map** $\hat{\boldsymbol{D}}_i \in \mathbb{R}^{H\times W}$: the depth of every pixel.
- **Point map** $\hat{\boldsymbol{X}}_i \in \mathbb{R}^{H\times W\times 3}$: the 3D point of every pixel, in the frame of camera 1, which is the world frame.
- **Feature map** $\boldsymbol{F}_i \in \mathbb{R}^{H\times W\times C}$: C numbers per pixel, used to track points across the images.

The outputs are redundant, since the cameras follow from the point maps (PnP), yet predicting all of them improves accuracy.

Example K.3. From field of view to focal lengthfov For $W = 518$ pixels and a predicted horizontal field of view $\phi_x = 60°$: $f_x = 518/(2\tan 30°) = 518/1.1547 \approx 449$ pixels. With the principal point at $(259, 259)$ this fully specifies $\boldsymbol{K}$ (no calibration needed, in contrast to Section 5.1).

Intuition

Classical SfM needs $\boldsymbol{K}$ from calibration or self-calibration (and fails when it is wrong). VGGT simply *guesses* the field of view from the image content as a human would (a wide-angle shot looks different from a telephoto one). Predicting the same geometry in several forms (cameras, depth, point maps) gives the network several consistent "views" of one underlying 3D structure to learn from.

K.5 Architecture: Alternating Attention *(slides 60)*

- **Image tokens,** as $\boldsymbol{H}^a$ in DUSt3R: a vision transformer (DINO) cuts each image $\boldsymbol{I}_i$ into patches and turns each patch into a token, a feature vector.
- **Camera token:** a transformer returns one vector per input token. A 518×518 image gives $37 \times 37 = 1369$ patch tokens, hence 1369 outputs, one per patch, for the per-pixel maps.

One extra learned token per image, the same for every image, yields one extra output: the camera head reads $\hat{\boldsymbol{g}}_i$ from it.

- **Alternating attention,** $L = 24$ times: *global attention* among the tokens of all images (as the cross-attention of DUSt3R but for N images), then *frame attention* among the tokens of one image.
- **Outputs:** the camera head turns each camera token into $\hat{\boldsymbol{g}}_i$; the DPT (Dense Prediction Transformer) head turns the image tokens into $\hat{\boldsymbol{D}}_i, \hat{\boldsymbol{X}}_i$ and $\boldsymbol{F}_i$.

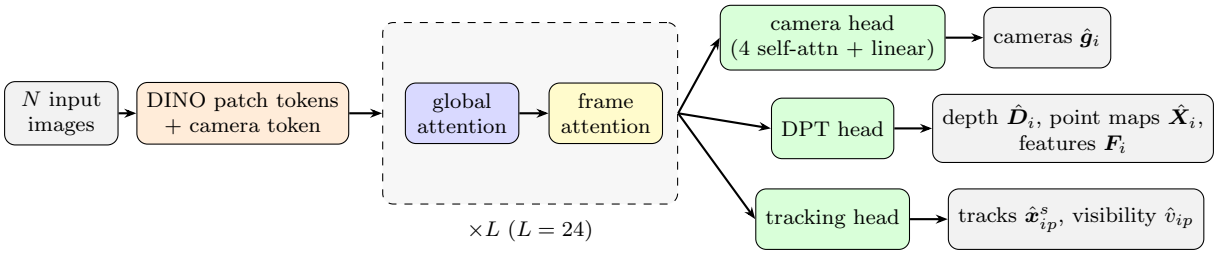


Figure 131: VGGT architecture (redrawn and simplified from slide 60).

Example K.4. Cost of global versus frame attentionattncost With $N = 10$ images and $T = 1369$ patch tokens per image there are $NT = 13{,}690$ tokens. Global attention scores $(NT)^2 \approx 1.87 \cdot 10^8$ token pairs per head and layer; frame attention scores $N \cdot T^2 \approx 1.87 \cdot 10^7$. Frame attention is $N = 10$ times cheaper.

Intuition

Global attention is where views "talk" to each other (it learns correspondence and relative pose implicitly); frame attention lets each image digest its own content cheaply. Alternating the two 24 times mimics what SfM does iteratively (match, then triangulate and refine), but inside one differentiable network with no hand-set thresholds. The camera token acts as a learned "summary slot" per image from which the camera is read out; no temporal order is assumed, so the images may be an unordered photo collection.

K.6 Prediction Heads and Tracking *(slides 61)*

- **Camera head:** four self-attention layers and a linear layer turn the camera token of image i into $\hat{\boldsymbol{g}}_i$.
- **DPT head,** the Dense Prediction Transformer decoder (Ranftl et al., ICCV 2021), turns tokens back into per-pixel maps: $\hat{\boldsymbol{D}}_i$, $\hat{\boldsymbol{X}}_i$, $\boldsymbol{F}_i$, and confidence maps $\boldsymbol{C}_i^D, \boldsymbol{C}_i^X \in \mathbb{R}^{H \times W}$ that say how much to trust each pixel of $\hat{\boldsymbol{D}}_i$ and $\hat{\boldsymbol{X}}_i$.
- **Tracking head,** as in CoTracker2 (a point tracker for videos): given the pixel of a point p in one image, it compares the feature of that pixel with the feature maps $\boldsymbol{F}_i$ of all images, and predicts where the point appears in each image, $\hat{\boldsymbol{x}}_{ip}^s$, and whether it is visible there, $\hat{v}_{ip} \in [0, 1]$.

K.7 Training Objective and Normalization *(slides 62)*

The four outputs are trained jointly, against ground truth written without a hat:

$$\mathcal{L} = \mathcal{L}_{\text{camera}} + \mathcal{L}_{\text{depth}} + \mathcal{L}_{\text{pmap}} + \lambda \mathcal{L}_{\text{track}}, \qquad \lambda = 0.05,$$

$$\mathcal{L}_{\text{depth}} = \sum_{i=1}^{N} \left\| \boldsymbol{C}_i^D \odot (\hat{\boldsymbol{D}}_i - \boldsymbol{D}_i) \right\| + \left\| \boldsymbol{C}_i^D \odot (\nabla \hat{\boldsymbol{D}}_i - \nabla \boldsymbol{D}_i) \right\| - \alpha \log \boldsymbol{C}_i^D .$$

Here $\odot$ is the per-pixel product, ∇ the image gradient, and α the weight of the term $-\alpha \log \boldsymbol{C}_i^D$, which keeps the confidence from dropping to zero. $\mathcal{L}_{\text{camera}}$ is a Huber loss, squared for small errors and linear for large ones, between $\hat{\boldsymbol{g}}_i$ and $\boldsymbol{g}_i$; $\mathcal{L}_{\text{pmap}}$ is like $\mathcal{L}_{\text{depth}}$ for $\hat{\boldsymbol{X}}_i$ with $\boldsymbol{C}_i^X$; $\mathcal{L}_{\text{track}}$ is the distance of $\hat{\boldsymbol{x}}_{ip}^s$ to the true positions, plus a visibility term.

Intuition

The confidence $\boldsymbol{C}$ weights each pixel's error: unreliable pixels (sky, reflections, moving objects) can lower their confidence and so pay less. Without the $-\alpha \log \boldsymbol{C}$ term the network could cheat by predicting $\boldsymbol{C} = 0$ everywhere. The term charges a price for low confidence, so confidence drops only where errors would be large. The gradient term in the depth loss encourages sharp, correct edges.

Proposition K.5. *Why a normalization is requirednormal Let $(\boldsymbol{T}_i, \boldsymbol{X})$ be a ground-truth reconstruction. Any similarity transform of the whole scene, applied consistently to cameras and points, produces the same images (Prop. **??**). A regression target that is not fixed by a convention is therefore not a function of the images.*

Proof. If two targets differ by a similarity, they map to identical inputs, so the network would be asked to output two different answers for the same input; the loss-minimizing prediction would blur between them. □

The images do not fix the global frame and scale (5.3). The ground truth is therefore expressed in the frame of camera 1 and divided by s, the mean distance of its 3D points to the origin; the network learns to predict in this normalized frame.

Intuition

It is the neural-network version of the gauge fixing of bundle adjustment: BA fixes the first pose and one distance; VGGT fixes the first pose and the average point distance ("scene scale = 1"). Every training scene is thus expressed in the same canonical units.

Training data and cost. 17 datasets; 64 A100 GPUs for nine days; 2 to 24 images per scene, at most 518 pixels per side (the slide quotes around 1.5M EUR for this compute).

K.8 Results: Camera Pose Estimation *(slides 63)*

10 random frames per scene; time on one H100 GPU. AUC@30: area under the accuracy curve of the relative rotation and translation errors, up to 30°. RealEstate10K: walkthrough videos of houses, used for training by none of the methods. CO3Dv2: videos around everyday objects. PixSfM: COLMAP with feature-metric refinement. DUSt3R and MASt3R with global alignment.

Method	RealEstate10K ↑	CO3Dv2 ↑	Time
COLMAP + SuperPoint + SuperGlue	45.2	25.3	≈ 15 s
PixSfM	49.4	30.1	> 20 s
DUSt3R	67.7	76.7	≈ 7 s
MASt3R	76.4	81.8	≈ 9 s
VGGSfM v2	78.9	83.4	≈ 10 s
VGGT, feed-forward	85.3	88.2	≈ 0.2 s
VGGT + bundle adjustment	**93.5**	**91.8**	≈ 1.8 s

Table 9: Camera pose estimation, AUC@30 (slide 63).

Intuition

Ten random frames is a sparse, wide-baseline setting, the weakest regime of classical SfM (few overlaps, little parallax per pair). That is where a learned prior helps most: COLMAP+SuperPoint+SuperGlue gets 45.2 on RealEstate10K, VGGT 85.3 in a fifth of a second. On dense captures with thousands of images, incremental SfM remains the reference (5.4). The speed difference (0.2 s versus ∼ 15 s) is a factor of about 75.

K.9 Results: Dense Geometry *(slides 64)*

DTU (lecture 04), Chamfer distance in mm (lower is better); GeoMVSNet is a learned MVS method given the true cameras.

DTU			ETH3D (laser-scanned GT)	
Method	GT cameras	Overall↓	Method	Overall↓
GeoMVSNet	yes	0.295	DUSt3R	1.005
MASt3R	yes	0.374	MASt3R	0.826
DUSt3R	no	1.741	VGGT, point head	0.709
VGGT	no	0.382	VGGT, depth + camera	**0.677**

Table 10: Dense geometry (slide 64).

- Depth unprojected with the predicted cameras beats the point head: the decomposition into depth and camera pays off.
- VGGT obtains dense depth without known cameras, whereas COLMAP's MVS needs SfM first (5.4); on DTU, VGGT (no GT cameras) is within 0.01 mm of MASt3R, which *uses* the true cameras.

[4.5cm]64Input, VGGT ("Ours") and DUSt3R: one image, two images without overlap, 32 images (reconstruction times shown on the slide).

K.10 Results: Matching and Tracking *(slides 65)*

ScanNet-1500: 1500 indoor image pairs; pose from the matches through the essential matrix (5.3); AUC of the pose error.

Method	AUC@5°	AUC@10°	AUC@20°
SuperGlue	16.2	33.8	51.8
LoFTR	22.1	40.8	57.6
RoMa	31.8	53.4	70.9
VGGT	**33.9**	**55.2**	**73.4**

Table 11: Two-view matching, pose from the essential matrix (slide 65).

RoMa is a dense learned matcher and the strongest baseline. The tracking head is *not trained* for two-view matching. The features also serve other tasks: a dynamic point tracker improves when given them, and novel view synthesis works without the input cameras.

Intuition

It is a surprising result: a network trained for 3D reconstruction produces features that outperform dedicated two-view matchers, because "being the same 3D point" is a stronger training signal than "looking alike". Note how the pipeline of Section 5.3 (matches $\rightarrow \mathbb{E} \rightarrow$ pose) is used here as an *evaluation protocol*, which shows the classical geometry remains the common language.

[4.4cm]65Point tracking: VGGT tracks (top) and a dynamic point tracker (CoTracker) given VGGT features (bottom) (Wang et al., CVPR 2025).

K.11 Bundle Adjustment from VGGT Predictions *(slides 66)*

Use the same objective as before,

$$\mathcal{T}^*, \mathcal{X}_w^* = \arg\min_{\mathcal{T}, \mathcal{X}_w} \sum_{i=1}^{N} \sum_{p=1}^{P} w_{ip} \left\| \boldsymbol{x}_{ip}^s - \pi_i(\boldsymbol{x}_p^w) \right\|_2^2,$$

with the following ingredients, all predicted:

- observations $\boldsymbol{x}_{ip}^s$ and weights w_{ip}: tracks and visibilities from the tracking head;
- initial poses $\boldsymbol{T}_i$: the camera head, $\hat{\boldsymbol{g}}_i$, which also gives the calibrations $\boldsymbol{K}_i$;
- initial points $\mathcal{X}_w$: the predicted depth maps $\hat{\boldsymbol{D}}_i$, unprojected with the predicted cameras.

Initialization, image registration and triangulation (5.4) are no longer needed; with bundle adjustment the reconstruction takes about 2 s. AUC@30 on RealEstate10K rises from 85.3 to 93.5 with bundle adjustment.

Intuition

The two worlds complement each other: the network gives a very good starting point in the right basin of attraction (solving the non-convexity problem of Section 5.4), and BA then enforces what the network cannot guarantee, namely that the output is *exactly* geometrically consistent with the observed tracks. This is the same "closed-form initialization + non-linear refinement" pattern as in Zhang's calibration.

K.12 Limitations *(slides 67)*

Stated in the paper:

- no fisheye or panoramic images;
- accuracy drops under extreme rotations between the input images;
- minor non-rigid motion is handled, substantial non-rigid deformation fails;
- runtime and memory grow with the number of frames: 200 frames take 8.75 s and 40.63 GB of GPU memory (one H100, images of 336×518 pixels).

Beyond the paper:

- training needs large 3D-labeled datasets and large compute: 17 datasets, 64 A100 GPUs for nine days;
- no geometric guarantee: the output is a prediction, not the optimum of an objective;
- bundle adjustment still improves the result: 85.3 to 93.5 AUC@30 on RealEstate10K.

K.13 Comparison of Incremental and Feed-Forward SfM *(slides 68)*

	Incremental SfM	VGGT
Camera model	perspective, calibrated or self-calibrated	perspective, principal point at the center
Views	one at a time	all jointly
Assumptions	overlap and texture, a good initial pair	scenes like the training data, mostly rigid
Solution	RANSAC and nonlinear least squares	one forward pass of a transformer
Runtime	minutes to hours	under a second for tens of frames
Scale	thousands of images	hundreds of frames, bounded by GPU memory
Output	sparse points and cameras; dense with MVS	cameras, depth, point maps, tracks
Failure modes	low overlap, little texture, drift	fisheye images, extreme rotations, non-rigid motion
Guarantees	local optimum of the reprojection error	none

Table 12: Incremental SfM (Schönberger and Frahm, CVPR 2016) versus VGGT (Wang et al., CVPR 2025), slide 68.

Intuition

The two are complementary rather than competing: use the feed-forward network when views are few, wide-baseline or the time budget is tight (robots, interactive tools), and use classical incremental or global SfM when you have thousands of images and need a certified, repeatable optimum. The hybrid VGGT + BA gives the best accuracy.

Summary *(slides 69)*

Comparison of the methods covered

Method	Input → output	Core idea	Main limitation
Zhang calibration (5.1)	plane images → $\boldsymbol{K}$, poses	homography constraints on $\boldsymbol{B}$, then non-linear LS	needs a known target
SIFT (5.2)	image → 128D features	DoG extrema in scale space, gradient histograms	hand-crafted; slow
SuperPoint (5.2)	image → keypoints + 256D descriptors	self-supervised, homographic adaptation	fails for rotations not in training data
8-point / 5-point (5.3)	≥ 8 / 5 matches → $\mathbb{E}$	linear epipolar constraint (+ rank-2 projection)	scale unobservable; 4-fold ambiguity
Triangulation (5.3)	2 views → 3D point	DLT or reprojection error	ill-conditioned for small parallax
Bundle adjustment (5.4)	observations → poses, points	Gauss–Newton on + Schur complement	needs good initialization
Incremental SfM / COLMAP (5.4)	image set → sparse model	seed from two views, register, triangulate, BA	slow; drift; many thresholds
VGGT (5.5)	N images → cameras, depth, points, tracks	alternating attention, trained on 17 datasets	no guarantees; GPU memory
VGGT + BA (5.5)	N images → refined model	learned initialization + BA	extra ~ 1.6 s

Table 13: Summary of the methods of Lecture 05.

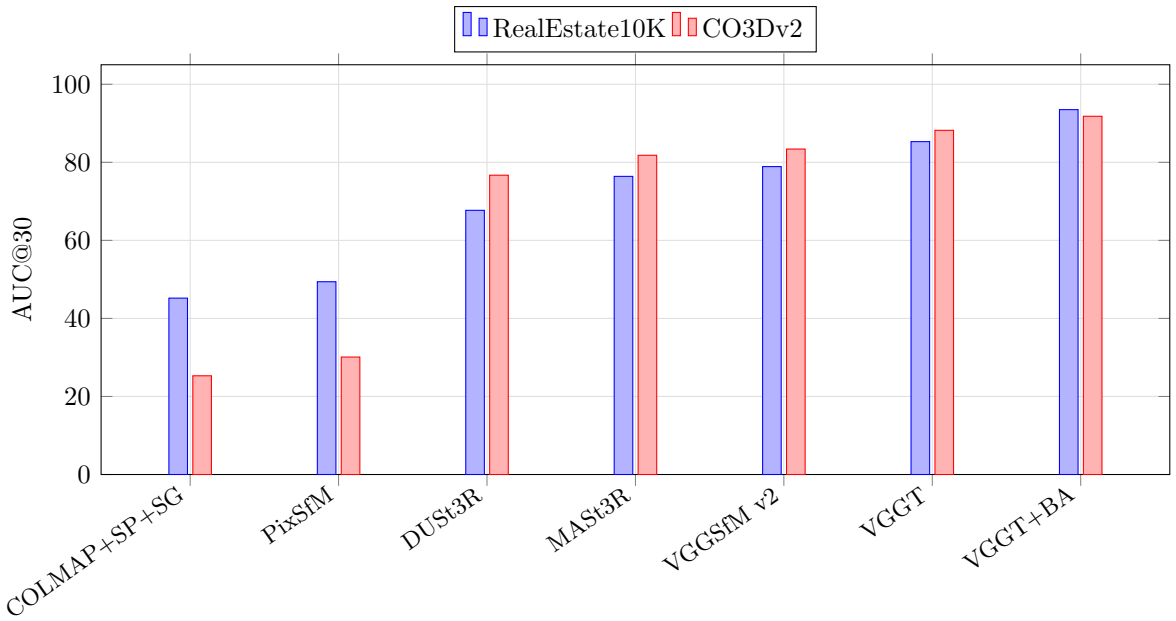


Figure 132: Camera pose accuracy of classical, learned two-view, and feed-forward methods (data of the table on slide 63; ten random frames per scene). The jump from the classical pipeline to learned methods is largest in the sparse regime, and bundle adjustment on top of VGGT adds the final gain.

Key take-aways

1. **Geometry supplies the constraints:** the epipolar constraint, triangulation and the reprojection error are the common language of every method in this lecture, classical or

learned (even VGGT is evaluated and refined with them).

2. **Classical SfM = robust estimation + non-linear least squares.** Sparsity and the Schur complement keep bundle adjustment tractable (a factor $\sim 10^8$ in our example); its hard part is the initialization, which is why reconstruction is built incrementally from a good seed.

3. **Scale and gauge are never observable from images alone:** the essential matrix is defined up to scale and the four-fold decomposition ambiguity is resolved by cheirality; BA fixes one pose and one distance; VGGT normalizes its training targets.

4. **VGGT learns the whole mapping** from images to cameras, depth, point maps and tracks in one forward pass of a transformer with alternating global and frame attention.

5. **The most accurate poses come from a learned initialization refined by bundle adjustment** (AUC@30: 85.3 $\rightarrow$ 93.5 on RealEstate10K).

Acknowledgments (slide 70): the slides are adapted from the Computer Vision course at the University of Tübingen (Prof. Andreas Geiger); Section 5.5 figures and results are from Wang et al., "VGGT: Visual Geometry Grounded Transformer", CVPR 2025.

Part V

Lecture 6

About these notes

The notes follow the slide deck in order; every `\section` and `\subsection` carries the title of the corresponding slide(s) and its slide range. Formulas on the slides are embedded as pictures, so the formal content (definitions, lemmas, theorems, proofs) was reconstructed from the references quoted in the slide footers (Thrun–Burgard–Fox, Stachniss, Dellaert–Kaess, Mur-Artal et al., Lovegrove et al. and others) and from lectures 01 and 05 (Lie groups, bundle adjustment). Dashed empty boxes mark photographs or screenshots that must be copied from the indicated slide; all other figures are redrawn in TikZ/pgfplots.

Notation. World-to-camera poses $_i \in$ (as in lecture 05); 3D points $_j$ (or $\mathbf{x}_j^w$); pixel observations $\mathbf{u}_{ij}$; $\pi(\cdot)$ is the pinhole projection including intrinsics; $\|\mathbf{v}\|\Omega^2 = \mathbf{v}^\top \Omega\, \mathbf{v}$ is the squared Mahalanobis norm. The letter is the Gauss–Newton matrix of Section 6.3, while the EKF measurement Jacobian is called $_t$.

Agenda and Learning Objectives *(slides 1–2)*

Agenda: 6.1 The SLAM problem; 6.2 Filtering; 6.3 Graph-based SLAM; 6.4 The visual SLAM pipeline; 6.5 Research directions.

After the lecture you should be able to:

1. formulate SLAM as joint estimation of trajectory and map, and derive the Kalman filter and EKF-SLAM together with their cost;
2. formulate graph-based SLAM as least squares over a factor graph, solve it with Gauss–Newton, and relate its sparsity to marginalization;
3. describe the threads of a keyframe-based visual SLAM system: tracking, local mapping, loop detection and loop correction.

L The SLAM Problem *(slides 3–6)*

L.1 The SLAM Problem 3

Definition L.1. SLAMslam *Simultaneous localization and mapping* computes, at the same time, the camera poses (the trajectory) $\boldsymbol{x}_{1:t} = \{\boldsymbol{x}_1, \dots, \boldsymbol{x}_t\}$ and the map of the environment , given the measurements $\boldsymbol{z}_{1:t}$ (and, if available, controls $_{1:t}$), i.e. the posterior

$$p(\boldsymbol{x}_{1:t}, \mid \boldsymbol{z}_{1:t}, _{1:t}).$$

Localization estimates the poses given the map; *mapping* builds the map given the poses.

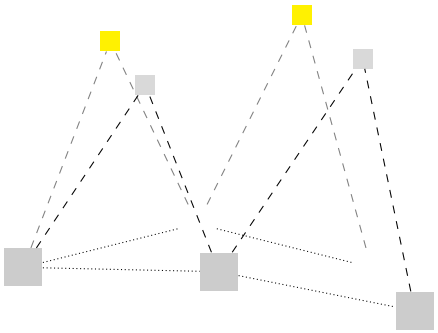
SLAM is a *chicken-or-egg problem*: a map is needed to localize, and a pose estimate is needed to build a map. Without any map, chaining relative motion estimates (*dead reckoning*) accumulates error without bound.

Proposition L.2. *Dead-reckoning driftdrift If each relative motion estimate carries independent zero-mean error of variance σ^2 per axis, the position variance after t steps is $t\sigma^2$, so the standard deviation grows as $\sigma\sqrt{t}$ and is unbounded.*

Proof. The estimated position is $\sum_{k=1}^{t}(\Delta_k + \varepsilon_k)$ with independent ε_k; variances of independent terms add. □

Intuition

With $\sigma = 1\,\text{cm}$ per step, after 1000 steps the 1σ error is $\sqrt{1000} \approx 32\,\text{cm}$, even though no single step was bad. Orientation errors are worse: a small heading error bends the whole subsequent path. A map that is *re-observed* gives absolute references and stops this growth.



star: landmark (yellow true, gray estimated)

circle: pose (white true, gray estimated)

dashed: observations

Figure 133: Redrawn from slide 3: true and estimated poses/landmarks and the observations linking them. Estimated poses drift away from the truth while the landmarks keep them mutually consistent.

L.2 Cameras for SLAM 4

Cameras are compact, accurate, non-invasive, well understood, and today cheap and ubiquitous. Images carry *appearance*: two rooms that look identical to a 2D laser scanner can be told apart by a camera. Applications range from augmented reality and vision-based navigation to self-driving cars, mobile devices, inspection drones and VR headsets. The cost: a camera senses geometry only indirectly, through photometric effects, and a single camera measures *no depth*.

Proposition L.3. *Scale ambiguity of a monocular camerascale Let a scene* $\{_j\}$ *and camera poses* $\{(\mathbf{R}_i, \mathbf{t}_i)\}$ *produce pixels* $\mathbf{u}_{ij} = \pi(\mathbf{R}_{ij} + \mathbf{t}_i)$. *For any* $s > 0$, *the scene* $\{s_j\}$ *with translations* $\{s\mathbf{t}_i\}$ *produces exactly the same pixels.*

Proof. $\mathbf{R}_i(s_j) + s\mathbf{t}_i = s(\mathbf{R}_{ij} + \mathbf{t}_i)$, and the perspective projection satisfies $\pi(s\mathbf{p}) = \pi(\mathbf{p})$ because it divides by the depth. □

Intuition

A photograph of a toy house and of a real house can be identical. A single camera therefore recovers the world only *up to a similarity* (rotation, translation *and* scale: 7 degrees of freedom), which is why monocular maps also drift in scale (Section 6.4).

[4.2cm]4Maps built from one camera: sparse features of a room (left) and a dense desktop (right).

L.3 SLAM and Structure-from-Motion 5

The unknowns and the error are those of lecture 05: camera poses $_i$ (world to camera), 3D points $_j$, and the reprojection error of every observation. Both problems stem from the same maximum a posteriori formulation; the *setting* differs.

Structure-from-motion (lecture 05)	SLAM
Unordered set of images, all available from the start	Frames arrive one at a time; solved incrementally as measurements arrive
Solved offline, analyzing all images at once	Pose and map needed at every frame, in real time
No notion of returning to a place	Returning to a visited place closes a loop and corrects long-term drift

Intuition

SfM is like developing all holiday photos and then assembling the 3D model at home; SLAM is like navigating a city while drawing the map as you walk. Same maths, very different time budget.

L.4 Running Example: VBR Campus 6

VBR (Vision Benchmark in Rome) was recorded hand-held and by car with a global-shutter stereo camera of wide baseline, a 3D LiDAR, an IMU and an RTK-GPS. Its ground truth refines the RTK-GPS with LiDAR bundle adjustment, accurate to ±3 cm over 1.5 km; its six datasets cover over 40 km. The running example of the lecture is a sequence of around 11 km.

[5cm]6VBR: sensor-rig reference frames (camera left/right CL, CR, LiDAR L, IMU I) and two point-cloud/map renderings of the Rome sequences.

M Filtering: From the Bayes Filter to EKF-SLAM *(slides 7–13)*

M.1 SLAM as a Graphical Model 8

The camera poses and the map are unknown; the measurements are observed. Without odometry, as with a single camera, a motion model alone links consecutive poses.

Definition M.1. Full and online SLAMfullonline *Full SLAM* estimates the whole trajectory and the map, $p(\boldsymbol{x}_{1:t}, \mid \boldsymbol{z}_{1:t,1:t})$. *Online SLAM* keeps only the current pose,

$$p(\boldsymbol{x}_t, \mid \boldsymbol{z}_{1:t,1:t}) = \int \cdots \int p(\boldsymbol{x}_{1:t}, \mid \boldsymbol{z}_{1:t,1:t})\, d\boldsymbol{x}_1 \cdots d\boldsymbol{x}_{t-1},$$

i.e. it *marginalizes* the past poses, and the integrals are solved recursively one step at a time.

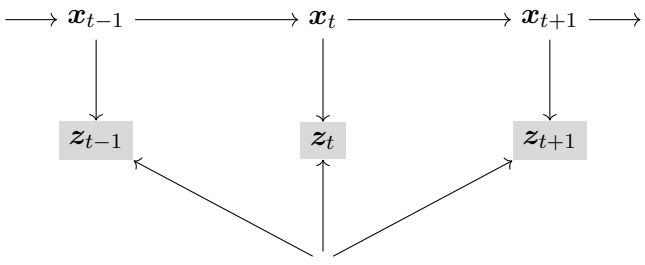


Figure 134: Graphical model of SLAM (slide 8): poses form a Markov chain, the map is static, and each (gray, observed) measurement depends on the current pose and the map. Controls are omitted, as in the monocular case.

Intuition

Full SLAM keeps the entire path (a growing problem, solved later by graphs); online SLAM forgets the past path but remembers what it taught us, packed inside a single Gaussian over the current pose

and the map. Marginalizing is "summing over everything I no longer care about".

M.2 The Bayes Filter 9

Let $_t = (\boldsymbol{x}_t,)$ be the state: camera pose and map.

Theorem M.2. *Recursive Bayes filterbayes Let* $\mathrm{bel}(_t) = p(_t|\ \boldsymbol{z}_{1:t,1:t}\ )$. *Under the Markov assumptions, one frame at a time:*

$$\overline{\mathrm{bel}}(_t) = \int p(_t|_{t-1},_t\ )\,\mathrm{bel}(_{t-1})\,d_{t-1} \qquad \textit{(prediction)}$$

$$\mathrm{bel}(_t) = \eta\ p(\boldsymbol{z}_t\ |_t)\,\overline{\mathrm{bel}}(_t) \qquad \textit{(update)}$$

where $p(_t|_{t-1},_t\ )$ *is the* motion model, $p(\boldsymbol{z}_t\ |_t)$ *the* measurement model *and* η *a normalizer.*

Proof. By Bayes' rule, $\mathrm{bel}(_t) = \eta\, p(\boldsymbol{z}_t\ |_t, \boldsymbol{z}_{1:t-1,1:t}\ )\, p(_t|\ \boldsymbol{z}_{1:t-1,1:t}\ )$. Given the state, the current measurement is independent of the past, so the first factor is $p(\boldsymbol{z}_t\ |_t)$. For the second factor, the law of total probability gives $\int p(_t|_{t-1}, \boldsymbol{z}_{1:t-1,1:t}\ )\, p(_{t-1}|\ \boldsymbol{z}_{1:t-1,1:t}\ )\, d_{t-1}$; the state is Markov, so $p(_t|_{t-1}, \dots\ ) = p(_t|_{t-1},_t\ )$, and $_t$ carries no information on $_{t-1}$, so the last factor is $\mathrm{bel}(_{t-1})$. □

Intuition

Two beats, forever: *predict* (move the belief with the motion model, which *widens* it, since motion adds uncertainty) and *update* (multiply by how well each hypothesis explains the new image, which *narrows* it). The one-dimensional example below follows slide 9: a camera on a line.

Example M.3. Camera on a lineline Prior $(0, 1)$. Motion model: move $+1$ with noise variance 1, so the prediction is $(1,\ 1+1) = (1, 2)$. Measurement $z = 2$ with noise variance 1. The posterior is the normalized product of the prediction and the measurement likelihood: $\sigma^2 = (2^{-1} + 1^{-1})^{-1} = 2/3$ and $\mu = \frac{2}{3}\left(\frac{1}{2}\cdot 1 + \frac{1}{1}\cdot 2\right) = 5/3$.

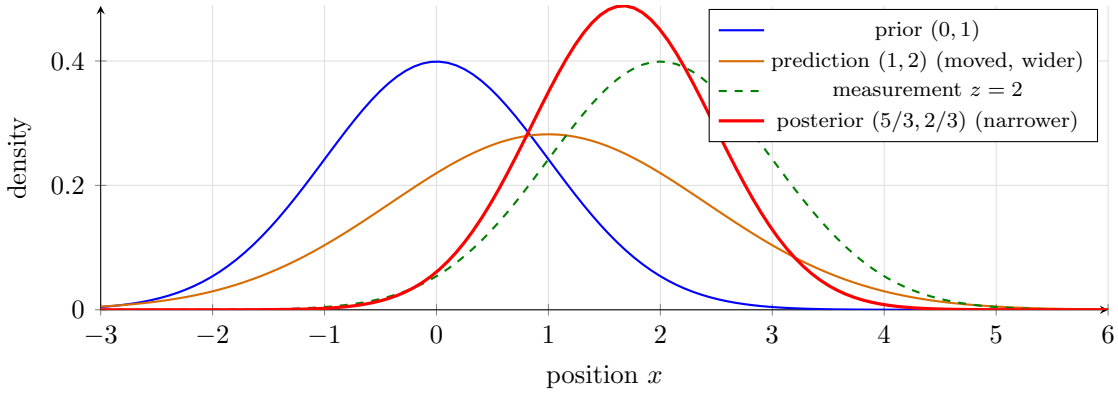


Figure 135: The Bayes filter in one dimension (slide 9): prediction shifts and widens the belief, the measurement narrows it.

M.3 Kalman Filter and Extended Kalman Filter 10

With Gaussian noise the belief stays Gaussian, $\mathrm{bel}(_t) = (\boldsymbol{\mu}_t, \Sigma_t)$. Let the models be $_t = g(_{t-1},_t\ ) + \boldsymbol{\varepsilon}_t$, $\boldsymbol{\varepsilon}_t \sim (0, \mathbf{R}_t)$ and $\boldsymbol{z}_t = h(_t) + \boldsymbol{\delta}_t$, $\boldsymbol{\delta}_t \sim (0, \mathbf{Q}_t)$. The *extended Kalman filter* (EKF) linearizes both around the current estimate, with Jacobians $_t = \partial g/\partial\,|_{\boldsymbol{\mu}_{t-1}}$ and $_t = \partial h/\partial\,|_{\bar{\boldsymbol{\mu}}_t}$.

Algorithm M.4. Extended Kalman filter stepekf **Predict:** $\bar{\boldsymbol{\mu}}_t = g(\boldsymbol{\mu}_{t-1}, _t)$, $\bar{\Sigma}_t = _t \Sigma_{t-1} {}_t^\top + \mathbf{R}_t$. **Update:** $_t = _t \bar{\Sigma}_t {}_t^\top + \mathbf{Q}_t$ (innovation covariance), $_t = \bar{\Sigma}_t {}_t^\top {}_t^{-1}$ (Kalman gain), $\boldsymbol{\mu}_t = \bar{\boldsymbol{\mu}}_t + _t \big(\boldsymbol{z}_t - h(\bar{\boldsymbol{\mu}}_t)\big)$, $\Sigma_t = (- _t)\bar{\Sigma}_t$.

Theorem M.5. *Kalman filter (linear case)kf If $g(,) = \mathbf{A} + \mathbf{B}$ and $h() = \mathbf{C}$, the Jacobians are $_t = \mathbf{A}$ and $_t = \mathbf{C}$, and the EKF equations give the* exact *posterior. This is Kalman's recursive filter for discrete-time linear systems.*

Proof. The prediction is a linear map of a Gaussian plus Gaussian noise, hence Gaussian with the stated mean and covariance. For the update, $(_t, \boldsymbol{z}_t)$ is jointly Gaussian with $\mathrm{cov}(\boldsymbol{z}_t) = \mathbf{C}\bar{\Sigma}\mathbf{C}^\top + \mathbf{Q} =$ and $\mathrm{cov}(_t, \boldsymbol{z}_t) = \bar{\Sigma}\mathbf{C}^\top$. Conditioning a joint Gaussian gives $\mathbb{E}[| \boldsymbol{z}] = \bar{\boldsymbol{\mu}} + \bar{\Sigma}\mathbf{C}^{\top -1}(\boldsymbol{z} - \mathbf{C}\bar{\boldsymbol{\mu}})$ and $\mathrm{cov} = \bar{\Sigma} - \bar{\Sigma}\mathbf{C}^{\top -1}\mathbf{C}\bar{\Sigma} = (-\mathbf{C})\bar{\Sigma}$. □

Intuition

The gain is a trust dial between model and sensor. In 1D with $\bar{\sigma}^2 = 2$ and measurement noise $q = 1$: $K = \bar{\sigma}^2/(\bar{\sigma}^2 + q) = 2/3$, so the posterior mean moves two thirds of the way from the prediction (1) toward the measurement (2), giving 5/3, exactly Example **??**; the variance shrinks to $(1-K)\bar{\sigma}^2 = 2/3$. For nonlinear g, h the EKF is only an approximation: it converges in the linear case but can *diverge* if nonlinearities are large, because Jacobians are evaluated once, at the current estimate.

M.4 The EKF-SLAM State 11

EKF-SLAM estimates the camera and the positions of all the points with *one* Gaussian: a single mean and a single joint covariance,

$$\boldsymbol{\mu} = \begin{pmatrix} \boldsymbol{x}_c \\ 1 \\ \vdots \\ N \end{pmatrix}, \qquad \Sigma = \begin{pmatrix} \Sigma_{cc} & \Sigma_{cm} \\ \Sigma_{mc} & \Sigma_{mm} \end{pmatrix}.$$

In MonoSLAM the camera block $\boldsymbol{x}_c$ has 13 numbers (position 3, orientation quaternion 4, linear velocity 3, angular velocity 3); each point adds 3, so the state has $13 + 3N$ entries and Σ has $(13 + 3N)^2$. The off-diagonal blocks are the *correlations* between the camera and the points and between the points. The filter cycle is: state prediction (the motion model moves the camera block only), measurement prediction, measurement, data association, update (all blocks change). The camera starts at the origin of its own frame; a new point enters the state when first observed.

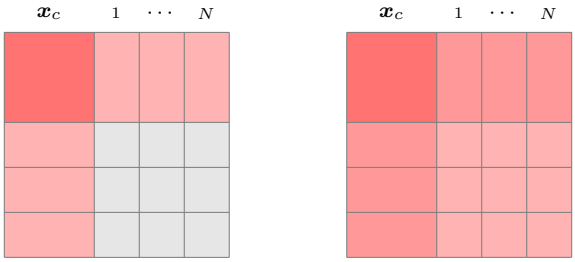


Figure 136: Which blocks of the EKF-SLAM covariance change (red). Left, prediction: only the camera block and its correlations, Σ_{mm} is untouched. Right, update: every block changes.

Proposition M.6. *Cost of EKF-SLAMekfcost The prediction touches only Σ_{cc} and Σ_{cm} (the Jacobian of g is* $\mathrm{diag}(_c,)$*), but the update has a dense gain , so it modifies the* whole *covariance: $O(N^2)$ time per step and $O(N^2)$ memory for N points.*

Proof. Since g moves only the camera, $_t = \mathrm{diag}(_c,)$, giving $\bar{\Sigma}_{cc} = _c \Sigma_{cc} {}_c^\top + \mathbf{R}$, $\bar{\Sigma}_{cm} = _c \Sigma_{cm}$ and $\bar{\Sigma}_{mm} = \Sigma_{mm}$. In the update, $\Sigma \leftarrow \Sigma - ^\top$ with $= \bar{\Sigma}^{\top -1}$; because all points are correlated with the camera, has no zero blocks and the rank-update is dense. □

Intuition

Numbers: with $N = 100$ points the state has 313 entries and Σ about 98,000 numbers: trivial. With $N = 10{,}000$ points, $30{,}013^2 \approx 9 \times 10^8$ entries, about 7.2 GB in double precision, *and* touched at every frame. Correlations are a feature (observing one point improves the others) but they are what makes the filter expensive.

M.5 Example: MonoSLAM 12

MonoSLAM is EKF-SLAM with a single camera: it recovers the 3D trajectory of a camera moving rapidly through an unknown scene, at 30 Hz on a standard PC, and drives live augmented reality from a hand-held camera.

- **Motion model:** constant linear and angular velocity, with unknown accelerations as Gaussian noise; a hand-held camera has no control input. With $\boldsymbol{x}_c = (\mathbf{r}, \mathbf{q}, \mathbf{v}, \boldsymbol{\omega})$: $\mathbf{r} \leftarrow \mathbf{r} + (\mathbf{v} + \mathbf{V})\Delta t$, $\mathbf{q} \leftarrow \mathbf{q} \otimes \mathbf{q}\big((\boldsymbol{\omega} + \boldsymbol{\Omega})\Delta t\big)$, $\mathbf{v} \leftarrow \mathbf{v} + \mathbf{V}$, $\boldsymbol{\omega} \leftarrow \boldsymbol{\omega} + \boldsymbol{\Omega}$, with $\mathbf{V}, \boldsymbol{\Omega}$ Gaussian impulses.
- **Measurement model:** pinhole projection of each point, with radial distortion.
- **Active search:** the innovation covariance defines an ellipse around each predicted feature, and only these ellipses are searched, for at most 10 or 12 features per frame.
- About 100 features at 30 Hz take 19 ms of the 33 ms of a frame.

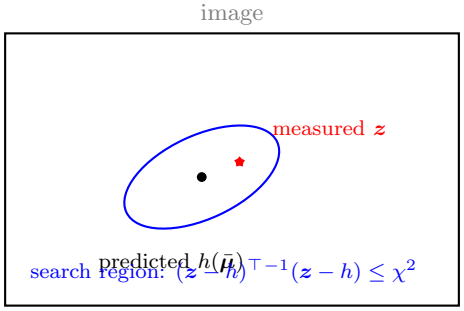


Figure 137: Active search (slide 12): the EKF predicts where a feature should appear and how uncertain that prediction is; only the ellipse is searched.

Intuition

Instead of scanning the whole image for each of 100 features, the filter says "it must be about *here*, give or take this much". Searching a few small ellipses is why a 1990s-style EKF reaches 30 Hz; and the ellipse shrinks as the map is better known.

[4.8cm]12MonoSLAM: search ellipses in a wide-angle image (top) and predicted camera paths of the motion model (bottom).

M.6 Limits of Filtering 13

- **Cost:** every update touches the whole covariance, $O(N^2)$ per step and in memory. The map must stay small: about 100 features in MonoSLAM at 30 Hz.
- **Linearization:** Jacobians are evaluated once, at the current estimate, and never recomputed; accuracy can fall well below bundle adjustment, and the EKF can diverge if nonlinearities are large.
- **Past poses:** all poses but the current one are marginalized after every frame; what they contributed survives only in the Gaussian over the current pose and the map.

Proposition M.7. *Marginalization creates fill-infillin In information form $\Lambda = \Sigma^{-1}$, partition the variables as $(\mathbf{a}, \mathbf{b})$. Marginalizing out $\mathbf{b}$ gives the information matrix $\Lambda_{aa} - \Lambda_{ab}\Lambda_{bb}^{-1}\Lambda_{ba}$ (Schur complement). Hence all variables that were connected to $\mathbf{b}$ become directly connected to each other.*

Proof. For a Gaussian with information matrix Λ, the marginal covariance of $\mathbf{a}$ is the corresponding block of Λ^{-1}, which by the block-inversion formula equals $(\Lambda_{aa} - \Lambda_{ab}\Lambda_{bb}^{-1}\Lambda_{ba})^{-1}$. The term $\Lambda_{ab}\Lambda_{bb}^{-1}\Lambda_{ba}$ is non-zero

in block (k, l) whenever k and l are both neighbours of $\mathbf{b}$. □

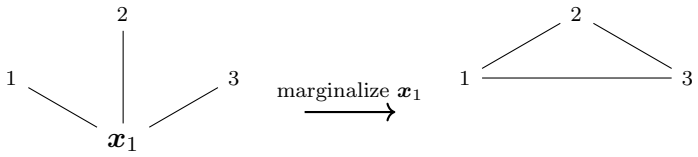


Figure 138: Marginalizing a pose that saw three points turns a sparse star into a dense clique: past poses cannot be dropped for free.

Intuition

Filtering throws away the past poses, and the price of forgetting is a dense belief over everything they saw. Graph-based SLAM (next section) keeps the past poses as explicit variables, so the system stays sparse and can be *relinearized*.

N Graph-Based SLAM: Factor Graphs and Least Squares *(slides 14–22)*

N.1 Graph-Based SLAM 15

Graph-based SLAM keeps *every keyframe pose* and represents the problem as a graph. Every node is a camera pose or a map point; every edge is a spatial constraint between two nodes, given by a (uncertain) measurement. Observing a previously seen area creates constraints between non-successive poses. The **front end** builds the graph from raw data; the **back end** finds the node configuration that minimizes the error of all constraints.

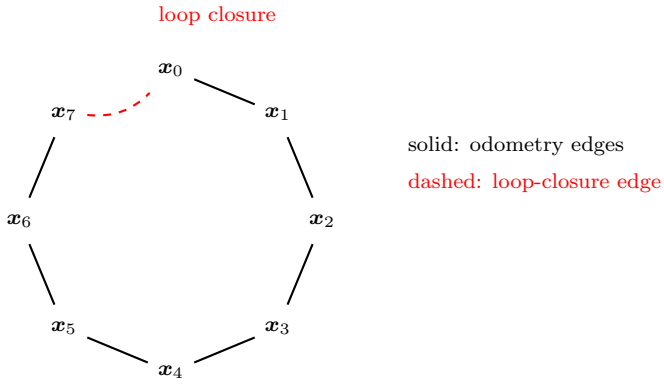


Figure 139: A pose graph (slide 15): odometry links consecutive poses; recognizing a visited place adds a constraint between non-successive poses.

N.2 Factor Graphs 16

Definition N.1. Factor graphfg A *factor graph* is a bipartite graph with *variable* nodes (circles: the unknown poses and points) and *factor* nodes (squares). Each factor ϕ_k is joined only to the variables Θ_k it depends on, and the global function is the product

$$\phi(\Theta) = \prod_k \phi_k(\Theta_k).$$

Given the measurements $\mathcal{Z}$, the posterior is proportional to such a product: $p(\Theta \mid \mathcal{Z}) \propto \prod_k \phi_k(\Theta_k)$.

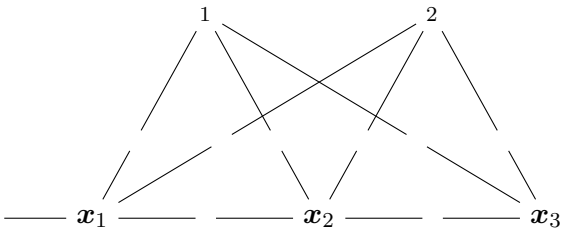


Figure 140: Toy factor graph (redrawn in the spirit of Dellaert and Kaess, Fig. 1.3): three poses, two points, nine factors, namely one prior, two motion factors, and one factor per measurement (six).

Intuition

Think of a factor as "one piece of evidence", and its edges as the unknowns it talks about. A GPS reading talks about one pose; an odometry reading about two consecutive poses; a feature observation about one pose and one point. The factor graph is just the table of contents of who-constrains-whom, which will turn out to be exactly the sparsity pattern of the linear system.

N.3 Visual SLAM as a Factor Graph 17

The variables are the camera poses $_i$ and the points $_j$. Each observation adds one *reprojection factor*, a Gaussian on the residual of lecture 05:

$$\phi_{ij}(_i,_j) \propto \exp\Big(-\tfrac{1}{2}\|\mathbf{r}_{ij}\|^2_{\Sigma_{ij}^{-1}}\Big), \qquad \mathbf{r}_{ij} = \pi(_{ij}) - \mathbf{u}_{ij},$$

with Σ_{ij} the covariance of the pixel noise. A *prior* factor on $_1$ anchors the solution in space; with one camera the scale stays free unless one distance is fixed too (lecture 05). With these Gaussian factors, the most probable variables are the bundle adjustment solution of lecture 05.

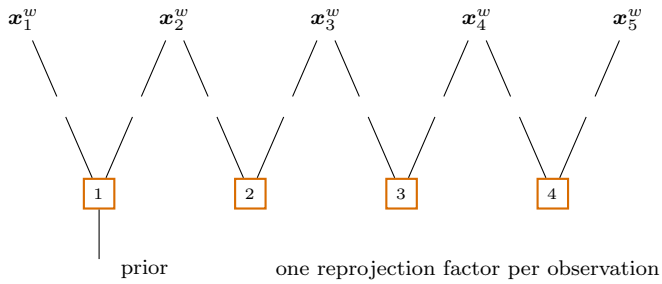


Figure 141: Visual SLAM factor graph (slide 17): each pose sees two neighbouring points.

Intuition

Bundle adjustment is not a different algorithm from SLAM's back end: it is the MAP estimate of this graph. The prior on $_1$ pins the *gauge*: the whole reconstruction could otherwise slide or rotate in space (and, for one camera, scale) with no change in error.

N.4 The Pose Graph 18

The pose graph keeps only the poses. Each edge is a measured *relative pose* between two keyframes i and j:

- **Odometry edge:** between consecutive keyframes, from the motion estimated between them.
- **Loop-closure edge:** between non-successive keyframes that observe the same part of the scene, a virtual measurement of keyframe j seen from keyframe i.

Each edge carries a measured relative pose $\mathbf{Z}_{ij}$ and an information matrix Ω_{ij}. Eliminating the points exactly would connect every pair of poses that saw a common point (Proposition **??**); the pose graph is kept sparse *by construction*, by using only a selected set of relative-pose edges.

N.5 From Factors to Least Squares 19

Each factor is a Gaussian in its residual, $\phi_k \propto \exp(-\frac{1}{2}{}_k^\top \Omega_{kk})$.

Theorem N.2. *MAP on a factor graph is nonlinear least squaresmap The most probable variables minimize a sum of squared Mahalanobis errors:*

$$\Theta^\star =_\Theta \prod_k \phi_k(\Theta_k) = \arg\min_\Theta \; F(\Theta), \qquad F(\Theta) = \sum_k {}_k(\Theta_k)^\top \Omega_k \, {}_k(\Theta_k),$$

where ${}_k$ is the residual of factor k (predicted minus actual measurement) and Ω_k the inverse covariance of the measurement. The sum runs over prior, motion and measurement terms alike.

Proof. $\log \prod_k \phi_k = \sum_k \log \phi_k = \text{const} - \frac{1}{2}\sum_k {}_k^\top \Omega_{kk}$, and log is monotone, so maximizing the product is minimizing the sum. □

Intuition

Logs turn products into sums, Gaussians into parabolas. Worked number: two measurements of the same 1D position, 10.0 with $\sigma = 1$ ($\Omega = 1$) and 10.6 with $\sigma = 0.5$ ($\Omega = 4$). Minimizing $1\,(x-10)^2 + 4\,(x-10.6)^2$ gives $x = \frac{1\cdot 10 + 4\cdot 10.6}{5} = 10.48$: the more certain reading pulls four times harder. This is bundle adjustment of lecture 05, *weighted*.

N.6 Error and Information Matrix of an Edge 20

An edge measures the pose of keyframe j seen from keyframe i, $\mathbf{Z}_{ij}$. Since each pose maps world to camera, the poses predict it as ${}_{ij}^{-1}$, which maps camera-j coordinates to camera i. The residual compares the two:

$${}_{ij} = \log\!\left(\mathbf{Z}_{ij}^{-1}\, {}_i\, {}_j^{-1}\right)^\vee \in \mathbb{R}^6, \qquad F = \sum_{(i,j)} {}_{ij}^\top \Omega_{ij}\, {}_{ij}.$$

Here $\mathbf{Z}_{ij}^{-1}$ maps from i to j, so ${}_{ij} = \mathbf{0}$ exactly when the poses agree with the measurement; Ω_{ij} is the inverse covariance of $\mathbf{Z}_{ij}$ and encodes its uncertainty: the larger Ω_{ij}, the more the edge matters in the optimization.

Intuition

Think of every edge as a spring whose rest length is the measurement and whose stiffness is Ω_{ij}. Odometry springs are stiff but there are many of them (errors accumulate); a loop-closure spring is softer but connects two far-away parts of the path. Numbers: $\sigma_{\text{odom}} = 0.1\,\text{m}$ gives $\Omega = 100$, while $\sigma_{\text{loop}} = 1\,\text{m}$ gives $\Omega = 1$: a loop edge is $100\times$ weaker per unit squared error, yet it can still bend a long chain because the chain's accumulated error is spread over hundreds of springs.

N.7 Gauss-Newton on the Pose Graph 21

Around the current poses, every residual is linearized in the increments of lecture 05, ${}_i \leftarrow \exp(\delta\boldsymbol{\xi}_i^\wedge)\,{}_i$: ${}_{ij}(\mathbf{x} \boxplus \delta\mathbf{x}) \approx {}_{ij} + {}_{ij}\,\delta\mathbf{x}$.

Algorithm N.3. Gauss–Newton / Levenberg–Marquardt on the pose graphgn Repeat until convergence, starting from a good initial guess (e.g. chained odometry):

1. $= \sum_{ij} {}_{ij}^\top \Omega_{ij}\, {}_{ij}, \quad = \sum_{ij} {}_{ij}^\top \Omega_{ij}\, {}_{ij}$.
2. Solve the normal equations $\delta\mathbf{x} = -$ (Gauss–Newton), or $(+\lambda)\,\delta\mathbf{x} = -$ (Levenberg–Marquardt).
3. Update $\mathbf{x} \leftarrow \mathbf{x} \boxplus \delta\mathbf{x}$. In LM, the damping λ is doubled when a step increases the error and halved otherwise.

Derivation of the normal equations. Substituting the linearization, $F(\mathbf{x} \boxplus \delta\mathbf{x}) \approx \sum(+\delta\mathbf{x})^\top \Omega (+\delta\mathbf{x}) = c + 2\,{}^\top \delta\mathbf{x} + \delta\mathbf{x}^\top \, \delta\mathbf{x}$. Setting the gradient $2\, + 2\delta\mathbf{x}$ to zero gives $\delta\mathbf{x} = -$. □

Proposition N.4. *Gauge freedomgauge The error F is unchanged if all poses move together, ${}_i \mapsto {}_i \mathbf{S}$ for any rigid $\mathbf{S}$. Hence is singular (null space of dimension 6), and the graph is anchored by holding ${}_1$ fixed, as the prior on the first pose did.*

Proof. $({}_i\mathbf{S})({}_j\mathbf{S})^{-1} = {}_i \, {}_j^{-1}$, so every ${}_{ij}$ is invariant. Differentiating along the 6-parameter family shows $\mathbf{v} = \mathbf{0}$ for its tangent directions $\mathbf{v}$, hence $\mathbf{v} = \sum^{\top} \Omega \mathbf{v} = \mathbf{0}$. □

Intuition

Pose graphs only know *relative* information, like a map with distances but no coordinates: you can pick the whole thing up and put it down elsewhere. Fixing one pose nails it to the table. LM damping is a safety belt: a large λ takes small, gradient-like steps; a small λ takes full Gauss–Newton steps. This handles highly non-convex problems better, and loop closures are exactly where the problem is non-convex.

N.8 Sparsity of the Normal Equations 22

A residual depends on ${}_i$ and ${}_j$ only, so its Jacobian has two non-zero blocks, ${}_{ij} = [\cdots \ \mathbf{A}_{ij} \ \cdots \ \mathbf{B}_{ij} \ \cdots]$, and each edge adds to *four blocks of and two of* :

$$ {}_{ii} \mathrel{+}= \mathbf{A}^{\top}\Omega\mathbf{A}, \quad {}_{ij} \mathrel{+}= \mathbf{A}^{\top}\Omega\mathbf{B}, \quad {}_{ji} \mathrel{+}= \mathbf{B}^{\top}\Omega\mathbf{A}, \quad {}_{jj} \mathrel{+}= \mathbf{B}^{\top}\Omega\mathbf{B}, \qquad {}_{i} \mathrel{+}= \mathbf{A}^{\top}\Omega, \quad {}_{j} \mathrel{+}= \mathbf{B}^{\top}\Omega. $$

Proposition N.5. *Sparsity pattern of sparse The block ${}_{ij}$ is non-zero if and only if there is an edge between i and j. The block pattern of is therefore the adjacency matrix of the graph plus the diagonal, and is assembled by summing the contribution of each edge.*

Proof. Off-diagonal blocks only receive the terms $\mathbf{A}^{\top}\Omega\mathbf{B}$ above, one per edge joining i and j; all other products ${}_{kl}^{\top}\Omega_{kl}$ are zero in block (i, j). □

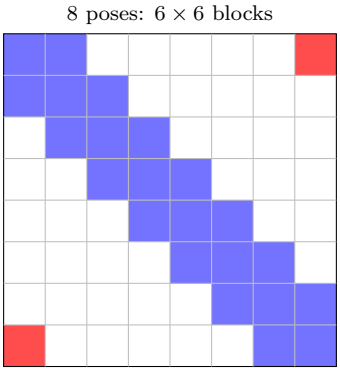


Figure 142: Block sparsity of for the ring pose graph of slide 15: diagonal and first off-diagonals come from odometry (blue), the two red blocks come from the single loop closure.

Intuition

Worked numbers: 1000 poses with 6 degrees of freedom give $\in \mathbb{R}^{6000\times6000}$, i.e. 36 million entries if dense. But there are only about 1000 diagonal and 2×999 off-diagonal blocks of 36 numbers, roughly 108,000 non-zeros (0.3%). Sparse solvers (sparse Cholesky, conjugate gradients) exploit this, which is why graph-based SLAM scales to the 11 km running example while a dense EKF covariance cannot.

O The Visual SLAM Pipeline: ORB-SLAM *(slides 23–32)*

O.1 Pipeline Overview 24

ORB-SLAM, a keyframe-based monocular SLAM system, runs three threads in parallel: **tracking** localizes the camera at every frame and decides when to insert a keyframe; **local mapping** processes each new keyframe and runs bundle adjustment around it; **loop closing** searches for a loop at every keyframe and corrects the drift by pose graph optimization. The *front end* abstracts the images into features and data associations; the *back end* estimates the variables from them (Section 6.3).

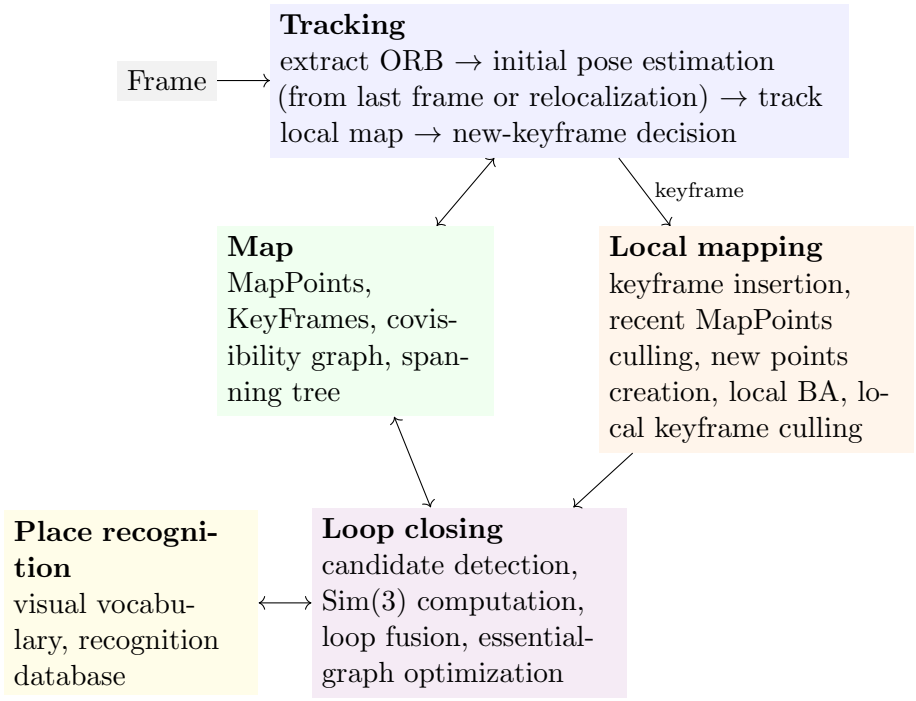


Figure 143: ORB-SLAM threads and data structures (redrawn from the slide 24 block diagram).

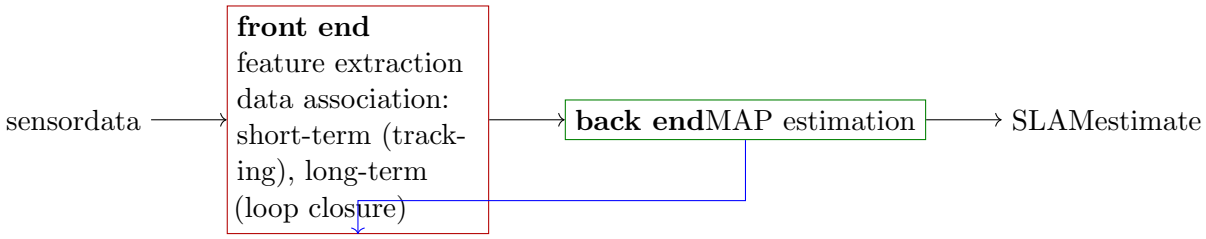


Figure 144: Front end / back end split (slide 24, right): the back end also feeds predictions back to guide data association.

Intuition

The split mirrors the lecture: Section 6.3 was the back end (sparse least squares, assumed data associations), and this section is a front end that produces those associations fast enough for 30 frames per second. Three threads exist because the three jobs have different deadlines: tracking must finish within one frame, local mapping within a keyframe interval, loop closing can take longer.

O.2 ORB Features and Matching 25

ORB-SLAM uses the same features for tracking, mapping, relocalization and loop detection. As in lecture 05, a feature is a keypoint with a descriptor.

- Extraction must take much less than the 33 ms between frames, which excludes SIFT at about 300 ms.
- **ORB**: corners from the FAST detector, given an orientation, found at 8 scale levels a factor 1.2 apart, each with a binary descriptor; the orientation makes it rotation invariant.
- 1000 keypoints per image up to 752×480 pixels, 2000 on larger images (1241×376), spread over a grid.

Definition O.1. Hamming distancehamming For binary descriptors $\mathbf{d}, \mathbf{d}' \in \{0,1\}^n$ the *Hamming distance* is the number of differing bits, $\|\mathbf{d} \oplus \mathbf{d}'\|_1 = \text{popcount}(\mathbf{d} \text{ XOR } \mathbf{d}')$.

Intuition

Matching two descriptors is one XOR and one popcount instruction: orders of magnitude cheaper than the Euclidean distance between 128 floats of SIFT. Example with 8 bits: `10110010` vs. `10011010` differ in 2 positions, so $d = 2$. The 8-level pyramid with factor 1.2 covers a total scale range of $1.2^7 \approx 3.6$, which is why ORB matches survive strong zoom changes.

[4.2cm]25ORB matches between a frame and a keyframe under a strong change of scale.

O.3 Map Initialization 26

The first map needs the relative pose of two frames and triangulated points, for planar and general scenes alike. Between a reference frame 1 and the current frame 2, two models are fitted in parallel with RANSAC: a homography $\mathbf{x}_2 \simeq_{21} \mathbf{x}_1$ (4 points, planar scene) and a fundamental matrix $\mathbf{x}_2^\top \mathbf{F}_{21} \mathbf{x}_1 = 0$ (8 points, general scene), each scored by its symmetric transfer error

$$S_M = \sum_i \Big[\rho_M\big(d^2(\mathbf{x}_1^i, M^{-1}\mathbf{x}_2^i)\big) + \rho_M\big(d^2(\mathbf{x}_2^i, M\mathbf{x}_1^i)\big) \Big].$$

Algorithm O.2. ORB-SLAM map initializationinit

1. Fit $_{21}$ and $\mathbf{F}_{21}$ with RANSAC and compute the scores S_H, S_F.
2. Select the homography if $S_H/(S_H + S_F) > 0.45$ (planar scenes and low parallax); otherwise the fundamental matrix.
3. Recover the motion hypotheses: 8 from the essential matrix or 4 from the homography (lecture 05); triangulate with all of them.
4. If there is no clear winner, *create no map* and try with new frames.
5. Refine the first map by full bundle adjustment; its scale is arbitrary (lecture 05).

Intuition

A homography explains almost anything seen with *no parallax* (a plane, or pure rotation), but then depth is ill-defined and triangulation is unreliable. ORB-SLAM prefers to *wait for parallax* rather than start from a wrong planar map; the slide shows two other systems that started wrongly and drifted, while ORB-SLAM, which waited, did not.

[5.2cm]26Some time after initialization: two other systems (top, middle) started from a wrong planar map; ORB-SLAM (bottom) waited for parallax.

O.4 Tracking 27

The tracking thread estimates the pose of every frame:

1. **Prediction:** a constant-velocity motion model, $_t^{\text{pred}} = (_{t-1}{}_{t-2}^{-1})\,_{t-1}$, predicts the pose and guides the search for the points seen in the last frame; if too few are found, a wider search follows.
2. **Local map:** the keyframes that share points with the frame, and their covisible neighbours; their points are projected into the frame and matched near the projection, at the predicted scale.
3. **Motion-only bundle adjustment:** the reprojection error of lecture 05 over the pose alone, points

fixed, with a robust (Huber) cost,

$$^{\star} = \arg\min \sum_{j} \rho_{\text{Huber}}\Big(\|\mathbf{u}_j - \pi(_j)\|\Sigma^{-1^2} \Big).$$

Intuition

Only 6 unknowns are optimized, so this runs in a few milliseconds per frame. The Huber cost behaves quadratically for small errors and linearly for large ones, so one wrong match (an outlier with a 50-pixel error) cannot dominate 100 good ones. **PTAM**, a monocular system for small augmented-reality workspaces, predicts with a decaying velocity and tracks coarse to fine: 50 points at the coarsest image scale, then up to 1000.

[4.8cm]27Keyframes (blue), current camera (green), map points (black) and local map (red).

O.5 Keyframes 28

PTAM splits SLAM into two threads that run in parallel: **tracking** processes every frame in real time (freed from updating the map, it can process each image more thoroughly), and **mapping** works on keyframes only: many video frames are redundant, and a smaller set of useful keyframes is refined by bundle adjustment without strict real-time limits.

- PTAM: a frame becomes a keyframe if tracking is good, more than twenty frames have passed, and the camera is a minimum distance (set by the mean scene depth) from the existing keyframes.
- ORB-SLAM instead inserts keyframes *as fast as possible*, which keeps tracking robust to rotations, and culls the redundant ones in local mapping.

Intuition

Keyframes are the "postcards" of the trip: you do not keep every frame of a 30 Hz video (11 km would be hundreds of thousands), only enough views to triangulate the scene. ORB-SLAM's strategy is generous insertion followed by pruning: it is easier to remove a redundant postcard than to recover a view you never kept.

[5cm]28A desk mapped by PTAM: two views of the map with its keyframes (top), and the 57 keyframes (bottom).

O.6 Local Mapping 29

Definition O.3. Covisibility graphcovis Nodes are keyframes; an edge joins two keyframes that share at least 15 map points, weighted by the number of shared points.

For every new keyframe the local mapping thread:

1. inserts it in the covisibility graph;
2. culls recent points: a point stays only if it is found in more than 25% of the frames predicted to see it, and from at least three keyframes;
3. triangulates new points from unmatched features in covisible keyframes, checking depth, parallax, reprojection error and scale;
4. runs **local bundle adjustment** on the keyframe, its covisible keyframes and all their points; the other keyframes that see these points stay fixed;
5. culls redundant keyframes, whose points are 90% seen by at least three other keyframes.

Intuition

The covisibility graph answers "who looks at the same things as me?". Optimizing only that neighbourhood keeps the cost bounded while the map grows to kilometres: *local* BA is constant-time per keyframe, unlike EKF-SLAM whose cost grows with the whole map. Fixing the outer keyframes plays the role of the prior on $_1$ for the local problem.

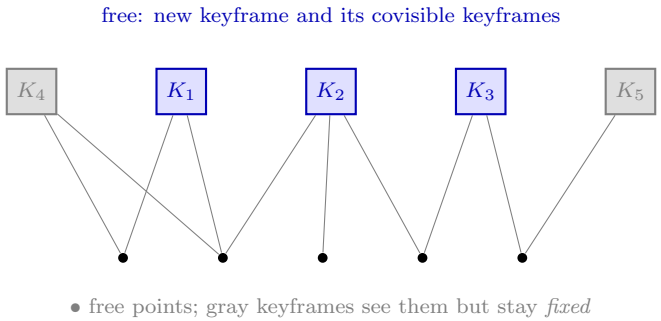


Figure 145: Local bundle adjustment (slide 29, step 4): only a small neighbourhood is optimized; fixed keyframes anchor it to the rest of the map.

[5cm]29Covisibility graph of an indoor handheld sequence (TUM RGB-D).

O.7 Loop Detection with Bags of Binary Words 30

A loop is detected by *appearance*: each image is compared with all the past ones, fast enough for every frame.

- **Vocabulary:** binary descriptors clustered offline into a tree of 10 branches and 6 levels whose leaves are the visual words; an image becomes a vector of word weights.
- **Inverse index:** for each word, the images that contain it, so a query visits only images sharing words with it.
- **Checks:** the score is normalized by that of the previous image; a match must agree with previous matches and be supported by a fundamental matrix with at least 12 RANSAC inliers.
- ORB-SLAM uses ORB words, groups candidates by covisibility rather than by time, and accepts a loop after three consecutive consistent candidates. 21.6 ms per image on average, over 26292 images.

Proposition O.4. *Why a hierarchical vocabulary is fastvocab A tree with branching factor k and L levels has k^L leaves (words), but quantizing a descriptor needs only kL comparisons.*

Proof. At each level the descriptor is compared with the k children of the current node and descends into the closest one; there are L levels. □

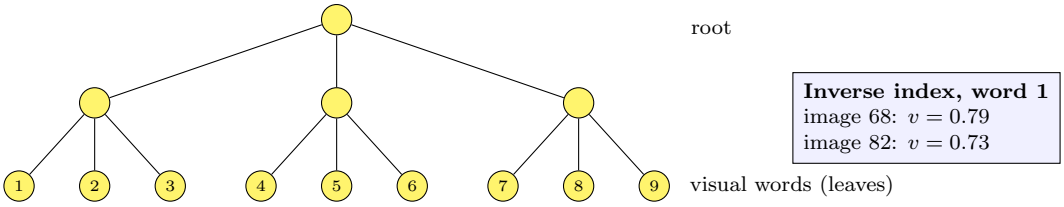


Figure 146: Vocabulary tree and inverse index (slide 30; the sketch shows $k = 3,\ L = 2$, the system uses $k = 10,\ L = 6$).

Intuition

Numbers: 10 branches and 6 levels give $10^6 = 1{,}000{,}000$ words, but a descriptor is quantized with only $10 \times 6 = 60$ comparisons. The inverse index is the book index idea: to find pages about "Cholesky" you do not read the whole book. Normalizing by the score of the previous image avoids false alarms in places that simply look similar to everything (corridors). The score between two images lies between 0 and 1.

O.8 Loop Correction and Scale Drift 31

One camera cannot observe scale, so the scale drifts too: a monocular map drifts in **seven** degrees of freedom (rotation, translation and scale). A pose graph over closes the loop but *keeps the scale drift*. The poses become similarities (lecture 01).

Definition O.5. Similarity transformation sim3 $\mathbf{S} = \begin{pmatrix} s\mathbf{R} & \mathbf{t} \\ \mathbf{0}^\top & 1 \end{pmatrix}$, with $\mathbf{R} \in SO(3)$, $\mathbf{t} \in \mathbb{R}^3$, $s > 0$; it acts on points as $\mapsto s\mathbf{R} + \mathbf{t}$ and has 7 degrees of freedom. The pose-graph residual has the same form as before, $_{ij} = \log(\mathbf{Z}_{ij}^{-1}\mathbf{S}_i\mathbf{S}_j^{-1})^\vee \in \mathbb{R}^7$.

- Odometry edges keep their measured motion with scale 1; the loop edge carries the measured change of scale.
- **ORB-SLAM:** the loop similarity comes from 3D–3D point matches, by RANSAC and Horn's method; duplicated points are fused; the pose graph over is optimized on the *essential graph*: a spanning tree, the covisibility edges with at least 100 shared points, and the loop edges.

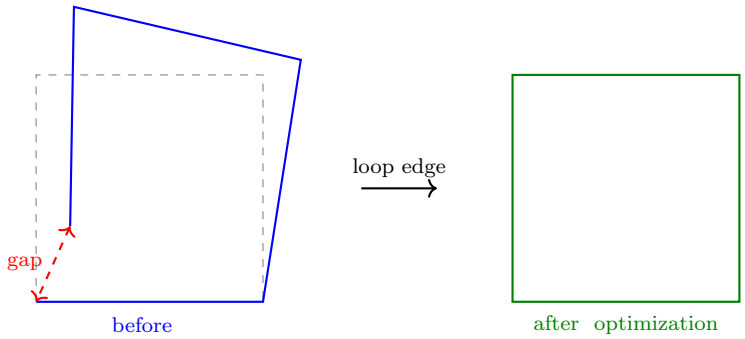


Figure 147: Loop closure (sketch of the slide 31 experiment): drift in rotation, translation and scale leaves the loop open; the loop edge and its scale are distributed along the cycle.

Intuition

Numbers: if after a loop the end of the trajectory is estimated at relative scale 1.10 with respect to the start, the loop edge says so, and the optimization spreads $\ln 1.10 \approx 0.0953$ over, say, 100 keyframes, about 0.00095 of log-scale each. A optimization has no scale variable, so it can bend the path but the scale error stays: the slide shows the 6 DoF result still distorted while the 7 DoF result matches the aerial photo.

[6.2cm]31A square circled by a hand-held camera: (a) before optimization, (b) over (6 DoF), (c) over (7 DoF), (d) aerial photo.

O.9 Relocalization 32

When tracking is lost, after an occlusion or an abrupt motion, the camera is localized again in the existing map:

1. the frame is converted into a bag of words, and the place recognition database returns candidate keyframes;
2. for each candidate, ORB features are matched to its map points, and RANSAC with PnP (lecture 05) estimates a camera pose;
3. a pose with enough inliers is refined and guides the search for more matches; with enough inliers, tracking resumes.

On indoor handheld sequences with ground truth (TUM RGB-D), 78.4% of 2769 frames are relocalized in a map built from the first 30 s; with people walking through the scene, 77.9% of 859 frames.

Intuition

Relocalization reuses everything already built: the same bag-of-words database that detects loops here answers "where am I?" from a single image. It is a *global* localization problem (no prior pose), solved by PnP inside RANSAC, which tolerates the moving people that spoil many of the matches.

[4.2cm]32A relocalization among moving people: ORB matches between the frame and a keyframe.

P Research Directions *(slides 33–40)*

P.1 Continuous Time: Rolling-Shutter Cameras 34

A rolling-shutter camera exposes every line of the image for a different period, each one more delayed than the last. When the camera moves during the exposure, the image appears distorted and skewed. So far every image had one pose, as with a global shutter, where all pixels are exposed over the same duration. Earlier work removed the distortion first, to recreate a global-shutter image. On a simulated monocular sequence, the loop-closure position error is 0.0472 of the distance travelled when time is modelled as for a global shutter, and 0.0057 when the rolling shutter is modelled.

Intuition

The numbers: 4.7% versus 0.57% of the path length, an 8× reduction ($0.0472/0.0057 \approx 8.3$) just from admitting that the rows of an image are taken at different times. Each scanline is a tiny camera with its own timestamp, so the *trajectory*, not a pose per image, becomes the unknown.

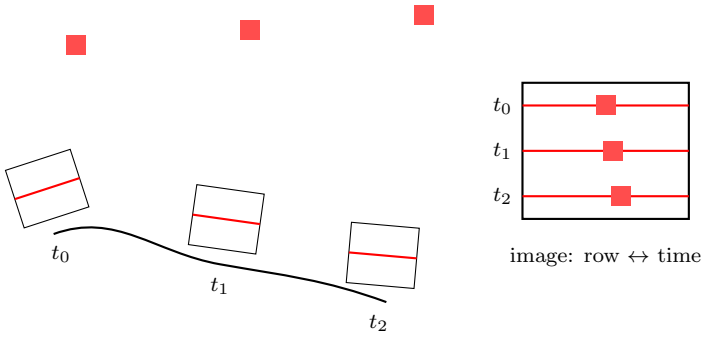


Figure 148: Each scanline is a camera with its own time along the trajectory (redrawn from slide 34); together they form the image.

[3.8cm]34A rolling-shutter warp (distorted and skewed building facade).

P.2 Continuous Time: Cumulative B-Splines on 35

The trajectory becomes a continuous function of time, a *cumulative cubic B-spline* built with the Lie algebra (lecture 05). Control poses $_i$ sit at uniform times t_i, Δt apart. For $t \in [t_i, t_{i+1})$ let $u = (t - t_i)/\Delta t \in [0, 1)$; the four poses $_{i-1}, \ldots, _{i+2}$ give

$$(t) = \exp\big(\tilde{B}_3(u)\,\boldsymbol{\delta}_3\big)\exp\big(\tilde{B}_2(u)\,\boldsymbol{\delta}_2\big)\exp\big(\tilde{B}_1(u)\,\boldsymbol{\delta}_1\big)\,{}_{i-1}, \qquad \boldsymbol{\delta}_j = \log\big({}_{i-1+j}{}^{-1}{}_{i-2+j}\big),$$

with cumulative basis functions

$$\tilde{B}_1 = \tfrac{1}{6}(5 + 3u - 3u^2 + u^3), \quad \tilde{B}_2 = \tfrac{1}{6}(1 + 3u + 3u^2 - 2u^3), \quad \tilde{B}_3 = \tfrac{1}{6}u^3.$$

(Spline written for world-to-camera poses with left-multiplied increments.) $\boldsymbol{\delta}_j$ is the increment from control pose $j-1$ to j, and the trajectory is twice continuously differentiable. Each pixel is projected with the pose at its own time: a point on image row v is observed at $t = t_0 + \frac{v}{h}\,t_{\mathrm{ro}}$, h being the image height and t_{ro} the frame readout time. The control poses are the unknowns of the least squares, optimized over a sliding window.

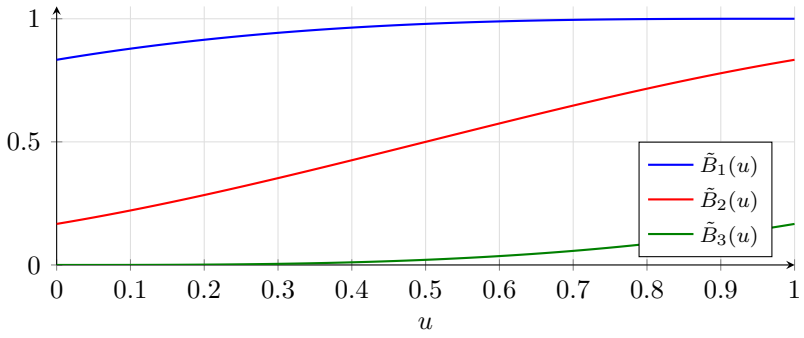


Figure 149: Cumulative cubic B-spline basis functions: at $u = 0$ the weights are $(5/6, 1/6, 0)$ and at $u = 1$ they are $(1, 5/6, 1/6)$, which equals the $u = 0$ weights of the next interval after shifting the control poses by one, so the curve is continuous.

Intuition

A spline is a flexible ruler pinned at control poses: you do not store a pose per row, only a handful of control poses per window, and any timestamp inside gives a smooth pose. Advantage: IMU, rolling-shutter and event-camera measurements at arbitrary timestamps can all be written as functions of the same few unknowns. Further reading: a survey of continuous-time state estimation in robotics (splines and Gaussian processes), Talbot et al., T-RO 2025.

P.3 Active SLAM 36

SLAM is *passive*: it does not guide the motion. **Active SLAM** plans and controls the motion of the robot to build the most accurate and complete model of the environment. It is a decision between two opposite goals, exploring new areas or revisiting those already seen (the *exploration–exploitation dilemma*). With high uncertainty, places where a loop could close carry more information than new frontiers.

Algorithm P.1. Active SLAM loopactive Repeat until a stopping condition is met: (1) identify candidate actions; (2) compute the utility of each; (3) select and execute the best.

The utility is the expected uncertainty of the pose and the map. For a Gaussian belief whose covariance $\Sigma \in \mathbb{R}^{n\times n}$ has eigenvalues $\lambda_1, \ldots, \lambda_n$, the *D-optimality* criterion measures the volume of the uncertainty ellipsoid,

$$\text{D-opt}(\Sigma) = \exp\Big(\tfrac{1}{n}\sum_{k=1}^{n} \log \lambda_k\Big) = \det(\Sigma)^{1/n},$$

to be minimized.

Intuition

Worked number: eigenvalues $(4, 1)$ give $\det^{1/2} = 2$; after an informative view that shrinks the long axis to $(1, 1)$ the criterion is 1. The geometric mean of the eigenvalues is used because it rewards shrinking *any* axis, not only the largest one.

P.4 Active SLAM with an Active Stereo Head 37

The work that named active SLAM: a wheeled robot carries a stereo head with four degrees of freedom. An EKF estimates the robot and a map of point features, and the head fixates one feature at a time; it decides where to look, not where to move. For each visible feature i the EKF predicts the measurement and its innovation covariance $_i$. The head fixates the feature whose position is hardest to predict, the one with the largest 3σ ellipsoid:

$$i^\star =_i \ \text{Vol}_{3\sigma}(_i) =_i \ \det(_i)^{1/2}.$$

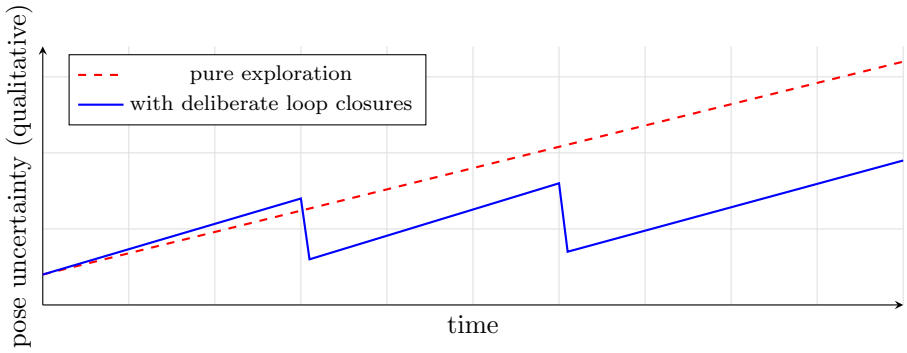


Figure 150: Exploration–exploitation (qualitative sketch): exploring grows uncertainty; revisiting a known place closes a loop and resets it.

Intuition

It is MonoSLAM's search ellipse turned into a decision: look at the thing you are *least sure about*, because a measurement there reduces uncertainty the most. The ellipsoid volume is $\frac{4}{3}\pi\, 3^3\sqrt{\det_i}$ for a 3D innovation, so ranking by $\det_i$ is the same as ranking by volume.

[4.2cm]37The robot and its active head (top); fixated views of features 0, 5, 11, 12 and 14, each marked by a white square (bottom).

P.5 Active Localization: Learning Where to Look 38

Active localization chooses where the camera should look so that it localizes accurately in a known map, built by structure-from-motion.

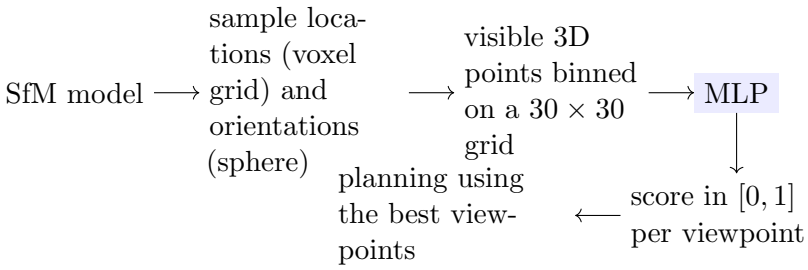


Figure 151: Active localization pipeline (redrawn from slide 38).

- **Candidate viewpoints:** positions on a voxel grid, each with orientations sampled on the sphere; for each, the visible 3D points are binned on a 30×30 grid and a small MLP scores the viewpoint between 0 and 1.
- **Self-supervised:** each training viewpoint is labelled by registering an image rendered from it against the map; one inference takes 0.02 s.
- 72.9% of 92k test viewpoints localize within 5 cm and 0.4°, against 60.7% for the best choice by Fisher information.

Intuition

That is about 67,000 good viewpoints against 56,000, roughly 11,000 more, and 0.02 s per scoring makes planning interactive. The lesson of the slide: *where the points lie in the image* matters more than the Fisher information matrix, which only counts how much information there is, not whether it is spread well enough to constrain pose.

P.6 Active View Selection with Radiance Fields 39

A **radiance field** is a scene model whose parameters $\boldsymbol{\theta}$ hold radiance and density at points in space, rendered into the image seen from a camera pose by volume rendering. **FisherRF** picks the next camera pose $\mathbf{x}$ by its expected information gain about $\boldsymbol{\theta}$. Let $\mathbf{I}_{\mathbf{x}} = {}_{\mathbf{x}}^{\top}{}_{\mathbf{x}}$ be the Fisher information of the view ($_{\mathbf{x}}$ the Jacobian of its rendered image with respect to $\boldsymbol{\theta}$; no image from that view is needed) and $\mathbf{I}_{\mathcal{D}} = \sum_{\mathbf{x}' \in \mathcal{D}} \mathbf{I}_{\mathbf{x}'}$ the sum over the views already captured.

Proposition P.2. *Trace bound on the information gainfisher For a candidate view* $\mathbf{x}$ *with* $\mathbf{I}_{\mathbf{x}} \succeq 0$ *and an (invertible, regularized)* $\mathbf{I}_{\mathcal{D}} \succ 0$*,*

$$\mathrm{IG}(\mathbf{x}) = \tfrac{1}{2} \log\det\big(+ \mathbf{I}_{\mathbf{x}} \mathbf{I}_{\mathcal{D}}^{-1}\big) \;\le\; \tfrac{1}{2}\big(\mathbf{I}_{\mathbf{x}} \mathbf{I}_{\mathcal{D}}^{-1}\big).$$

Proof. $\mathbf{I}_{\mathbf{x}} \mathbf{I}_{\mathcal{D}}^{-1}$ is similar to the PSD matrix $\mathbf{I}_{\mathcal{D}}^{-1/2} \mathbf{I}_{\mathbf{x}} \mathbf{I}_{\mathcal{D}}^{-1/2}$, so it has eigenvalues $\mu_k \ge 0$. Then $\log\det(+\cdot) = \sum_k \log(1+\mu_k) \le \sum_k \mu_k = (\cdot)$ because $\log(1+\mu) \le \mu$. □

With over 200 million parameters, both matrices are kept *diagonal*, so the bound becomes $\frac{1}{2} \sum_k (\mathbf{I}_{\mathbf{x}})_{kk} / (\mathbf{I}_{\mathcal{D}})_{kk}$. With 3D Gaussian Splatting (a scene of 3D Gaussians), views are evaluated at 70 fps.

Intuition

Toy numbers for 3 parameters: $\mathrm{diag}(\mathbf{I}_{\mathbf{x}}) = (1, 4, 0.1)$ and $\mathrm{diag}(\mathbf{I}_{\mathcal{D}}) = (10, 1, 0.1)$ give ratios $(0.1, 4, 1)$, bound $\frac{1}{2} \cdot 5.1 = 2.55$. The view is valuable mostly because it informs the second parameter, which previous views barely constrained. In words: *go where the model is least certain and the new image would speak the loudest.*

[4.5cm]39Information gain over the viewing sphere of a scene, with candidate views (frusta) and four of their renders.

P.7 Outlook: Learned Front Ends 40

Two recent systems keep the back-end machinery of this lecture, least squares over a graph of relative constraints on a Lie group, and replace the feature front end by a network from earlier lectures.

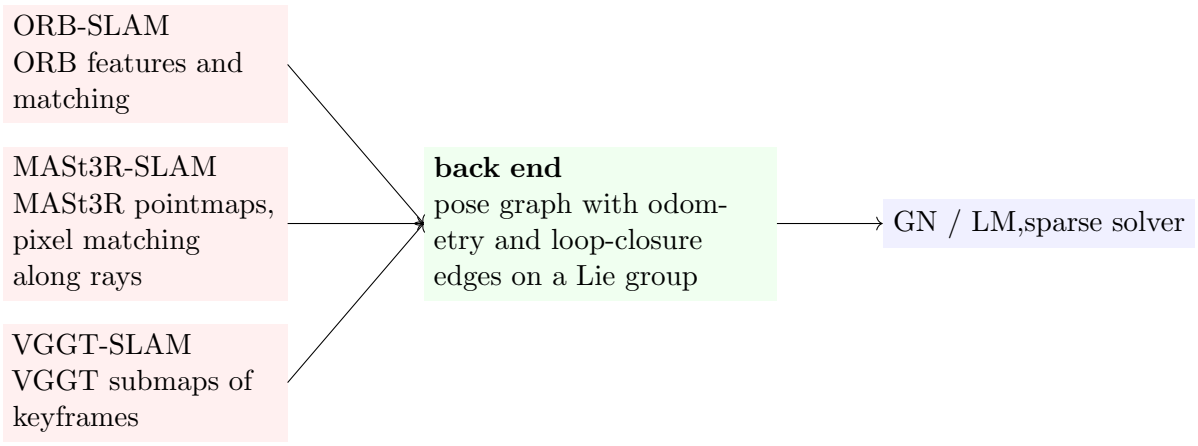


Figure 152: Same back end, different front ends (slide 40).

	MASt3R-SLAM	VGGT-SLAM
Front end	MASt3R (lecture 04) predicts pointmaps of each frame and its keyframe; pixels matched along rays	VGGT (lecture 05) reconstructs a submap of keyframes in one pass
Group	keyframe poses in	each submap gets a homography in (15 DoF)
Solver	Gauss–Newton, sparse Cholesky, on ray errors; loops by image retrieval	pose graph with odometry and loop-closure edges, Levenberg–Marquardt
Speed / camera	about 15 FPS on one GPU; no camera model beyond a single centre	without calibration, submaps agree only up to a projective transformation

Intuition

Degrees of freedom of the group tell how much ambiguity the front end leaves: has 6 (metric, e.g. stereo), has 7 (one camera: unknown scale), and has 15 (an *uncalibrated* reconstruction, where even angles and parallelism are not trustworthy and each submap needs a full projective correction). The back end is the same sparse least squares of Section 6.3; only the group and the residual change.

[4.2cm]40MASt3R-SLAM pipeline (left) and VGGT-SLAM Sim(3) vs. SL(4) comparison renderings (right).

Q Summary 41

Method	Estimate / variables	Cost	Loops / drift	Notes
EKF-SLAM, MonoSLAM	current pose (13) + N points, one Gaussian	$O(N^2)$ per step and memory; ≈ 100 features at 30 Hz	no explicit loop mechanism; past poses marginalized	linearized once; may diverge
Factor graph / pose graph	all keyframe poses (+ points), MAP = sparse nonlinear LS	GN/LM on sparse (Cholesky, CG)	loop edge between non-successive poses	relinearizes; needs good initial guess
PTAM	tracking thread + mapping thread, BA on keyframes	BA outside the real-time loop	no large-scale loop closing	small AR workspaces
ORB-SLAM	ORB features; 3 threads; local BA; essential graph over	local BA bounded; 21.6 ms/image loop detection	bags of binary words; corrects scale drift	relocalization $\approx 78\%$ of frames
Continuous-time splines	control poses of a cumulative B-spline	sliding window of control poses	–	rolling shutter: $4.7\% \to 0.57\%$ error
Active SLAM	choose actions by utility (D-opt)	utility per candidate action	revisit vs. explore	active head: largest 3σ ellipsoid
MASt3R-SLAM / VGGT-SLAM	keyframes on / submaps on	GN+Cholesky / LM; ≈ 15 FPS (MASt3R)	image retrieval / pose graph loops	learned front end, same back end

Key take-aways

- **SLAM = joint MAP estimation** of trajectory and map from sequential frames; it is SfM with a real-time budget and loop closure.
- **Filtering** (Bayes filter $\to$ KF $\to$ EKF-SLAM) keeps one Gaussian over the current pose and map; marginalizing past poses makes the covariance dense, so the cost is $O(N^2)$ and the map must stay tiny (about 100 features at 30 Hz).

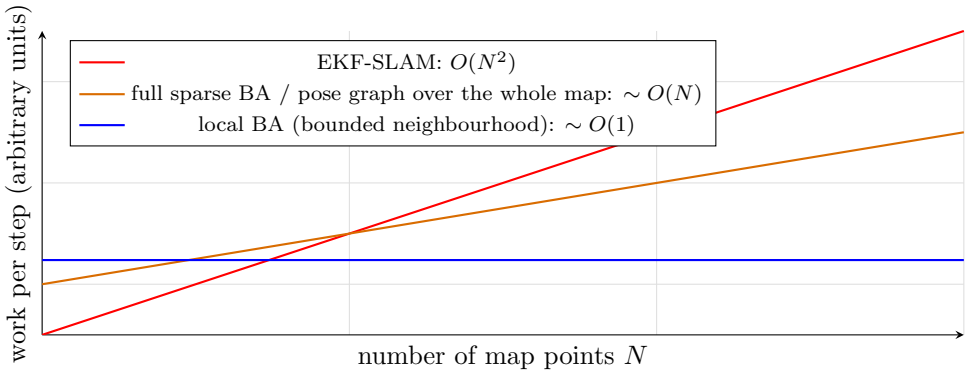


Figure 153: Qualitative scaling of the per-step cost with the map size: the filter pays quadratically for correlations, sparse least squares roughly linearly, and keyframe-based local BA stays bounded (constants are illustrative only).

- **Graph-based SLAM** keeps the keyframes: MAP on a factor graph is sparse nonlinear least squares, solved by Gauss–Newton/LM; the sparsity pattern of equals the graph adjacency.
- **A modern pipeline** (ORB-SLAM) runs tracking (every frame), local mapping (local BA on the covisibility graph) and loop closing (bags of binary words, pose graph to fix scale drift) in parallel.
- **Research directions** change the front end or the problem statement: continuous time for rolling shutter, active viewpoint selection, and learned front ends (MASt3R, VGGT) on top of the same least-squares back end.